

前端面试八股文

1. 讲讲 flex:1 和 flex:auto 有什么区别？

flex: 1 和 flex: auto 都是 Flexbox 布局中常用的简写属性，但它们的行为有重要区别。

核心区别

flex: 1 相当于：

- flex-grow: 1 (可以放大)
- flex-shrink: 1 (可以缩小)
- flex-basis: 0% (基准尺寸为 0)

flex: auto 相当于：

- flex-grow: 1 (可以放大)
- flex-shrink: 1 (可以缩小)
- flex-basis: auto (基准尺寸为元素原始尺寸)

关键差异对比

特性	flex: 1	flex: auto
初始尺寸	忽略元素原有尺寸，从0开始分配空间	尊重元素原有尺寸，作为分配起点
空间分配	按比例严格平分剩余空间	先满足元素自身尺寸，再按比例分配剩余空间
表现	类似表格的等宽列	类似浮动布局的弹性列

点击图片可查看完整电子表格

实际例子

```
XML
<div class="container">
```

```
<div class="item" style="width: 100px">A</div>
<div class="item">B</div>
<div class="item">C</div>
</div>
```

使用 `flex: 1`

```
JavaScript
.container { display: flex; width: 600px; }
.item { flex: 1; }
```

结果：三个元素完全等宽 (200px)，忽略第一个元素的 `width: 100px`

使用 `flex: auto`

```
JavaScript
.container { display: flex; width: 600px; }
.item { flex: auto; }
```

结果：

- 第一个元素：保持 100px
- 剩余 500px 空间：第二个和第三个元素平分 (各 250px)

适用场景

使用 `flex: 1` 时：

- 需要严格的等分容器空间
- 元素内容宽度不重要或一致
- 创建响应式的网格系统

使用 `flex: auto` 时：

- 需要尊重元素原有尺寸
- 内容宽度很重要（如图片、固定宽度的按钮）
- 需要混合固定宽度和弹性宽度的布局

记忆技巧

- `flex: 1` = "从零开始，完全弹性" (所有元素平等)

- `flex: auto` = "先满足自己，再分享剩余" (尊重个体差异)

这两个属性都能实现弹性布局，但选择哪一个取决于你是否需要尊重元素的初始尺寸。

2. `Promise.all`, `Promise.race`, `Promise.allSettled`, 和 `Promise.any` 的区别和各自的应用场景是什么？

`Promise.all`

```
JavaScript
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = Promise.reject('错误');

// 所有成功才成功，有一个失败就立即失败
Promise.all([p1, p2])
  .then(results => {
    console.log(results); // [1, 2]
  });

Promise.all([p1, p2, p3])
  .catch(error => {
    console.log(error); // '错误'
  });

// 应用场景：多个并行请求都成功才能继续
```

`Promise.race`

```
JavaScript
const p1 = new Promise(resolve => setTimeout(() => resolve('第一个'), 1000));
const p2 = new Promise(resolve => setTimeout(() => resolve('第二个'), 500));

// 第一个完成的结果（无论成功失败）
Promise.race([p1, p2])
```

```
.then(result => {
  console.log(result); // '第二个'
});

// 应用场景：请求超时控制
function withTimeout(promise, timeout) {
  return Promise.race([
    promise,
    new Promise((_, reject) =>
      setTimeout(() => reject('超时'), timeout)
    )
  ]);
}
```

Promise.allSettled

```
JavaScript
const p1 = Promise.resolve(1);
const p2 = Promise.reject('错误');
const p3 = Promise.resolve(3);

// 等待所有 Promise 完成，无论成功失败
Promise.allSettled([p1, p2, p3])
  .then(results => {
    console.log(results);
    // [
    //   { status: 'fulfilled', value: 1 },
    //   { status: 'rejected', reason: '错误' },
    //   { status: 'fulfilled', value: 3 }
    // ]
  });
```

// 应用场景：需要知道所有请求结果的场景

Promise.any

```
JavaScript
const p1 = Promise.reject('错误 1');
const p2 = Promise.resolve('成功 1');
const p3 = Promise.resolve('成功 2');

// 第一个成功的 Promise
```

```
Promise.any([p1, p2, p3])
  .then(result => {
    console.log(result); // '成功 1'
  });

// 全部失败时
Promise.any([p1, Promise.reject('错误 2')])
  .catch(error => {
    console.log(error.errors); // ['错误 1', '错误 2']
  });

// 应用场景：多个备用方案，只要有一个成功即可
```

3. **async/await** 是如何工作的？它和 **Promise** 的关系是什么？

基本用法

```
JavaScript
// async 函数总是返回 Promise
async function hello() {
  return 'Hello';
}
hello().then(console.log); // 'Hello'

// await 等待 Promise 完成
async function fetchData() {
  try {
    const response = await fetch('/api/data');
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('请求失败:', error);
  }
}
```

async/await 与 **Promise** 的关系

JavaScript

// async/await 是 Promise 的语法糖

```
async function example() {  
  const result = await somePromise;  
  return result + 1;  
}
```

// 等价于

```
function example() {  
  return somePromise.then(result => {  
    return result + 1;  
  });  
}
```

// 错误处理对比

// Promise 方式

```
function fetchUserPromise(id) {  
  return fetch(`/users/${id}`)  
    .then(response => response.json())  
    .then(user => fetch(`/posts/${user.id}`))  
    .then(posts => ({ user, posts }))  
    .catch(error => console.error('Error:', error));  
}
```

// async/await 方式

```
async function fetchUserAsync(id) {  
  try {  
    const userResponse = await fetch(`/users/${id}`);  
    const user = await userResponse.json();  
  
    const postsResponse = await fetch(`/posts/${user.id}`);  
    const posts = await postsResponse.json();  
  
    return { user, posts };  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}
```

并行执行优化

JavaScript


```
// 串行执行（慢）
async function serialFetch() {
  const user = await fetch('/user');
  const posts = await fetch('/posts');
  const comments = await fetch('/comments');
  return { user, posts, comments };
}

// 并行执行（快）
async function parallelFetch() {
  const [user, posts, comments] = await Promise.all([
    fetch('/user'),
    fetch('/posts'),
    fetch('/comments')
  ]);
  return { user, posts, comments };
}

// 混合执行
async function mixedFetch() {
  const user = await fetch('/user');
  const [posts, comments] = await Promise.all([
    fetch('/posts'),
    fetch('/comments')
  ]);
  return { user, posts, comments };
}
```

4. 请解释 JavaScript 中的闭包，并举一个实际的应用场景。

闭包定义

闭包是指函数能够访问并记住其词法作用域，即使该函数在其作用域之外执行。

```
JavaScript
function createCounter() {
  let count = 0; // 私有变量

  // 返回的函数形成了闭包
```

```

return {
  increment: function() {
    count++;
    return count;
  },
  decrement: function() {
    count--;
    return count;
  },
  getValue: function() {
    return count;
  }
};
}

// 使用闭包
const counter = createCounter();
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.decrement()); // 1
console.log(counter.getValue()); // 1

// 每个闭包都有自己的独立状态
const counter2 = createCounter();
console.log(counter2.getValue()); // 0 - 独立的状态

```

实际应用场景

1. 模块模式

```

JavaScript
// 使用闭包创建私有变量和公共接口
const userModule = (function() {
  let users = []; // 私有变量

  function isValidUser(user) {
    return user && user.name && user.email;
  }

  // 公共接口
  return {
    addUser: function(user) {
      if (isValidUser(user)) {

```



```

        users.push(user);
        return true;
    }
    return false;
},
getUsers: function() {
    return [...users]; // 返回副本，保护原始数据
},
getUserCount: function() {
    return users.length;
}
};
})();

userModule.addUser({ name: 'Alice', email: 'alice@example.com' });
console.log(userModule.getUserCount()); // 1
console.log(userModule.users); // undefined - 无法直接访问私有变量

```

1. 事件处理和回调

JavaScript

```

function createButtonHandlers() {
    const buttons = document.querySelectorAll('button');

    buttons.forEach((button, index) => {
        // 每个回调函数都记住了自己的 index
        button.addEventListener('click', function() {
            console.log(`按钮 ${index} 被点击了`);
            // 可以访问外部的 index 变量
        });
    });
}

// 另一种常见用法 - 函数工厂
function createMultiplier(factor) {
    return function(number) {
        return number * factor;
    };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10

```

```
console.log(triple(5)); // 15
```

5. **this** 的指向在不同场景下（普通函数、箭头函数、构造函数、**call**/**apply**/**bind**）是如何决定的？

this 绑定规则

JavaScript

// 1. 默认绑定（普通函数调用）

```
function showThis() {  
  console.log(this); // 浏览器中：Window, Node.js 中：global  
}  
showThis();
```

// 严格模式下

```
function strictShowThis() {  
  'use strict';  
  console.log(this); // undefined  
}  
strictShowThis();
```

// 2. 隐式绑定（方法调用）

```
const obj = {  
  name: 'Alice',  
  sayHello: function() {  
    console.log(`Hello, ${this.name}`);  
  }  
};  
obj.sayHello(); // Hello, Alice
```

// 隐式绑定丢失问题

```
const helloFunc = obj.sayHello;  
helloFunc(); // Hello, undefined - this 指向全局
```

// 3. 显式绑定（**call**/**apply**/**bind**）

```
function introduce(language, hobby) {  
  console.log(`${this.name} 喜欢 ${language} 和 ${hobby}`);  
}
```

```
const person = { name: 'Bob' };

// call - 立即调用，参数逐个传递
introduce.call(person, 'JavaScript', '编程');

// apply - 立即调用，参数数组传递
introduce.apply(person, ['Python', '阅读']);

// bind - 返回新函数，延迟调用
const boundIntroduce = introduce.bind(person, 'Java');
boundIntroduce('游泳');

// 4. new 绑定（构造函数）
function Person(name) {
  this.name = name;
  this.sayName = function() {
    console.log(this.name);
  };
}
const john = new Person('John');
john.sayName(); // John

// 5. 箭头函数（无自己的 this，继承外层）
const arrowObj = {
  name: 'Arrow',
  traditional: function() {
    console.log('传统函数:', this.name); // Arrow
  },
  arrow: () => {
    console.log('箭头函数:', this.name); // undefined 或外层 this
  },
  nested: function() {
    const innerArrow = () => {
      console.log('嵌套箭头:', this.name); // Arrow - 继承外层函数的 this
    };
    innerArrow();
  }
};
arrowObj.traditional();
arrowObj.arrow();
arrowObj.nested();
```

this 绑定优先级

new 绑定 > 显式绑定 > 隐式绑定 > 默认绑定

```
JavaScript
// 绑定优先级示例
function test() {
  console.log(this.name);
}

const obj1 = { name: 'obj1' };
const obj2 = { name: 'obj2' };

// 显式绑定优先级高于隐式绑定
obj1.test = test;
obj1.test.call(obj2); // obj2

// new 绑定优先级最高
const boundTest = test.bind(obj1);
const instance = new boundTest(); // undefined - new 覆盖了 bind
```

6. 请解释什么是原型和原型链。proto 和 prototype 有什么区别？

原型基础

```
JavaScript
// 构造函数
function Person(name) {
  this.name = name;
}

// 在原型上添加方法
Person.prototype.sayHello = function() {
  console.log(`Hello, I'm ${this.name}`);
};

// 创建实例
const alice = new Person('Alice');
```

```
const bob = new Person('Bob');

alice.sayHello(); // Hello, I'm Alice
bob.sayHello();   // Hello, I'm Bob

// 所有实例共享原型方法
console.log(alice.sayHello === bob.sayHello); // true
```

proto vs prototype

```
JavaScript
function Animal(type) {
  this.type = type;
}

Animal.prototype.breathe = function() {
  console.log('呼吸中...');
};

const dog = new Animal('哺乳动物');

// __proto__ 和 prototype 的关系
console.log(dog.__proto__ === Animal.prototype); // true
console.log(Animal.prototype.constructor === Animal); // true

// 原型链
console.log(dog.__proto__.__proto__ === Object.prototype); // true
console.log(Object.prototype.__proto__); // null

// 区别总结:
// - prototype 是函数特有的属性, 指向原型对象
// - __proto__ 是每个对象都有的属性, 指向构造函数的原型
```

完整的原型链示例

```
JavaScript
function Vehicle(brand) {
  this.brand = brand;
}

Vehicle.prototype.start = function() {
```

```

    console.log(`${this.brand} 启动`);
  };

  function Car(brand, model) {
    Vehicle.call(this, brand);
    this.model = model;
  }

  // 设置原型链继承
  Car.prototype = Object.create(Vehicle.prototype);
  Car.prototype.constructor = Car;

  Car.prototype.drive = function() {
    console.log(`${this.brand} ${this.model} 行驶中`);
  };

  const myCar = new Car('Toyota', 'Camry');

  // 原型链: myCar -> Car.prototype -> Vehicle.prototype ->
  // Object.prototype -> null
  console.log(myCar.__proto__ === Car.prototype); // true
  console.log(Car.prototype.__proto__ === Vehicle.prototype); //
  true
  console.log(Vehicle.prototype.__proto__ === Object.prototype); //
  true

  myCar.start(); // Toyota 启动 (来自 Vehicle)
  myCar.drive(); // Toyota Camry 行驶中 (来自 Car)

```

7. 请实现一个深拷贝函数。

基础深拷贝实现

```

JavaScript
function deepClone(obj, hash = new WeakMap()) {
  // 处理基本类型和 null
  if (obj === null || typeof obj !== 'object') {
    return obj;
  }

```



```

// 处理 Date 对象
if (obj instanceof Date) {
  return new Date(obj.getTime());
}

// 处理 RegExp 对象
if (obj instanceof RegExp) {
  return new RegExp(obj);
}

// 处理数组
if (Array.isArray(obj)) {
  return obj.map(item => deepClone(item, hash));
}

// 处理循环引用
if (hash.has(obj)) {
  return hash.get(obj);
}

// 处理普通对象
const cloneObj = Object.create(Object.getPrototypeOf(obj));
hash.set(obj, cloneObj);

for (let key in obj) {
  if (obj.hasOwnProperty(key)) {
    cloneObj[key] = deepClone(obj[key], hash);
  }
}

return cloneObj;
}

```

更完整的深拷贝实现

```

JavaScript
function deepCloneComplete(target, map = new WeakMap()) {
  // 基本数据类型直接返回
  if (target === null || typeof target !== 'object') {
    return target;
  }

  // 防止循环引用

```

```
if (map.has(target)) {
  return map.get(target);
}

// 处理各种对象类型
const constructor = target.constructor;

// 处理 Date
if (target instanceof Date) {
  return new Date(target);
}

// 处理 RegExp
if (target instanceof RegExp) {
  return new RegExp(target);
}

// 处理 Set
if (target instanceof Set) {
  const cloneSet = new Set();
  map.set(target, cloneSet);
  target.forEach(value => {
    cloneSet.add(deepCloneComplete(value, map));
  });
  return cloneSet;
}

// 处理 Map
if (target instanceof Map) {
  const cloneMap = new Map();
  map.set(target, cloneMap);
  target.forEach((value, key) => {
    cloneMap.set(key, deepCloneComplete(value, map));
  });
  return cloneMap;
}

// 处理 ArrayBuffer
if (target instanceof ArrayBuffer) {
  return target.slice(0);
}

// 处理 TypedArray
if (ArrayBuffer.isView(target)) {
```

```

    return new target.constructor(target);
  }

  // 处理普通对象和数组
  const cloneObj = Array.isArray(target) ? [] :
Object.create(Object.getPrototypeOf(target));
  map.set(target, cloneObj);

  // 使用 Reflect.ownKeys 获取所有属性（包括 Symbol）
  const keys = Reflect.ownKeys(target);

  for (let key of keys) {
    if (target.hasOwnProperty(key)) {
      cloneObj[key] = deepCloneComplete(target[key], map);
    }
  }

  return cloneObj;
}

```

测试深拷贝

```

JavaScript
// 测试数据
const original = {
  name: 'Alice',
  age: 25,
  hobbies: ['reading', 'coding'],
  address: {
    city: 'Beijing',
    street: 'Main St'
  },
  date: new Date(),
  regex: /hello/gi,
  set: new Set([1, 2, 3]),
  map: new Map([['key', 'value']]),
  func: function() { return this.name; },
  [Symbol('id')]: 123
};

// 添加循环引用
original.self = original;

```

```
// 执行深拷贝
const cloned = deepCloneComplete(original);

// 验证
console.log(cloned !== original); // true
console.log(cloned.address !== original.address); // true
console.log(cloned.hobbies !== original.hobbies); // true
console.log(cloned.set !== original.set); // true
console.log(cloned.self === cloned); // true - 循环引用正确处理
console.log(cloned.func()); // Alice - 函数正常工作
```

8. 谈谈你对 ES6 模块和 CommonJS 规范的理解，它们有什么主要差异？

ES6 模块 (ESM)

```
JavaScript
// math.js - 导出
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

// 默认导出
export default class Calculator {
  static version = '1.0';
}

// app.js - 导入
import Calculator, { PI, add, multiply } from './math.js';
// 或者全部导入
import * as math from './math.js';

console.log(PI); // 3.14159
```

```
console.log(add(2, 3)); // 5
console.log(Calculator.version); // 1.0
```

CommonJS

```
JavaScript
// math.js - 导出
const PI = 3.14159;

function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

// 方式 1: module.exports
module.exports = {
  PI,
  add,
  multiply
};

// 方式 2: exports
exports.PI = PI;
exports.add = add;

// app.js - 导入
const math = require('./math.js');
// 或者解构
const { PI, add } = require('./math.js');

console.log(math.PI); // 3.14159
console.log(add(2, 3)); // 5
```

主要差异对比

特性	ES6 模块 (ESM)	CommonJS
加载时机	编译时静态分析	运行时动态加载
导入/导出	import / export	require() / module.exports
值类型	值的引用 (动态绑定)	值的拷贝 (静态缓存)
异步支持	原生支持	不支持 (需要 Promise)

点击图片可查看完整电子表格

值引用 vs 值拷贝示例

```
JavaScript
// ES6 模块 - 值引用
// counter.js
export let count = 0;
export function increment() {
  count++;
}

// app.js
import { count, increment } from './counter.js';
console.log(count); // 0
increment();
console.log(count); // 1 - 值被更新

// CommonJS - 值拷贝
// counter.js
let count = 0;
function increment() {
  count++;
}
module.exports = { count, increment };

// app.js
const { count, increment } = require('./counter.js');
console.log(count); // 0
increment();
```



```
console.log(count); // 0 - 原始值不变（拷贝的是初始值）
```

在 Node.js 中使用 ESM

```
JavaScript
// package.json
{
  "type": "module", // 启用 ESM
  "scripts": {
    "start": "node app.js"
  }
}

// 或者使用 .mjs 扩展名
// app.mjs
import { add } from './math.mjs';

// 动态导入（两者都支持）
const math = await import('./math.mjs');
```

9. Map 和 Set 与 Object 和 Array 相比，有什么优势？

Map vs Object

```
JavaScript
// Map 的优势
const map = new Map();
const obj = {};

// 1. 键的类型：Map 的键可以是任意类型
map.set(1, '数字键');
map.set('1', '字符串键');
map.set({}, '对象键');
map.set([], '数组键');
map.set(() => {}, '函数键');

// Object 的键只能是字符串或 Symbol
```

```
obj[1] = '数字键'; // 会被转换为字符串 '1'
obj['1'] = '字符串键';
console.log(obj); // { '1': '字符串键' } - 数字键被覆盖

// 2. 顺序保证: Map 保持插入顺序
map.set('a', 1);
map.set('b', 2);
map.set('c', 3);
console.log([...map.keys()]); // ['a', 'b', 'c'] - 保持顺序

// Object 的键顺序不保证 (虽然现代引擎通常按插入顺序, 但非标准)
obj.a = 1;
obj.b = 2;
obj.c = 3;
console.log(Object.keys(obj)); // ['a', 'b', 'c'] - 但特殊键 (如数字) 会重新排序

// 3. 性能: 频繁增删的场景下 Map 性能更好
const performanceTest = () => {
  const map = new Map();
  const obj = {};
  const size = 100000;

  // 添加性能
  console.time('Map 添加');
  for (let i = 0; i < size; i++) {
    map.set(i, i);
  }
  console.timeEnd('Map 添加');

  console.time('Object 添加');
  for (let i = 0; i < size; i++) {
    obj[i] = i;
  }
  console.timeEnd('Object 添加');

  // 删除性能
  console.time('Map 删除');
  for (let i = 0; i < size; i++) {
    map.delete(i);
  }
  console.timeEnd('Map 删除');
```

```
console.time('Object 删除');
for (let i = 0; i < size; i++) {
  delete obj[i];
}
console.timeEnd('Object 删除');
};

// 4. 大小获取: Map 有 size 属性
console.log(map.size); // 直接获取

// Object 需要计算
console.log(Object.keys(obj).length);

// 5. 迭代: Map 原生可迭代
for (let [key, value] of map) {
  console.log(key, value);
}

// Object 需要先获取键数组
for (let key in obj) {
  if (obj.hasOwnProperty(key)) {
    console.log(key, obj[key]);
  }
}
```

Set vs Array

```
C++
// Set 的优势
const set = new Set();
const arr = [];

// 1. 唯一性: Set 自动去重
set.add(1);
set.add(2);
set.add(2); // 重复, 不会被添加
set.add(3);
console.log(set.size); // 3

// Array 需要手动去重
arr.push(1, 2, 2, 3);
const uniqueArr = [...new Set(arr)]; // 需要额外处理
```

```
console.log(uniqueArr); // [1, 2, 3]

// 2. 存在性检查: Set 有 has 方法, 性能 O(1)
console.log(set.has(2)); // true
console.log(set.has(4)); // false

// Array 需要 indexOf 或 includes, 性能 O(n)
console.log(arr.includes(2)); // true
console.log(arr.indexOf(4) !== -1); // false

// 3. 删除操作: Set 删除性能更好
set.delete(2); // 快速删除
console.log(set.has(2)); // false

// Array 删除需要查找索引或过滤
const index = arr.indexOf(2);
if (index > -1) {
  arr.splice(index, 1); // 性能较差, 需要移动元素
}

// 4. 应用场景
// 数据去重
const duplicateData = [1, 2, 2, 3, 4, 4, 5];
const uniqueData = [...new Set(duplicateData)];

// 集合运算
const setA = new Set([1, 2, 3]);
const setB = new Set([2, 3, 4]);

// 并集
const union = new Set([...setA, ...setB]);

// 交集
const intersection = new Set([...setA].filter(x => setB.has(x)));

// 差集
const difference = new Set([...setA].filter(x => !setB.has(x)));

console.log(union); // Set(4) {1, 2, 3, 4}
console.log(intersection); // Set(2) {2, 3}
console.log(difference); // Set(1) {1}
```

10. 请解释一下浏览器中的同源策略以及如何解决跨域问题。

同源策略定义

同源策略要求协议、域名、端口完全相同

```
JavaScript
// 同源检查
function isSameOrigin(url1, url2) {
  const a = new URL(url1);
  const b = new URL(url2);

  return a.protocol === b.protocol &&
    a.hostname === b.hostname &&
    a.port === b.port;
}

// 示例
console.log(isSameOrigin(
  'https://example.com:443/page1',
  'https://example.com:443/page2'
)); // true

console.log(isSameOrigin(
  'https://example.com',
  'http://example.com'
)); // false - 协议不同

console.log(isSameOrigin(
  'https://example.com',
  'https://api.example.com'
)); // false - 域名不同

console.log(isSameOrigin(
  'https://example.com:443',
  'https://example.com:8080'
)); // false - 端口不同
```

跨域解决方案

1. CORS (主流方案)

```

JavaScript
// 服务端设置响应头
app.use((req, res, next) => {
  // 允许的源
  res.header('Access-Control-Allow-Origin',
'https://example.com');
  // 允许的请求方法
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT,
DELETE, OPTIONS');
  // 允许的请求头
  res.header('Access-Control-Allow-Headers', 'Content-Type,
Authorization');
  // 允许携带凭证
  res.header('Access-Control-Allow-Credentials', 'true');
  // 预检请求缓存时间
  res.header('Access-Control-Max-Age', '86400');

  if (req.method === 'OPTIONS') {
    return res.sendStatus(200);
  }
  next();
});

```

1. JSONP (传统方案)

```

JavaScript
// 前端实现
function jsonp(url, callbackName) {
  return new Promise((resolve, reject) => {
    // 创建 script 标签
    const script = document.createElement('script');

    // 定义回调函数
    window[callbackName] = (data) => {
      resolve(data);
      document.body.removeChild(script);
      delete window[callbackName];
    };

    // 设置 script 的 src
    script.src = `${url}?callback=${callbackName}`;
    script.onerror = reject;
  });
}

```



```

    document.body.appendChild(script);
  });
}

// 使用
jsonp('https://api.example.com/data', 'handleData')
  .then(data => console.log(data))
  .catch(err => console.error(err));

// 服务端响应
app.get('/data', (req, res) => {
  const callback = req.query.callback;
  const data = { message: 'Hello JSONP' };
  res.send(` ${callback} (${JSON.stringify(data)}) `);
});

```

1. 代理服务器

```

Java
// 开发环境代理配置 (webpack)
module.exports = {
  devServer: {
    proxy: {
      '/api': {
        target: 'https://api.example.com',
        changeOrigin: true,
        pathRewrite: {
          '^/api': ''
        }
      }
    }
  }
};

// Nginx 代理配置
// location /api/ {
//   proxy_pass https://api.example.com/;
//   proxy_set_header Host $host;
//   proxy_set_header X-Real-IP $remote_addr;
// }

```

1. 其他方案

JavaScript

```
// postMessage - 跨窗口通信
// 窗口 A
const iframe = document.getElementById('iframe');
iframe.contentWindow.postMessage('Hello', 'https://target-origin.com');

// 窗口 B
window.addEventListener('message', (event) => {
  if (event.origin !== 'https://trusted-origin.com') return;
  console.log('收到消息:', event.data);
});

// document.domain - 仅适用于子域名
// 主域名相同的情况下，设置相同的 document.domain
// 在 a.example.com 和 b.example.com 中都设置：
document.domain = 'example.com';
```

11. 什么是 CORS？请描述一下它的工作原理，特别是预检请求。

简单请求 vs 预检请求

```
SQL
// 简单请求条件：
// 1. 方法为 GET、POST、HEAD
// 2. 请求头只能是：Accept、Accept-Language、Content-Language、Content-Type
// 3. Content-Type 只能是：application/x-www-form-urlencoded、multipart/form-data、text/plain

// 简单请求示例
fetch('https://api.example.com/data', {
  method: 'GET',
  headers: {
    'Content-Type': 'text/plain'
  }
});
```

```
// 复杂请求 - 会触发预检请求
fetch('https://api.example.com/data', {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json',
    'X-Custom-Header': 'value'
  },
  body: JSON.stringify({ data: 'test' })
});
```

预检请求详细流程

YAML

```
// 1. 浏览器发送 OPTIONS 预检请求
// 请求头包含:
OPTIONS /data HTTP/1.1
Host: api.example.com
Origin: https://example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: content-type,x-custom-header

// 2. 服务器响应预检请求
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, X-Custom-Header
Access-Control-Max-Age: 86400
Access-Control-Allow-Credentials: true

// 3. 浏览器发送实际请求
PUT /data HTTP/1.1
Host: api.example.com
Origin: https://example.com
Content-Type: application/json
X-Custom-Header: value

// 4. 服务器响应实际请求
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://example.com
Content-Type: application/json

{"status": "success"}
```

CORS 错误处理

JavaScript

// 前端错误处理

```
async function makeCorsRequest() {
  try {
    const response = await fetch('https://api.example.com/data', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      credentials: 'include', // 携带 cookie
      body: JSON.stringify({ data: 'test' })
    });

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    if (error.name === 'TypeError' &&
    error.message.includes('Failed to fetch')) {
      console.error('CORS 错误: 请求被阻止');
    } else {
      console.error('其他错误: ', error);
    }
  }
}
```

// 服务端完整的 CORS 中间件

```
function corsMiddleware(req, res, next) {
  const allowedOrigins = ['https://example.com',
    'https://app.example.com'];
  const origin = req.headers.origin;

  // 检查请求来源是否在允许列表中
  if (allowedOrigins.includes(origin)) {
    res.header('Access-Control-Allow-Origin', origin);
  }

  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT,
```

```
DELETE, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type,
Authorization, X-Requested-With');
  res.header('Access-Control-Allow-Credentials', 'true');
  res.header('Access-Control-Max-Age', '86400');

  // 处理预检请求
  if (req.method === 'OPTIONS') {
    return res.status(200).end();
  }

  next();
}
```

12. 描述一下从用户在浏览器地址栏输入 URL 到页面完整展示的整个过程。

```
JavaScript
// 整个过程的伪代码表示
async function browserNavigation(url) {
  // 1. 用户输入 URL
  console.log('1. 用户输入 URL:', url);

  // 2. URL 解析
  const parsedUrl = new URL(url);

  // 3. DNS 解析
  const ipAddress = await dnsLookup(parsedUrl.hostname);
  console.log('3. DNS 解析完成, IP:', ipAddress);

  // 4. TCP 连接
  const tcpConnection = await establishTCPConnection(ipAddress,
443);
  console.log('4. TCP 连接建立');

  // 5. TLS 握手 (HTTPS)
  if (parsedUrl.protocol === 'https:') {
    await performTLSHandshake(tcpConnection);
    console.log('5. TLS 握手完成');
```

```
}

// 6. HTTP 请求
const httpResponse = await sendHttpRequest(tcpConnection,
parsedUrl);
console.log('6. HTTP 响应接收, 状态码:',
httpResponse.statusCode);

// 7. 处理响应
if (httpResponse.statusCode === 301 || httpResponse.statusCode
=== 302) {
  // 重定向
  console.log('7. 重定向到:', httpResponse.headers.location);
  return browserNavigation(httpResponse.headers.location);
}

// 8. 解析 HTML
const domTree = parseHTML(httpResponse.body);
console.log('8. DOM 树构建完成');

// 9. 解析 CSS
const cssom = parseCSS(extractCSS(httpResponse.body));
console.log('9. CSSOM 构建完成');

// 10. 构建渲染树
const renderTree = buildRenderTree(domTree, cssom);
console.log('10. 渲染树构建完成');

// 11. 布局
const layout = calculateLayout(renderTree);
console.log('11. 布局计算完成');

// 12. 绘制
const paintCommands = generatePaintCommands(renderTree, layout);
console.log('12. 绘制命令生成完成');

// 13. 合成与显示
await compositeAndDisplay(paintCommands);
console.log('13. 页面显示完成');

// 14. 加载后续资源
await loadAdditionalResources(domTree);
```



```
console.log('14. 所有资源加载完成');

// 15. 执行 JavaScript
executeJavaScript(extractJavaScript(httpResponse.body));
console.log('15. JavaScript 执行完成');
}
```

关键阶段详解

1. 构建 DOM 树

```
JavaScript
// HTML 解析过程
function parseHTML(html) {
  // 字节 → 字符 → 令牌 → 节点 → DOM
  const tokens = tokenize(html);
  const domTree = buildDOM(tokens);

  // 遇到 script 标签会暂停解析，执行 JavaScript
  // async 和 defer 属性改变执行时机
  return domTree;
}
```

1. 构建 CSSOM

```
JavaScript
function parseCSS(cssText) {
  // CSS 解析是渲染阻塞的
  const cssRules = parseCSSRules(cssText);
  const cssom = buildCSSOM(cssRules);
  return cssom;
}
```

1. 渲染树 → 布局 → 绘制

```
JavaScript
function renderPipeline(dom, cssom) {
  // 构建渲染树（只包含可见元素）
  const renderTree = domTree + cssom;

  // 布局（重排）
  const layout = calculateLayout(renderTree);
}
```

```
// 绘制（重绘）
const paint = generatePaintCommands(layout);

// 合成
const composite = layerComposition(paint);

return composite;
}
```

1. 关键渲染路径优化

```
JavaScript
// 优化建议
const optimizationTips = {
  // 1. 减少关键资源数量
  criticalResources: '延迟加载非关键 CSS/JS',

  // 2. 优化关键资源大小
  resourceSize: '压缩、Tree Shaking、代码分割',

  // 3. 缩短关键路径长度
  criticalPath: '并行下载、预连接、CDN',

  // 4. 避免阻塞渲染
  renderBlocking: 'CSS 放头部、JS 放底部或使用 async/defer'
};

// 使用 defer 和 async
<script src="script.js" defer></script> // DOM 解析完成后执行
<script src="script.js" async></script> // 下载完成后立即执行
```

13. 什么是 Tree Shaking? 它的工作原理是什么?

基本概念

Tree Shaking 是通过静态分析移除 JavaScript 中未使用的代码

```
JavaScript
// 源代码
// math.js
```

```
export function add(a, b) {  
  return a + b;  
}  
  
export function multiply(a, b) {  
  return a * b;  
}  
  
export function subtract(a, b) {  
  return a - b;  
}  
  
// app.js  
import { add, multiply } from './math.js';  
  
console.log(add(1, 2));  
console.log(multiply(3, 4));  
  
// Tree Shaking 后  
// 只会保留 add 和 multiply 函数
```

工作原理

1. 静态分析阶段

```
JavaScript  
// Webpack 的 Tree Shaking 过程  
function treeShaking(moduleGraph) {  
  // 标记所有导出  
  const exports = collectExports(moduleGraph);  
  
  // 从入口开始追踪使用情况  
  const usedExports = traceUsageFromEntry(moduleGraph);  
  
  // 标记未使用的导出  
  const unusedExports = findUnusedExports(exports, usedExports);  
  
  // 移除未使用的代码  
  return removeUnusedCode(moduleGraph, unusedExports);  
}
```

1. 实际示例

```
JavaScript
// 场景 1: 直接导入使用
import { a, b } from './module';
console.log(a); // b 会被摇掉

// 场景 2: 命名空间导入
import * as utils from './utils';
console.log(utils.a); // 只有 a 被保留

// 场景 3: 副作用处理
import { Button } from 'ui-library';
// 如果 ui-library 有副作用, 需要特殊处理
```

配置和使用

Webpack 配置

```
Java
// webpack.config.js
module.exports = {
  mode: 'production', // 生产模式自动启用 Tree Shaking

  optimization: {
    usedExports: true, // 标记已使用的导出
    minimize: true,    // 启用代码压缩
    sideEffects: false // 假设所有模块都没有副作用
  }
};
```

package.json 配置

```
JSON
{
  "name": "my-package",
  "sideEffects": [
    "*.css",
    "*.scss",
    "./src/polyfill.js"
  ]
}
```

副作用处理

```
JavaScript
// 有副作用的模块需要特殊标记
// polyfill.js - 有副作用
import 'core-js/stable';
import 'regenerator-runtime/runtime';
// 这个文件没有导出，但有执行代码

// utils.js - 纯函数（无副作用）
export function pureFunction(a, b) {
  return a + b;
}

// 有副作用的函数
let counter = 0;
export function withSideEffect() {
  counter++; // 修改外部状态
  console.log('Called', counter, 'times');
}
```

最佳实践

```
JavaScript
// 1. 使用 ES6 模块语法
// 推荐 - 静态分析友好
import { specificFunction } from './module';
export const namedExport = () => {};

// 避免 - 动态分析困难
const dynamicImport = require('./module');
module.exports = something;

// 2. 避免副作用
// 推荐 - 纯函数
export function calculate(a, b) {
  return a * b;
}

// 避免 - 有副作用的代码
export function initialize() {
  window.myApp = {}; // 修改全局对象
}
```

```

}

// 3. 正确标记副作用
// package.json
{
  "sideEffects": [
    "**/*.css",
    "src/setup.js"
  ]
}

// 4. 使用工具分析
const { BundleAnalyzerPlugin } = require('webpack-bundle-analyzer');
module.exports = {
  plugins: [
    new BundleAnalyzerPlugin()
  ]
};

```

14. 防抖节流的区别是什么？请分别实现这两个函数。

核心区别

特性	防抖 (Debounce)
定义	在事件触发后等待一段时间执行，若在此期间再次触发则重新计时
执行时机	连续触发结束后执行
适用场景	搜索框输入、窗口调整大小

点击图片可查看完整电子表格

实现代码

14.1.1 防抖函数 (Debounce)

```

JavaScript
function debounce(func, wait, immediate = false) {

```



```

let timeout;

return function executedFunction(...args) {
  const context = this;

  const later = function() {
    timeout = null;
    if (!immediate) func.apply(context, args);
  };

  const callNow = immediate && !timeout;

  clearTimeout(timeout);
  timeout = setTimeout(later, wait);

  if (callNow) func.apply(context, args);
};
}

// 使用示例
const handleSearch = debounce((query) => {
  console.log('搜索:', query);
  // 实际搜索逻辑
}, 300);

// 输入时频繁调用，但只会在停止输入 300ms 后执行
input.addEventListener('input', (e) => {
  handleSearch(e.target.value);
});

```

14.1.2 节流函数 (Throttle)

```

JavaScript
function throttle(func, limit) {
  let inThrottle;

  return function(...args) {
    const context = this;

    if (!inThrottle) {
      func.apply(context, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}

```

```

    }
  };
}

// 时间戳版本（立即执行）
function throttleTimestamp(func, limit) {
  let lastCall = 0;

  return function(...args) {
    const context = this;
    const now = Date.now();

    if (now - lastCall >= limit) {
      func.apply(context, args);
      lastCall = now;
    }
  };
}

// 使用示例
const handleScroll = throttle(() => {
  console.log('处理滚动事件');
  // 滚动处理逻辑
}, 100);

window.addEventListener('scroll', handleScroll);

```

高级实现（支持配置选项）

```

JavaScript
// 高级防抖函数
function advancedDebounce(func, wait, options = {}) {
  const {
    leading = false,    // 是否在开始时执行
    trailing = true,    // 是否在结束时执行
    maxWait            // 最大等待时间
  } = options;

  let timeout;
  let lastCallTime;
  let lastInvokeTime = 0;
  let result;

```

```

function invokeFunc(time) {
  const args = arguments;
  result = func.apply(this, args);
  lastInvokeTime = time;
  return result;
}

function shouldInvoke(time) {
  const timeSinceLastCall = time - lastCallTime;
  const timeSinceLastInvoke = time - lastInvokeTime;

  return (lastCallTime === undefined) ||
    (timeSinceLastCall >= wait) ||
    (maxWait !== undefined && timeSinceLastInvoke >=
maxWait);
}

return function(...args) {
  const time = Date.now();
  const isInvoking = shouldInvoke(time);

  lastCallTime = time;

  if (isInvoking) {
    if (leading) {
      return invokeFunc(time);
    }
  }

  if (timeout === undefined && trailing) {
    timeout = setTimeout(() => {
      timeout = undefined;
      if (trailing && lastCallTime !== time) {
        return invokeFunc(Date.now());
      }
    }, wait);
  }

  return result;
};
}

```

实际应用场景

防抖应用

JavaScript

// 搜索框防抖

```
const searchInput = document.getElementById('search');
const debouncedSearch = debounce((value) => {
  fetch(`/api/search?q=${value}`)
    .then(response => response.json())
    .then(data => console.log(data));
}, 500);
```

```
searchInput.addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});
```

// 窗口调整防抖

```
const debouncedResize = debounce(() => {
  console.log('窗口大小:', window.innerWidth, window.innerHeight);
}, 250);
```

```
window.addEventListener('resize', debouncedResize);
```

节流应用

JavaScript

// 滚动加载更多

```
const throttledScroll = throttle(() => {
  const scrollTop = window.scrollY;
  const windowHeight = window.innerHeight;
  const documentHeight = document.documentElement.scrollHeight;

  if (scrollTop + windowHeight >= documentHeight - 100) {
    loadMoreContent();
  }
}, 200);
```

```
window.addEventListener('scroll', throttledScroll);
```

// 按钮防重复点击

```
const throttledClick = throttle(() => {
  submitForm();
}, 1000);
```

```
submitButton.addEventListener('click', throttledClick);
```

总结

- 防抖：适合处理"最终状态"，如搜索输入完成、窗口调整结束
- 节流：适合处理"过程状态"，如滚动、鼠标移动等连续事件

两者都是优化性能的重要手段，根据具体场景选择合适的策略可以显著提升用户体验。

15. 在 React (或 Vue) 中，父子组件之间如何通信？ 兄弟组件之间呢？

React 中的通信

父子组件通信：

1. 父传子：通过 props 传递数据和函数

```
JavaScript
// 父组件
<ChildComponent data={data} onEvent={handleEvent} />

// 子组件
function ChildComponent({ data, onEvent }) {
  return <button onClick={() => onEvent('data')}>Click</button>;
}
```

2. 子传父：父组件通过 props 传递回调函数，子组件调用该函数

```
JavaScript
// 父组件
function Parent() {
  const handleChildData = (data) => { /* 处理数据 */ };
  return <Child onSendData={handleChildData} />;
}

// 子组件
function Child({ onSendData }) {
  return <button onClick={() =>
```

```
onSendData('hello')}}>Send</button>;  
}
```

兄弟组件通信:

1. 状态提升: 将共享状态提升到最近共同父组件
2. Context API: 创建全局状态上下文
3. Redux/Mobx: 使用状态管理库
4. 事件总线/自定义事件 (较少使用)

Vue 中的通信

父子组件通信:

1. 父传子: 通过 props

```
XML  
<!-- 父组件 -->  
<ChildComponent :data="data" @custom-event="handleEvent" />  
  
<!-- 子组件 -->  
<script>  
export default {  
  props: ['data'],  
  emits: ['custom-event'],  
  methods: {  
    sendData() {  
      this.$emit('custom-event', 'data');  
    }  
  }  
}  
</script>
```

2. 子传父: 通过 `$emit` 触发自定义事件

兄弟组件通信:

1. Event Bus: Vue 2 中使用, Vue 3 推荐使用 mitt 等库
2. Vuex/Pinia: 状态管理
3. 状态提升: 通过共同的父组件
4. provide/inject: 依赖注入 (适合深层嵌套)

16. React Hooks 解决了什么问题？

Hooks 解决的问题：

1. 状态逻辑复用困难：解决 Class 组件中 mixins、HOC、render props 的复杂性问题
2. 复杂组件难以理解：将相互关联的代码拆分到不同生命周期的问题
3. Class 的学习成本：避免 this 绑定、理解 JavaScript Class 的复杂性
4. 函数组件的功能增强：让函数组件拥有状态和生命周期能力

常用 Hooks：

1. useState - 状态管理

```
JavaScript
const [count, setCount] = useState(0);
// 管理组件内部状态
```

2. useEffect - 副作用处理

```
JavaScript
useEffect(() => {
  // 相当于 componentDidMount + componentDidUpdate
  fetchData();
  return () => {
    // 清理函数，相当于 componentWillUnmount
  };
}, [dependency]); // 依赖数组
```

3. useContext - 上下文访问

```
JavaScript
const theme = useContext(ThemeContext);
// 避免多层 props 传递
```

4. useReducer - 复杂状态管理

```
JavaScript
const [state, dispatch] = useReducer(reducer, initialState);
// 适合复杂状态逻辑，类似 Redux
```

5. useCallback - 记忆函数

```
JavaScript
const memoizedCallback = useCallback(
  () => { doSomething(a, b); },
  [a, b], // 依赖项
);
// 避免不必要的重渲染
```

6. useMemo - 记忆值

```
JavaScript
const memoizedValue = useMemo(
  () => computeExpensiveValue(a, b),
  [a, b],
);
// 缓存计算结果
```

7. useRef - DOM 引用/持久化值

```
JavaScript
const inputRef = useRef(null);
// 访问 DOM 元素或保存 mutable 值
```

8. 自定义 Hooks - 逻辑复用

```
JavaScript
function useLocalStorage(key, initialValue) {
  // 封装可复用的逻辑
}
```

17. 请简述其 Diff 算法的工作原理。

虚拟 DOM 是什么？

虚拟 DOM 是一个轻量级的 JavaScript 对象，它是对真实 DOM 的抽象表示。React/Vue 等框架使用虚拟 DOM 作为中间层，用于描述 UI 应该是什么样子。

优点：

1. 性能优化：减少直接操作真实 DOM 的次数
2. 跨平台能力：可以渲染到不同平台（Web、Native、Canvas）

3. 声明式编程：开发者只需关心状态，框架负责更新 UI

Diff 算法工作原理

核心思想：

- 同级比较，不跨级比较
- 不同类型的组件会完全重建
- 通过 key 属性识别元素稳定性

Diff 过程：

1. Tree Diff（树比较）

```
JavaScript
// React 只比较同一层级的节点
// O(n) 复杂度，而非传统的 O(n³)
```

2. Component Diff（组件比较）

- 相同类型组件：更新 props，递归比较子节点
- 不同类型组件：直接替换整个组件

3. Element Diff（元素比较）

当比较同一层级的子节点列表时，React 使用以下策略：

传统列表比较（没有 key）：

旧: A B C D

新: A C B E

比较过程：

A 相同 → 保留

B ≠ C → 删除 B，创建 C

C ≠ B → 删除 C，创建 B

D ≠ E → 删除 D，创建 E

使用 key 优化：

旧: A(key=1) B(key=2) C(key=3) D(key=4)

新: A(key=1) C(key=3) B(key=2) E(key=5)

React 通过 key 识别:

A(key=1) → 相同, 保留

找到 C(key=3) → 移动位置

找到 B(key=2) → 移动位置

D(key=4) 不存在 → 删除

E(key=5) 新元素 → 创建

具体算法步骤:

1. 第一轮遍历: 按索引顺序比较新旧节点
 - 如果类型不同, 结束比较, 执行重建
 - 如果类型相同, 更新属性, 递归比较子节点
2. 第二轮遍历 (处理列表):
 - 收集所有变更 (插入、移动、删除)
 - 使用最长递增子序列算法 (LIS) 最小化移动操作

示例代码说明:

```
JavaScript
// 更新前
<ul>
  <li key="a">A</li>
  <li key="b">B</li>
  <li key="c">C</li>
</ul>

// 更新后
<ul>
  <li key="c">C</li>
  <li key="a">A</li>
  <li key="b">B</li>
</ul>

// Diff 结果: 移动 C 到首位, A、B 顺序后移
```

优化策略：

1. 批量更新：将多个 DOM 操作合并为一次更新
2. 时间分片：将渲染工作分割到多个帧中
3. 优先级调度：高优先级更新优先处理

与传统 DOM 操作对比：

```
JavaScript
// 传统方式：直接操作 DOM（性能差）
element.removeChild(child1);
element.insertBefore(child3, child1);
element.insertBefore(child2, child3);

// 虚拟 DOM：计算最小变更（性能好）
patches = diff(oldVNode, newVNode);
applyPatches(dom, patches); // 仅更新变化部分
```

总结：虚拟 DOM 通过高效的 Diff 算法，计算出最小变更集，然后批量更新真实 DOM，从而提升性能并简化开发。

18. 由 Hooks 引起的闭包陷阱

18.1.1 什么是闭包陷阱？

闭包陷阱是指由于函数组件每次渲染都会创建新的闭包，导致某些回调函数或副作用函数中引用的变量“冻结”在创建时的状态值，无法获取到最新的状态值。

18.1.2 常见的闭包陷阱场景

场景 1：setInterval 中的旧值

```
JavaScript
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      // 问题：count 永远都是 0
      console.log(count); // 一直是 0
    }, 1000);
  }, [count]);
}
```

```
    setCount(count + 1); // 永远设置的是 0 + 1
  }, 1000);

  return () => clearInterval(timer);
}, []); // 空依赖数组

return <div>Count: {count}</div>;
}
```

场景 2：事件监听器的旧值

```
JavaScript
function Counter() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    // 问题：总是输出点击时的初始值
    console.log(count);
  };

  return <button onClick={handleClick}>Click me</button>;
}
```

18.1.3 为什么会发生闭包陷阱？

React 函数组件的工作原理：

- 每次渲染都会创建新的函数作用域
- 每次渲染都会重新执行组件函数
- Hooks 会捕获当前渲染的状态值
- 回调函数创建时“记住”了当时的变量值

18.1.4 解决闭包陷阱的几种方法

方法 1：使用函数式更新（针对 `setState`）

```
JavaScript
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
```



```
const timer = setInterval(() => {
  // ✔ 使用函数式更新，获取最新的 state
  setCount(prevCount => prevCount + 1);
}, 1000);

return () => clearInterval(timer);
}, []); // 依赖为空，只在 mount 时执行

return <div>Count: {count}</div>;
}
```

方法 2：正确设置依赖数组

```
JavaScript
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      console.log(count);
      setCount(count + 1);
    }, 1000);

    return () => clearInterval(timer);
  }, [count]); // ✔ 将 count 加入依赖

  return <div>Count: {count}</div>;
}
```

方法 3：使用 useRef 保存可变值

```
JavaScript
function Timer() {
  const [count, setCount] = useState(0);
  const countRef = useRef(count);

  // 保持 ref 与 state 同步
  useEffect(() => {
    countRef.current = count;
  }, [count]);

  useEffect(() => {
```

```

const timer = setInterval(() => {
  // ✓ 通过 ref 获取最新值
  console.log(countRef.current);
  setCount(countRef.current + 1);
}, 1000);

return () => clearInterval(timer);
}, []);

return <div>Count: {count}</div>;
}

```

方法 4: 使用 `useCallback` 和 `useMemo` 优化

```

JavaScript
function Parent() {
  const [count, setCount] = useState(0);

  // ✓ 正确: 将依赖项传入
  const increment = useCallback(() => {
    setCount(prev => prev + 1);
  }, []);

  // 或者明确依赖
  const logCount = useCallback(() => {
    console.log(count);
  }, [count]); // ✓ count 变化时重新创建函数

  return <Child onClick={increment} />;
}

```

18.1.5 复杂场景示例

示例 1: 异步操作中的闭包

```

JavaScript
function ChatRoom({ roomId }) {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    const connection = createConnection(roomId);
    connection.connect();
  }, [roomId]);
}

```

```

connection.on('message', (msg) => {
  // 问题: 如果 roomId 变化, 这里还是旧的 roomId
  setMessages([...messages, msg]); // messages 也是旧的
});

return () => connection.disconnect();
}, [roomId]); // 缺少 messages 依赖

// ✓ 正确写法
useEffect(() => {
  const connection = createConnection(roomId);
  connection.connect();

  connection.on('message', (msg) => {
    setMessages(prev => [...prev, msg]); // 使用函数式更新
  });

  return () => connection.disconnect();
}, [roomId]); // 现在依赖正确
}

```

示例 2: 表单提交的闭包问题

```

JavaScript
function Form() {
  const [formData, setFormData] = useState({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  // 问题: handleSubmit 创建时捕获了旧的 formData
  const handleSubmit = async () => {
    setIsSubmitting(true);
    try {
      await submitToServer(formData); // 可能不是最新的 formData
    } finally {
      setIsSubmitting(false);
    }
  };

  // ✓ 正确: 使用 ref 或 useEffect 同步
  const formDataRef = useRef(formData);
  useEffect(() => {
    formDataRef.current = formData;
  });
}

```

```
    }, [formData]));

const handleSubmit = async () => {
  setIsSubmitting(true);
  try {
    await submitToServer(formDataRef.current); // 获取最新值
  } finally {
    setIsSubmitting(false);
  }
};
}
```

18.1.6 ESLint 规则的帮助

使用 `eslint-plugin-react-hooks` 可以自动检测依赖问题：

```
JSON
{
  "plugins": ["react-hooks"],
  "rules": {
    "react-hooks/rules-of-hooks": "error",
    "react-hooks/exhaustive-deps": "warn"
  }
}
```

18.1.7 最佳实践总结

1. 始终添加正确的依赖项到 `useEffect`、`useCallback`、`useMemo` 的依赖数组中
2. 使用函数式更新来避免依赖 `state` 当前值
3. `useRef` 用于存储可变值，特别是在回调中访问的最新值
4. 避免在依赖数组中使用对象/数组字面量，它们每次都会重新创建
5. 使用 `useCallback` 包装函数，避免不必要的重新渲染
6. 复杂状态逻辑考虑使用 `useReducer`

18.1.8 调试技巧

```
JavaScript
function Component() {
  const [state, setState] = useState(0);
```

```

useEffect(() => {
  console.log('Current state:', state);

  // 使用 chrome devtools 调试
  debugger;

  const timer = setTimeout(() => {
    console.log('In timeout:', state); // 检查是否是期望的值
  }, 1000);

  return () => clearTimeout(timer);
}, [state]);
}

```

19. 如何从 Class 组件重构为 Hooks 组件

19.1 重构的基本原则

对比表：Class vs Hooks

Class 组件	Hooks 组件
state 定义在构造函数中	useState 定义状态
this.setState 更新状态	setState 函数更新状态
生命周期方法	useEffect 处理副作用
绑定 this	无需绑定，函数自然作用域
高阶组件/HOC	自定义 Hook 复用逻辑

点击图片可查看完整电子表格

19.2 状态管理重构

Class 组件：

```

JavaScript
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,

```

```
    name: 'John',
    items: []
  };
}

increment = () => {
  this.setState({
    count: this.state.count + 1
  });
};
}
```

Hooks 重构:

```
JavaScript
function Counter(props) {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('John');
  const [items, setItems] = useState([]);

  const increment = () => {
    setCount(prevCount => prevCount + 1);
    // 或者 setCount(count + 1);
  };
}
```

复杂状态的重构:

```
JavaScript
// Class 组件
class Form extends React.Component {
  state = {
    formData: {
      username: '',
      email: '',
      age: 0
    },
    errors: {},
    isSubmitting: false
  };

  // Hooks 重构
```



```

function Form() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    age: 0
  });
  const [errors, setErrors] = useState({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  // 更新嵌套状态
  const updateFormData = (field, value) => {
    setFormData(prev => ({
      ...prev,
      [field]: value
    }));
  };
}

```

19.3 生命周期方法重构

componentDidMount → useEffect

```

JavaScript
// Class 组件
class UserProfile extends React.Component {
  componentDidMount() {
    console.log('组件已挂载');
    this.fetchUserData();
  }

  fetchUserData = async () => {
    const response = await fetch('/api/user');
    const data = await response.json();
    this.setState({ userData: data });
  };
}

// Hooks 重构
function UserProfile() {
  const [userData, setUserData] = useState(null);

```

```

useEffect(() => {
  console.log('组件已挂载');
  fetchUserData();

  // 清理函数（对应 componentWillUnmount）
  return () => {
    console.log('组件即将卸载');
  };
}, []); // 空数组表示只在 mount 时执行

const fetchUserData = async () => {
  const response = await fetch('/api/user');
  const data = await response.json();
  setUserData(data);
};
}

```

componentDidUpdate → useEffect

```

JavaScript
// Class 组件
class UserProfile extends React.Component {
  componentDidUpdate(prevProps, prevState) {
    if (prevProps.userId !== this.props.userId) {
      this.fetchUserData(this.props.userId);
    }
  }
}

// Hooks 重构
function UserProfile({ userId }) {
  const [userData, setUserData] = useState(null);

  useEffect(() => {
    if (userId) {
      fetchUserData(userId);
    }
  }, [userId]); // 依赖数组：当 userId 变化时重新执行

  // 或者使用 useRef 处理第一次不执行
  const isFirstRender = useRef(true);

  useEffect(() => {

```

```

    if (isFirstRender.current) {
      isFirstRender.current = false;
      return;
    }
    // 只在更新时执行的逻辑
  }, [userId]);
}

```

componentWillUnmount → useEffect 返回清理函数

```

JavaScript
// Class 组件
class Timer extends React.Component {
  componentDidMount() {
    this.timerId = setInterval(() => {
      console.log('tick');
    }, 1000);
  }

  componentWillUnmount() {
    clearInterval(this.timerId);
  }
}

// Hooks 重构
function Timer() {
  useEffect(() => {
    const timerId = setInterval(() => {
      console.log('tick');
    }, 1000);

    return () => {
      clearInterval(timerId);
    };
  }, []);
}

```

shouldComponentUpdate → React.memo + useMemo/useCallback

```

JavaScript
// Class 组件

```

```

class ExpensiveComponent extends React.Component {
  shouldComponentUpdate(nextProps, nextState) {
    return nextProps.value !== this.props.value;
  }

  render() {
    // 昂贵的计算
    const result = expensiveCalculation(this.props.value);
    return <div>{result}</div>;
  }
}

// Hooks 重构
const ExpensiveComponent = React.memo(({ value }) => {
  // 使用 useMemo 缓存昂贵的计算结果
  const result = useMemo(() => {
    return expensiveCalculation(value);
  }, [value]);

  return <div>{result}</div>;
});

```

19.4 事件处理函数重构

绑定 this 的问题消除

```

JavaScript
// Class 组件
class Form extends React.Component {
  constructor(props) {
    super(props);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  handleSubmit(event) {
    event.preventDefault();
    // 使用 this
  }

  // 或者使用箭头函数
  handleClick = () => {
    // 自动绑定 this
  }
}

```

```

    };
  }

  // Hooks 重构 - 无需绑定
  function Form() {
    const handleSubmit = (event) => {
      event.preventDefault();
      // 直接使用变量和函数, 无需 this
    };

    const handleClick = () => {
      // 函数作用域, 无需绑定
    };
  }

```

使用 useCallback 优化

```

JavaScript
function ParentComponent() {
  const [count, setCount] = useState(0);

  // 当依赖变化时重新创建函数
  const increment = useCallback(() => {
    setCount(prev => prev + 1);
  }, []);

  return <ChildComponent onIncrement={increment} />;
}

const ChildComponent = React.memo(({ onIncrement }) => {
  return <button onClick={onIncrement}>Increment</button>;
});

```

19.5 实例属性重构 → useRef

```

JavaScript
// Class 组件
class Timer extends React.Component {
  timerId = null; // 实例属性

  startTimer = () => {

```

```

    this.timerId = setInterval(() => {
      console.log('tick');
    }, 1000);
  };

  stopTimer = () => {
    clearInterval(this.timerId);
  };
}

// Hooks 重构
function Timer() {
  const timerId = useRef(null); // 使用 useRef 替代实例属性

  const startTimer = () => {
    timerId.current = setInterval(() => {
      console.log('tick');
    }, 1000);
  };

  const stopTimer = () => {
    clearInterval(timerId.current);
  };
}

```

19.6 上下文 (Context) 重构

```

JavaScript
// Class 组件
class ThemedButton extends React.Component {
  static contextType = ThemeContext;

  render() {
    const theme = this.context;
    return (
      <button style={{ background: theme.background }}>
        {this.props.children}
      </button>
    );
  }
}

// Hooks 重构

```



```
function ThemedButton(props) {
  const theme = useContext(ThemeContext);

  return (
    <button style={{ background: theme.background }}>
      {props.children}
    </button>
  );
}
```

19.7 高阶组件 (HOC) 重构 → 自定义 Hook

```
JavaScript
// 传统 HOC 模式
function withWindowSize(WrappedComponent) {
  return class extends React.Component {
    state = { width: window.innerWidth, height:
window.innerHeight };

    componentDidMount() {
      window.addEventListener('resize', this.updateSize);
    }

    componentWillUnmount() {
      window.removeEventListener('resize', this.updateSize);
    }

    updateSize = () => {
      this.setState({
        width: window.innerWidth,
        height: window.innerHeight
      });
    };

    render() {
      return <WrappedComponent
        {...this.props}
        windowSize={this.state}
      />;
    }
  };
}
```

```

// 使用 Hooks 重构为自定义 Hook
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({
        width: window.innerWidth,
        height: window.innerHeight
      });
    };

    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize',
handleResize);
  }, []); // 空数组确保只添加一次事件监听

  return size;
}

// 使用自定义 Hook
function MyComponent() {
  const { width, height } = useWindowSize();

  return (
    <div>
      Window size: {width} x {height}
    </div>
  );
}

```

19.8 完整重构示例

Class 组件:

```

JavaScript
class UserDashboard extends React.Component {
  constructor(props) {
    super(props);
  }
}

```

```
this.state = {
  user: null,
  posts: [],
  loading: true,
  error: null
};
}

componentDidMount() {
  this.fetchUserAndPosts();
}

componentDidUpdate(prevProps) {
  if (prevProps.userId !== this.props.userId) {
    this.fetchUserAndPosts();
  }
}

componentWillUnmount() {
  // 清理操作
}

fetchUserAndPosts = async () => {
  this.setState({ loading: true, error: null });

  try {
    const [userResponse, postsResponse] = await Promise.all([
      fetch(`/api/users/${this.props.userId}`),
      fetch(`/api/users/${this.props.userId}/posts`)
    ]);

    const user = await userResponse.json();
    const posts = await postsResponse.json();

    this.setState({ user, posts, loading: false });
  } catch (error) {
    this.setState({ error, loading: false });
  }
};

handleRetry = () => {
  this.fetchUserAndPosts();
};
```

```

render() {
  const { user, posts, loading, error } = this.state;

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <button onClick={this.handleRetry}>Retry</button>
      <ul>
        {posts.map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  );
}
}

```

Hooks 重构:

JavaScript

```

function UserDashboard({ userId }) {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchUserAndPosts = useCallback(async () => {
    setLoading(true);
    setError(null);

    try {
      const [userResponse, postsResponse] = await Promise.all([
        fetch(`/api/users/${userId}`),
        fetch(`/api/users/${userId}/posts`)
      ]);

      const userData = await userResponse.json();
      const postsData = await postsResponse.json();

      setUser(userData);
    } catch (error) {
      setError(error.message);
    }
  }, [userId]);

  useEffect(() => {
    fetchUserAndPosts();
  }, [fetchUserAndPosts]);

  return (
    <div>
      <h1>{user.name}</h1>
      <button onClick={this.handleRetry}>Retry</button>
      <ul>
        {posts.map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  );
}

```

```

        setPosts(postsData);
    } catch (err) {
        setError(err);
    } finally {
        setLoading(false);
    }
}, [userId]);

useEffect(() => {
    fetchUserAndPosts();
}, [fetchUserAndPosts]);

const handleRetry = () => {
    fetchUserAndPosts();
};

if (loading) return <div>Loading...</div>;
if (error) return (
    <div>
        Error: {error.message}
        <button onClick={handleRetry}>Retry</button>
    </div>
);

return (
    <div>
        <h1>{user.name}</h1>
        <button onClick={handleRetry}>Refresh</button>
        <ul>
            {posts.map(post => (
                <li key={post.id}>{post.title}</li>
            ))}
        </ul>
    </div>
);
}

```

19.9 重构最佳实践和注意事项

注意事项：

1. 逐块重构：不要一次性重构整个大型组件，先拆分小部分
2. 测试驱动：确保重构前后功能一致

3. 性能考虑: Hooks 可能导致不必要的重新渲染, 使用 `useMemo/useCallback` 优化
4. 依赖数组: 正确设置 `useEffect` 的依赖数组, 避免无限循环

重构步骤:

1. 分析现有组件: 理解 `state`、生命周期、方法的作用
2. 创建函数组件框架: 基本的函数组件结构
3. 迁移状态: 将 `this.state` 转换为 `useState`
4. 迁移生命周期: 将生命周期方法转换为 `useEffect`
5. 迁移方法: 将类方法转换为函数, 注意 `this` 绑定问题
6. 优化性能: 添加 `React.memo`、`useMemo`、`useCallback`
7. 测试验证: 确保功能完全一致

常见问题解决:

```
JavaScript
// 问题: 多个 state 更新导致的渲染问题
// Class 组件: setState 是批量更新
this.setState({ count: count + 1 });
this.setState({ name: 'New Name' });

// Hooks: 每个 useState 都是独立的
setCount(count + 1);
setName('New Name'); // 可能触发两次渲染

// 解决方案: 合并状态或使用 useReducer
const [state, setState] = useState({ count: 0, name: '' });
// 或者使用 useReducer 管理复杂状态逻辑
```

19.10 迁移工具和资源

官方迁移指南:

- React 官方文档提供了详细的迁移路径
- 可以使用 `react-codemod` 工具辅助迁移

渐进式迁移：

```
JavaScript
// 可以在同一个项目中同时使用 Class 和 Hooks 组件
// 逐渐将 Class 组件重构为 Hooks 组件

// 也可以在 Class 组件中使用 Hooks
// （通过将部分逻辑提取到自定义 Hook）
class MyComponent extends React.Component {
  render() {
    // 在 render 方法中无法直接使用 Hooks
    // 但可以通过子组件使用 Hooks
    return <MyHooksComponent {...this.props} />;
  }
}
```

20. 如何在 JavaScript 中实现一个发布-订阅模式

基础实现

```
JavaScript
class EventEmitter {
  constructor() {
    this.events = new Map(); // 存储事件和对应的回调列表
  }

  // 订阅事件
  on(event, callback) {
    if (!this.events.has(event)) {
      this.events.set(event, []);
    }
    this.events.get(event).push(callback);

    // 返回取消订阅函数
    return () => this.off(event, callback);
  }

  // 发布事件
  emit(event, ...args) {
```

```
if (!this.events.has(event)) return false;

const callbacks = this.events.get(event);
callbacks.forEach(callback => {
  try {
    callback.apply(this, args);
  } catch (error) {
    console.error(`Error in event handler for ${event}:`,
error);
  }
});

return true;
}

// 取消订阅
off(event, callback) {
  if (!this.events.has(event)) return false;

  const callbacks = this.events.get(event);
  const index = callbacks.indexOf(callback);

  if (index > -1) {
    callbacks.splice(index, 1);
    if (callbacks.length === 0) {
      this.events.delete(event);
    }
    return true;
  }

  return false;
}

// 一次性订阅
once(event, callback) {
  const onceCallback = (...args) => {
    callback.apply(this, args);
    this.off(event, onceCallback);
  };
  this.on(event, onceCallback);
}

// 获取所有事件
eventNames() {
```

```
    return Array.from(this.events.keys());
  }

  // 清空所有事件
  clear() {
    this.events.clear();
  }
}
```

使用示例

```
JavaScript
// 创建事件中心
const eventBus = new EventEmitter();

// 订阅事件
const unsubscribe = eventBus.on('message', (data) => {
  console.log('收到消息:', data);
});

// 发布事件
eventBus.emit('message', 'Hello World!'); // 输出: 收到消息: Hello
World!

// 取消订阅
unsubscribe(); // 或 eventBus.off('message', callback);

// 一次性订阅
eventBus.once('login', (user) => {
  console.log('用户登录:', user.name);
});

eventBus.emit('login', { name: 'Alice' }); // 输出: 用户登录: Alice
eventBus.emit('login', { name: 'Bob' }); // 没有输出
```

更高级的实现（支持命名空间和通配符）

```
JavaScript
class AdvancedEventEmitter {
  constructor() {
    this.events = new Map();
  }
}
```

```
this.wildcardEvents = new Map(); // 通配符事件
}

on(event, callback) {
  // 处理通配符
  if (event.includes('*')) {
    const pattern = event.replace(/\*/g, '.*');
    const regex = new RegExp(`^${pattern}$`);

    if (!this.wildcardEvents.has(pattern)) {
      this.wildcardEvents.set(pattern, []);
    }
    this.wildcardEvents.get(pattern).push({ regex, callback });
    return () => this.off(event, callback);
  }

  // 普通事件
  if (!this.events.has(event)) {
    this.events.set(event, []);
  }
  this.events.get(event).push(callback);
  return () => this.off(event, callback);
}

emit(event, ...args) {
  let emitted = false;

  // 触发精确匹配的事件
  if (this.events.has(event)) {
    this.events.get(event).forEach(callback => {
      callback.apply(this, args);
    });
    emitted = true;
  }

  // 触发通配符事件
  this.wildcardEvents.forEach((handlers) => {
    handlers.forEach(({ regex, callback }) => {
      if (regex.test(event)) {
        callback.apply(this, args);
        emitted = true;
      }
    });
  });
}
```

```
    return emitted;
  }

  // ... 其他方法类似
}
```

在 React/Vue 中的应用

```
JavaScript
// React 中创建全局事件总线
export const EventBus = new EventEmitter();

// React 组件中使用
function ComponentA() {
  useEffect(() => {
    const unsubscribe = EventBus.on('dataUpdate', (data) => {
      console.log('数据更新:', data);
    });
    return () => unsubscribe(); // 组件卸载时取消订阅
  }, []);

  const sendData = () => {
    EventBus.emit('dataUpdate', { value: 123 });
  };
}

// Vue 3 中使用
// 可以使用 mitt 库或类似的实现
```

21. 谈谈你对 WeakMap 和 WeakSet 的理解，它们与 Map 和 Set 的主要区别是什么？

WeakMap

特性：

1. 键必须是对象（不能是原始值）
2. 键是弱引用：不影响垃圾回收
3. 不可遍历：没有 `keys()`、`values()`、`entries()`、`size` 属性

4. 自动清理：当键对象被垃圾回收时，对应的键值对会被自动删除

```
JavaScript
let obj = { id: 1 };
const weakMap = new WeakMap();

// 设置值
weakMap.set(obj, 'secret data');

// 获取值
console.log(weakMap.get(obj)); // 'secret data'

// 删除对象引用
obj = null;

// 垃圾回收后，weakMap 中的条目会被自动清除
// 但具体清除时间由垃圾回收机制决定
```

使用场景：

1. 私有属性存储

```
JavaScript
const privateData = new WeakMap();

class Person {
  constructor(name) {
    privateData.set(this, { name });
  }

  getName() {
    return privateData.get(this).name;
  }
}

const person = new Person('Alice');
console.log(person.getName()); // 'Alice'
// 当 person 被回收时，对应的私有数据也会被自动清理
```

2. DOM 元素关联数据

```
JavaScript
const domData = new WeakMap();
```



```
const element = document.getElementById('myElement');
domData.set(element, { clicked: 0 });

element.addEventListener('click', () => {
  const data = domData.get(element);
  data.clicked++;
  domData.set(element, data);
});

// 当 element 从 DOM 移除并被垃圾回收时，关联数据自动清理
```

WeakSet

特性：

1. 值必须是对象
2. 值是弱引用
3. 不可遍历：没有 `values()`、`entries()`、`size` 属性
4. 只有三个方法：`add()`、`has()`、`delete()`

```
JavaScript
let obj1 = { id: 1 };
let obj2 = { id: 2 };

const weakSet = new WeakSet();

weakSet.add(obj1);
weakSet.add(obj2);

console.log(weakSet.has(obj1)); // true

obj1 = null;
// 垃圾回收后，obj1 会从 weakSet 中自动移除
```

使用场景：

1. 避免内存泄漏的集合

```
JavaScript
// 记录已处理过的对象
const processedObjects = new WeakSet();

function processObject(obj) {
```

```

if (processedObjects.has(obj)) {
  console.log('已处理过，跳过');
  return;
}

// 处理对象...
processedObjects.add(obj);
}

```

2. 检查循环引用

```

JavaScript
function hasCircularReference(obj, seen = new WeakSet()) {
  if (obj && typeof obj === 'object') {
    if (seen.has(obj)) return true;
    seen.add(obj);

    for (let key in obj) {
      if (hasCircularReference(obj[key], seen)) {
        return true;
      }
    }
  }
  return false;
}

```

主要区别对比

特性	Map/Set	WeakMap/ WeakSet
键/值类型	任意类型	只能是对象
垃圾回收	强引用，会阻止垃圾回收	弱引用，不影响垃圾回收
可遍历性	可遍历，有 size 属性	不可遍历，无 size
性能	需要维护引用，内存占用较大	内存占用较小，自动清理

点击图片可查看完整电子表格

内存管理示例

```
JavaScript
// Map 会导致内存泄漏
function createMemoryLeak() {
  let obj = { data: 'large data' };
  const map = new Map();
  const weakMap = new WeakMap();

  map.set('key', obj);      // 强引用, obj 不会被回收
  weakMap.set(obj, 'meta'); // 弱引用, 不影响 obj 回收

  // 即使 obj = null, Map 中仍然保留引用
  obj = null;
  // weakMap 中的引用会在垃圾回收时自动清除
}

// 测试
function testMemory() {
  const elements = [];
  const map = new Map();
  const weakMap = new WeakMap();

  for (let i = 0; i < 1000; i++) {
    const element = document.createElement('div');
    elements.push(element);

    // 两种方式存储数据
    map.set(element, { index: i });
    weakMap.set(element, { index: i });
  }

  // 移除所有元素
  elements.forEach(el => el.remove());

  // 清空数组
  elements.length = 0;

  // 此时:
  // map.size 仍然是 1000 (内存泄漏)
  // weakMap 中的条目会被逐渐垃圾回收
}
```

22. 如何判断一个 JavaScript 对象是否为数组？请列出至少三种方法。

方法 1: `Array.isArray()` (推荐)

```
JavaScript
const arr = [1, 2, 3];
const obj = { length: 3 };

console.log(Array.isArray(arr)); // true
console.log(Array.isArray(obj)); // false
console.log(Array.isArray([])); // true
console.log(Array.isArray(null)); // false
console.log(Array.isArray(undefined)); // false

// ES5+ 支持，最可靠的方法
```

方法 2: `Object.prototype.toString.call()`

```
JavaScript
const arr = [1, 2, 3];
const obj = { length: 3 };

console.log(Object.prototype.toString.call(arr) === '[object Array]'); // true
console.log(Object.prototype.toString.call(obj) === '[object Array]'); // false
console.log(Object.prototype.toString.call('') === '[object Array]'); // false

// 原理：调用 Object.prototype.toString 方法
// 通用性最好，支持所有类型判断
function getType(value) {
  return Object.prototype.toString.call(value).slice(8, -1);
}

console.log(getType([])); // 'Array'
console.log(getType({})); // 'Object'
console.log(getType(null)); // 'Null'
```

```
console.log(getType(undefined)); // 'Undefined'
```

方法 3: instanceof Array

```
JavaScript
const arr = [1, 2, 3];
const obj = { length: 3 };

console.log(arr instanceof Array); // true
console.log(obj instanceof Array); // false
console.log([] instanceof Array); // true

// 注意: 跨框架/iframe 问题
const iframe = document.createElement('iframe');
document.body.appendChild(iframe);
const iframeArray = iframe.contentWindow.Array;

const myArray = new iframeArray(1, 2, 3);
console.log(myArray instanceof Array); // false (跨窗口问题)
console.log(Array.isArray(myArray)); // true (正确识别)
```

方法 4: constructor 属性 (不推荐)

```
JavaScript
const arr = [1, 2, 3];

console.log(arr.constructor === Array); // true

// 问题: constructor 可以被修改
arr.constructor = Object;
console.log(arr.constructor === Array); // false (错误结果)

// 或者对象根本没有 constructor 属性
const obj = Object.create(null);
console.log(obj.constructor === Array); // false (obj.constructor 是 undefined)
```

方法 5: Array.prototype.isPrototypeOf()

```
JavaScript
```

```
const arr = [1, 2, 3];
const obj = { length: 3 };

console.log(Array.prototype.isPrototypeOf(arr)); // true
console.log(Array.prototype.isPrototypeOf(obj)); // false

// 同样有跨框架问题
```

方法 6: ES6 Symbol.toStringTag (高级)

```
JavaScript
// 正常数组
const arr = [1, 2, 3];
console.log(arr[Symbol.toStringTag]); // undefined (但 toString 仍然返回 [object Array])

// 自定义类数组对象
const arrayLike = {
  length: 3,
  [Symbol.toStringTag]: 'Array'
};

console.log(Object.prototype.toString.call(arrayLike)); // [object Array]
console.log(Array.isArray(arrayLike)); // false (能正确识别)
```

综合比较

方法	优点	缺点	推荐程度
Array.isArray()	ES5+, 最可靠, 跨框架安全	IE8- 不支持 (需 polyfill)	★★★★★
Object.prototype.toString.call()	最通用, 支持所有类型	代码较长	★★★★☆
instanceof Array	简单直观	跨框架/iframe 问题	★★★☆☆
constructor	简单	易被修改, 不可靠	★☆☆☆☆

点击图片可查看完整电子表格

兼容性处理

JavaScript

// Polyfill for Array.isArray (ES5 之前)

```
if (!Array.isArray) {
```

```
    Array.isArray = function(arg) {
```

```
        return Object.prototype.toString.call(arg) === '[object Array]';
```

```
    };
```

```
}
```

// 综合判断函数

```
function isArray(value) {
```

```
    // 优先使用原生方法
```

```
    if (typeof Array.isArray === 'function') {
```

```
        return Array.isArray(value);
```

```
    }
```

```
    // 备用方案
```

```
    return Object.prototype.toString.call(value) === '[object Array]';
```

```
}
```

// 严格检查（排除类数组对象）

```
function isStrictArray(value) {
```

```
    if (!Array.isArray(value)) return false;
```

```
    // 额外检查：确保有正确的 length 属性
```

```
    if (typeof value.length !== 'number') return false;
```

```
    // 检查是否可以被展开
```

```
    try {
```

```
        [...value];
```

```
        return true;
```

```
    } catch {
```

```
        return false;
```

```
    }
```

```
}
```

// 测试

```
console.log(isStrictArray([1, 2, 3])); // true
```

```
console.log(isStrictArray({ length: 3 })); // false
```

```
console.log(isStrictArray(document.querySelectorAll('div'))); // false
```

实际应用场景

JavaScript

// 1. 函数参数类型检查

```
function processData(data) {  
  if (!Array.isArray(data)) {  
    throw new TypeError('Expected an array');  
  }  
  return data.map(item => item * 2);  
}
```

// 2. 安全地遍历

```
function safeForEach(collection, callback) {  
  if (Array.isArray(collection)) {  
    collection.forEach(callback);  
  } else if (collection && typeof collection[Symbol.iterator] ===  
'function') {  
    // 处理其他可迭代对象  
    for (const item of collection) {  
      callback(item);  
    }  
  }  
}
```

// 3. 深度检查嵌套数组

```
function deepContainsArray(obj, seen = new WeakSet()) {  
  if (seen.has(obj)) return false;  
  seen.add(obj);  
  
  if (Array.isArray(obj)) {  
    return true;  
  }  
  
  if (obj && typeof obj === 'object') {  
    for (const key in obj) {  
      if (deepContainsArray(obj[key], seen)) {  
        return true;  
      }  
    }  
  }  
  
  return false;  
}
```

23. `(a == 1 && a == 2 && a == 3)` 这个表达式有没有可能为 `true`? 请说明如何实现

实现方式 1: `valueOf` / `toString` 方法重写

```
JavaScript
// 方法 1.1: 使用对象和 valueOf 方法
let a = {
  value: 0,
  valueOf() {
    return ++this.value; // 每次比较时 valueOf 被调用, 返回递增的值
  }
};

console.log(a == 1 && a == 2 && a == 3); // true

// 方法 1.2: 使用 toString 方法
let b = {
  value: 0,
  toString() {
    return ++this.value;
  }
};

console.log(b == 1 && b == 2 && b == 3); // true

// 方法 1.3: 同时重写 valueOf 和 toString
let c = {
  value: 0,
  valueOf() {
    return ++this.value;
  },
  toString() {
    return this.value.toString();
  }
};

console.log(c == 1 && c == 2 && c == 3); // true
```

实现方式 2: 使用数组和 `shift` 方法

JavaScript

// 利用数组在比较时会被转换为字符串的特性

```
let a = [1, 2, 3];
```

```
a.toString = a.shift; // 每次比较时调用 shift，返回并删除第一个元素
```

```
console.log(a == 1 && a == 2 && a == 3); // true
```

实现方式 3：使用 Proxy 代理

JavaScript

// ES6 Proxy 可以拦截属性的读取操作

```
let value = 0;
```

```
let a = new Proxy({}, {
```

```
  get(target, prop) {
```

```
    if (prop === Symbol.toPrimitive) {
```

```
      // 当需要转换为原始值时，返回一个函数
```

```
      return () => ++value;
```

```
    }
```

```
    return Reflect.get(target, prop);
```

```
  }
```

```
});
```

```
console.log(a == 1 && a == 2 && a == 3); // true
```

// 另一种 Proxy 实现

```
let b = new Proxy({}, {
```

```
  i: 0,
```

```
  get(target, prop) {
```

```
    if (prop === Symbol.toPrimitive) {
```

```
      return () => ++this.i;
```

```
    }
```

```
    return target[prop];
```

```
  }
```

```
});
```

```
console.log(b == 1 && b == 2 && b == 3); // true
```

实现方式 4：使用 Object.defineProperty（全局变量）

JavaScript

// 在全局作用域中定义 a

```

let value = 0;
Object.defineProperty(window, 'a', {
  get() {
    return ++value;
  },
  configurable: true
});

console.log(a == 1 && a == 2 && a == 3); // true

// 或者使用 let/const（但需要注意作用域）
{
  let _value = 0;
  Object.defineProperty(globalThis, 'a', {
    get() {
      return ++_value;
    },
    configurable: true
  });
}

console.log(a == 1 && a == 2 && a == 3); // true

```

实现方式 5：使用 Symbol.toPrimitive

```

JavaScript
let a = {
  value: 0,
  [Symbol.toPrimitive](hint) {
    // hint 可以是 'number', 'string', 'default'
    return ++this.value;
  }
};

console.log(a == 1 && a == 2 && a == 3); // true

// 更精细的控制
let b = {
  counter: 0,
  [Symbol.toPrimitive](hint) {
    if (hint === 'number' || hint === 'default') {
      return ++this.counter;
    }
  }
};

```

```
        return this.counter.toString();
    }
};

console.log(b == 1 && b == 2 && b == 3); // true
```

实现方式 6：使用 **with** 语句和变量作用域

```
JavaScript
// 不推荐，仅作展示
let value = 0;
with({
    get a() {
        return ++value;
    }
}) {
    console.log(a == 1 && a == 2 && a == 3); // true
}
```

实现方式 7：使用 **Generator** 函数

```
JavaScript
function* createA() {
    let i = 0;
    while (true) {
        yield ++i;
    }
}

const gen = createA();
const a = {
    [Symbol.toPrimitive]() {
        return gen.next().value;
    }
};

console.log(a == 1 && a == 2 && a == 3); // true
```

原理分析

这些实现都利用了 JavaScript 的隐式类型转换机制：

1. 宽松相等 (==) 会触发类型转换

- 当比较 `a == 1` 时，如果 `a` 不是原始值，JavaScript 会尝试将其转换为原始值
- 转换顺序：`valueOf()` → `toString()`

2. 原始值转换的优先级

```
JSON
// JavaScript 调用顺序:
// 1. 如果有 Symbol.toPrimitive, 调用它
// 2. 否则, 如果 hint 是 'number' 或 'default':
//    a. 先调用 valueOf()
//    b. 如果返回值不是原始值, 再调用 toString()
// 3. 如果 hint 是 'string':
//    a. 先调用 toString()
//    b. 如果返回值不是原始值, 再调用 valueOf()
```

3. 每次比较都会重新求值

- 在 `a == 1 && a == 2 && a == 3` 中
- `a` 被访问了 3 次，每次都会调用转换方法
- 如果转换方法有副作用（如递增计数器），就能实现不同的返回值

扩展：严格相等 (===) 版本

严格相等不会进行类型转换，所以更复杂，但仍然可能：

```
JavaScript
// 方法 1: 使用 async/await
async function test() {
  let i = 0;
  const a = {
    get [Symbol.toPrimitive]() {
      return () => Promise.resolve(++i);
    }
  };
};

// 注意: 这里需要使用 await
console.log(
  (await a) === 1 &&
  (await a) === 2 &&
  (await a) === 3
);
```

```

    ); // true
  }
  test();

  // 方法 2: 每次访问返回不同的值 (使用 Proxy)
  {
    let i = 0;
    const a = new Proxy({}, {
      get(target, prop) {
        if (prop === Symbol.toPrimitive) {
          return () => ++i;
        }
        return target[prop];
      }
    });

    // 需要转换为数字
    console.log(
      +a === 1 &&
      +a === 2 &&
      +a === 3
    ); // true
  }

```

实际应用（不推荐在生产环境中使用）

```

JavaScript
// 恶作剧代码: 让 Math.random() 总是返回递增的值
let counter = 0;
const originalRandom = Math.random;
Math.random = function() {
  return ++counter / 100;
};

// 或者让 Date.now() 返回固定序列
let time = 0;
const originalNow = Date.now;
Date.now = function() {
  return ++time * 1000;
};

```

安全建议

这种技巧虽然有趣，但在实际开发中：

1. 不要修改内置对象的默认行为
2. 避免使用隐式类型转换，尽量使用 `===`
3. 代码可读性比炫技更重要

24. CSS 的 `box-sizing: border-box`；有什么作用？

基本概念

`box-sizing` 是 CSS 属性，用于控制元素宽度和高度的计算方式。它有两个主要值：

- `content-box`（默认值）
- `border-box`

对比说明

24.1.1.1 `content-box`（默认）

```
JavaScript
/* 默认情况：元素宽度 = 内容宽度 */
div {
  box-sizing: content-box;
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}

/*
实际占用空间计算：
内容宽度：200px
+ 内边距：20px × 2 = 40px
+ 边框：5px × 2 = 10px
= 元素总宽度：250px

可视宽度：250px
总占用宽度（含外边距）：250px + 20px = 270px
```

```
*/
```

24.1.1.2 border-box

JavaScript

```
/* border-box: 元素宽度 = 内容宽度 + 内边距 + 边框 */
```

```
div {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid black;  
  margin: 10px;  
}
```

```
/*
```

实际占用空间计算:

元素总宽度: 200px (固定)

- 内边距: $20\text{px} \times 2 = 40\text{px}$

- 边框: $5\text{px} \times 2 = 10\text{px}$

= 内容宽度: 150px

可视宽度: 200px

总占用宽度 (含外边距): $200\text{px} + 20\text{px} = 220\text{px}$

```
*/
```

视觉对比

XML

```
<div class="content-box">content-box</div>
```

```
<div class="border-box">border-box</div>
```

```
<style>
```

```
div {  
  width: 200px;  
  height: 100px;  
  padding: 20px;  
  border: 5px solid #333;  
  margin: 20px;  
  background: #f0f0f0;  
}
```

```
.content-box {
```

```
box-sizing: content-box;
/* 实际宽度: 200 + 40 + 10 = 250px */
}

.border-box {
  box-sizing: border-box;
  /* 实际宽度: 200px, 内容区宽度: 200 - 40 - 10 = 150px */
}
</style>
```

border-box 的优势

1. 直观的尺寸控制

```
JavaScript
/* 更容易创建固定布局 */
.container {
  width: 1200px;
  padding: 0 20px;
  box-sizing: border-box;
  /* 不需要计算: 1200px 就包含了 padding */
}

.col-3 {
  width: 25%;
  padding: 15px;
  box-sizing: border-box;
  /* 列宽总是父容器的 25%, padding 不会破坏布局 */
}
```

2. 简化响应式设计

```
JavaScript
/* 栅格系统 */
.row {
  display: flex;
  flex-wrap: wrap;
}

.col {
  box-sizing: border-box;
  padding: 15px;
```

```
width: 33.333%; /* 始终保持 1/3, 不受 padding 影响 */
}

/* 媒体查询中不需要重新计算宽度 */
@media (max-width: 768px) {
  .col {
    width: 50%;
    /* padding 仍然包含在 50% 内 */
  }
}
```

3. 避免数学计算

```
JavaScript
/* 没有 border-box */
.old-way {
  width: 300px;
  padding: 20px;
  /* 如果想要总宽度 300px, 需要计算:
    width: 300px - 40px = 260px */
}

/* 使用 border-box */
.new-way {
  width: 300px;
  padding: 20px;
  box-sizing: border-box;
  /* 直接设置 300px, padding 包含在内 */
}
```

4. 一致的组件尺寸

```
YAML
/* 按钮组件 */
.btn {
  display: inline-block;
  padding: 10px 20px;
  border: 2px solid;
  box-sizing: border-box;
  text-align: center;
}
```



```
.btn-small {
  width: 100px; /* 总宽度 100px, 包含 padding 和 border */
}

.btn-large {
  width: 200px; /* 总宽度 200px */
}
```

实际应用场景

场景 1：全局重置

```
JavaScript
/* 最常用的做法：全局设置 border-box */
*, *::before, *::after {
  box-sizing: border-box;
}

/* 或者使用更细粒度的控制 */
html {
  box-sizing: border-box;
}

*, *::before, *::after {
  box-sizing: inherit;
}
```

场景 2：表单元素兼容

```
JavaScript
/* 表单元素有默认的 box-sizing 值 */
input, textarea, select, button {
  box-sizing: border-box; /* 确保一致性 */
}

/* 特别是 file input 在不同浏览器中表现不同 */
input[type="file"] {
  width: 100%;
  box-sizing: border-box;
}
```

场景 3: Flex/Grid 布局

```
JavaScript
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.grid-item {
  padding: 20px;
  box-sizing: border-box; /* gap 不会影响 item 的尺寸计算 */
}
```

场景 4: 边框动画

```
JavaScript
/* 悬停时添加边框，不会导致布局跳动 */
.button {
  width: 200px;
  padding: 10px;
  border: 2px solid transparent;
  box-sizing: border-box;
  transition: border-color 0.3s;
}

.button:hover {
  border-color: red;
  /* 不会改变元素尺寸，因为 border 已经包含在宽度内 */
}
```

注意事项

1. margin 不包含在内

```
JavaScript
/* box-sizing: border-box 只包含 content, padding, border */
.element {
  width: 200px;
  padding: 20px;
  border: 5px solid;
  margin: 30px;
}
```

```
box-sizing: border-box;
/*
元素宽度: 200px
总占用宽度: 200px + 60px = 260px
*/
}
```

2. height 也受影响

```
JavaScript
/* border-box 同时影响 width 和 height */
.box {
width: 200px;
height: 100px;
padding: 20px;
border: 5px solid;
box-sizing: border-box;
/*
总高度: 100px
内容高度: 100px - 40px - 10px = 50px
*/
}
```

3. 与 min-width/max-width 的关系

```
JavaScript
.container {
width: 100%;
max-width: 1200px;
padding: 0 20px;
box-sizing: border-box;
/*
最大宽度 1200px 包含 padding
在窄屏幕上, 宽度会自适应
*/
}
```

4. 浏览器兼容性

```
JavaScript
/* 虽然现代浏览器都支持, 但可以添加前缀 */
```

```
.element {
  -webkit-box-sizing: border-box;
  -moz-box-sizing: border-box;
  box-sizing: border-box;
}

/* IE8+ 支持 box-sizing */
```

最佳实践

```
Python
/* 推荐：全局使用 border-box */
html {
  box-sizing: border-box;
}

*, *::before, *::after {
  box-sizing: inherit;
}

/* 例外情况：第三方组件可能需要 content-box */
.third-party-component {
  box-sizing: content-box !important;
}

/* 对于特定元素保留默认行为 */
textarea,
input[type="text"],
input[type="password"],
input[type="email"],
input[type="number"] {
  box-sizing: border-box; /* 确保一致性 */
}
```

25. 在 React (或 Vue) 中，key 的作用是什么？为什么不建议使用数组的索引 index 作为 key？

key 的作用

1. 标识元素身份

```
JavaScript
// 没有 key 时，React 无法高效识别元素
<ul>
  <li>苹果</li>
  <li>香蕉</li>
  <li>橙子</li>
</ul>

// 有 key 时，React 可以追踪每个元素
<ul>
  <li key="fruit-1">苹果</li>
  <li key="fruit-2">香蕉</li>
  <li key="fruit-3">橙子</li>
</ul>
```

2. 优化 Diff 算法性能

```
JavaScript
// React 的 Diff 算法使用 key 来:
// 1. 快速识别哪些元素是新增的
// 2. 哪些元素被移动了
// 3. 哪些元素被删除了

// 没有 key:  $O(n^3)$  复杂度（理论上）
// 有 key:  $O(n)$  复杂度
```

3. 维持组件状态

```
JavaScript
// 有 key 时，组件状态可以正确关联
{items.map(item => (
  <TodoItem
    key={item.id} // 相同的 id 保持相同的组件实例
    todo={item}
  />
))}

// 当 items 重新排序时，每个 TodoItem 的状态（如输入框内容）保持不变
```

为什么不应该使用索引作为 key

问题 1：列表重新排序时的性能问题

```
JavaScript
// 初始列表
const initialItems = [
  { id: 'a', text: '苹果' },
  { id: 'b', text: '香蕉' },
  { id: 'c', text: '橙子' }
];

// 使用 index 作为 key
function ListWithIndexKey() {
  return (
    <ul>
      {initialItems.map((item, index) => (
        <li key={index}>{item.text}</li> // key: 0, 1, 2
      ))}
    </ul>
  );
}

// 新列表：在开头插入一个新项
const newItems = [
  { id: 'd', text: '葡萄' }, // 新增
  { id: 'a', text: '苹果' },
  { id: 'b', text: '香蕉' },
  { id: 'c', text: '橙子' }
];

// React 看到的 key 变化：
// 旧：0(a), 1(b), 2(c)
// 新：0(d), 1(a), 2(b), 3(c)
// 结果：所有 li 元素都会被重新渲染，因为 key 对应的内容都变了
```

问题 2：组件状态丢失

```
JavaScript
// 待办事项列表，每个项有输入框
function TodoList() {
```



```

const [todos, setTodos] = useState([
  { id: 1, text: '学习 React' },
  { id: 2, text: '学习 Vue' }
]);

return (
  <div>
    {todos.map((todo, index) => (
      <TodoItem
        key={index} // ✕ 使用索引作为 key
        todo={todo}
      />
    ))}
    <button onClick={() => {
      // 在开头添加新待办事项
      setTodos([{ id: 3, text: '新任务' }, ...todos]);
    }}>
      添加新任务
    </button>
  </div>
);
}

```

```

// TodoItem 组件
function TodoItem({ todo }) {
  const [inputValue, setInputValue] = useState('');

  return (
    <div>
      <span>{todo.text}</span>
      <input
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
    </div>
  );
}

```

// 问题：当添加新任务时，原本输入框中的内容会错位！
 // key=0 的输入框原来显示"学习 React"的输入内容，现在变成了"新任务"的输入框

问题 3：动画问题

```

JavaScript
// 使用 CSS 过渡动画
<TransitionGroup>
  {items.map((item, index) => (
    <CSSTransition
      key={index} // ✗ 索引变化会导致动画异常
      timeout={300}
      classNames="item"
    >
      <div>{item.text}</div>
    </CSSTransition>
  ))}
</TransitionGroup>

// 当删除中间项时，后面的项索引都会改变
// 导致 React 认为所有后续项都被替换，而不是移动
// 动画效果会出错

```

正确使用 key 的示例

示例 1：使用唯一 ID

```

JavaScript
// 最佳实践：使用数据的唯一标识
function UserList({ users }) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

// 如果没有 id，可以生成一个（确保稳定性）
function ItemList({ items }) {
  return (
    <ul>
      {items.map(item => (
        <li key={` ${item.category}-${item.name}`}>
          {item.name}
        </li>
      ))}
    </ul>
  );
}

```

```
    )})  
  </ul>  
);  
}
```

示例 2：没有 ID 时的替代方案

```
JavaScript  
// 方案 1：使用内容哈希（如果内容是唯一的）  
function HashKeyList({ items }) {  
  return (  
    <ul>  
      {items.map(item => (  
        <li key={hashCode(item.text)}>{item.text}</li>  
      ))}  
    </ul>  
  );  
}  
  
// 方案 2：使用 nanoid 或 uuid（客户端生成）  
import { nanoid } from 'nanoid';  
  
function GeneratedKeyList({ items }) {  
  const itemsWithKey = items.map(item => ({  
    ...item,  
    _key: nanoid() // 只在客户端使用  
  }));  
  
  return (  
    <ul>  
      {itemsWithKey.map(item => (  
        <li key={item._key}>{item.text}</li>  
      ))}  
    </ul>  
  );  
}
```

示例 3：复杂列表的正确处理

```
JavaScript  
// 嵌套列表  
function NestedList({ categories }) {
```

```

return (
  <div>
    {categories.map(category => (
      <div key={category.id}>
        <h3>{category.name}</h3>
        <ul>
          {category.items.map(item => (
            <li key={item.id}>
              {item.name}
            </li>
          ))}
        </ul>
      </div>
    ))}
  </div>
);
}

```

// 动态列表

```

function DynamicList({ items, onReorder }) {
  // 使用 Draggable 组件时, key 尤为重要
  return (
    <DragDropContext onDragEnd={onReorder}>
      <Droppable droppableId="list">
        {(provided) => (
          <ul ref={provided.innerRef}
            {...provided.droppableProps}>
            {items.map((item, index) => (
              <Draggable
                key={item.id} // ✔ 正确的 key
                draggableId={item.id}
                index={index}
              >
                {(provided) => (
                  <li
                    ref={provided.innerRef}
                    {...provided.draggableProps}
                    {...provided.dragHandleProps}
                  >
                    {item.text}
                  </li>
                )}
              </Draggable>
            ))}
          </ul>
        )}
      </Droppable>
    )}
  )}

```

```

        {provided.placeholder}
      </ul>
    })
  </Draggable>
</DragDropContext>
);
}

```

Vue 中的 key

XML

```

<!-- Vue 2/3 中的 key -->
<template>
  <ul>
    <!-- 使用唯一 key -->
    <li v-for="item in items" :key="item.id">
      {{ item.text }}
    </li>

    <!-- v-if/v-else 也需要 key -->
    <div v-if="isLoading" key="loading">加载中...</div>
    <div v-else key="content">{{ content }}</div>

    <!-- 强制重新渲染组件 -->
    <MyComponent :key="componentKey" />
  </ul>
</template>

<script>
export default {
  methods: {
    forceRerender() {
      this.componentKey += 1; // 改变 key 会强制组件重新创建
    }
  }
}
</script>

```

特殊情况处理

情况 1：静态列表（永远不会改变）

```
JavaScript
// 如果列表永远不会改变（排序、过滤、增删），可以使用索引
const STATIC_ITEMS = ['首页', '关于', '联系'];

function Navigation() {
  return (
    <nav>
      {STATIC_ITEMS.map((item, index) => (
        <a key={index} href="#">{item}</a> // ✔ 静态列表可以使用索引
      ))}
    </nav>
  );
}
```

情况 2：没有合适的唯一标识

```
JavaScript
// 如果数据真的没有唯一标识，且列表会改变
function NoIdList({ items }) {
  // 使用 index 作为最后的选择
  // 但需要清楚潜在的问题
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>
          {item}
          /* 不要在这个组件内部维护状态！ */
        </li>
      ))}
    </ul>
  );
}
```

情况 3：key 的稳定性

```
JavaScript
// 错误的做法：key 在每次渲染时都改变
function UnstableKeyList({ items }) {
  return (
    <ul>
```



```

    {items.map(item => (
      <li key={Math.random()}> // ✗ 每次渲染 key 都不同
        {item.text}
      </li>
    ))}
  </ul>
);
}

// 正确的做法: 保持 key 的稳定性
function StableKeyList({ items }) {
  // 使用 useMemo 或 useCallback 保持 key 稳定
  const itemsWithStableKey = useMemo(() =>
    items.map(item => ({
      ...item,
      // 只在第一次映射时生成 key
      _key: item.id || generateStableId(item)
    })),
    [items] // 只有当 items 改变时才重新计算
  );

  return (
    <ul>
      {itemsWithStableKey.map(item => (
        <li key={item._key}>{item.text}</li>
      ))}
    </ul>
  );
}

```

性能测试示例

```

JavaScript
// 测试不同 key 策略的性能
function PerformanceTest() {
  const [items, setItems] = useState(
    Array.from({ length: 1000 }, (_, i) => ({
      id: i,
      value: `Item ${i}`,
      checked: false
    }))
  );
}

```

```
const shuffle = () => {
  setItems(prev => [...prev].sort(() => Math.random() - 0.5));
};

return (
  <div>
    <button onClick={shuffle}>随机排序</button>

    {/* 测试 1: 使用 id 作为 key */}
    <div>
      <h3>使用 id 作为 key</h3>
      {items.map(item => (
        <Item key={item.id} item={item} />
      ))}
    </div>

    {/* 测试 2: 使用 index 作为 key */}
    <div>
      <h3>使用 index 作为 key</h3>
      {items.map((item, index) => (
        <Item key={index} item={item} />
      ))}
    </div>
  </div>
);
}

// 使用 React DevTools Profiler 可以明显看到性能差异
// 使用 id 作为 key: 只有必要的组件重新渲染
// 使用 index 作为 key: 所有组件都可能重新渲染
```

总结

方面	使用唯一 ID 作为 key	使用索引作为 key
性能	最优，最小化重新渲染	较差，可能导致大量重新渲染
状态保持	正确保持组件状态	状态可能错乱或丢失
列表变化	增删改查都高效	除末尾操作外都低效
动画	平滑正确	可能产生奇怪效果

点击图片可查看完整电子表格

最佳实践：始终优先使用数据的唯一标识作为 key，只有在列表绝对静态且简单时才考虑使用索引。

26. 什么是事件委托？

什么是事件委托？

事件委托是一种利用事件冒泡机制的技术，将单个子元素的事件处理程序绑定到父元素上，通过判断事件目标（event.target）来统一处理多个子元素的事件。

基本示例

```
XML
<!-- 传统方式：为每个按钮绑定事件 -->
<ul id="list">
  <li><button class="btn">按钮 1</button></li>
  <li><button class="btn">按钮 2</button></li>
  <li><button class="btn">按钮 3</button></li>
</ul>

<script>
// ✖ 传统方式：每个按钮单独绑定
document.querySelectorAll('.btn').forEach(button => {
  button.addEventListener('click', function(e) {
    console.log('点击了按钮:', this.textContent);
  });
});
```

```
});

// ✔ 事件委托：只需绑定一次
document.getElementById('list').addEventListener('click',
function(e) {
    // 检查点击的是否是按钮
    if (e.target.classList.contains('btn')) {
        console.log('点击了按钮:', e.target.textContent);
    }
});
</script>
```

事件委托的原理

```
JavaScript
// 事件流的三个阶段
// 1. 捕获阶段 (Capturing Phase): 从 window → target
// 2. 目标阶段 (Target Phase): 到达目标元素
// 3. 冒泡阶段 (Bubbling Phase): 从 target → window

document.addEventListener('click', function(e) {
    console.log('捕获阶段', e.eventPhase); // 1
}, true); // true 表示在捕获阶段监听

document.addEventListener('click', function(e) {
    console.log('冒泡阶段', e.eventPhase); // 3
}, false); // false 表示在冒泡阶段监听（默认）
```

事件委托的实现方式

方式 1：使用 `closest()` 方法

```
JavaScript
document.getElementById('container').addEventListener('click',
function(e) {
    // 查找最近的匹配元素
    const button = e.target.closest('.btn');
    if (button) {
        console.log('点击了按钮:', button.dataset.id);
    }
}
```

```
// 检查是否点击了特定区域
const item = e.target.closest('.item');
if (item) {
  console.log('点击了项目:', item.textContent);
}
});
```

方式 2：使用 matches() 方法

```
JavaScript
document.getElementById('menu').addEventListener('click',
function(e) {
  // 检查目标元素是否匹配选择器
  if (e.target.matches('.menu-item')) {
    const id = e.target.dataset.id;
    console.log('菜单项被点击:', id);
  }

  // 可以处理嵌套结构
  if (e.target.matches('.menu-item .icon')) {
    console.log('图标被点击');
  }
});
```

方式 3：处理动态添加的元素

```
JavaScript
// 父容器的事件委托可以处理动态添加的子元素
const container = document.getElementById('dynamic-list');

container.addEventListener('click', function(e) {
  if (e.target.classList.contains('dynamic-item')) {
    console.log('动态元素被点击:', e.target.textContent);
  }
});

// 动态添加元素
function addItem(text) {
  const item = document.createElement('div');
  item.className = 'dynamic-item';
  item.textContent = text;
  container.appendChild(item);
}
```

```
// 无需为新元素单独绑定事件!  
}
```

```
addItem('新项目 1');  
addItem('新项目 2');
```

事件委托的优点

1. 减少内存消耗

```
JavaScript  
// 传统方式: 每个元素都绑定事件  
const buttons = document.querySelectorAll('.btn');  
buttons.forEach(btn => {  
  btn.addEventListener('click', handler); // 1000 个元素 = 1000 个监  
听器  
});  
  
// 事件委托: 只有一个监听器  
document.addEventListener('click', function(e) {  
  if (e.target.classList.contains('btn')) {  
    handler(e);  
  }  
});
```

2. 简化动态元素的事件处理

```
JavaScript  
// 动态表格行处理  
const table = document.getElementById('data-table');  
  
table.addEventListener('click', function(e) {  
  const row = e.target.closest('tr');  
  if (row && row.dataset.id) {  
    console.log('行被点击:', row.dataset.id);  
  }  
}  
  
// 处理删除按钮  
if (e.target.classList.contains('delete-btn')) {  
  const id = e.target.closest('tr').dataset.id;  
  deleteRow(id);  
}
```



```

    }

    // 处理编辑按钮
    if (e.target.classList.contains('edit-btn')) {
        const id = e.target.closest('tr').dataset.id;
        editRow(id);
    }
});

// Ajax 加载新数据后, 无需重新绑定事件
function loadMoreData() {
    fetch('/api/data')
        .then(res => res.json())
        .then(data => {
            // 添加新行
            data.forEach(item => {
                const row = createRow(item);
                table.appendChild(row);
            });
            // 无需为新增的行绑定事件!
        });
}

```

3. 提高性能（减少事件绑定时间）

```

JavaScript
// 测试性能差异
console.time('传统绑定');
const items = document.querySelectorAll('.item');
items.forEach(item => {
    item.addEventListener('click', () => {});
});
console.timeEnd('传统绑定'); // 大约 5-10ms (1000 个元素)

console.time('事件委托');
document.addEventListener('click', (e) => {
    if (e.target.classList.contains('item')) {
        // 处理逻辑
    }
});
console.timeEnd('事件委托'); // 大约 0.1ms

```

4. 代码更简洁易维护

```
JavaScript
// 传统方式：分散的事件处理
document.getElementById('btn1').addEventListener('click',
handleBtn1);
document.getElementById('btn2').addEventListener('click',
handleBtn2);
document.getElementById('btn3').addEventListener('click',
handleBtn3);

// 事件委托：集中处理
document.getElementById('button-group').addEventListener('click',
function(e) {
  switch(e.target.id) {
    case 'btn1':
      handleBtn1();
      break;
    case 'btn2':
      handleBtn2();
      break;
    case 'btn3':
      handleBtn3();
      break;
    default:
      // 处理其他情况
  }
});
```

事件委托的注意事项

1. 事件目标可能不是期望的元素

```
JavaScript
// 问题：点击按钮内的文本，target 是文本节点，不是按钮
document.getElementById('list').addEventListener('click',
function(e) {
  // e.target 可能是 <button> 或 <span> 或文本节点
  if (e.target.classList.contains('btn')) {
    // 如果点击按钮内的图标，这个条件可能为 false
  }
});
```

```
// 解决方案: 使用 closest()
document.getElementById('list').addEventListener('click',
function(e) {
  const button = e.target.closest('.btn');
  if (button) {
    // 总是能获取到最近的按钮元素
    console.log('按钮被点击');
  }
});
```

2. 事件冒泡可能被阻止

```
JavaScript
// 如果子元素阻止了事件冒泡
document.querySelector('.child').addEventListener('click',
function(e) {
  e.stopPropagation(); // 阻止冒泡
});

// 父元素的事件委托将无法捕获该事件
document.querySelector('.parent').addEventListener('click',
function(e) {
  // 这行代码不会执行
  console.log('父元素捕获到点击事件');
});
```

3. 不适用于所有事件类型

```
JavaScript
// 有些事件不会冒泡
// focus、blur、load、unload、mouseenter、mouseleave 等事件不会冒泡

// 解决方案: 使用 focusin 和 focusout (会冒泡)
document.getElementById('form').addEventListener('focusin',
function(e) {
  if (e.target.tagName === 'INPUT') {
    e.target.parentElement.classList.add('focused');
  }
});

document.getElementById('form').addEventListener('focusout',
```

```
function(e) {  
  if (e.target.tagName === 'INPUT') {  
    e.target.parentElement.classList.remove('focused');  
  }  
});
```

实际应用案例

案例 1：任务列表应用

JavaScript

```
class TaskManager {  
  constructor(containerId) {  
    this.container = document.getElementById(containerId);  
    this.bindEvents();  
    this.tasks = [];  
  }  
  
  bindEvents() {  
    // 使用事件委托处理所有交互  
    this.container.addEventListener('click', (e) => {  
      // 完成任务  
      if (e.target.classList.contains('complete-btn')) {  
        this.completeTask(e.target.dataset.taskId);  
      }  
  
      // 删除任务  
      if (e.target.classList.contains('delete-btn')) {  
        this.deleteTask(e.target.dataset.taskId);  
      }  
  
      // 编辑任务  
      if (e.target.classList.contains('task-text')) {  
        this.editTask(e.target.dataset.taskId);  
      }  
    });  
  
    // 键盘事件委托  
    this.container.addEventListener('keydown', (e) => {  
      if (e.target.classList.contains('task-input') && e.key ===  
        'Enter') {  
        this.saveTask(e.target.dataset.taskId, e.target.value);  
      }  
    });  
  }  
}
```

```

    }
  });
}

completeTask(taskId) {
  // 完成任务逻辑
  const taskElement = this.container.querySelector(`[data-task-id="${taskId}"]`);
  taskElement.classList.toggle('completed');
}

// 其他方法...
}

```

案例 2：无限滚动列表

```

JavaScript
class InfiniteScroll {
  constructor(listId) {
    this.list = document.getElementById(listId);
    this.setupEventDelegation();
  }

  setupEventDelegation() {
    // 处理列表项点击
    this.list.addEventListener('click', (e) => {
      const item = e.target.closest('.list-item');
      if (item) {
        this.handleItemClick(item);
      }
    });

    // 处理图片懒加载
    const observer = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          const img = entry.target;
          img.src = img.dataset.src;
          observer.unobserve(img);
        }
      });
    });
  }
}

```

```

// 观察所有需要懒加载的图片
this.list.querySelectorAll('img[data-src]').forEach(img => {
  observer.observe(img);
});
}

// 动态加载更多内容
loadMore() {
  fetch('/more-items')
    .then(res => res.json())
    .then(items => {
      items.forEach(item => {
        const itemElement = this.createItemElement(item);
        this.list.appendChild(itemElement);
        // 无需为新元素单独绑定事件！
      });
    });
}
}

```

React/Vue 中的事件委托

React 中的事件委托

```

JavaScript
// React 自动使用事件委托：所有事件都委托到 document（React 17 之前）
或 root（React 17+）
function List({ items }) {
  // React 在内部处理事件委托
  const handleClick = (itemId) => {
    console.log('Item clicked:', itemId);
  };

  return (
    <ul>
      {items.map(item => (
        <li key={item.id} onClick={() => handleClick(item.id)}>
          {item.name}
        </li>
      ))}
    </ul>
  );
}

```



```

}

// 手动实现事件委托（在某些场景下有用）
function DelegatedList({ items }) {
  const containerRef = useRef();

  useEffect(() => {
    const handleClick = (e) => {
      if (e.target.matches('.list-item')) {
        const id = e.target.dataset.id;
        console.log('Clicked:', id);
      }
    };

    containerRef.current.addEventListener('click', handleClick);
    return () => {
      containerRef.current?.removeEventListener('click',
handleClick);
    };
  }, []);

  return (
    <div ref={containerRef}>
      {items.map(item => (
        <div key={item.id} className="list-item" data-
id={item.id}>
          {item.name}
        </div>
      ))}
    </div>
  );
}

```

Vue 中的事件委托

```

XML
<template>
  <!-- Vue 也自动处理事件绑定 -->
  <ul @click="handleClick">
    <li v-for="item in items"
      :key="item.id"
      :data-id="item.id"
      class="item">

```

```
    {{ item.name }}
  </li>
</ul>
</template>

<script>
export default {
  methods: {
    handleClick(event) {
      // 手动实现事件委托
      const item = event.target.closest('.item');
      if (item) {
        const id = item.dataset.id;
        console.log('Item clicked:', id);
      }
    }
  }
}
</script>
```

性能测试和最佳实践

```
JavaScript
// 性能测试函数
function testPerformance() {
  const container = document.getElementById('test-container');
  const count = 1000;

  // 创建测试元素
  for (let i = 0; i < count; i++) {
    const btn = document.createElement('button');
    btn.className = 'test-btn';
    btn.textContent = `Button ${i}`;
    container.appendChild(btn);
  }

  // 测试传统绑定
  console.time('传统绑定');
  const buttons = container.querySelectorAll('.test-btn');
  buttons.forEach(btn => {
    btn.addEventListener('click', () => {});
  });
  console.timeEnd('传统绑定');
```

```
// 测试事件委托
console.time('事件委托');
container.addEventListener('click', (e) => {
  if (e.target.classList.contains('test-btn')) {
    // 模拟处理逻辑
  }
});
console.timeEnd('事件委托');

// 测试事件处理速度
console.time('传统触发');
buttons[0].click();
console.timeEnd('传统触发');

console.time('委托触发');
buttons[0].click();
console.timeEnd('委托触发');
}

// 最佳实践总结：
// 1. 对于大量相似元素，使用事件委托
// 2. 使用 closest() 或 matches() 精确匹配目标
// 3. 注意事件冒泡被阻止的情况
// 4. 对于不会冒泡的事件，使用捕获阶段或替代方案
// 5. 在动态内容场景下，事件委托是必须的
```

27. 谈谈你对浏览器渲染过程的理解，包括关键渲染路径、重绘和回流

27.1.1 关键渲染路径 (Critical Rendering Path, CRP)

关键渲染路径是浏览器将 HTML、CSS 和 JavaScript 转换为屏幕上像素的过程。

详细步骤：

步骤 1：构建 DOM（文档对象模型）

XML

<!-- 浏览器解析 HTML -->

```
<html>
  <head>
    <title>页面标题</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <p>Hello <span>world</span></p>
    <script src="app.js"></script>
  </body>
</html>
```

```
/*
DOM 树结构:
- html
  - head
    - title
      - "页面标题"
    - link
  - body
    - p
      - "Hello "
      - span
        - "world"
    - script
*/
```

步骤 2: 构建 CSSOM (CSS 对象模型)

```
JavaScript
/* styles.css */
body { font-size: 16px; }
p { color: red; }
span { font-weight: bold; }

/*
CSSOM 树结构:
- body
  - font-size: 16px
- p
  - color: red
- span
  - font-weight: bold
*/
```

步骤 3: 构建渲染树 (Render Tree)

```
JavaScript
// 渲染树 = DOM 树 + CSSOM 树, 但只包含可见元素
// 不包含: display: none、head、script 等
/*
渲染树结构:
- html
  - body
    - p
      - "Hello "
    - span
      - "world"
*/
```

步骤 4: 布局 (Layout / Reflow)

```
YAML
// 计算每个元素在屏幕上的确切位置和大小
// 从根元素开始, 递归计算
const layout = {
  body: { x: 0, y: 0, width: 1200, height: 800 },
  p: { x: 20, y: 50, width: 500, height: 30 },
  span: { x: 100, y: 50, width: 50, height: 30 }
};
```

步骤 5: 绘制 (Paint)

```
JavaScript
// 将布局计算的每个节点转换为屏幕上的实际像素
// 包括文本、颜色、边框、阴影等视觉部分
paint(context, {
  element: 'p',
  text: 'Hello ',
  color: 'red',
  position: { x: 20, y: 50 }
});
```

步骤 6: 合成 (Compositing)

```
JavaScript
// 将各个图层合并, 显示在屏幕上
// 现代浏览器使用 GPU 加速合成
```

```
composite(layers => {  
  // 图层 1: 背景  
  // 图层 2: 文本  
  // 图层 3: 动画元素  
});
```

27.1.2 回流 (Reflow) 与重绘 (Repaint)

回流 (Reflow / Layout)

回流是当元素的尺寸、位置或某些属性改变时，浏览器需要重新计算布局的过程。

触发回流的操作：

```
JavaScript  
// 1. 几何属性改变  
element.style.width = '100px';  
element.style.height = '100px';  
element.style.padding = '10px';  
element.style.margin = '20px';  
element.style.borderWidth = '2px';  
  
// 2. 位置改变  
element.style.position = 'absolute';  
element.style.left = '50px';  
element.style.top = '50px';  
  
// 3. 内容改变  
element.innerHTML = '新内容';  
element.textContent = '新文本';  
  
// 4. 添加/删除可见 DOM 元素  
document.body.appendChild(newElement);  
element.removeChild(childElement);  
  
// 5. 读取某些几何属性（强制同步回流）  
const width = element.offsetWidth; // 触发回流  
const height = element.offsetHeight;  
const scrollTop = element.scrollTop;  
  
// 6. 窗口变化  
window.addEventListener('resize', () => {
```



```
// 触发整个页面的回流
});

// 7. 字体改变
element.style.fontSize = '20px';
```

重绘 (Repaint)

重绘是当元素的外观改变但不影响布局时，浏览器只需要重新绘制受影响的部分。

触发重绘的操作：

```
Java
// 颜色相关
element.style.color = 'red';
element.style.backgroundColor = '#fff';
element.style.borderColor = 'blue';

// 背景相关
element.style.backgroundImage = 'url(new-image.jpg)';
element.style.opacity = '0.5';

// 其他视觉属性
element.style.outline = '1px solid red';
element.style.boxShadow = '0 0 10px #000';
element.style.visibility = 'hidden'; // 注意：与 display: none 不同

// 不触发回流的变换
element.style.transform = 'translateX(100px)';
element.style.opacity = '0.8';
```

27.1.3 性能影响对比

```
JavaScript
// 测试回流和重绘的性能差异
function testPerformance() {
  const div = document.createElement('div');
  div.style.width = '100px';
  div.style.height = '100px';
  document.body.appendChild(div);

  console.time('回流测试');
```

```
for (let i = 0; i < 1000; i++) {
  // 触发回流: 改变宽度
  div.style.width = `${100 + i}px`;
  // 强制同步布局
  const width = div.offsetWidth;
}
console.timeEnd('回流测试'); // 约 50-100ms

console.time('重绘测试');
for (let i = 0; i < 1000; i++) {
  // 只触发重绘: 改变背景色
  div.style.backgroundColor = `rgb(${i % 255}, 100, 100)`;
}
console.timeEnd('重绘测试'); // 约 5-10ms

console.time('合成层测试');
for (let i = 0; i < 1000; i++) {
  // 只触发合成: 使用 transform
  div.style.transform = `translateX(${i}px)`;
}
console.timeEnd('合成层测试'); // 约 1-2ms
}
```

27.1.4 优化策略

策略 1: 减少回流次数

JavaScript

```
// ✘ 错误做法: 多次修改样式, 触发多次回流
element.style.width = '100px'; // 回流
element.style.height = '200px'; // 回流
element.style.left = '10px'; // 回流
```

```
// ✔ 正确做法 1: 使用 cssText 或 class
element.style.cssText = 'width: 100px; height: 200px; left: 10px;';
```

```
// ✔ 正确做法 2: 添加 class
element.classList.add('new-style');
```

```
// ✔ 正确做法 3: 使用 DocumentFragment
```

```
const fragment = document.createDocumentFragment();
for (let i = 0; i < 100; i++) {
  const div = document.createElement('div');
  fragment.appendChild(div);
}
document.body.appendChild(fragment); // 只触发一次回流
```

策略 2：批量读取和写入

```
Java
// ✗ 错误做法：读取和写入交替进行
const width = element.offsetWidth; // 读取，触发回流
element.style.width = width + 10 + 'px'; // 写入，触发回流
const height = element.offsetHeight; // 读取，触发回流
element.style.height = height + 10 + 'px'; // 写入，触发回流

// ✔ 正确做法：先读取，后写入（批量处理）
const width = element.offsetWidth;
const height = element.offsetHeight;
element.style.width = width + 10 + 'px';
element.style.height = height + 10 + 'px';
```

策略 3：使用合成层（GPU 加速）

```
JavaScript
// 这些属性会创建独立的合成层，避免回流和重绘
element.style.transform = 'translateZ(0)'; // 开启 GPU 加速
element.style.willChange = 'transform'; // 提示浏览器将要改变
element.style.backfaceVisibility = 'hidden';

// 使用 transform 和 opacity 实现动画
function animateWithTransform() {
  // 使用 transform 而不是 left/top
  element.style.transform = `translate(${x}px, ${y}px)`;
  // 使用 opacity 而不是 visibility
  element.style.opacity = '0.5';
}
```

策略 4：离线 DOM 操作

```
JavaScript
```

```
// ✖ 错误做法：在线修改
for (let i = 0; i < 100; i++) {
  const list = document.getElementById('list');
  const item = document.createElement('li');
  item.textContent = `Item ${i}`;
  list.appendChild(item); // 每次都会触发回流
}

// ✔ 正确做法 1：隐藏元素进行修改
const list = document.getElementById('list');
list.style.display = 'none'; // 触发一次回流
for (let i = 0; i < 100; i++) {
  const item = document.createElement('li');
  item.textContent = `Item ${i}`;
  list.appendChild(item); // 不触发回流（因为元素隐藏）
}
list.style.display = 'block'; // 触发一次回流

// ✔ 正确做法 2：使用 cloneNode
const list = document.getElementById('list');
const clone = list.cloneNode(false); // 浅克隆
for (let i = 0; i < 100; i++) {
  const item = document.createElement('li');
  item.textContent = `Item ${i}`;
  clone.appendChild(item);
}
list.parentNode.replaceChild(clone, list); // 只触发一次回流
```

27.1.5 现代浏览器的优化

分层（Layer）和合成（Composite）

```
JavaScript
// 浏览器自动分层优化
/*
页面被分成多个层：
1. 根层（Root Layer）
2. 文本层（Text Layer）
3. 图片层（Image Layer）
4. 合成层（Composited Layer）
```

分层依据:

- 3D 变换: `transform: translate3d()`
 - 视频、Canvas、WebGL
 - CSS 滤镜: `filter: blur()`
 - `will-change` 属性
- */

// 检查元素是否在独立图层

`console.log(getComputedStyle(element).transform);` // 非 `none` 值可能表示独立图层

请求动画帧 (`requestAnimationFrame`)

JavaScript

// ✖ 错误做法: 使用 `setTimeout/setInterval`

```
function animate() {  
  element.style.left = (parseFloat(element.style.left) + 1) +  
  'px';  
  setTimeout(animate, 16); // 约 60fps  
}
```

// ✔ 正确做法: 使用 `requestAnimationFrame`

```
function animate() {  
  element.style.left = (parseFloat(element.style.left) + 1) +  
  'px';  
  requestAnimationFrame(animate); // 与浏览器刷新率同步  
}
```

// 在 `rAF` 中批量样式修改

```
function updateStyles() {  
  requestAnimationFrame(() => {  
    // 所有样式修改在这里进行  
    element1.style.width = '100px';  
    element2.style.height = '200px';  
    element3.style.transform = 'translateX(50px)';  
  });  
}
```

27.1.6 调试工具

Chrome DevTools 性能分析

```

JavaScript
// 1. 性能面板 (Performance)
// 可以录制页面活动，分析回流和重绘

// 2. 渲染面板 (Rendering)
// - Paint flashing: 高亮显示重绘区域
// - Layout Shift Regions: 显示布局偏移
// - FPS meter: 帧率监控

// 3. 命令行 API
// 监控回流
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log('布局变化:', entry);
  }
});
observer.observe({ entryTypes: ['layout-shift'] });

// 监控重绘
const paintObserver = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log('绘制:', entry.name, entry.startTime);
  }
});
paintObserver.observe({ entryTypes: ['paint'] });

```

27.1.7 实际案例分析

案例 1: 无限滚动列表优化

```

JavaScript
class OptimizedInfiniteScroll {
  constructor() {
    this.items = [];
    this.container = document.getElementById('list');
    this.setupOptimizations();
  }

  setupOptimizations() {
    // 使用虚拟滚动: 只渲染可见区域
    this.virtualScroll();
  }
}

```



```

    // 使用 transform 实现滚动
    this.container.style.willChange = 'transform';
  }

  virtualScroll() {
    // 只创建可见区域的 DOM 元素
    const visibleCount = Math.ceil(window.innerHeight / 50);
    this.visibleItems = this.items.slice(0, visibleCount);

    // 使用绝对定位
    this.container.style.position = 'relative';
    this.visibleItems.forEach((item, index) => {
      const element = this.createElement(item);
      element.style.position = 'absolute';
      element.style.top = `${index * 50}px`;
      this.container.appendChild(element);
    });
  }

  createElement(item) {
    const div = document.createElement('div');
    div.className = 'list-item';
    div.textContent = item.text;

    // 预定义尺寸，避免回流
    div.style.width = '100%';
    div.style.height = '50px';
    div.style.boxSizing = 'border-box';

    return div;
  }
}

```

案例 2：动画性能优化

```

JavaScript
class SmoothAnimation {
  constructor(element) {
    this.element = element;
    this.init();
  }
}

```

```

init() {
  // 创建独立的合成层
  this.element.style.transform = 'translateZ(0)';
  this.element.style.willChange = 'transform, opacity';

  // 使用 CSS 变量减少样式计算
  this.element.style.setProperty('--x', '0px');
  this.element.style.setProperty('--y', '0px');
}

animateTo(x, y) {
  // 使用 transform 而不是 left/top
  // 触发合成，而不是回流
  this.element.style.transform = `translate(${x}px, ${y}px)`;

  // 或者使用 CSS 变量 + transform
  this.element.style.setProperty('--x', `${x}px`);
  this.element.style.setProperty('--y', `${y}px`);
  this.element.style.transform = 'translate(var(--x), var(--y))';
}

// 使用 Web Animations API
animateWithWebAPI() {
  this.element.animate([
    { transform: 'translateX(0px)', opacity: 1 },
    { transform: 'translateX(100px)', opacity: 0.5 }
  ], {
    duration: 1000,
    easing: 'ease-in-out',
    fill: 'forwards'
  });
}
}

```

总结对比表

阶段	触发条件	性能成本	优化策略
回流 (Reflow)	尺寸、位置、内容变化	高（重新计算布局）	批量修改、离线DOM、避免强制同步布局
重绘 (Repaint)	颜色、背景等外观变化	中（重新绘制像素）	减少重绘区域、使用CSS精灵图
合成 (Composite)	transform、opacity变化	低（GPU加速合成）	使用transform/opacity动画，开启

点击图片可查看完整电子表格

28. Web 性能优化有哪些常见的手段？请至少列举五种。

28.1.1 资源加载优化

压缩和最小化

```

Java
// 构建工具配置示例（webpack）
module.exports = {
  optimization: {
    minimize: true, // 压缩代码
    minimizer: [
      new TerserPlugin(), // 压缩 JavaScript
      new CssMinimizerPlugin(), // 压缩 CSS
      new ImageMinimizerPlugin({ // 压缩图片
        minimizer: {
          implementation: ImageMinimizerPlugin.imageminMinify,
          options: {
            plugins: ['imagemin-webp', 'imagemin-mozjpeg']
          }
        }
      })
    ]
  }
};

```

```
// Gzip/Brotli 压缩
// 服务器配置 (Nginx)
/*
gzip on;
gzip_types text/plain text/css application/json
application/javascript text/xml;
gzip_min_length 1000;
brotli on;
brotli_types text/plain text/css application/json
application/javascript;
*/
```

代码分割和懒加载

```
JavaScript
// React 懒加载
const LazyComponent = React.lazy(() => import('./LazyComponent'));

function MyComponent() {
  return (
    <Suspense fallback={<Loading />}>
      <LazyComponent />
    </Suspense>
  );
}

// 动态导入 (Webpack 自动代码分割)
button.addEventListener('click', async () => {
  const module = await import('./heavyModule.js');
  module.doSomething();
});

// 路由懒加载 (React Router)
const routes = [
  {
    path: '/dashboard',
    element: React.lazy(() => import('./Dashboard'))
  }
];

// 图片懒加载
 {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      observer.unobserve(img);
    }
  });
});

document.querySelectorAll('img.lazy').forEach(img => {
  observer.observe(img);
});

```

预加载和预连接

```

XML

<!-- 预加载关键资源 -->
<link rel="preload" href="critical.css" as="style">
<link rel="preload" href="main.js" as="script">

<!-- 预加载字体 -->
<link rel="preload" href="font.woff2" as="font" type="font/woff2"
crossorigin>

<!-- 预连接重要第三方域名 -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="dns-prefetch" href="https://api.example.com">

<!-- 预获取可能需要的资源 -->
<link rel="prefetch" href="/next-page.html" as="document">
<link rel="prerender" href="/next-page.html">

<!-- 预加载关键图片（LCP 优化） -->
<link rel="preload" as="image" href="hero-image.jpg"
imagesrcset="hero-480.jpg 480w, hero-800.jpg 800w">

```

28.1.2 网络请求优化

HTTP/2 和多路复用

```
JSON
// HTTP/2 带来的优化:
// 1. 多路复用: 一个连接并行多个请求
// 2. 服务器推送: 服务器主动推送资源
// 3. 头部压缩: HPACK 压缩

// 利用 HTTP/2 优化资源加载
// 将小文件合并为大文件的策略不再必要
// 反而应该拆分资源, 利用并行加载

// Nginx 启用 HTTP/2
/*
server {
    listen 443 ssl http2;
    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    # 服务器推送
    http2_push /style.css;
    http2_push /app.js;
}
*/
```

CDN 和缓存策略

```
JavaScript
// 缓存策略配置
// Service Worker 缓存
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open('v1').then(cache => {
            return cache.addAll([
                '/',
                '/index.html',
                '/styles.css',
                '/app.js',
                '/manifest.json'
            ]);
        })
    );
});
```



```

    })
  );
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response => {
      return response || fetch(event.request);
    })
  );
});

// HTTP 缓存头
// 服务器设置
/*
# 长期缓存（哈希文件名）
location ~*
\. (css|js|woff2?|eot|ttf|otf|svg|png|jpg|jpeg|gif|ico|webp)$ {
  expires 1y;
  add_header Cache-Control "public, immutable";
}

# HTML 文件不缓存或短期缓存
location ~* \.html$ {
  expires 1h;
  add_header Cache-Control "public, must-revalidate";
}
*/

```

请求合并和减少

```

JavaScript
// GraphQL 减少请求次数
const query = `
  query GetUserData($id: ID!) {
    user(id: $id) {
      name
      email
      posts {
        title
        content
      }
      comments {

```

```

        text
      }
    }
  }
};

// REST API 批量请求
fetch('/api/batch', {
  method: 'POST',
  body: JSON.stringify({
    requests: [
      { path: '/users/1', method: 'GET' },
      { path: '/posts', method: 'GET', params: { userId: 1 } }
    ]
  })
});

// 使用 WebSocket 减少 HTTP 请求
const ws = new WebSocket('wss://api.example.com/ws');
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  updateUI(data);
};

```

28.1.3 渲染性能优化

关键渲染路径优化

```

HTML
<!DOCTYPE html>
<html>
<head>
  <!-- 1. 关键 CSS 内联 -->
  <style>
    /* Above-the-fold 关键样式 */
    body { font-family: sans-serif; }
    .hero { background: #f0f0f0; }
  </style>

  <!-- 2. 非关键 CSS 异步加载 -->
  <link rel="stylesheet" href="non-critical.css" media="print"
onload="this.media='all'">

```

```

<!-- 3. 关键字体预加载 -->
<link rel="preload" href="font.woff2" as="font" crossorigin>
</head>
<body>
<!-- 4. 关键内容优先 -->
<div class="hero">
  <h1>主要标题</h1>
  <p>关键内容...</p>
</div>

<!-- 5. 非关键内容延后 -->
<div class="secondary-content">
  <!-- 次要内容 -->
</div>

<!-- 6. JavaScript 异步/延迟加载 -->
<script src="critical.js" defer></script>
<script src="non-critical.js" async></script>
</body>
</html>

```

防抖和节流

```

JavaScript
// 防抖：最后一次操作后执行
function debounce(func, wait) {
  let timeout;
  return function executedFunction(...args) {
    const later = () => {
      clearTimeout(timeout);
      func(...args);
    };
    clearTimeout(timeout);
    timeout = setTimeout(later, wait);
  };
}

// 节流：固定时间间隔执行
function throttle(func, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {

```

```

        func.apply(this, args);
        inThrottle = true;
        setTimeout(() => inThrottle = false, limit);
    }
};
}

// 实际应用
const searchInput = document.getElementById('search');
searchInput.addEventListener('input', debounce(function(e) {
    // 搜索请求，避免频繁调用
    fetchResults(e.target.value);
}, 300));

window.addEventListener('scroll', throttle(function() {
    // 滚动事件处理，避免频繁执行
    updateScrollPosition();
}, 100));

// 使用 requestAnimationFrame 优化动画
function smoothScroll() {
    let start = null;
    const duration = 500;

    function step(timestamp) {
        if (!start) start = timestamp;
        const progress = timestamp - start;
        const position = Math.min(progress / duration * 100, 100);
        window.scrollTo(0, position);

        if (progress < duration) {
            requestAnimationFrame(step);
        }
    }

    requestAnimationFrame(step);
}

```

虚拟滚动和列表优化

```

JavaScript
class VirtualScroll {

```

```
constructor(container, items, itemHeight) {
  this.container = container;
  this.items = items;
  this.itemHeight = itemHeight;
  this.visibleCount = Math.ceil(container.clientHeight /
itemHeight);
  this.startIndex = 0;
  this.endIndex = this.startIndex + this.visibleCount;

  this.init();
}

init() {
  // 设置容器高度
  this.container.style.height = `${this.items.length *
this.itemHeight}px`;
  this.container.style.position = 'relative';
  this.container.style.overflow = 'auto';

  // 创建可视区域
  this.viewport = document.createElement('div');
  this.viewport.style.position = 'absolute';
  this.viewport.style.top = '0';
  this.viewport.style.left = '0';
  this.viewport.style.width = '100%';
  this.container.appendChild(this.viewport);

  this.render();
  this.container.addEventListener('scroll',
this.handleScroll.bind(this));
}

handleScroll() {
  const scrollTop = this.container.scrollTop;
  this.startIndex = Math.floor(scrollTop / this.itemHeight);
  this.endIndex = Math.min(
    this.startIndex + this.visibleCount + 2, // 缓冲
    this.items.length
  );

  this.render();
}

render() {
```

```

// 只渲染可见项
const visibleItems = this.items.slice(this.startIndex,
this.endIndex);

// 更新视口位置
this.viewport.style.top = `${this.startIndex *
this.itemHeight}px`;
this.viewport.style.height = `${visibleItems.length *
this.itemHeight}px`;

// 渲染项
this.viewport.innerHTML = '';
visibleItems.forEach((item, index) => {
  const element = this.createItem(item);
  element.style.position = 'absolute';
  element.style.top = `${index * this.itemHeight}px`;
  element.style.height = `${this.itemHeight}px`;
  this.viewport.appendChild(element);
});
}

createItem(item) {
  const div = document.createElement('div');
  div.textContent = item.text;
  return div;
}
}

```

28.1.4 图片和媒体优化

响应式图片

```

XML
<!-- srcset 和 sizes -->
<img
  src="image-400.jpg"
  srcset="
    image-400.jpg 400w,
    image-800.jpg 800w,
    image-1200.jpg 1200w
  "
  sizes="(max-width: 600px) 100vw, 50vw"

```



```

    alt="响应式图片"
  >

  <!-- picture 元素 -->
  <picture>
    <!-- WebP 格式（现代浏览器） -->
    <source
      srcset="image.webp"
      type="image/webp"
    >
    <!-- 回退方案 -->
    <source
      srcset="image.jpg"
      type="image/jpeg"
    >
    <!-- 最终回退 -->
    
  </picture>

  <!-- 懒加载和占位符 -->
  

```

现代图片格式

```

JavaScript
// WebP 检测和回退
function supportsWebP() {
  return new Promise(resolve => {
    const img = new Image();
    img.onload = () => resolve(true);
    img.onerror = () => resolve(false);
    img.src =
'
4HAA==';
  });
}

```

```
// 根据支持情况提供不同格式
async function loadOptimalImage(element) {
  const supports = await supportsWebP();
  const src = element.dataset.src;

  if (supports && src) {
    element.src = src.replace(/\.(\jpg|png)$/, '.webp');
  } else {
    element.src = src;
  }
}

// AVIF 格式（更高效的下一代格式）
<picture>
  <source srcset="image.avif" type="image/avif">
  <source srcset="image.webp" type="image/webp">
  
</picture>
```

视频优化

```
XML
<!-- 视频懒加载 -->
<video
  preload="metadata"
  poster="video-poster.jpg"
  controls
  class="lazy-video"
>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
</video>

<script>
// 滚动到视口时加载视频
const videoObserver = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const video = entry.target;
      video.load();
      videoObserver.unobserve(video);
    }
  })
})
```

```
});  
});  
  
document.querySelectorAll('.lazy-video').forEach(video => {  
  videoObserver.observe(video);  
});  
</script>
```

28.1.5 性能监控和分析

核心 Web 指标 (Core Web Vitals)

```
JavaScript  
// 监控 LCP (最大内容绘制)  
const observer = new PerformanceObserver((entryList) => {  
  const entries = entryList.getEntries();  
  const lastEntry = entries[entries.length - 1];  
  console.log('LCP:', lastEntry.startTime);  
  // 目标: 小于 2.5 秒  
});  
  
observer.observe({ entryTypes: ['largest-contentful-paint'] });  
  
// 监控 FID (首次输入延迟)  
new PerformanceObserver((entryList) => {  
  const entries = entryList.getEntries();  
  entries.forEach(entry => {  
    const delay = entry.processingStart - entry.startTime;  
    console.log('FID:', delay);  
    // 目标: 小于 100 毫秒  
  });  
}).observe({ type: 'first-input', buffered: true });  
  
// 监控 CLS (累积布局偏移)  
let clsValue = 0;  
let clsEntries = [];  
  
new PerformanceObserver((entryList) => {  
  for (const entry of entryList.getEntries()) {  
    if (!entry.hadRecentInput) {  
      clsValue += entry.value;  
      clsEntries.push(entry);  
    }  
  }  
}).observe({ type: 'layout-shift', buffered: true });
```

```
    console.log('当前 CLS 值:', clsValue);
    // 目标: 小于 0.1
  }
}
}).observe({ type: 'layout-shift', buffered: true });
```

真实用户监控 (RUM)

JavaScript

// 简单的性能监控脚本

```
class PerformanceMonitor {
  constructor() {
    this.metrics = {};
    this.init();
  }
```

```
  init() {
    // 页面加载性能
    window.addEventListener('load', () => {
      setTimeout(() => {
        this.captureMetrics();
        this.sendMetrics();
      }, 0);
    });
  }
```

// 用户交互监控

```
this.monitorUserInteractions();
```

// 错误监控

```
window.addEventListener('error',
this.captureError.bind(this));
window.addEventListener('unhandledrejection',
this.capturePromiseError.bind(this));
}
```

```
captureMetrics() {
```

// 导航计时

```
const timing = performance.timing;
```

```
this.metrics = {
```

```
  dns: timing.domainLookupEnd - timing.domainLookupStart,
```

```
  tcp: timing.connectEnd - timing.connectStart,
```

```
  ttfb: timing.responseStart - timing.requestStart,
```

```
  domReady: timing.domContentLoadedEventEnd -
```

```
timing.navigationStart,
  pageLoad: timing.loadEventEnd - timing.navigationStart,

  // 资源加载时间
  resources:
performance.getEntriesByType('resource').map(resource => ({
  name: resource.name,
  duration: resource.duration,
  type: resource.initiatorType
})),

  // 核心 Web 指标
  fcp: performance.getEntriesByName('first-contentful-
paint')[0]?.startTime,
  lcp: performance.getEntriesByType('largest-contentful-
paint')[0]?.startTime
});
}

monitorUserInteractions() {
  let lastInteractionTime = Date.now();

  // 监控用户活动
  ['click', 'scroll', 'keydown', 'mousemove'].forEach(event => {
    document.addEventListener(event, () => {
      lastInteractionTime = Date.now();
    }, { passive: true });
  });

  // 空闲时间计算
  setInterval(() => {
    const idleTime = Date.now() - lastInteractionTime;
    if (idleTime > 30000) { // 30 秒空闲
      this.sendIdleEvent(idleTime);
    }
  }, 5000);
}

sendMetrics() {
  // 发送到监控服务器
  navigator.sendBeacon('/api/metrics',
JSON.stringify(this.metrics));
}
```

```
}
```

28.1.6 其他重要优化手段

服务器端渲染 (SSR)

```
JavaScript
// Next.js SSR 示例
export async function getServerSideProps(context) {
  const data = await fetchData();
  return {
    props: { data } // 在服务端获取数据并渲染
  };
}

function Page({ data }) {
  return (
    <div>
      <h1>服务端渲染的页面</h1>
      <p>{data}</p>
    </div>
  );
}

// Vue Nuxt.js SSR
export default {
  async asyncData({ params }) {
    const data = await fetchData();
    return { data };
  }
}
```

渐进式 Web 应用 (PWA)

```
JavaScript
// manifest.json
{
  "name": "我的 PWA",
  "short_name": "PWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
}
```



```

"theme_color": "#000000",
"icons": [
  {
    "src": "/icon-192.png",
    "sizes": "192x192",
    "type": "image/png"
  }
]
}

// Service Worker 缓存策略
const CACHE_NAME = 'v1';
const urlsToCache = ['/', '/styles.css', '/app.js'];

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(urlsToCache))
  );
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request)
      .then(response => {
        if (response) {
          return response;
        }
        return fetch(event.request).then(response => {
          if (!response || response.status !== 200 ||
response.type !== 'basic') {
            return response;
          }
          const responseToCache = response.clone();
          caches.open(CACHE_NAME)
            .then(cache => cache.put(event.request,
responseToCache));
          return response;
        });
      })
  );
});

```

Web Workers

```

TypeScript
// 主线程
const worker = new Worker('worker.js');

// 发送数据给 Worker
worker.postMessage({ type: 'process', data: largeData });

// 接收 Worker 处理结果
worker.onmessage = (event) => {
  console.log('Worker 结果:', event.data);
};

// worker.js
self.onmessage = (event) => {
  const { type, data } = event.data;

  if (type === 'process') {
    // 执行耗时操作，不阻塞主线程
    const result = heavyProcessing(data);
    self.postMessage(result);
  }
};

function heavyProcessing(data) {
  // 复杂计算
  return data.map(item => item * 2);
}

```

优化检查清单

```

Java
// 性能优化检查清单
const checklist = {
  // 加载优化
  loading: [
    '压缩资源（JS/CSS/图片）',
    '启用 Gzip/Brotli 压缩',
    '使用 HTTP/2',
    '代码分割和懒加载',
    '预加载关键资源',
    '使用 CDN',
  ],
};

```

```
    '优化缓存策略'
  ],

  // 渲染优化
  rendering: [
    '优化关键渲染路径',
    '减少重绘和回流',
    '使用 transform/opacity 动画',
    '虚拟滚动长列表',
    '防抖和节流事件',
    '使用 requestAnimationFrame',
    '避免强制同步布局'
  ],

  // 资源优化
  resources: [
    '使用 WebP/AVIF 图片格式',
    '实现响应式图片',
    '图片懒加载',
    '字体优化和 subset',
    '视频优化和懒加载',
    '移除未使用的代码',
    'Tree-shaking 和代码分割'
  ],

  // 监控和持续优化
  monitoring: [
    '监控 Core Web Vitals',
    '真实用户监控 (RUM)',
    '性能预算设定',
    'A/B 测试优化方案',
    '定期性能审计',
    '错误监控和上报'
  ]
};
```

29. null 和 undefined 有什么区别? typeof null 的结果是什么, 为什么?

null 和 undefined 的区别

29.1.1.1 基本定义

```
JavaScript
// undefined 表示"未定义"
let a;           // 变量声明但未赋值
console.log(a);  // undefined

function foo(b) {
  console.log(b); // 未传递参数时 b 为 undefined
}
foo();

const obj = {};
console.log(obj.notExist); // 访问不存在的属性

// null 表示"空值", 是主动赋值的
let b = null;      // 明确表示 b 为空
console.log(b);    // null
```

29.1.1.2 区别对比表

特性	undefined	null
类型	全局对象的一个属性	表示空值的原始值
表示含义	变量未定义或未赋值	表示空值或空对象指针
类型检查	typeof undefined → "undefined"	typeof null → "object"
转换为数字	Number(undefined) → NaN	Number(null) → 0
转换为布尔	Boolean(undefined)	Boolean(null)

点击图片可查看完整电子表格

29.1.1.3 实际使用场景

```
JavaScript
// 场景 1: 函数参数默认值
```

```

function greet(name = 'Guest') {
  // 只有 undefined 会触发默认值
  console.log(`Hello, ${name}!`);
}
greet();           // Hello, Guest! (undefined 触发默认值)
greet(null);       // Hello, null! (null 不会触发默认值)

// 场景 2: 对象属性删除
const obj = { a: 1, b: 2 };
obj.a = undefined; // 属性仍然存在, 值为 undefined
obj.b = null;       // 属性存在, 值为 null
delete obj.a;       // 完全删除属性

console.log(obj);    // { a: undefined, b: null }
console.log('a' in obj); // true
console.log('b' in obj); // true

// 场景 3: 数组中的表现
const arr = [undefined, null];
console.log(arr[0]); // undefined
console.log(arr[1]); // null
console.log(arr.length); // 2
console.log(0 in arr); // true (索引 0 存在)
console.log(arr.hasOwnProperty(0)); // true

const sparseArr = [, ,]; // 稀疏数组
console.log(sparseArr[0]); // undefined
console.log(0 in sparseArr); // false (索引 0 不存在)

```

29.1.1.4 类型检查实践

```

JavaScript
// 各种检查方式对比
const value = null;

// typeof 检查
console.log(typeof value); // "object" (问题所在!)
console.log(typeof undefined); // "undefined"

// 正确检查 null
console.log(value === null); // true
console.log(value == null); // true (但也会匹配 undefined)

```

```

// 检查 undefined
let undef;
console.log(undef === undefined); // true
console.log(typeof undef === 'undefined'); // true

// 通用检查函数
function isNullOrUndefined(value) {
  return value == null; // 注意：使用 == 而非 ===
}

// 现代 JavaScript 的检查
console.log(Object.is(value, null)); // true
console.log(Object.is(undefined, null)); // false

// 可选链操作符
const obj = { a: { b: null } };
console.log(obj.a.b); // null
console.log(obj.a.b?.c); // undefined (不会报错)
console.log(obj.a.b.c); // TypeError

```

typeof null 的结果和原因

29.1.1.1 令人困惑的结果

```

JavaScript
console.log(typeof null); // "object" 而不是 "null"
console.log(typeof undefined); // "undefined"

// 历史原因造成的设计缺陷
// 这是 JavaScript 最初实现时的错误，现在已无法修复

```

29.1.1.2 历史原因深度解析

```

JSON
// JavaScript 类型标签的底层表示
// 在 V8 引擎中，对象使用 32 位表示：
// 000: object pointer
// 001: integer
// 010: double
// 100: string

```



```

// 110: boolean
// 111: undefined/null

// null 在底层被表示为全 0: 0x00000000
// 而 typeof 检查第一个比特位（最低位）
// 由于历史原因，null 的位模式被错误地解释为对象

// 简单来说：
// null 的机器码：00000000 00000000 00000000 00000000
// 对象标签：xxxxxxx xxxxxxxx xxxxxxxx xxxxxxx0（最后一位是 0）
// typeof 看到最后一位是 0，就认为是对象

```

29.1.1.3 为什么不能修复

```

JavaScript
// 1. 向后兼容性问题
// 大量的现有代码依赖这个行为
if (typeof value === 'object') {
  // 如果修复，这里的逻辑会被破坏
}

// 2. 提案尝试
// 曾有提案想修改 typeof null 的行为
// 但被拒绝，因为破坏性太大

// 3. 替代方案
// 使用新的检查方式
console.log(Object.prototype.toString.call(null)); // "[object Null]"

// 或者使用 Symbol.toStringTag
class NullObject {
  get [Symbol.toStringTag]() {
    return 'Null';
  }
}

```

29.1.1.4 实际影响和解决方案

```

JavaScript
// 问题：无法用 typeof 区分 null 和对象

```

```

const values = [null, {}, undefined, []];

values.forEach(value => {
  console.log(`${value} -> typeof: ${typeof value}`);
  // null -> typeof: object
  // {} -> typeof: object
  // undefined -> typeof: undefined
  // [] -> typeof: object
});

// 解决方案 1: 组合检查
function getType(value) {
  if (value === null) return 'null';
  if (value === undefined) return 'undefined';
  return typeof value;
}

// 解决方案 2: 使用 Object.prototype.toString
function accurateTypeOf(value) {
  return Object.prototype.toString.call(value)
    .slice(8, -1)
    .toLowerCase();
}

console.log(accurateTypeOf(null));           // "null"
console.log(accurateTypeOf(undefined));      // "undefined"
console.log(accurateTypeOf([]));             // "array"
console.log(accurateTypeOf({}));             // "object"
console.log(accurateTypeOf(123));            // "number"

// 解决方案 3: 使用现代 API
console.log(Object.prototype.toString.call(null)); // "[object Null]"
console.log(Object.prototype.toString.call(undefined)); // "[object Undefined]"

```

在框架和库中的处理

React 中的处理

```

JavaScript
// React 组件 props 的默认值
function Component({ value = 'default' }) {

```

```

// undefined 会使用默认值, null 不会
return <div>{value}</div>;
}

<Component />           // 显示 "default" (undefined)
<Component value={null} /> // 显示 "null"
<Component value={undefined} /> // 显示 "default"

// React 的状态更新
const [state, setState] = useState(null);

setState(null);           // 允许设置为 null
setState(undefined);      // 也允许, 但可能会触发默认值

// React 的条件渲染
function UserProfile({ user }) {
  // 检查 null 和 undefined
  if (!user) {
    return <div>Loading...</div>;
  }

  // 或者使用可选链
  return <div>{user?.name}</div>;
}

```

Vue 中的处理

```

XML
<template>
  <!-- Vue 模板中, null 和 undefined 都会被忽略 -->
  <div v-if="value">{{ value }}</div>

  <!-- 使用默认值 -->
  <div>{{ value || 'default' }}</div>
</template>

<script>
export default {
  props: {
    // 只有 undefined 会触发默认值
    value: {
      type: String,

```

```
    default: 'default'
  }
},

data() {
  return {
    // 可以设置为 null
    user: null
  };
},

methods: {
  fetchUser() {
    this.user = null; // 重置为 null
    // 或者设置为 undefined
  }
}
};
</script>
```

最佳实践

```
JavaScript
// 1. 明确区分使用场景
const config = {
  // undefined: 属性不存在或未设置
  // null: 属性存在但为空
  timeout: undefined, // 未设置超时（使用默认值）
  retry: null,        // 明确设置为不重试
  maxConnections: 10  // 明确设置值
};

// 2. 使用严格相等检查
function processValue(value) {
  if (value === null) {
    // 明确处理 null
    return '处理 null 的情况';
  }

  if (value === undefined) {
    // 明确处理 undefined
    return '使用默认值';
  }
}
```

```

    }

    return `处理值: ${value}`;
  }

// 3. 使用默认参数和默认值
function createUser({
  name = 'Anonymous',
  age = 0,
  email = null // 明确允许为 null
} = {}) {
  return { name, age, email };
}

// 4. JSON 序列化处理
const data = {
  a: undefined, // 会被忽略
  b: null,      // 会被保留
  c: 'value'
};

console.log(JSON.stringify(data)); // '{"b":null,"c":"value"}'

// 5. 数据库和 API 交互
async function saveToDatabase(data) {
  // undefined 通常表示"跳过此字段"
  // null 通常表示"显式设置为空"
  const payload = {
    name: data.name || null, // 空字符串转为 null
    age: data.age || undefined, // 未提供则跳过
    email: data.email // 保留原始值
  };

  // 过滤掉 undefined 值
  const cleanPayload = Object.fromEntries(
    Object.entries(payload).filter(([_, v]) => v !== undefined)
  );

  return await api.save(cleanPayload);
}

```

总结

TypeScript

// 快速参考表

```
const nullVsUndefined = {
  undefined: {
    meaning: "变量已声明但未赋值",
    typeof: "undefined",
    numberConversion: NaN,
    whenToUse: "表示未定义的状态",
    check: "value === undefined 或 typeof value === 'undefined'"
  },
  null: {
    meaning: "表示空值或无对象",
    typeof: "object (历史 bug)",
    numberConversion: 0,
    whenToUse: "主动设置空值",
    check: "value === null"
  },
  common: {
    bothFalsy: true,
    looseEquality: true,
    defaultParameters: "只有 undefined 触发默认值",
    json: "undefined 被忽略, null 被保留"
  }
};

// 实用工具函数
const TypeUtils = {
  isNull(value) {
    return value === null;
  },

  isUndefined(value) {
    return value === undefined;
  },

  isNullOrUndefined(value) {
    return value == null; // 使用 == 同时检查 null 和 undefined
  },

  getType(value) {
    if (value === null) return 'null';
    const type = typeof value;
    if (type === 'object' && Array.isArray(value)) return 'array';
  }
};
```



```

    return type;
  },

  // 安全获取嵌套属性
  getSafe(obj, path, defaultValue = undefined) {
    return path.split('.').reduce((acc, key) => {
      if (acc == null) return defaultValue;
      return acc[key];
    }, obj);
  }
};

```

30. Array.prototype.map 和 Array.prototype.forEach 有什么本质区别？

核心区别总结

特性	map()	forEach()
返回值	返回新数组	返回 undefined
目的	转换/映射数组元素	遍历/执行副作用
是否改变原数组	否（纯函数）	否（但回调可以改变）
链式调用	支持	不支持（返回 undefined）
性能	稍慢（需要创建新数组）	稍快
使用场景	数据转换、提取属性	执行操作、副作用、修改外部变量

点击图片可查看完整电子表格

详细对比和示例

30.1.1.1 返回值差异

```

JavaScript
// map() 返回新数组
const numbers = [1, 2, 3, 4];

const doubledMap = numbers.map(num => num * 2);
console.log(doubledMap); // [2, 4, 6, 8]
console.log(numbers);    // [1, 2, 3, 4] (原数组不变)

```

```
// forEach() 返回 undefined
const doubledForEach = numbers.forEach(num => num * 2);
console.log(doubledForEach); // undefined
console.log(numbers);        // [1, 2, 3, 4] (原数组不变)
```

30.1.1.2 使用场景对比

场景 1: 数据转换 (适合 map)

```
YAML
// 提取对象数组中的特定属性
const users = [
  { id: 1, name: 'Alice', age: 25 },
  { id: 2, name: 'Bob', age: 30 },
  { id: 3, name: 'Charlie', age: 35 }
];

// 使用 map 提取名称数组
const names = users.map(user => user.name);
console.log(names); // ['Alice', 'Bob', 'Charlie']

// 使用 map 转换数据结构
const userMap = users.map(user => ({
  userId: user.id,
  fullName: user.name.toUpperCase(),
  isAdult: user.age >= 18
}));
console.log(userMap);
// [
//   { userId: 1, fullName: 'ALICE', isAdult: true },
//   { userId: 2, fullName: 'BOB', isAdult: true },
//   { userId: 3, fullName: 'CHARLIE', isAdult: true }
// ]
```

场景 2: 执行副作用 (适合 forEach)

```
JavaScript
// 更新 DOM 元素
const items = ['apple', 'banana', 'orange'];

// 使用 forEach 更新 UI
const list = document.getElementById('fruit-list');
items.forEach(item => {
```

```

const li = document.createElement('li');
li.textContent = item;
list.appendChild(li);
});

// 修改外部变量
let total = 0;
const prices = [10, 20, 30];

prices.forEach(price => {
  total += price; // 副作用：修改外部变量
});
console.log(total); // 60

// 发送 API 请求（并行）
const userIds = [1, 2, 3];
userIds.forEach(userId => {
  fetch(`/api/users/${userId}`)
    .then(res => res.json())
    .then(data => console.log(data));
});

```

30.1.3 链式调用能力

```

JavaScript
const numbers = [1, 2, 3, 4, 5];

// map() 支持链式调用
const result = numbers
  .map(x => x * 2)      // [2, 4, 6, 8, 10]
  .filter(x => x > 5)   // [6, 8, 10]
  .reduce((sum, x) => sum + x, 0); // 24

console.log(result); // 24

// forEach() 不支持链式调用（返回 undefined）
const brokenChain = numbers
  .forEach(x => console.log(x)) // 打印每个元素，返回 undefined
  .map(x => x * 2); // TypeError: Cannot read property 'map' of undefined

// 但可以在 map() 之后使用 forEach()
numbers

```

```
.map(x => x * 2)          // 转换
.forEach(x => console.log(x)); // 执行副作用
```

30.1.4 性能比较

JavaScript

// 性能测试函数

```
function performanceTest() {
  const largeArray = Array.from({ length: 1000000 }, (_, i) => i);
```

```
  console.time('map');
  const mapResult = largeArray.map(x => x * 2);
  console.timeEnd('map');
```

```
  console.time('forEach');
  const forEachResult = [];
  largeArray.forEach(x => {
    forEachResult.push(x * 2);
  });
  console.timeEnd('forEach');
```

```
  console.time('for loop');
  const loopResult = [];
  for (let i = 0; i < largeArray.length; i++) {
    loopResult.push(largeArray[i] * 2);
  }
  console.timeEnd('for loop');
```

/*

典型结果:

map: 约 30-50ms

forEach: 约 25-40ms

for loop: 约 15-25ms

注意: 这些方法创建新数组的方式不同

*/

}

// 纯遍历性能测试（不创建新数组）

```
function traversalPerformance() {
  const largeArray = Array.from({ length: 1000000 }, (_, i) => i);
  let sum = 0;
```

```
console.time('map traversal');
largeArray.map(x => { sum += x; });
console.timeEnd('map traversal');

sum = 0;
console.time('forEach traversal');
largeArray.forEach(x => { sum += x; });
console.timeEnd('forEach traversal');

sum = 0;
console.time('for loop traversal');
for (let i = 0; i < largeArray.length; i++) {
  sum += largeArray[i];
}
console.timeEnd('for loop traversal');
}
```

30.1.5 高级用法和技巧

map() 的高级用法

```
JavaScript
// 1. 映射到多个值
const users = [
  { id: 1, firstName: 'John', lastName: 'Doe' },
  { id: 2, firstName: 'Jane', lastName: 'Smith' }
];

const fullNames = users.map(user => ({
  id: user.id,
  fullName: `${user.firstName} ${user.lastName}`,
  initials: `${user.firstName[0]}${user.lastName[0]}`
})));

// 2. 有条件地映射
const numbers = [1, 2, 3, 4, 5];
const evenSquared = numbers.map(num => {
  return num % 2 === 0 ? num * num : num;
});
console.log(evenSquared); // [1, 4, 3, 16, 5]

// 3. 使用索引参数
```

```
const items = ['a', 'b', 'c'];
const indexedItems = items.map((item, index) => ({
  value: item,
  position: index,
  isEven: index % 2 === 0
})));

// 4. 稀疏数组处理
const sparseArray = [1, , 3, , 5]; // 两个空位
const mappedSparse = sparseArray.map(x => x * 2);
console.log(mappedSparse); // [2, empty, 6, empty, 10]
```

forEach() 的高级用法

```
JavaScript
// 1. 提前终止（使用异常）
try {
  [1, 2, 3, 4, 5].forEach(num => {
    if (num === 3) {
      throw new Error('Stop iteration');
    }
    console.log(num);
  });
} catch (e) {
  if (e.message !== 'Stop iteration') throw e;
}
// 输出: 1, 2

// 2. 修改原数组（不推荐，但可能有用）
const array = [1, 2, 3];
array.forEach((num, index, arr) => {
  arr[index] = num * 2; // 直接修改原数组
});
console.log(array); // [2, 4, 6]

// 3. 异步操作
async function processItems(items) {
  const results = [];

  // 顺序执行（forEach 不会等待异步）
  items.forEach(async (item) => {
    const result = await processItem(item);
    results.push(result);
  });
}
```



```
});

// 注意: forEach 不会等待异步完成
// 如果需要顺序执行, 使用 for...of

// 并行执行 (使用 Promise.all 更好)
await Promise.all(items.map(async (item) => {
  return await processItem(item);
})));
}
```

30.1.6 常见误用和正确用法

误用 1: 用 forEach 代替 map

```
JavaScript
// ✗ 误用: 用 forEach 创建新数组
const numbers = [1, 2, 3];
const doubled = [];

numbers.forEach(num => {
  doubled.push(num * 2);
});
// 需要额外创建数组并 push

// ✔ 正确: 直接用 map
const doubledCorrect = numbers.map(num => num * 2);
```

误用 2: 用 map 代替 forEach

```
JavaScript
// ✗ 误用: 用 map 只为了副作用
const elements = document.querySelectorAll('.item');

Array.from(elements).map(element => {
  element.classList.add('active'); // 副作用
  return element; // 返回未使用的数组
});

// ✔ 正确: 用 forEach
Array.from(elements).forEach(element => {
```

```
element.classList.add('active');
});
```

误用 3: 忽略 map 的返回值

```
JavaScript
// ✗ 误用: 调用 map 但忽略返回值
const users = [{ name: 'Alice' }, { name: 'Bob' }];
users.map(user => {
  console.log(user.name); // 只有副作用
});
// 创建了未使用的数组: [undefined, undefined]

// ✔ 正确: 如果只需要副作用, 用 forEach
users.forEach(user => {
  console.log(user.name);
});
```

30.1.7 现代 JavaScript 的替代方案

使用 for...of 循环

```
C++
const numbers = [1, 2, 3, 4];

// 替代 forEach (更灵活, 可以 break/continue)
for (const num of numbers) {
  if (num === 3) break; // 可以提前终止
  console.log(num);
}

// 替代 map (需要手动收集结果)
const doubled = [];
for (const num of numbers) {
  doubled.push(num * 2);
}
```

使用 reduce 实现复杂转换

```
JavaScript
// 使用 reduce 替代 map + filter
```

```
const numbers = [1, 2, 3, 4, 5];

// 传统方式: 先过滤再映射
const result1 = numbers
  .filter(x => x % 2 === 0)
  .map(x => x * 2);

// 使用 reduce 一次完成
const result2 = numbers.reduce((acc, x) => {
  if (x % 2 === 0) {
    acc.push(x * 2);
  }
  return acc;
}, []);

console.log(result1); // [4, 8]
console.log(result2); // [4, 8]
```

30.1.8 实际应用示例

示例 1: React 组件中的使用

```
JavaScript
// 使用 map 渲染列表
function UserList({ users }) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>
          {user.name} - {user.email}
        </li>
      ))}
    </ul>
  );
}

// 使用 forEach 处理副作用
function ShoppingCart({ items }) {
  const [total, setTotal] = useState(0);

  useEffect(() => {
    let sum = 0;
    items.forEach(item => {
```

```
    sum += item.price * item.quantity;
  });
  setTotal(sum);
}, [items]);

return <div>Total: ${total}</div>;
}
```

示例 2：数据管道处理

```
JavaScript
// 复杂数据处理管道
function processSalesData(transactions) {
  return transactions
    // 1. 过滤无效交易
    .filter(t => t.amount > 0 && t.status === 'completed')

    // 2. 转换数据结构
    .map(t => ({
      id: t.transactionId,
      date: new Date(t.timestamp),
      amount: t.amount,
      currency: t.currency.toUpperCase(),
      category: t.category || 'uncategorized'
    }))

    // 3. 按日期分组
    .reduce((groups, transaction) => {
      const dateStr =
transaction.date.toISOString().split('T')[0];
      if (!groups[dateStr]) {
        groups[dateStr] = [];
      }
      groups[dateStr].push(transaction);
      return groups;
    }, {});

  // 注意：forEach 不适合这种链式处理
}
```

30.1.9 最佳实践总结

JavaScript

// 决策流程图

```
function chooseArrayMethod(array, purpose) {  
  /*  
  决策逻辑:  
  1. 需要新数组吗? → 是 → 使用 map()  
  2. 需要提前终止吗? → 是 → 使用 for...of  
  3. 需要索引吗? → 是 → map() 或 forEach()  
  4. 只是副作用吗? → 是 → 使用 forEach()  
  5. 需要链式调用吗? → 是 → 使用 map()/filter()/reduce()  
  */  
}
```

// 实用工具函数

```
const ArrayUtils = {  
  // 安全的 map, 处理 null/undefined  
  safeMap(array, callback) {  
    if (!array || !Array.isArray(array)) return [];  
    return array.map(callback);  
  },  
  
  // 带索引的 forEach (类似 map 但无返回值)  
  forEachWithIndex(array, callback) {  
    if (!array || !Array.isArray(array)) return;  
    array.forEach((item, index) => callback(item, index));  
  },  
  
  // 批量处理 (分块)  
  chunkMap(array, chunkSize, mapCallback) {  
    const chunks = [];  
    for (let i = 0; i < array.length; i += chunkSize) {  
      const chunk = array.slice(i, i + chunkSize);  
      chunks.push(chunk.map(mapCallback));  
    }  
    return chunks;  
  }  
};
```

// 性能建议:

```
// 1. 大数组: 优先考虑性能, for 循环最快  
// 2. 小数组: 使用 map/forEach, 代码更清晰  
// 3. 链式操作: 使用 map/filter/reduce
```

```
// 4. 异步操作：使用 Promise.all + map
```

31. Webpack 或 Vite 等构建工具的理解，它们解决了什么问题？

构建工具解决的问题

31.1.1.1 模块化支持

```
JavaScript
// 问题：浏览器原生不支持模块导入导出
// ✕ 浏览器中无法直接使用
import { utils } from './utils.js';
export default function() { ... }

// ✔ 构建工具将其转换为浏览器支持的代码
// Webpack 输出：
(function(modules) {
  // webpack 运行时
})(function(modules) {
  "._/src/utils.js": function(module, exports) {
    // 转换后的代码
  }
});
```

31.1.1.2 资源管理和依赖处理

```
XML
// 问题：手动管理依赖和加载顺序
<!-- ✕ 传统方式 -->
<script src="jquery.js"></script>
<script src="plugin.js"></script> <!-- 依赖 jquery -->
<script src="app.js"></script> <!-- 依赖 plugin -->

// ✔ 构建工具自动处理
// app.js
import $ from 'jquery';
import plugin from './plugin';
import './styles.css';
```



```
import logo from './logo.png';
```

31.1.1.3 代码转换和编译

JavaScript

// 问题：浏览器兼容性和现代语法

// ❑ 源代码（使用现代语法）

```
const component = () => {  
  const [state, setState] = useState(null);  
  return <div>Hello</div>;  
};
```

// ✔ 构建工具转换

```
var component = function component() {  
  var _useState = useState(null),  
      state = _useState[0],  
      setState = _useState[1];  
  
  return React.createElement("div", null, "Hello");  
};
```

31.1.1.4 性能优化

XML

// 问题：未优化资源影响性能

<!-- ❌ 未优化 -->

<script src="large-library.js"></script> <!-- 1MB -->

<link rel="stylesheet" href="unused.css"> <!-- 未使用 -->

// ✔ 构建工具优化

// 1. 代码分割（Code Splitting）

// 2. 摇树优化（Tree Shaking）

// 3. 懒加载（Lazy Loading）

// 4. 压缩（Minification）

Webpack 核心工作流程

整体架构

详细工作流程

```

JavaScript
// webpack.config.js
module.exports = {
  entry: './src/index.js', // 入口起点
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.js'
  },
  mode: 'production', // 模式: development/production
  // 初始化 Compiler 实例
};

// Webpack 内部:
class Compiler {
  constructor(config) {
    this.options = config;
    this.hooks = {
      initialize: new SyncHook(),
      beforeRun: new AsyncSeriesHook(),
      run: new AsyncSeriesHook(),
      // ... 更多生命周期钩子
    };
  }
}

```

阶段 2: 编译构建

```

JavaScript
// 1. 从入口开始解析
// index.js
import utils from './utils';
import './styles.css';

// 2. 创建模块依赖图
const dependencyGraph = {
  './src/index.js': {
    id: './src/index.js',
    dependencies: ['./src/utils.js', './src/styles.css'],
    code: 'import utils from "./utils"; import "./styles.css";',
    ast: AbstractSyntaxTree // 抽象语法树
  },
  './src/utils.js': {
    id: './src/utils.js',

```

```

    dependencies: [],
    code: 'export default function() {}'
  }
};

// 3. 加载器 (Loaders) 处理
// webpack.config.js
module: {
  rules: [
    {
      test: /\.css$/,
      use: ['style-loader', 'css-loader'] // 从右到左执行
    },
    {
      test: /\.js$/,
      use: 'babel-loader' // ES6+ → ES5
    }
  ]
}

// Loader 工作流程:
// 1. css-loader: 解析 CSS 中的 @import 和 url()
// 2. style-loader: 将 CSS 插入到 DOM

```

阶段 3: 代码生成

```

JavaScript
// 1. 模块合并
// 原始模块
// module1.js
export const a = 1;

// module2.js
import { a } from './module1';
console.log(a);

// 2. Webpack 生成的代码
(function(modules) {
  // webpackBootstrap 运行时
  var installedModules = {};

  function __webpack_require__(moduleId) {
    if (installedModules[moduleId]) {

```

```

        return installedModules[moduleId].exports;
    }
    var module = installedModules[moduleId] = {
        i: moduleId,
        l: false,
        exports: {}
    };
    modules[moduleId].call(module.exports, module, module.exports,
__webpack_require__);
    module.l = true;
    return module.exports;
}

return __webpack_require__((__webpack_require__.s =
"./src/index.js"));
})(({
    "./src/module1.js": function(module, exports) {
        exports.a = 1;
    },

    "./src/module2.js": function(module, exports,
__webpack_require__) {
        var _module1__WEBPACK_IMPORTED_MODULE_0__ =
__webpack_require__("./src/module1.js");
        console.log(_module1__WEBPACK_IMPORTED_MODULE_0__.a);
    },

    "./src/index.js": function(module, exports, __webpack_require__)
{
    __webpack_require__("./src/module2.js");
}
}));

```

阶段4：优化处理

```

JavaScript
// 插件（Plugins）在生命周期中执行
class MyPlugin {
    apply(compiler) {
        compiler.hooks.compilation.tap('MyPlugin', compilation => {
            // 优化钩子
            compilation.hooks.optimizeChunkAssets.tapAsync('MyPlugin',
(chunks, callback) => {
                // 优化资源
            });
        });
    }
}

```

```

    });
  });
}
}

// 常见优化:
// 1. 摇树优化 (Tree Shaking)
// 移除未使用的代码
export const used = 'used';
export const unused = 'unused'; // 被移除

// 2. 代码分割 (Code Splitting)
// webpack.config.js
optimization: {
  splitChunks: {
    chunks: 'all',
    cacheGroups: {
      vendors: {
        test: /[\\/]node_modules[\\/]/,
        name: 'vendors'
      }
    }
  }
}

// 3. 作用域提升 (Scope Hoisting)
// 转换前:
import { a } from './module';
console.log(a);

// 转换后:
console.log(1); // 直接内联值

```

Vite 的核心工作流程

与 Webpack 的对比

```

JavaScript
// Vite 采用不同的架构
/*
Webpack:
构建时编译 (打包时处理所有模块)

```

- 首屏加载慢
- 热更新需要重建整个包

Vite:

基于 ES Modules 的按需编译

- 开发时快速启动
 - 热更新只更新修改的模块
- */

Vite 工作流程

开发模式 (Dev Server)

TypeScript

```
// 1. 启动开发服务器
// vite dev 命令执行时:
// - 启动 Koa 服务器
// - 不打包, 直接提供 ESM

// 2. 浏览器请求模块
// index.html
<script type="module" src="/src/main.js"></script>

// main.js
import { createApp } from 'vue';
import App from './App.vue'; // .vue 文件请求

// 3. Vite 中间件处理
// 伪代码:
app.use(async (ctx, next) => {
  if (ctx.path.endsWith('.vue')) {
    // 编译 .vue 文件
    const content = await compileVue(ctx.path);
    ctx.type = 'application/javascript';
    ctx.body = content;
  } else if (ctx.path.endsWith('.js')) {
    // 使用 esbuild 转译
    const result = await esbuild.transform(code, {
      loader: 'js',
      target: 'esnext'
    });
    ctx.body = result.code;
  }
});
```



```

    }
  });

  // 4. 热更新 (HMR)
  // Vite 内置 HMR API
  if (import.meta.hot) {
    import.meta.hot.accept('./module.js', (newModule) => {
      // 模块更新时的回调
    });
  }
}

```

生产构建

```

TypeScript
// vite.config.js
export default {
  build: {
    // 使用 Rollup 进行生产构建
    rollupOptions: {
      input: 'index.html',
      output: {
        format: 'es', // 输出 ESM
        dir: 'dist'
      }
    }
  },

  // 预构建依赖
  optimizeDeps: {
    include: ['vue', 'vue-router', 'axios']
  }
};

// 构建过程:
// 1. 依赖预构建 (使用 esbuild, 速度极快)
// 2. Rollup 打包 (更好的 tree-shaking)
// 3. 输出优化后的代码

```

核心概念对比

Webpack 核心概念

```
JavaScript
// 1. 入口 (Entry)
entry: {
  main: './src/index.js',
  admin: './src/admin.js'
}

// 2. 输出 (Output)
output: {
  path: path.resolve(__dirname, 'dist'),
  filename: '[name].[contenthash].js',
  chunkFilename: '[name].[contenthash].chunk.js'
}

// 3. 加载器 (Loaders)
module: {
  rules: [
    {
      test: /\.jsx?$/,
      exclude: /node_modules/,
      use: {
        loader: 'babel-loader',
        options: { presets: ['@babel/preset-react'] }
      }
    }
  ]
}

// 4. 插件 (Plugins)
plugins: [
  new HtmlWebpackPlugin({
    template: './public/index.html'
  }),
  new MiniCssExtractPlugin(),
  new CleanWebpackPlugin()
]

// 5. 模式 (Mode)
mode: 'production' // 或 'development'

// 6. 代码分割 (Code Splitting)
optimization: {
  splitChunks: {
```

```
    chunks: 'all'
  },
  runtimeChunk: 'single'
}
```

Vite 核心概念

```
C++
// vite.config.js
export default {
  // 1. 根目录
  root: process.cwd(),

  // 2. 开发服务器
  server: {
    port: 3000,
    open: true,
    proxy: {
      '/api': 'http://localhost:8080'
    }
  },

  // 3. 构建配置
  build: {
    outDir: 'dist',
    assetsDir: 'assets',
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['vue', 'vue-router']
        }
      }
    }
  },

  // 4. 插件系统
  plugins: [
    vue(),
    vueJsx(),
    legacy({
      targets: ['defaults', 'not IE 11']
    })
  ],
```

```
// 5. 解析别名
resolve: {
  alias: {
    '@': path.resolve(__dirname, 'src')
  }
}
};
```

实际配置示例

Webpack 完整配置

```
JavaScript
// webpack.config.js
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');
const MiniCssExtractPlugin = require('mini-css-extract-plugin');

module.exports = (env, argv) => {
  const isProduction = argv.mode === 'production';

  return {
    entry: './src/index.js',

    output: {
      path: path.resolve(__dirname, 'dist'),
      filename: isProduction
        ? '[name].[contenthash].js'
        : '[name].js',
      clean: true
    },

    module: {
      rules: [
        {
          test: /\.jsx?$/,
          exclude: /node_modules/,
          use: 'babel-loader'
        },
        {
          test: /\.css$/,
          use: [
```

```

        isProduction ? MiniCssExtractPlugin.loader : 'style-
loader',
        'css-loader',
        'postcss-loader'
    ]
  },
  {
    test: /\. (png|jpg|gif|svg) $/,
    type: 'asset',
    parser: {
      dataUrlCondition: {
        maxSize: 10 * 1024 // 10KB 以下转为 base64
      }
    }
  }
]
},
plugins: [
  new HtmlWebpackPlugin({
    template: './public/index.html',
    favicon: './public/favicon.ico'
  }),
  ...(isProduction ? [new MiniCssExtractPlugin()] : [])
],

optimization: {
  splitChunks: {
    cacheGroups: {
      commons: {
        test: /[\\/]node_modules[\\/]$/,
        name: 'vendors',
        chunks: 'all'
      }
    }
  },
  minimizer: [
    '...', // 使用默认的 TerserPlugin
    new CssMinimizerPlugin()
  ]
},

devServer: {
  static: './dist',

```

```

    hot: true,
    historyApiFallback: true,
    compress: true
  },

  resolve: {
    extensions: ['.js', '.jsx', '.json'],
    alias: {
      '@': path.resolve(__dirname, 'src')
    }
  },

  devtool: isProduction ? 'source-map' : 'cheap-module-source-map'
};
};

```

Vite 完整配置

```

JavaScript
// vite.config.js
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';
import { createSvgIconsPlugin } from 'vite-plugin-svg-icons';
import path from 'path';

export default defineConfig(({ mode }) => {
  const isProduction = mode === 'production';

  return {
    root: '.',

    server: {
      port: 3000,
      open: true,
      proxy: {
        '/api': {
          target: 'http://localhost:8080',
          changeOrigin: true
        }
      }
    },

    build: {

```



```
    outDir: 'dist',
    assetsDir: 'assets',
    sourcemap: isProduction,
    minify: 'terser',
    rollupOptions: {
      output: {
        manualChunks: {
          vue: ['vue', 'vue-router', 'pinia'],
          ui: ['element-plus'],
          utils: ['axios', 'lodash-es']
        },
        chunkFileNames: 'js/[name]-[hash].js',
        entryFileNames: 'js/[name]-[hash].js',
        assetFileNames: 'assets/[name]-[hash].[ext]'
      }
    },
  },

  plugins: [
    vue(),
    createSvgIconsPlugin({
      iconDirs: [path.resolve(__dirname, 'src/assets/icons')],
      symbolId: 'icon-[dir]-[name]'
    })
  ],

  resolve: {
    alias: {
      '@': path.resolve(__dirname, 'src'),
      '#': path.resolve(__dirname, 'types')
    }
  },

  css: {
    preprocessorOptions: {
      scss: {
        additionalData: `@import "@/styles/variables.scss";`
      }
    },
    postcss: {
      plugins: [
        require('autoprefixer'),
        require('cssnano')({
          preset: 'default'
        })
      ]
    }
  }
}
```

```
    })
  ]
}
},

optimizeDeps: {
  include: ['vue', 'vue-router'],
  exclude: ['some-big-dependency']
}
};
});
```

现代构建工具生态

工具链选择指南

```
Java
// 项目类型与工具选择
const buildToolSelection = {
  // 小型项目/简单原型
  simple: {
    tools: ['Vite', 'Parcel'],
    reasons: ['快速启动', '零配置', '开发体验好']
  },

  // 大型企业应用
  enterprise: {
    tools: ['Webpack', 'Rollup'],
    reasons: ['成熟生态', '高度可配置', '复杂优化']
  },

  // 库/包开发
  library: {
    tools: ['Rollup', 'tsup', 'Microbundle'],
    reasons: ['Tree-shaking 好', '输出多种格式', '体积小']
  },

  // 全栈框架
  fullstack: {
    tools: ['Next.js (Webpack)', 'Nuxt.js (Vite/Webpack)'],
    reasons: ['一体化', '服务端渲染', '约定优于配置']
  }
}
```

```

};

// 性能对比
const performanceComparison = {
  startup: {
    webpack: '慢（需要构建整个依赖图）',
    vite: '快（基于原生 ESM）',
    snowpack: '快（类似 Vite）'
  },

  hmr: {
    webpack: '中等（需要重建模块图）',
    vite: '快（基于 ESM HMR）',
    snowpack: '快'
  },

  production: {
    webpack: '优秀（优化成熟）',
    vite: '良好（Rollup 构建）',
    snowpack: '良好'
  },

  bundleSize: {
    webpack: '优秀（高级优化）',
    vite: '优秀（Rollup tree-shaking）',
    rollup: '优秀（tree-shaking 先驱）'
  }
};

```

未来趋势

新兴工具和模式

```

JavaScript
// 1. 无打包开发（Bundle-less）
// 基于原生 ESM 的 CDN 服务
import { createApp } from 'https://unpkg.com/vue@3/dist/vue.esm-browser.js';

// 2. 构建工具融合
// Vite: 开发用 esbuild, 生产用 Rollup

```

```
// wmr: Preact 团队的轻量级工具

// 3. Rust 编写的工具
// - swc: 超快的 JavaScript/TypeScript 编译器
// - esbuild: Go 编写, 速度极快
// - Turbopack: Webpack 作者用 Rust 重写

// 4. 一体化框架
// Next.js, Nuxt.js, SvelteKit
// 内置构建、路由、渲染等全套解决方案
```

配置简化趋势

```
C++
// 现代工具的配置越来越简单
// Vite 默认配置
export default defineConfig({
  // 很多配置都有智能默认值
});

// Next.js 几乎零配置
// 基于文件系统的路由
// 内置 TypeScript 支持
// 自动代码分割

// 云原生构建
// 构建在云端进行
// 分布式缓存
// 增量构建
```

32. Promise 有哪些状态？如何捕获 Promise 中的错误？

Promise 的状态

Promise 对象有三种状态：

1. pending（等待中）：初始状态，既不是成功，也不是失败
2. fulfilled（已完成）：操作成功完成

3. rejected (已拒绝)：操作失败

状态转换图示

```
JavaScript
// Promise 状态转换
const promise = new Promise((resolve, reject) => {
  // 此时状态为 pending
  setTimeout(() => {
    // 状态变为 fulfilled
    resolve('成功');

    // 或者状态变为 rejected
    // reject(new Error('失败'));
  }, 1000);
});
```

状态特性

```
JavaScript
// 1. 状态不可逆
const p = new Promise((resolve, reject) => {
  resolve('第一次成功'); // 状态变为 fulfilled
  reject('第二次失败'); // 这个调用会被忽略
  resolve('第三次成功'); // 这个调用也会被忽略
});

// 2. 状态是私有的
// 无法从外部直接访问 Promise 的状态
console.log(p); // Promise {<fulfilled>: "第一次成功"}
```

捕获 Promise 错误的方法

方法 1：使用 catch() 方法

```
JavaScript
// 基础用法
fetch('/api/data')
  .then(response => response.json())
  .then(data => console.log(data))
```

```

    .catch(error => {
        console.error('请求失败:', error);
        // 可以在这里进行错误恢复或用户提示
    });

// catch() 会捕获所有前面的错误
promise
    .then(result => {
        throw new Error('then 中的错误'); // 这个错误会被 catch 捕获
    })
    .catch(error => {
        console.error('捕获到错误:', error);
        return '默认值'; // 可以返回一个恢复值
    })
    .then(value => {
        console.log('继续执行:', value); // "默认值"
    });

```

方法 2: then() 的第二个参数

```

JavaScript
// then() 的第二个参数专门处理错误
promise.then(
    value => {
        // 成功回调
        console.log('成功:', value);
    },
    error => {
        // 错误回调（只捕获这个 promise 的拒绝，不捕获 then 回调中的错误）
        console.error('失败:', error);
    }
);

// 注意：这种方法不会捕获 then 回调中抛出的错误
promise.then(
    value => {
        throw new Error('这个错误不会被第二个参数捕获');
    },
    error => {
        // 这里不会执行
        console.error('不会执行到这里');
    }
);

```



```
).catch(error => {  
  // 需要额外的 catch 来捕获 then 回调中的错误  
  console.error('这里会捕获:', error);  
});
```

方法 3: 使用 `async/await` 和 `try-catch`

```
JavaScript  
async function fetchData() {  
  try {  
    const response = await fetch('/api/data');  
    const data = await response.json();  
    console.log('数据:', data);  
  
    // 可以继续 await 其他 Promise  
    const result2 = await anotherAsyncOperation(data);  
    return result2;  
  } catch (error) {  
    // 捕获所有 await 表达式的错误  
    console.error('发生错误:', error);  
  
    // 可以抛出错误或返回恢复值  
    throw error; // 重新抛出  
    // 或 return '默认值'; // 返回恢复值  
  } finally {  
    // 无论成功失败都会执行  
    console.log('清理工作');  
  }  
}  
  
// 调用 async 函数返回的是 Promise  
fetchData()  
  .then(result => console.log('最终结果:', result))  
  .catch(error => console.error('最终错误:', error));
```

方法 4: `Promise.all()` 的错误处理

```
JavaScript  
// Promise.all() 遇到第一个错误就会立即拒绝  
const promises = [  
  fetch('/api/1'),
```

```

    fetch('/api/2'),
    fetch('/api/3')
  ];

  // 方法 4.1: 使用 catch
  Promise.all(promises)
    .then(responses => {
      // 所有 promise 都成功
    })
    .catch(error => {
      // 任何一个 promise 失败
      console.error('有一个请求失败:', error);
    });

  // 方法 4.2: 使用 Promise.allSettled() (ES2020)
  Promise.allSettled(promises)
    .then(results => {
      results.forEach((result, index) => {
        if (result.status === 'fulfilled') {
          console.log(`Promise ${index} 成功:`, result.value);
        } else {
          console.log(`Promise ${index} 失败:`, result.reason);
        }
      });
    });

  // 方法 4.3: 让每个 promise 都自己处理错误
  const safePromises = promises.map(p =>
    p.catch(error => {
      console.error('单个请求失败:', error);
      return null; // 返回默认值, 不让整个 Promise.all 失败
    })
  );

  Promise.all(safePromises)
    .then(results => {
      // 所有 promise 都完成了, 即使有失败的
      const validResults = results.filter(r => r !== null);
      console.log('有效结果:', validResults);
    });

```

方法 5: 全局错误处理

JavaScript

// 浏览器环境: unhandledrejection 事件

```
window.addEventListener('unhandledrejection', event => {
```

```
  // event.preventDefault() 可以阻止默认的错误输出
```

```
  console.warn('未处理的 Promise 拒绝:', event.reason);
```

```
  // 可以在这里进行错误上报
```

```
  reportError(event.reason);
```

```
  // 阻止浏览器控制台的默认错误输出
```

```
  event.preventDefault();
```

```
});
```

// Node.js 环境

```
process.on('unhandledRejection', (reason, promise) => {
```

```
  console.error('未处理的 Promise 拒绝:', reason);
```

```
  // 应用程序应该在这里退出或进行恢复
```

```
});
```

// 还可以监听 rejectionhandled 事件

```
window.addEventListener('rejectionhandled', event => {
```

```
  console.log('之前未处理的拒绝现在被处理了:', event.reason);
```

```
});
```

方法 6: 封装安全的 Promise 工具

JavaScript

// 工具函数: 确保 Promise 总是解决 (不拒绝)

```
function safePromise(promise, defaultValue = null) {
```

```
  return promise
```

```
    .then(value => ({ value, error: null }));
```

```
    .catch(error => ({ value: defaultValue, error }));
```

```
}
```

// 使用

```
const result = await safePromise(fetch('/api/data'));
```

```
if (result.error) {
```

```
  console.error('请求失败:', result.error);
```

```
  // 使用 defaultValue
```

```
} else {
```

```
  console.log('请求成功:', result.value);
```

```
}
```

```
// 工具函数：带重试的 Promise
function retryPromise(fn, retries = 3, delay = 1000) {
  return new Promise((resolve, reject) => {
    const attempt = (n) => {
      fn()
        .then(resolve)
        .catch(error => {
          if (n <= 0) {
            reject(error);
          } else {
            console.log(`重试 ${retries - n + 1}/${retries}`);
            setTimeout(() => attempt(n - 1), delay);
          }
        });
    };
    attempt(retries);
  });
}

// 使用重试
const data = await retryPromise(() => fetch('/api/data'), 3);
```

常见错误处理模式

模式 1：错误冒泡

```
JavaScript
async function processUserData(userId) {
  try {
    const user = await getUser(userId); // 可能失败
    const profile = await getProfile(user.id); // 可能失败
    const posts = await getPosts(user.id); // 可能失败
    return { user, profile, posts };
  } catch (error) {
    // 任何一个步骤失败都会到这里
    console.error('处理用户数据失败:', error);
    throw new Error(`无法处理用户 ${userId} 的数据`, { cause:
error });
  }
}
```

```
// 调用
processUserData(123)
  .then(data => console.log('成功:', data))
  .catch(error => console.error('最终失败:', error));
```

模式 2：并行处理与错误隔离

```
JavaScript
async function loadDashboard() {
  const [userResult, notificationsResult, statsResult] = await
  Promise.all([
    getUser().catch(error => ({ error, data: null })),
    getNotifications().catch(error => ({ error, data: [] })),
    getStats().catch(error => ({ error, data: {} }))
  ]);

  // 每个部分独立处理错误
  return {
    user: userResult.error ? '用户加载失败' : userResult.data,
    notifications: notificationsResult.data, // 失败时是 []
    stats: statsResult.error ? {} : statsResult.data
  };
}
```

模式 3：超时处理

```
JavaScript
function timeoutPromise(promise, timeoutMs, timeoutMessage = '操作
超时') {
  let timeoutId;

  const timeoutPromise = new Promise((_, reject) => {
    timeoutId = setTimeout(() => {
      reject(new Error(timeoutMessage));
    }, timeoutMs);
  });

  return Promise.race([promise, timeoutPromise])
    .finally(() => clearTimeout(timeoutId));
}

// 使用
```

```
try {
  const data = await timeoutPromise(fetch('/api/slow'), 5000);
  console.log('数据:', data);
} catch (error) {
  if (error.message === '操作超时') {
    console.error('请求超时');
    // 显示重试按钮
  } else {
    console.error('其他错误:', error);
  }
}
```

最佳实践

JavaScript

// 1. 总是返回错误对象，而不是字符串

// ✖ 不好

```
reject('发生错误');
```

// ✔ 好

```
reject(new Error('发生错误'));
```

// 2. 使用 Error 的子类提供更多上下文

```
class ApiError extends Error {
  constructor(message, statusCode, originalError) {
    super(message);
    this.name = 'ApiError';
    this.statusCode = statusCode;
    this.originalError = originalError;
  }
}
```

// 3. 在 async 函数中明确处理错误

```
async function criticalOperation() {
  try {
    const result = await doSomething();
    return result;
  } catch (error) {
    // 记录错误
    logger.error('关键操作失败', error);

    // 执行恢复操作
  }
}
```



```

    await cleanup();

    // 重新抛出或返回安全值
    throw error; // 或 return safeDefault;
  }
}

// 4. 避免 Promise 地狱
// ✖ 不好
getUser()
  .then(user => {
    getProfile(user.id)
      .then(profile => {
        getPosts(user.id)
          .then(posts => {
            // 嵌套太深
          });
      });
  });

// ✔ 好: 使用 async/await
async function getUserData() {
  const user = await getUser();
  const [profile, posts] = await Promise.all([
    getProfile(user.id),
    getPosts(user.id)
  ]);
  return { user, profile, posts };
}

// 5. 始终添加 catch 处理
// 即使是简单的脚本
fetch('/api/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(error => {
    console.error('发生错误:', error);
    process.exit(1); // 非正常退出
  });

```

33. 请解释什么是 XSS (跨站脚本攻击) 和 CSRF (跨站

请求伪造)，以及如何防范它们。

XSS (跨站脚本攻击)

什么是 XSS?

XSS 攻击是指攻击者向网页中注入恶意脚本，当其他用户访问该网页时，恶意脚本会在用户的浏览器中执行。

XSS 类型

1. 反射型 XSS

```
JavaScript
// 攻击方式：用户点击恶意链接
// 恶意 URL：
http://example.com/search?q=<script>alert('XSS')</script>

// 有漏洞的代码
const searchQuery = req.query.q; // 直接使用用户输入
res.send(`<h1>搜索结果：${searchQuery}</h1>`);

// 防御前：用户输入会直接执行
// 防御后：需要对用户输入进行转义
```

2. 存储型 XSS

```
JavaScript
// 攻击方式：恶意脚本存储在服务器（如数据库）
// 有漏洞的评论系统
const comment = req.body.comment; // 用户评论：
<script>stealCookie()</script>
db.saveComment(comment); // 存储到数据库

// 其他用户查看评论时，脚本执行
const comments = db.getComments();
res.send(`<div>${comments}</div>`); // 恶意脚本被执行
```

3. DOM 型 XSS

```
JavaScript
// 攻击方式：恶意脚本通过修改 DOM 执行
```

```
// 有漏洞的前端代码
const input = document.getElementById('search').value;
document.getElementById('result').innerHTML = input; // 直接设置 innerHTML

// 用户输入: 
// 脚本会被执行
```

XSS 防御措施

1. 输入验证和过滤

```
JavaScript
// 白名单过滤: 只允许特定的 HTML 标签和属性
function sanitizeHTML(input) {
  const allowedTags = ['b', 'i', 'em', 'strong', 'a'];
  const allowedAttributes = ['href', 'title', 'target'];

  // 使用 DOMPurify 等库
  return DOMPurify.sanitize(input, {
    ALLOWED_TAGS: allowedTags,
    ALLOWED_ATTR: allowedAttributes
  });
}

// 在服务器端验证
const validator = require('validator');
const cleanInput = validator.escape(userInput); // 转义特殊字符
```

2. 输出编码

```
JavaScript
// 根据输出位置进行不同的编码
function encodeForHTML(text) {
  return text
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#x27;');
}

// 现代框架自动编码
```

```
// React 自动转义
function Component({ userInput }) {
  return <div>{userInput}</div>; // 自动转义
}

// Vue 自动转义
<template>
  <div>{{ userInput }}</div> <!-- 自动转义 -->
</template>
```

3. 内容安全策略 (CSP)

```
XML
<!-- 在 HTTP 头中设置 -->
<!-- Content-Security-Policy: default-src 'self'; script-src
'self' https://trusted.cdn.com -->

<!-- 或 meta 标签 -->
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self';
              script-src 'self' 'unsafe-inline' 'unsafe-eval';
              style-src 'self' 'unsafe-inline';
              img-src 'self' data: https:">
```

4. HttpOnly Cookie

```
Java
// 设置 Cookie 时启用 HttpOnly
res.cookie('sessionId', 'abc123', {
  httpOnly: true, // JavaScript 无法访问
  secure: true,   // 仅 HTTPS
  sameSite: 'strict'
});

// 这样 JavaScript 就不能通过 document.cookie 访问
```

5. 其他防御措施

```
XML
// 使用安全的 API
// ✗ 不安全
element.innerHTML = userInput;
```

```
// ✔ 安全
element.textContent = userInput;
element.setAttribute('data-value', userInput);

// 使用模板引擎的安全配置
// EJS
<%- userInput %> <!-- 转义输出 -->
<%= userInput %> <!-- 非转义输出（危险） -->

// 设置正确的 Content-Type
res.setHeader('Content-Type', 'application/json');
// 避免将用户输入作为 JavaScript 执行
```

CSRF (跨站请求伪造)

什么是 CSRF?

CSRF 攻击诱使用户在已登录的网站上执行非本意的操作。

CSRF 攻击示例

```
XML
<!-- 恶意网站中的代码 -->


<!-- 或使用表单 -->
<form action="http://bank.com/transfer" method="POST" id="csrf-
form">
  <input type="hidden" name="to" value="attacker">
  <input type="hidden" name="amount" value="10000">
</form>
<script>
  document.getElementById('csrf-form').submit();
</script>
```

CSRF 防御措施

1. CSRF Token

```
JavaScript
// 服务器生成 Token
```

```
const csrfToken = crypto.randomBytes(32).toString('hex');
req.session.csrfToken = csrfToken;

// 发送给客户端
res.render('form', { csrfToken });

// 在表单中包含 Token
// <input type="hidden" name="_csrf" value="<%= csrfToken %>">

// 验证 Token
app.post('/transfer', (req, res) => {
  const clientToken = req.body._csrf;
  const serverToken = req.session.csrfToken;

  if (!clientToken || clientToken !== serverToken) {
    return res.status(403).send('CSRF 验证失败');
  }

  // 处理请求
});
```

2. SameSite Cookie

```
Java
// 设置 Cookie 时
res.cookie('sessionId', 'abc123', {
  sameSite: 'strict', // 或 'lax'
  httpOnly: true,
  secure: true
});

/*
SameSite 选项:
- strict: 完全禁止跨站携带 Cookie
- lax: 允许部分跨站请求（如导航）携带 Cookie
- none: 允许所有跨站请求（需配合 secure）
*/
```

3. 验证 Referer/Origin 头

```
JavaScript
app.use((req, res, next) => {
  const referer = req.get('Referer');
```



```
const origin = req.get('Origin');

// 验证请求来源
const allowedOrigins = ['https://example.com',
'https://www.example.com'];

if (req.method !== 'GET' && req.method !== 'HEAD') {
  if (origin && !allowedOrigins.includes(origin)) {
    return res.status(403).send('非法来源');
  }

  // 或者验证 Referer
  if (referer && !referer.startsWith('https://example.com/')) {
    return res.status(403).send('非法来源');
  }
}

next();
});
```

4. 双重提交 Cookie

```
JavaScript
// 客户端从 Cookie 中读取 Token
function getCSRFToken() {
  const name = 'csrfToken=';
  const cookies = document.cookie.split(';');

  for (let cookie of cookies) {
    cookie = cookie.trim();
    if (cookie.startsWith(name)) {
      return cookie.substring(name.length);
    }
  }
  return '';
}

// 将 Token 添加到请求头
fetch('/api/transfer', {
  method: 'POST',
  headers: {
    'X-CSRF-Token': getCSRFToken(),
    'Content-Type': 'application/json'
  },
```

```

    body: JSON.stringify(data)
  });

  // 服务器验证 Cookie 和 Header 中的 Token 是否一致
  app.post('/api/transfer', (req, res) => {
    const cookieToken = req.cookies.csrfToken;
    const headerToken = req.get('X-CSRF-Token');

    if (!cookieToken || cookieToken !== headerToken) {
      return res.status(403).send('CSRF 验证失败');
    }
  });

```

5. 自定义请求头

```

JavaScript
// 前端设置自定义请求头
axios.defaults.headers.common['X-Requested-With'] =
'XMLHttpRequest';

// 服务器验证
app.use((req, res, next) => {
  if (['POST', 'PUT', 'DELETE', 'PATCH'].includes(req.method)) {
    const requestedWith = req.get('X-Requested-With');

    if (requestedWith !== 'XMLHttpRequest') {
      return res.status(403).send('非法请求');
    }
  }
  next();
});

```

综合防御策略

后端防御层

```

JavaScript
// Express 中间件示例
const csrfProtection = require('csurf');
const helmet = require('helmet');

app.use(helmet({

```

```

    contentSecurityPolicy: {
      directives: {
        defaultSrc: ['"self"'],
        scriptSrc: ['"self"', "'unsafe-inline'"],
        styleSrc: ['"self"', "'unsafe-inline'"]
      }
    }
  }));

app.use(csrfProtection({ cookie: true }));

// 在所有路由中应用 CSRF 保护
app.use((req, res, next) => {
  res.locals.csrfToken = req.csrfToken();
  next();
});

// API 路由单独处理
app.use('/api', (req, res, next) => {
  // 验证 Origin/Referer
  const origin = req.get('Origin');
  if (origin && !origin.includes('example.com')) {
    return res.status(403).json({ error: '非法请求' });
  }
  next();
});

```

前端安全实践

```

JavaScript
// React 安全组件
import React from 'react';
import DOMPurify from 'dompurify';

const SafeHTML = ({ html, ...props }) => {
  const cleanHTML = DOMPurify.sanitize(html, {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'p', 'br'],
    ALLOWED_ATTR: []
  });

  return <div {...props} dangerouslySetInnerHTML={{ __html:
cleanHTML }} />;
};

```

```

// 使用
<SafeHTML html={userContent} />

// 安全的 HTTP 客户端
const apiClient = axios.create({
  baseURL: '/api',
  timeout: 10000,
  headers: {
    'X-Requested-With': 'XMLHttpRequest'
  }
});

// 自动添加 CSRF Token
apiClient.interceptors.request.use(config => {
  const token = document.querySelector('meta[name="csrf-token"]')?.content;
  if (token) {
    config.headers['X-CSRF-Token'] = token;
  }
  return config;
});

```

安全检测工具

```

Java
// 1. 使用安全相关的 npm 包
const dependencies = {
  security: [
    'helmet',           // 安全 HTTP 头
    'csurf',            // CSRF 保护
    'xss-clean',        // XSS 防护
    'express-rate-limit', // 限流
    'validator'         // 输入验证
  ]
};

// 2. 代码扫描工具
// - ESLint 安全规则
// - SonarQube
// - Snyk

```

```
// 3. 自动化安全测试
// - OWASP ZAP
// - Burp Suite
```

34. ES6 中的箭头函数和普通函数有什么主要区别？

主要区别总结

特性	箭头函数	普通函数
this 绑定	继承父作用域的 this	有自己的 this（动态绑定）
arguments 对象	没有	有
constructor	不能作为构造函数	可以作为构造函数
prototype	没有	有
super	没有	有（在类方法中）
new.target	没有	有
yield	不能用作生成器	可以用作生成器
重复命名参数	不允许	严格模式下不允许

点击图片可查看完整电子表格

详细对比

34.1.1.1 this 绑定的区别

```
JavaScript
// 普通函数的 this
const obj1 = {
  name: '普通函数',
  sayName: function() {
    console.log(this.name); // this 指向 obj1
  }
};

obj1.sayName(); // "普通函数"

// 箭头函数的 this
const obj2 = {
  name: '箭头函数',
  sayName: () => {
```

```

    console.log(this.name); // this 继承自父作用域（此处是
window/global）
  }
};

obj2.sayName(); // undefined（浏览器中）或 空字符串（Node.js）

// 实际场景：事件处理
const button = document.getElementById('btn');

// ✕ 普通函数的问题
button.addEventListener('click', function() {
  console.log(this); // 指向 button
  setTimeout(function() {
    console.log(this); // 指向 window（不是 button！）
    this.style.color = 'red'; // TypeError
  }, 100);
});

// ✔ 箭头函数的解决方案
button.addEventListener('click', function() {
  console.log(this); // 指向 button
  setTimeout(() => {
    console.log(this); // 继承父作用域的 this，指向 button
    this.style.color = 'red'; // 正常工作
  }, 100);
});

```

34.1.1.2 构造函数和 prototype

```

JavaScript
// 普通函数可以作为构造函数
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function() {
  console.log(`Hello, ${this.name}`);
};

const person = new Person('Alice'); // 可以
person.sayHello(); // "Hello, Alice"

```



```
// 箭头函数不能作为构造函数
const Animal = (name) => {
  this.name = name; // TypeError: 箭头函数没有 this
};

const animal = new Animal('Dog'); // TypeError: Animal is not a
constructor

// 箭头函数没有 prototype
console.log(Person.prototype); // {sayHello: f, constructor: f}
console.log(Animal.prototype); // undefined
```

34.1.1.3 arguments 对象

```
JavaScript
// 普通函数有 arguments 对象
function sum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}

console.log(sum(1, 2, 3, 4)); // 10

// 箭头函数没有 arguments 对象
const sumArrow = () => {
  console.log(arguments); // ReferenceError: arguments is not
defined
};

// 使用 rest 参数替代
const sumArrow2 = (...args) => {
  return args.reduce((total, num) => total + num, 0);
};

console.log(sumArrow2(1, 2, 3, 4)); // 10
```

34.1.1.4 方法定义中的区别

```

JavaScript
class Calculator {
  constructor() {
    this.value = 0;
  }

  // 普通函数方法
  add1(num) {
    this.value += num;
    return this;
  }

  // 箭头函数方法（实例属性）
  add2 = (num) => {
    this.value += num; // this 绑定正确
    return this;
  }

  // 延迟执行的场景
  delayedAdd1(num) {
    setTimeout(function() {
      this.value += num; // this 丢失，指向 window
    }, 100);
  }

  delayedAdd2(num) {
    setTimeout(() => {
      this.value += num; // this 正确指向实例
    }, 100);
  }
}

const calc = new Calculator();
calc.add1(5); // 正常
calc.delayedAdd1(5); // 错误，this 丢失
calc.delayedAdd2(5); // 正常

```

使用场景建议

适合使用箭头函数的场景

JavaScript

```
// 1. 回调函数
const numbers = [1, 2, 3, 4, 5];

// ✕ 普通函数（需要额外处理 this）
const doubled1 = numbers.map(function(num) {
  return num * 2;
});

// ✔ 箭头函数（简洁）
const doubled2 = numbers.map(num => num * 2);

// 2. 需要继承外层 this 的场景
class Timer {
  constructor() {
    this.seconds = 0;
    // 箭头函数正确绑定 this
    this.timer = setInterval(() => {
      this.seconds++;
      console.log(this.seconds);
    }, 1000);
  }
}

// 3. 立即执行函数
const result = (() => {
  const a = 1;
  const b = 2;
  return a + b;
})();

// 4. 返回对象的简洁写法
const createUser = (name, age) => ({ name, age });

// 5. 函数式编程
const operations = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
  multiply: (a, b) => a * b
};
```

适合使用普通函数的场景

JavaScript

```
// 1. 构造函数
function User(name, email) {
  this.name = name;
  this.email = email;
}

User.prototype.sendEmail = function(message) {
  console.log(`Sending email to ${this.email}: ${message}`);
};

// 2. 需要 arguments 对象的函数
function max() {
  return Math.max.apply(null, arguments);
}

// 3. 对象方法（需要访问对象本身）
const obj = {
  items: [],
  addItem(item) { // 普通函数方法
    this.items.push(item);
    return this;
  },
  // 或者
  removeItem: function(item) {
    const index = this.items.indexOf(item);
    if (index > -1) {
      this.items.splice(index, 1);
    }
    return this;
  }
};

// 4. 生成器函数
function* numberGenerator() {
  let i = 0;
  while (true) {
    yield i++;
  }
}

// 5. 递归函数（需要函数名）
const factorial = function(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
};
```

```
};
```

特殊注意事项

34.1.1.1 箭头函数不能绑定 this

```
C++
// 无法通过 call、apply、bind 改变箭头函数的 this
const obj1 = { value: 1 };
const obj2 = { value: 2 };

const arrowFunc = () => console.log(this.value);

// 这些调用都无法改变 this
arrowFunc.call(obj1); // undefined (this 仍然是父作用域的 this)
arrowFunc.apply(obj2); // undefined
const bound = arrowFunc.bind(obj1);
bound(); // undefined
```

34.1.1.2 多层箭头函数的 this

```
JavaScript
const obj = {
  name: 'outer',
  method: function() {
    return () => {
      console.log(this.name); // 'outer'
      return () => {
        console.log(this.name); // 仍然是 'outer' (继承自上一层箭头函数)
      };
    };
  }
};

const inner = obj.method();
const innermost = inner();
innermost(); // 打印两次 'outer'
```

34.1.1.3 箭头函数和普通函数混用

```
JavaScript
// 在类中混用的例子
class MixedExample {
  constructor() {
    this.count = 0;
  }

  // 箭头函数：绑定到实例
  increment = () => {
    this.count++;
  }

  // 普通函数：在原型上
  decrement() {
    this.count--;
  }

  // 静态方法（只能是普通函数）
  static helper() {
    return '帮助函数';
  }
}

const instance = new MixedExample();
console.log(instance.increment); // 箭头函数，在实例上
console.log(instance.decrement); // 普通函数，在原型上
console.log(MixedExample.helper()); // 静态方法
```

性能考虑

```
JavaScript
// 性能测试
function testPerformance() {
  const iterations = 1000000;

  // 普通函数
  console.time('普通函数');
  function regularAdd(a, b) {
    return a + b;
  }
  for (let i = 0; i < iterations; i++) {
    regularAdd(i, i + 1);
  }
}
```



```

}
console.timeEnd('普通函数');

// 箭头函数
console.time('箭头函数');
const arrowAdd = (a, b) => a + b;
for (let i = 0; i < iterations; i++) {
  arrowAdd(i, i + 1);
}
console.timeEnd('箭头函数');

// 方法调用
const obj = {
  regularMethod: function() { return this.value; },
  arrowMethod: () => this.value,
  value: 1
};

console.time('普通方法');
for (let i = 0; i < iterations; i++) {
  obj.regularMethod();
}
console.timeEnd('普通方法');

console.time('箭头方法');
for (let i = 0; i < iterations; i++) {
  obj.arrowMethod();
}
console.timeEnd('箭头方法');
}

/*
现代 JavaScript 引擎中，性能差异很小
但在某些情况下：
1. 箭头函数创建速度稍快
2. 普通方法调用稍快（原型链查找优化）
实际选择应基于语义而非性能
*/

```

35. CSS 选择器的优先级是如何计算的？

优先级计算规则

CSS 选择器的优先级由四个级别组成，通常表示为 (a, b, c, d):

1. 内联样式 (style 属性) : a = 1
2. ID 选择器: b = ID 选择器的数量
3. 类、伪类、属性选择器: c = 类选择器、伪类选择器、属性选择器的数量
4. 元素、伪元素选择器: d = 元素选择器、伪元素选择器的数量

计算示例

```
C++
/* 示例 1: (0,0,0,1) */
div { color: red; }

/* 示例 2: (0,0,1,0) */
.container { color: blue; }

/* 示例 3: (0,1,0,0) */
#header { color: green; }

/* 示例 4: (0,0,1,1) */
div.container { color: yellow; }

/* 示例 5: (0,1,0,1) */
div#header { color: purple; }

/* 示例 6: (0,1,1,3) */
body div#header ul li.active { color: orange; }

/* 示例 7: (0,0,2,2) */
ul.menu li.item a { color: pink; }
```

优先级比较规则

```
JavaScript
// 优先级比较函数
function compareSpecificity(selector1, selector2) {
  // 将选择器转换为 (a, b, c, d) 格式
  const spec1 = calculateSpecificity(selector1);
  const spec2 = calculateSpecificity(selector2);
```

```
// 逐级比较
if (spec1.a !== spec2.a) return spec1.a > spec2.a;
if (spec1.b !== spec2.b) return spec1.b > spec2.b;
if (spec1.c !== spec2.c) return spec1.c > spec2.c;
if (spec1.d !== spec2.d) return spec1.d > spec2.d;

// 如果优先级完全相同，后定义的样式胜出
return false;
}
```

特殊选择器的优先级

35.1.1.1 通配符选择器

```
JavaScript
/* 优先级: (0,0,0,0) */
* { margin: 0; }
```

35.1.1.2 组合选择器

```
C++
/* 后代选择器: 计算所有部分 */
/* (0,1,1,3) = id + class + 3 个元素 */
#nav .menu li a:hover { }

/* 子选择器: 同样计算所有部分 */
/* (0,0,2,2) */
ul > li > a.active { }

/* 相邻兄弟选择器: 计算所有部分 */
/* (0,0,1,2) */
h1 + p.intro { }

/* 通用兄弟选择器: 计算所有部分 */
/* (0,0,1,2) */
h1 ~ p.intro { }
```

35.1.1.3 伪类和伪元素

```
JavaScript
```

```

/* :hover, :focus 等伪类参与 c 的计算 */
/* (0,0,2,1) */
a:hover:focus { }

/* ::before, ::after 等伪元素参与 d 的计算 */
/* (0,0,1,2) */
p::before { }

/* :not() 伪类本身不增加优先级，但内部的选择器参与计算 */
/* :not(.ignore) 只计算 .ignore，所以是 (0,0,1,0) */
div:not(.ignore) { }

```

35.1.1.4 属性选择器

```

Go
/* 属性选择器参与 c 的计算 */
/* (0,0,2,1) */
input[type="text"].required { }

/* 属性存在选择器 */
/* (0,0,1,1) */
a[target] { }

/* 属性值匹配选择器 */
/* (0,0,1,1) */
input[name^="user_"] { }

```

实际应用示例

```

C++
/* 样式表 A */
#content div.box { color: red; } /* (0,1,1,1) */

/* 样式表 B（后来加载） */
div#content .box { color: blue; } /* (0,1,1,1) - 相同优先级，后定义的胜出 */

/* 内联样式总是最高 */
<div style="color: green;">文本</div> /* (1,0,0,0) */

/* !important 的例外情况 */

```

```
.special { color: orange !important; } /* 无视优先级，最高 */
```

优先级计算工具函数

JavaScript

// 计算选择器优先级的函数

```
function calculateSpecificity(selector) {
```

```
  let a = 0; // 内联样式
```

```
  let b = 0; // ID 选择器
```

```
  let c = 0; // 类、伪类、属性选择器
```

```
  let d = 0; // 元素、伪元素选择器
```

// 简单解析器（实际实现更复杂）

```
const regex = {
```

```
  id: /#([\w-]+)/g,
```

```
  class: /\.[\w-]+)/g,
```

```
  attribute: /\[[^\]]+\]/g,
```

```
  pseudoClass: /:([\w-]+)(?:\[^\]]*\)/g,
```

```
  pseudoElement: /::([\w-]+)/g,
```

```
  element: /^(?:[\s>+~])([\w-]+)(?:\.[\w-]+)/g
```

```
};
```

// 统计 ID 选择器

```
const idMatches = selector.match(regex.id);
```

```
b = idMatches ? idMatches.length : 0;
```

// 统计类选择器

```
const classMatches = selector.match(regex.class);
```

```
const classCount = classMatches ? classMatches.length : 0;
```

// 统计属性选择器

```
const attrMatches = selector.match(regex.attribute);
```

```
const attrCount = attrMatches ? attrMatches.length : 0;
```

// 统计伪类（排除伪元素）

```
const pseudoClassMatches = selector.match(regex.pseudoClass);
```

```
const pseudoClassCount = pseudoClassMatches ?
```

```
pseudoClassMatches.length : 0;
```

// 计算 c

```
c = classCount + attrCount + pseudoClassCount;
```

```

// 统计伪元素
const pseudoElementMatches =
selector.match(regex.pseudoElement);
const pseudoElementCount = pseudoElementMatches ?
pseudoElementMatches.length : 0;

// 统计元素选择器
const elementMatches = selector.match(regex.element);
const elementCount = elementMatches ? elementMatches.length : 0;

// 计算 d
d = elementCount + pseudoElementCount;

return { a, b, c, d };
}

// 使用示例
console.log(calculateSpecificity('#nav .menu li a:hover'));
// { a: 0, b: 1, c: 2, d: 3 }

```

最佳实践和避免优先级问题

35.1.1.1 使用低优先级

```

JavaScript
/* 避免过度使用高优先级选择器 */
/* ✗ 不好：优先级过高，难以覆盖 */
#header #nav #menu li.active a#current { }

/* ✓ 好：使用合适的优先级 */
.nav-menu .active > a { }

/* ✓ 更好：使用 BEM 命名约定 */
.nav-menu__item--active .nav-menu__link { }

```

35.1.1.2 避免 !important

```

YAML
/* ✗ 滥用 !important */
.btn { color: red !important; }
.btn-primary { color: blue !important; } /* 需要更重要的 !important

```



```
*/

/* ✔ 合理使用 !important */
/* 只在真正需要覆盖第三方样式时使用 */
.third-party-widget .btn {
  /* 第三方样式优先级很高 */
  color: green !important; /* 不得已使用 */
}
```

35.1.1.3 使用 CSS 自定义属性

```
JavaScript
/* 利用 CSS 变量降低优先级需求 */
:root {
  --primary-color: #007bff;
  --button-color: var(--primary-color);
}

.btn {
  color: var(--button-color); /* 易于覆盖 */
}

.btn.special {
  --button-color: #ff0000; /* 轻松覆盖 */
}
```

35.1.1.4 合理组织 CSS

```
JavaScript
/* 基础样式（低优先级） */
.button { padding: 8px 16px; }

/* 修饰符样式（中等优先级） */
.button.primary { background: blue; }

/* 状态样式（稍高优先级） */
.button.primary:hover { background: darkblue; }

/* 覆盖样式（最高，放在最后） */
.container .button.primary { background: green; }
```

调试优先级问题

```
SQL
/* 调试技巧 */

/* 1. 使用特异性标识 */
#header { /* specificity: 0100 */ }
.nav { /* specificity: 0010 */ }
div { /* specificity: 0001 */ }

/* 2. Chrome DevTools 显示优先级 */
/*
在 Elements 面板中，查看 Styles 标签
每个样式旁边会显示优先级，如：
(0, 1, 0, 0)
(0, 0, 2, 1)
*/

/* 3. 使用 CSS @layer 控制优先级（CSS Cascading Layers） */
/* 定义图层顺序 */
@layer base, components, utilities;

/* 基础层（优先级最低） */
@layer base {
  .btn { color: black; }
}

/* 组件层 */
@layer components {
  .btn { color: blue; } /* 即使后定义，优先级也低于 utilities */
}

/* 工具层（优先级最高） */
@layer utilities {
  .btn { color: red; }
}
```

36. 移动端 H5 页面进行适配时，有哪些常见的方案？

36.1.1 视口设置（基础配置）

XML

```
<!-- 标准视口配置 -->
<meta name="viewport"
      content="width=device-width,
              initial-scale=1.0,
              maximum-scale=1.0,
              minimum-scale=1.0,
              user-scalable=no,
              viewport-fit=cover">

<!-- 针对不同设备的优化 -->
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="format-detection" content="telephone=no, email=no">
```

36.1.2 媒体查询（Media Queries）

Python

```
/* 基于设备宽度的断点 */
/* 超小屏幕（手机，竖屏） */
@media (max-width: 575.98px) {
  .container { padding: 0 15px; }
}

/* 小屏幕（手机，横屏） */
@media (min-width: 576px) and (max-width: 767.98px) {
  .container { max-width: 540px; }
}

/* 中等屏幕（平板） */
@media (min-width: 768px) and (max-width: 991.98px) {
  .container { max-width: 720px; }
}

/* 大屏幕（平板横屏/小桌面） */
@media (min-width: 992px) and (max-width: 1199.98px) {
  .container { max-width: 960px; }
}
```

```

/* 特大屏幕（桌面） */
@media (min-width: 1200px) {
  .container { max-width: 1140px; }
}

/* 基于设备特性的查询 */
/* 深色模式 */
@media (prefers-color-scheme: dark) {
  body { background: #121212; color: #fff; }
}

/* 减少动画（用户偏好） */
@media (prefers-reduced-motion: reduce) {
  * { animation-duration: 0.01ms !important; }
}

/* 高对比度模式 */
@media (prefers-contrast: high) {
  .btn { border: 2px solid currentColor; }
}

```

36.1.3 弹性布局（Flexbox）

```

YAML
/* 常用 Flex 布局模式 */
.container {
  display: flex;
  flex-direction: column; /* 移动端常用列布局 */
  min-height: 100vh; /* 视口高度 */
}

.header, .footer {
  flex: 0 0 auto; /* 固定高度 */
}

.main {
  flex: 1 1 auto; /* 占据剩余空间 */
  overflow-y: auto;
}

/* 响应式导航 */
.nav {

```

```

display: flex;
flex-wrap: wrap; /* 允许换行 */
justify-content: space-around;
}

.nav-item {
  flex: 1 0 100px; /* 最小宽度 100px, 可伸缩 */
  max-width: 200px;
}

/* 移动端卡片布局 */
.card-list {
  display: flex;
  flex-direction: column;
  gap: 16px; /* 卡片间距 */
}

@media (min-width: 768px) {
  .card-list {
    flex-direction: row;
    flex-wrap: wrap;
  }

  .card {
    flex: 1 0 calc(50% - 16px); /* 两列布局 */
  }
}

@media (min-width: 1024px) {
  .card {
    flex: 1 0 calc(33.333% - 16px); /* 三列布局 */
  }
}

```

36.1.4 网格布局 (Grid)

```

C++
/* 响应式网格 */
.grid-container {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
  padding: 20px;
}

```

```
}

/* 移动端优先的网格 */
.product-grid {
  display: grid;
  grid-template-columns: 1fr; /* 默认单列 */
  gap: 16px;
}

@media (min-width: 640px) {
  .product-grid {
    grid-template-columns: repeat(2, 1fr); /* 平板: 两列 */
  }
}

@media (min-width: 1024px) {
  .product-grid {
    grid-template-columns: repeat(4, 1fr); /* 桌面: 四列 */
  }
}

/* 复杂网格布局 */
.dashboard {
  display: grid;
  grid-template-areas:
    "header"
    "sidebar"
    "main"
    "footer";
  grid-template-rows: auto auto 1fr auto;
  min-height: 100vh;
}

@media (min-width: 768px) {
  .dashboard {
    grid-template-areas:
      "header header"
      "sidebar main"
      "footer footer";
    grid-template-columns: 250px 1fr;
    grid-template-rows: auto 1fr auto;
  }
}
```


36.1.5 相对单位和响应式 typography

```
SQL
/* 根元素字体设置 */
html {
  font-size: 16px; /* 基准字体 */
}

/* 使用 rem 单位 */
body {
  font-size: 1rem; /* 16px */
  line-height: 1.5;
}

h1 {
  font-size: 2rem; /* 32px */
  margin-bottom: 1rem; /* 16px */
}

/* 使用视口单位进行响应式 typography */
.title {
  font-size: calc(1.5rem + 2vw); /* 动态字体大小 */
  line-height: calc(1.3em + 0.5vw);
}

/* 响应式间距 */
.section {
  padding: 2rem 1rem;
}

@media (min-width: 768px) {
  .section {
    padding: 3rem 2rem;
  }
}

/* 使用 clamp() 函数限制范围 */
.heading {
  font-size: clamp(1.5rem, 4vw, 3rem); /* 最小 1.5rem, 最大 3rem */
}
```

36.1.6 基于 JavaScript 的动态适配方案

JavaScript

// 动态设置根字体大小（rem 适配）

```
(function setRootFontSize() {
```

```
  const designWidth = 750; // 设计稿宽度
```

```
  const baseFontSize = 100; // 1rem = 100px
```

```
  function refreshRem() {
```

```
    const width = document.documentElement.clientWidth;
```

```
    const fontSize = (width / designWidth) * baseFontSize;
```

```
    document.documentElement.style.fontSize = `${fontSize}px`;
```

```
  }
```

// 初始设置

```
refreshRem();
```

// 防抖处理

```
let timeoutId;
```

```
window.addEventListener('resize', () => {
```

```
  clearTimeout(timeoutId);
```

```
  timeoutId = setTimeout(refreshRem, 300);
```

```
});
```

```
})();
```

// VW 适配方案（不需要 JavaScript）

// 使用 postcss-px-to-viewport 插件自动转换

/*

PostCSS 配置：

```
{
```

```
  'postcss-px-to-viewport': {
```

```
    viewportWidth: 750,
```

```
    viewportHeight: 1334,
```

```
    unitPrecision: 5,
```

```
    viewportUnit: 'vw',
```

```
    selectorBlackList: [],
```

```
    minPixelValue: 1,
```

```
    mediaQuery: false
```

```
  }
```

```
}
```

```
*/
```

36.1.7 图片和媒体适配

XML

```
<!-- 响应式图片 -->


<!-- picture 元素 -->
<picture>
  <!-- 小屏幕：方形裁剪 -->
  <source media="(max-width: 767px)"
          srcset="image-square-400.jpg 1x,
                  image-square-800.jpg 2x">

  <!-- 大屏幕：宽屏裁剪 -->
  <source media="(min-width: 768px)"
          srcset="image-wide-800.jpg 1x,
                  image-wide-1600.jpg 2x">

  <!-- 回退 -->
  
</picture>

<!-- 背景图片适配 -->
<div class="hero"
      style="background-image: url('image-mobile.jpg');">
</div>

<style>
.hero {
  background-size: cover;
  background-position: center;
  aspect-ratio: 16/9; /* 保持比例 */
}

@media (min-width: 768px) {
  .hero {
    background-image: url('image-desktop.jpg');
```

```
    }  
  }  
</style>
```

36.1.8 移动端特有优化

1px 边框问题

YAML

```
/* 解决移动端 1px 边框变粗的问题 */  
.border-1px {  
  position: relative;  
}  
  
.border-1px::after {  
  content: '';  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 200%;  
  height: 200%;  
  border: 1px solid #ddd;  
  transform: scale(0.5);  
  transform-origin: 0 0;  
  box-sizing: border-box;  
  pointer-events: none;  
}  
  
/* 或者使用媒体查询和 viewport 的 devicePixelRatio */  
@media (-webkit-min-device-pixel-ratio: 2), (min-resolution:  
2dppx) {  
  .border-thin {  
    border-width: 0.5px;  
  }  
}
```

点击延迟和点击高亮

JavaScript

```
/* 移除点击高亮 */  
button, a {  
  -webkit-tap-highlight-color: transparent;
```

```
-webkit-touch-callout: none;
}

/* 提高点击区域 */
.btn {
  min-height: 44px; /* 苹果推荐的最小点击区域 */
  min-width: 44px;
  padding: 12px 24px;
}

/* 禁用用户缩放 */
html {
  touch-action: manipulation; /* 禁用双击缩放 */
}
```

安全区域适配 (iPhone X+)

```
SQL
/* 适配刘海屏和底部安全区域 */
.safe-area {
  /* 上边距适配刘海 */
  padding-top: env(safe-area-inset-top);

  /* 底部适配 Home Indicator */
  padding-bottom: env(safe-area-inset-bottom);

  /* 左边距适配 */
  padding-left: env(safe-area-inset-left);

  /* 右边距适配 */
  padding-right: env(safe-area-inset-right);
}

/* 固定底部导航适配 */
.bottom-nav {
  position: fixed;
  bottom: 0;
  left: 0;
  right: 0;
  padding-bottom: env(safe-area-inset-bottom);
  background: white;
}
```

```
/* 全屏背景适配 */
.fullscreen {
  padding: env(safe-area-inset-top)
           env(safe-area-inset-right)
           env(safe-area-inset-bottom)
           env(safe-area-inset-left);
}
```

36.1.9 实用工具类和框架

实用 CSS 工具类

```
SQL
/* 间距工具类 */
.mt-1 { margin-top: 0.25rem; }
.mt-2 { margin-top: 0.5rem; }
.mt-4 { margin-top: 1rem; }

/* 响应式显示/隐藏 */
.mobile-only {
  display: block;
}

.tablet-only {
  display: none;
}

.desktop-only {
  display: none;
}

@media (min-width: 768px) {
  .mobile-only { display: none; }
  .tablet-only { display: block; }
}

@media (min-width: 1024px) {
  .tablet-only { display: none; }
  .desktop-only { display: block; }
}

/* 弹性工具类 */
```



```
.flex { display: flex; }
.flex-col { flex-direction: column; }
.items-center { align-items: center; }
.justify-between { justify-content: space-between; }
.flex-1 { flex: 1 1 0%; }
```

使用 CSS-in-JS 方案

Python

// styled-components 示例

```
import styled from 'styled-components';
```

```
const Container = styled.div`
  max-width: 100%;
  padding: 0 16px;
```

```
  @media (min-width: 768px) {
    max-width: 720px;
    margin: 0 auto;
  }
```

```
  @media (min-width: 1024px) {
    max-width: 960px;
  }
```

```
`;
```

// 使用 ThemeProvider 统一断点

```
const theme = {
  breakpoints: {
    mobile: '576px',
    tablet: '768px',
    desktop: '992px',
  }
};
```

```
const ResponsiveBox = styled.div`
  width: 100%;
```

```
  ${props => props.theme.breakpoints.tablet} {
    width: 50%;
  }
```

```
  ${props => props.theme.breakpoints.desktop} {
```

```
width: 25%;  
}  
`;  
`;
```

36.1.10 性能优化考虑

图片优化策略

```
JavaScript  
// 图片懒加载  
const lazyImages = document.querySelectorAll('img[data-src]');  
  
const imageObserver = new IntersectionObserver((entries) => {  
  entries.forEach(entry => {  
    if (entry.isIntersecting) {  
      const img = entry.target;  
      img.src = img.dataset.src;  
      img.classList.add('loaded');  
      imageObserver.unobserve(img);  
    }  
  });  
}, {  
  rootMargin: '50px' // 提前 50px 加载  
});  
  
lazyImages.forEach(img => imageObserver.observe(img));  
  
// 响应式图片选择  
function getOptimalImageSize() {  
  const viewportWidth = window.innerWidth;  
  const devicePixelRatio = window.devicePixelRatio || 1;  
  
  if (viewportWidth <= 375 && devicePixelRatio >= 2) {  
    return 'small@2x'; // iPhone 小屏高清  
  } else if (viewportWidth <= 768) {  
    return 'medium'; // 平板  
  } else {  
    return 'large'; // 桌面  
  }  
}
```

按需加载组件

JavaScript

// React 懒加载组件

```
const LazyComponent = React.lazy(() =>
import('./HeavyComponent'));
```

```
function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <LazyComponent />
    </Suspense>
  );
}
```

// 基于路由的代码分割

```
const routes = [
  {
    path: '/dashboard',
    component: React.lazy(() => import('./Dashboard'))
  }
];
```

// 条件加载 polyfill

```
if (!('IntersectionObserver' in window)) {
  import('intersection-observer').then(() => {
    // 初始化懒加载
  });
}
```

最佳实践总结

SQL

/* 移动端适配最佳实践示例 */

```
:root {
  /* 设计变量 */
  --breakpoint-mobile: 576px;
  --breakpoint-tablet: 768px;
  --breakpoint-desktop: 992px;

  /* 间距系统 */
  --space-xs: 0.25rem;
  --space-sm: 0.5rem;
  --space-md: 1rem;
```

```
--space-lg: 1.5rem;
--space-xl: 2rem;

/* 字体系统 */
--font-size-xs: 0.75rem;
--font-size-sm: 0.875rem;
--font-size-base: 1rem;
--font-size-lg: 1.125rem;
--font-size-xl: 1.25rem;
}

/* 移动优先的基类 */
* {
  box-sizing: border-box;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

html {
  font-size: 16px;
  overflow-x: hidden;
}

body {
  font-size: var(--font-size-base);
  line-height: 1.5;
  margin: 0;
  padding: 0;
  min-height: 100vh;
  min-height: -webkit-fill-available; /* 移动端全屏高度 */
}

/* 容器 */
.container {
  width: 100%;
  padding-right: var(--space-md);
  padding-left: var(--space-md);
  margin-right: auto;
  margin-left: auto;
}

/* 响应式调整 */
@media (min-width: 576px) {
  .container {
```

```
    max-width: 540px;
  }
}

@media (min-width: 768px) {
  .container {
    max-width: 720px;
  }
}

@media (min-width: 992px) {
  .container {
    max-width: 960px;
  }
}

@media (min-width: 1200px) {
  .container {
    max-width: 1140px;
  }
}
```

37. 请实现一个函数，将一个扁平的数组转换为树形结构。

问题描述

将一个包含父子关系的扁平数组转换成嵌套的树形结构。通常每个元素包含 `id` 和 `parentId` 字段。

基本实现

方法 1：递归实现

```
YAML
function arrayToTreeRecursive(items, parentId = null) {
  return items
    .filter(item => item.parentId === parentId)
    .map(item => ({
```

```
        ...item,
        children: arrayToTreeRecursive(items, item.id)
    }));
}
```

// 使用示例

```
const flatArray = [
  { id: 1, name: 'Node 1', parentId: null },
  { id: 2, name: 'Node 2', parentId: 1 },
  { id: 3, name: 'Node 3', parentId: 1 },
  { id: 4, name: 'Node 4', parentId: 2 },
  { id: 5, name: 'Node 5', parentId: null },
  { id: 6, name: 'Node 6', parentId: 5 }
];
```

```
const tree = arrayToTreeRecursive(flatArray);
console.log(JSON.stringify(tree, null, 2));
```

方法 2: 迭代实现 ($O(n)$ 时间复杂度)

JavaScript

```
function arrayToTree(items) {
```

```
    // 创建 id 到节点的映射
```

```
    const itemMap = {};
```

```
    const tree = [];
```

```
    // 第一次遍历: 创建所有节点的映射
```

```
    items.forEach(item => {
```

```
        itemMap[item.id] = {
```

```
            ...item,
```

```
            children: []
```

```
        };
```

```
    });
```

```
    // 第二次遍历: 构建树结构
```

```
    items.forEach(item => {
```

```
        const node = itemMap[item.id];
```

```
        if (item.parentId === null || item.parentId === undefined) {
```

```
            // 根节点
```

```
            tree.push(node);
```

```
        } else {
```

```
            // 找到父节点并添加为子节点
```



```

    const parent = itemMap[item.parentId];
    if (parent) {
      parent.children.push(node);
    } else {
      // 如果没有找到父节点，作为根节点处理
      tree.push(node);
    }
  }
});

return tree;
}

```

方法 3：使用 Map 的优化版本

```

JavaScript
function arrayToTreeOptimized(items, options = {}) {
  const {
    idKey = 'id',
    parentKey = 'parentId',
    childrenKey = 'children'
  } = options;

  const nodeMap = new Map();
  const tree = [];

  // 创建节点映射
  items.forEach(item => {
    nodeMap.set(item[idKey], {
      ...item,
      [childrenKey]: []
    });
  });

  // 构建树
  items.forEach(item => {
    const node = nodeMap.get(item[idKey]);
    const parentId = item[parentKey];

    if (parentId === null || parentId === undefined) {
      tree.push(node);
    } else {
      const parent = nodeMap.get(parentId);

```

```

    if (parent) {
      parent[childrenKey].push(node);
    } else {
      // 孤儿节点作为根节点
      tree.push(node);
    }
  }
});

return tree;
}

```

高级功能实现

37.1.1.1 支持自定义键名和排序

JavaScript

```

function arrayToTreeAdvanced(items, options = {}) {
  const {
    idKey = 'id',
    parentKey = 'parentId',
    childrenKey = 'children',
    sortKey = null,
    sortOrder = 'asc' // 'asc' 或 'desc'
  } = options;

  const nodeMap = new Map();
  const tree = [];

  // 创建节点映射
  items.forEach(item => {
    nodeMap.set(item[idKey], {
      ...item,
      [childrenKey]: []
    });
  });

  // 构建树
  items.forEach(item => {
    const node = nodeMap.get(item[idKey]);
    const parentId = item[parentKey];

    if (parentId === null || parentId === undefined) {

```

```

    tree.push(node);
  } else {
    const parent = nodeMap.get(parentId);
    if (parent) {
      parent[childrenKey].push(node);
    } else {
      tree.push(node);
    }
  }
});

// 递归排序
function sortTree(nodes) {
  if (sortKey) {
    nodes.sort((a, b) => {
      const aVal = a[sortKey];
      const bVal = b[sortKey];

      if (typeof aVal === 'string' && typeof bVal === 'string')
      {
        return sortOrder === 'asc'
          ? aVal.localeCompare(bVal)
          : bVal.localeCompare(aVal);
      }

      return sortOrder === 'asc' ? aVal - bVal : bVal - aVal;
    });
  }

  nodes.forEach(node => {
    if (node[childrenKey].length > 0) {
      sortTree(node[childrenKey]);
    }
  });
}

sortTree(tree);
return tree;
}

```

37.1.1.2 保留原始引用和循环引用检测

JavaScript

```
function arrayToTreeSafe(items, options = {}) {
  const {
    idKey = 'id',
    parentKey = 'parentId',
    childrenKey = 'children'
  } = options;

  const nodeMap = new Map();
  const tree = [];
  const visited = new Set();

  // 检测循环引用
  function hasCycle(node, path = new Set()) {
    if (path.has(node[idKey])) {
      return true;
    }

    path.add(node[idKey]);

    for (const child of node[childrenKey]) {
      if (hasCycle(child, new Set(path))) {
        return true;
      }
    }

    return false;
  }

  // 第一次遍历
  items.forEach(item => {
    nodeMap.set(item[idKey], {
      ...item,
      [childrenKey]: []
    });
  });

  // 第二次遍历
  items.forEach(item => {
    const node = nodeMap.get(item[idKey]);
    const parentId = item[parentKey];

    if (parentId === null || parentId === undefined) {
      tree.push(node);
    } else {
```

```

const parent = nodeMap.get(parentId);
if (parent) {
  // 检查是否会造成循环引用
  const tempChildren = [...parent[childrenKey], node];
  const tempParent = { ...parent, [childrenKey]:
tempChildren };

  if (hasCycle(tempParent)) {
    console.warn(`检测到循环引用: ${node[idKey]} ->
${parentId}`);
    tree.push(node); // 作为根节点处理
  } else {
    parent[childrenKey].push(node);
  }
} else {
  tree.push(node);
}
});

return tree;
}

```

37.1.1.3 反向查找：从树到扁平数组

```

JavaScript
function treeToArray(tree, options = {}) {
  const {
    childrenKey = 'children',
    parentKey = 'parentId',
    parentId = null
  } = options;

  const result = [];

  function flatten(node, currentParentId) {
    const { [childrenKey]: children, ...rest } = node;

    result.push({
      ...rest,
      [parentKey]: currentParentId
    });
  }

```

```
    if (children && children.length > 0) {
      children.forEach(child => {
        flatten(child, node.id);
      });
    }
  }

  tree.forEach(node => {
    flatten(node, parentId);
  });

  return result;
}
```

实际应用示例

示例 1：菜单系统

YAML

```
const menuItems = [
  { id: 1, title: '首页', path: '/', parentId: null, order: 1 },
  { id: 2, title: '产品', path: '/products', parentId: null,
    order: 2 },
  { id: 3, title: '产品列表', path: '/products/list', parentId: 2,
    order: 1 },
  { id: 4, title: '产品详情', path: '/products/detail', parentId:
    2, order: 2 },
  { id: 5, title: '关于我们', path: '/about', parentId: null,
    order: 3 },
  { id: 6, title: '公司历史', path: '/about/history', parentId: 5,
    order: 1 },
  { id: 7, title: '团队介绍', path: '/about/team', parentId: 5,
    order: 2 }
];

const menuTree = arrayToTreeAdvanced(menuItems, {
  sortKey: 'order',
  sortOrder: 'asc'
});

console.log(menuTree);
```


示例 2：组织架构

YAML

```
const employees = [
  { id: 1, name: 'CEO', title: '首席执行官', parentId: null },
  { id: 2, name: 'CTO', title: '首席技术官', parentId: 1 },
  { id: 3, name: 'CFO', title: '首席财务官', parentId: 1 },
  { id: 4, name: '开发经理', title: '开发部经理', parentId: 2 },
  { id: 5, name: '财务经理', title: '财务部经理', parentId: 3 },
  { id: 6, name: '前端开发', title: '前端工程师', parentId: 4 },
  { id: 7, name: '后端开发', title: '后端工程师', parentId: 4 }
];

function buildOrgChart(items) {
  const tree = arrayToTree(items);

  // 添加深度信息
  function addDepth(node, depth = 0) {
    node.depth = depth;
    if (node.children) {
      node.children.forEach(child => addDepth(child, depth + 1));
    }
  }

  tree.forEach(root => addDepth(root));
  return tree;
}

const orgChart = buildOrgChart(employees);
console.log(orgChart);
```

示例 3：评论系统

JavaScript

```
const comments = [
  { id: 1, content: '评论 1', postId: 1, parentId: null, createdAt: '2024-01-01' },
  { id: 2, content: '回复评论 1', postId: 1, parentId: 1, createdAt: '2024-01-02' },
  { id: 3, content: '评论 2', postId: 1, parentId: null, createdAt: '2024-01-01' },
  { id: 4, content: '回复评论 2', postId: 1, parentId: 3,
```

```
createdAt: '2024-01-03' },
  { id: 5, content: '回复的回复', postId: 1, parentId: 2,
    createdAt: '2024-01-04' }
];

function buildCommentTree(comments) {
  const tree = arrayToTree(comments);

  // 按时间排序每个层级的评论
  function sortComments(node) {
    if (node.children) {
      node.children.sort((a, b) =>
        new Date(a.createdAt) - new Date(b.createdAt)
      );
      node.children.forEach(sortComments);
    }
  }

  tree.forEach(sortComments);
  return tree;
}

const commentTree = buildCommentTree(comments);
console.log(commentTree);
```

测试用例

```
JavaScript
// 测试函数
function testArrayToTree() {
  console.log('=== 测试开始 ===\n');

  // 测试数据
  const testData = [
    { id: 'root', name: 'Root', parentId: null },
    { id: 'child1', name: 'Child 1', parentId: 'root' },
    { id: 'child2', name: 'Child 2', parentId: 'root' },
    { id: 'grandchild1', name: 'Grandchild 1', parentId:
      'child1' },
    { id: 'grandchild2', name: 'Grandchild 2', parentId:
      'child1' },
    { id: 'orphan', name: 'Orphan', parentId: 'nonexistent' }
  ];
```

```
// 测试基本功能
console.log('1. 测试基本功能:');
const result1 = arrayToTree(testData);
console.log(JSON.stringify(result1, null, 2));

// 测试循环引用检测
console.log('\n2. 测试循环引用检测:');
const circularData = [
  { id: 'a', name: 'A', parentId: 'c' },
  { id: 'b', name: 'B', parentId: 'a' },
  { id: 'c', name: 'C', parentId: 'b' }
];

const result2 = arrayToTreeSafe(circularData);
console.log(JSON.stringify(result2, null, 2));

// 测试性能
console.log('\n3. 测试性能 (10000 个节点):');
const largeData = [];
for (let i = 0; i < 10000; i++) {
  largeData.push({
    id: `node_${i}`,
    name: `Node ${i}`,
    parentId: i === 0 ? null : `node_${Math.floor(Math.random()
* i)}`
  });
}

console.time('arrayToTree');
const result3 = arrayToTree(largeData);
console.timeEnd('arrayToTree');
console.log(`生成了 ${result3.length} 棵树的森林`);

// 测试反向转换
console.log('\n4. 测试树到数组的转换:');
const flatAgain = treeToArray(result1);
console.log(flatAgain);

console.log('\n=== 测试结束 ===');
}

testArrayToTree();
```

在 React 中的应用

JavaScript

// TreeView 组件

```
function TreeView({ data }) {  
  const tree = arrayToTree(data);  
  
  const TreeNode = ({ node, depth = 0 }) => (  
    <div style={{ marginLeft: `${depth * 20}px` }}>  
      <div>{node.name}</div>  
      {node.children?.map(child => (  
        <TreeNode key={child.id} node={child} depth={depth + 1} />  
      ))}  
    </div>  
  );  
  
  return (  
    <div>  
      {tree.map(node => (  
        <TreeNode key={node.id} node={node} />  
      ))}  
    </div>  
  );  
}
```

// 可折叠的树

```
function CollapsibleTree({ data }) {  
  const [expanded, setExpanded] = useState(new Set());  
  
  const tree = arrayToTree(data);  
  
  const TreeNode = ({ node, depth = 0 }) => {  
    const hasChildren = node.children && node.children.length > 0;  
    const isExpanded = expanded.has(node.id);  
  
    const toggle = () => {  
      const newExpanded = new Set(expanded);  
      if (isExpanded) {  
        newExpanded.delete(node.id);  
      } else {  
        newExpanded.add(node.id);  
      }  
      setExpanded(newExpanded);  
    };  
  };  
}
```

```

return (
  <div style={{ marginLeft: `${depth * 20}px` }}>
    <div onClick={hasChildren ? toggle : undefined}>
      {hasChildren && (
        <span>{isExpanded ? '▼' : '▶'}</span>
      )}
      {node.name}
    </div>
    {hasChildren && isExpanded && node.children.map(child => (
      <TreeNode key={child.id} node={child} depth={depth + 1}
    />
    ))}
  </div>
);
};

return (
  <div>
    {tree.map(node => (
      <TreeNode key={node.id} node={node} />
    ))}
  </div>
);
}

```

性能优化建议

JavaScript

```

// 1. 使用 Map 而不是对象
// Map 在频繁增删时性能更好

// 2. 避免递归过深
// 如果树可能很深，使用迭代方法

// 3. 使用尾递归优化（如果支持）
function arrayToTreeTailRecursive(items, parentId = null, result = []) {
  const children = items.filter(item => item.parentId ===
parentId);

  for (const child of children) {
    const node = { ...child, children: [] };

```

```

    result.push(node);
    arrayToTreeTailRecursive(items, child.id, node.children);
  }

  return result;
}

// 4. 处理大型数据集时使用分页或虚拟化
async function buildLargeTree(itemIds, fetchItem, batchSize = 100)
{
  const tree = [];
  const itemMap = new Map();

  // 分批获取数据
  for (let i = 0; i < itemIds.length; i += batchSize) {
    const batchIds = itemIds.slice(i, i + batchSize);
    const items = await Promise.all(batchIds.map(fetchItem));

    items.forEach(item => {
      itemMap.set(item.id, { ...item, children: [] });
    });
  }

  // 构建树
  for (const item of itemMap.values()) {
    if (item.parentId === null || item.parentId === undefined) {
      tree.push(item);
    } else {
      const parent = itemMap.get(item.parentId);
      if (parent) {
        parent.children.push(item);
      } else {
        tree.push(item);
      }
    }
  }

  return tree;
}

```

38. 图片标签 `img` 和 `picture` 的区别

 标签

基本图片显示标签

HTML

```

```

特点:

- 最简单的图片嵌入方式
- 只能加载一个图片源
- 通过 `srcset` 和 `sizes` 属性可实现基本的响应式图片
- 所有浏览器都支持

<picture> 标签

响应式图片容器

XML

```
<picture>
  <source media="(min-width: 1200px)" srcset="large.jpg">
  <source media="(min-width: 768px)" srcset="medium.jpg">
  
</picture>
```

特点:

- 是一个容器元素，内部包含多个 `<source>` 和一个 `<img>`
- 允许基于不同条件（设备、屏幕尺寸、像素密度等）提供不同的图片
- 支持现代图片格式（WebP、AVIF）并提供降级方案

主要区别对比

特性		<picture>
主要用途	显示单张图片	响应式图片和格式适配
艺术方向	不支持	✅ 支持（不同裁剪的图片）
格式降级	有限支持	✅ 完善支持
代码结构	单个标签	容器 + 多个 <code><source></code> + <code></code>
浏览器支持	所有浏览器	现代浏览器（IE 不支持）

点击图片可查看完整电子表格

使用场景示例

1. 艺术方向适配（最适合用 <picture>）

XML

```
<picture>
  <!-- 大屏幕：显示横版裁剪的图片 -->
  <source media="(min-width: 1200px)" srcset="landscape.jpg">
  <!-- 中等屏幕：显示方形裁剪 -->
  <source media="(min-width: 768px)" srcset="square.jpg">
  <!-- 小屏幕：显示竖版裁剪 -->
  
</picture>
```

2. 现代格式降级（最适合用 <picture>）

XML

```
<picture>
  <!-- 支持 AVIF 格式的浏览器 -->
  <source srcset="image.avif" type="image/avif">
  <!-- 支持 WebP 格式的浏览器 -->
  <source srcset="image.webp" type="image/webp">
  <!-- 所有浏览器都支持的回退方案 -->
  
</picture>
```

3. 像素密度适配（两者都可）

XML

```
<!-- 使用 img -->


<!-- 使用 picture -->
<picture>
  <source srcset="image@2x.jpg 2x, image@3x.jpg 3x">
  
```

```
</picture>
```

最佳实践建议

1. 使用 `<img>` 当：
 - 只需要显示一张简单图片
 - 仅需要基于视口宽度调整图片分辨率时
 - 兼容性要求极高（需要支持旧浏览器）
2. 使用 `<picture>` 当：
 - 需要为不同屏幕尺寸提供不同裁剪的图片（艺术方向）
 - 希望使用现代图片格式（WebP/AVIF）并优雅降级
 - 需要基于设备特性（如 `prefers-color-scheme`）提供不同图片
3. 重要提示：
 - `<picture>` 中的 `<img>` 标签是必需的，用作回退
 - 始终提供 `alt` 属性
 - 使用正确的 `type` 属性指定图片格式
 - 在 `<source>` 中指定条件，浏览器会从上到下匹配第一个符合条件的

总结

- `<img>` 是基础，`<picture>` 是增强
- `<img>` 适合简单响应式，`<picture>` 适合复杂场景
- 使用 `<picture>` 的主要目的是：艺术方向控制和格式适配

39. 在 React 中，PureComponent 和 memo 是如何进行性能优化的？

PureComponent

基本原理

PureComponent 通过浅比较（shallow compare）`props` 和 `state` 的变化来决定是否

需要重新渲染。

```
JavaScript
// PureComponent 示例
class MyComponent extends React.PureComponent {
  render() {
    console.log('MyComponent 渲染');
    return <div>{this.props.value}</div>;
  }
}

// 使用
class ParentComponent extends React.Component {
  state = { value: 1 };

  handleClick = () => {
    this.setState({ value: 1 }); // 相同的值，PureComponent 不会重新渲染
  };

  render() {
    return (
      <div>
        <MyComponent value={this.state.value} />
        <button onClick={this.handleClick}>更新</button>
      </div>
    );
  }
}
```

浅比较的实现原理

```
JavaScript
// 简化的浅比较实现
function shallowEqual(objA, objB) {
  if (Object.is(objA, objB)) {
    return true;
  }

  if (
    typeof objA !== 'object' ||
    objA === null ||
    typeof objB !== 'object' ||
  )
```

```

    objB === null
  ) {
    return false;
  }

  const keysA = Object.keys(objA);
  const keysB = Object.keys(objB);

  if (keysA.length !== keysB.length) {
    return false;
  }

  for (let i = 0; i < keysA.length; i++) {
    const key = keysA[i];
    if (
      !Object.prototype.hasOwnProperty.call(objB, key) ||
      !Object.is(objA[key], objB[key])
    ) {
      return false;
    }
  }

  return true;
}

// React 的 shallowCompare 简版
function shouldComponentUpdate(nextProps, nextState) {
  return !shallowEqual(this.props, nextProps) ||
    !shallowEqual(this.state, nextState);
}

```

PureComponent 的限制

```

JavaScript
// 问题 1: 引用类型的变化
class UserList extends React.PureComponent {
  render() {
    return (
      <ul>
        {this.props.users.map(user => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    )
  }
}

```

```

    });
  }
}

class App extends React.Component {
  state = {
    users: [
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' }
    ]
  };

  addUser = () => {
    // ✖ 错误: 直接修改原数组
    this.state.users.push({ id: 3, name: 'Charlie' });
    this.setState({ users: this.state.users }); // 引用没变,
    PureComponent 不会更新
  };

  addUserCorrect = () => {
    // ✔ 正确: 创建新数组
    this.setState({
      users: [...this.state.users, { id: 3, name: 'Charlie' }]
    });
  };

  render() {
    return (
      <div>
        <UserList users={this.state.users} />
        <button onClick={this.addUserCorrect}>添加用户</button>
      </div>
    );
  }
}

```

React.memo

函数组件的 PureComponent

```

JavaScript
// 基本用法
const MyComponent = React.memo(function MyComponent(props) {

```



```

    console.log('MyComponent 渲染');
    return <div>{props.value}</div>;
  });

  // 使用
  function ParentComponent() {
    const [value, setValue] = useState(1);

    return (
      <div>
        <MyComponent value={value} />
        <button onClick={() => setValue(1)}>更新（相同值）</button>
      </div>
    );
  }
}

```

自定义比较函数

```

JavaScript
// 自定义 props 比较
const UserProfile = React.memo(
  function UserProfile({ user, onUpdate }) {
    console.log('UserProfile 渲染');
    return (
      <div>
        <h2>{user.name}</h2>
        <p>年龄: {user.age}</p>
        <button onClick={() => onUpdate(user.id)}>更新</button>
      </div>
    );
  },
  // 自定义比较函数
  (prevProps, nextProps) => {
    // 只比较 user 对象的基本属性
    return (
      prevProps.user.id === nextProps.user.id &&
      prevProps.user.name === nextProps.user.name &&
      prevProps.user.age === nextProps.user.age
    );
    // 忽略 onUpdate 函数的比较（通常函数引用会变化）
  }
);

```

```
// 使用 useCallback 避免函数引用变化
function UserList({ users }) {
  const handleUpdate = useCallback((userId) => {
    console.log('更新用户:', userId);
  }, []);

  return (
    <div>
      {users.map(user => (
        <UserProfile
          key={user.id}
          user={user}
          onUpdate={handleUpdate}
        />
      ))}
    </div>
  );
}
```

性能优化实践

39.1.1.1 避免不必要的重新渲染

```
JavaScript
// ✖ 不好的示例：每次渲染都创建新对象
function Parent() {
  const [count, setCount] = useState(0);

  const config = { theme: 'dark', size: 'large' }; // 每次渲染都创建新对象

  return (
    <div>
      <ExpensiveComponent config={config} />
      <button onClick={() => setCount(count + 1)}>计数: {count}</button>
    </div>
  );
}

// ✔ 优化：使用 useMemo 和 useCallback
```

```

function ParentOptimized() {
  const [count, setCount] = useState(0);

  // 缓存配置对象
  const config = useMemo(() => ({
    theme: 'dark',
    size: 'large'
  }), []);

  // 缓存事件处理函数
  const handleClick = useCallback(() => {
    setCount(prev => prev + 1);
  }, []);

  return (
    <div>
      <ExpensiveComponent config={config} />
      <button onClick={handleClick}>计数: {count}</button>
    </div>
  );
}

```

39.1.1.2 列表项的优化

```

JavaScript
// 优化列表渲染
const UserItem = React.memo(({ user, onSelect }) => {
  console.log(`UserItem ${user.id} 渲染`);
  return (
    <li onClick={() => onSelect(user.id)}>
      {user.name}
    </li>
  );
});

function UserList({ users, selectedId, onSelect }) {
  // 使用 useCallback 缓存回调
  const handleSelect = useCallback((userId) => {
    onSelect(userId);
  }, [onSelect]);

  return (
    <ul>

```

```

    {users.map(user => (
      <UserItem
        key={user.id}
        user={user}
        isSelected={user.id === selectedId}
        onSelect={handleSelect}
      />
    ))}
  </ul>
);
}

// 进一步优化：虚拟滚动
import { FixedSizeList } from 'react-window';

function VirtualUserList({ users, selectedId, onSelect }) {
  const Row = ({ index, style }) => {
    const user = users[index];
    return (
      <div style={style}>
        <UserItem
          user={user}
          isSelected={user.id === selectedId}
          onSelect={onSelect}
        />
      </div>
    );
  };
  return (
    <FixedSizeList
      height={400}
      width={300}
      itemCount={users.length}
      itemSize={50}
    >
      {Row}
    </FixedSizeList>
  );
}

```

39.1.1.3 使用 Context 时的优化

JavaScript

// 避免 Context 导致的不必要渲染

```
const UserContext = React.createContext();
```

// ✘ 问题: 任何 Context 值变化都会导致所有 Consumer 重新渲染

```
function App() {  
  const [user, setUser] = useState({ name: 'Alice', age: 25 });  
  const [theme, setTheme] = useState('light');  
  
  return (  
    <UserContext.Provider value={{ user, theme }}>  
      <UserProfile />  
      <ThemeSwitcher />  
    </UserContext.Provider>  
  );  
}
```

// ✔ 解决方案 1: 拆分 Context

```
const UserContext = React.createContext();  
const ThemeContext = React.createContext();
```

```
function App() {  
  const [user, setUser] = useState({ name: 'Alice', age: 25 });  
  const [theme, setTheme] = useState('light');  
  
  return (  
    <UserContext.Provider value={user}>  
      <ThemeContext.Provider value={theme}>  
        <UserProfile />  
        <ThemeSwitcher />  
      </ThemeContext.Provider>  
    </UserContext.Provider>  
  );  
}
```

// ✔ 解决方案 2: 使用 memo 和 useMemo

```
const UserProfile = React.memo(() => {  
  const user = useContext(UserContext);  
  // 只有当 user 变化时才重新渲染  
  return <div>{user.name}</div>;  
});
```

```
function App() {
```

```
const [user, setUser] = useState({ name: 'Alice', age: 25 });
const [theme, setTheme] = useState('light');

// 缓存 Context 值
const userValue = useMemo(() => ({ user, setUser }), [user]);
const themeValue = useMemo(() => ({ theme, setTheme }),
[theme]);

return (
  <UserContext.Provider value={userValue}>
    <ThemeContext.Provider value={themeValue}>
      <UserProfile />
      <ThemeSwitcher />
    </ThemeContext.Provider>
  </UserContext.Provider>
);
}
```

性能分析工具

React DevTools Profiler

```
JavaScript
// 使用 Profiler 组件测量性能
import { Profiler } from 'react';

function App() {
  const onRender = (id, phase, actualDuration, baseDuration,
startTime, commitTime) => {
    console.log({
      id, // Profiler 树的 "id"
      phase, // "mount" 或 "update"
      actualDuration, // 实际渲染时间
      baseDuration, // 估计不使用 memoization 的渲染时间
      startTime, // 开始渲染的时间戳
      commitTime // React 提交更新的时间戳
    });
  };

  return (
    <Profiler id="App" onRender={onRender}>
      <MyComponent />
    </Profiler>
  );
}
```



```
    </Profiler>
  );
}
```

性能测量 Hook

```
JavaScript
// 自定义性能测量 Hook
function useRenderTime(name) {
  const startTime = useRef(performance.now());

  useEffect(() => {
    const endTime = performance.now();
    console.log(`${name} 渲染时间:`, endTime - startTime.current,
'ms');
  });

  useEffect(() => {
    startTime.current = performance.now();
  });
}

function MyComponent() {
  useRenderTime('MyComponent');
  return <div>组件内容</div>;
}
```

最佳实践总结

```
JavaScript
// 性能优化清单
const PerformanceChecklist = {
  1: '使用 React.memo 包装函数组件',
  2: '对于类组件，继承 PureComponent 而不是 Component',
  3: '使用 useMemo 缓存计算昂贵的值',
  4: '使用 useCallback 缓存事件处理函数',
  5: '避免在渲染函数中创建新的对象/数组/函数',
  6: '列表项使用稳定的 key',
  7: '大型列表使用虚拟滚动',
  8: '拆分大型 Context，避免不必要的渲染',
  9: '使用代码分割（React.lazy）',
}
```

```
10: '定期使用 React DevTools Profiler 分析性能'
});

// 实际示例：优化后的组件
const OptimizedComponent = React.memo(function
OptimizedComponent({
  data,
  onAction
}) {
  // 使用 useMemo 缓存计算结果
  const processedData = useMemo(() => {
    return data.map(item => ({
      ...item,
      formattedDate: new Date(item.date).toLocaleDateString()
    }));
  }, [data]);

  // 使用 useMemo 缓存 JSX
  const content = useMemo(() => (
    <div>
      {processedData.map(item => (
        <DataItem key={item.id} item={item} />
      ))}
    </div>
  ), [processedData]);

  return (
    <div>
      <h2>数据列表</h2>
      {content}
      <ActionButton onAction={onAction} />
    </div>
  );
});

// 进一步优化的 DataItem
const DataItem = React.memo(function DataItem({ item }) {
  return (
    <div>
      <span>{item.name}</span>
      <span>{item.formattedDate}</span>
    </div>
  );
});
```

```
// 使用 useCallback 优化的 ActionButton
const ActionButton = React.memo(function
ActionButton({ onAction }) {
  const handleClick = useCallback(() => {
    onAction('button_clicked');
  }, [onAction]);

  return <button onClick={handleClick}>执行操作</button>;
});
```

40. localStorage, sessionStorage, 和 Cookie 之间有什么区别？

三者对比总览

特性	localStorage	sessionStorage	Cookie
存储容量	5-10MB	5-10MB	4KB
生命周期	永久（除非手动清除）	浏览器标签页关闭时	可设置过期时间
作用域	同源的所有标签页	单个标签页	同源的所有标签页
服务器通信	不自动发送	不自动发送	每次请求自动发送
访问方式	JavaScript API	JavaScript API	JavaScript 或 HTTP 头

点击图片可查看完整电子表格

详细区别和用法

40.1.1.1 localStorage

```
JavaScript
// 基本操作
// 设置
localStorage.setItem('username', 'john_doe');
```

```

localStorage.setItem('user', JSON.stringify({ id: 1, name:
'John' }));

// 获取
const username = localStorage.getItem('username');
const user = JSON.parse(localStorage.getItem('user') || '{}');

// 删除
localStorage.removeItem('username');
localStorage.removeItem('user');

// 清空
localStorage.clear();

// 遍历
for (let i = 0; i < localStorage.length; i++) {
  const key = localStorage.key(i);
  const value = localStorage.getItem(key);
  console.log(`${key}: ${value}`);
}

// 事件监听（同源页面间通信）
window.addEventListener('storage', (event) => {
  console.log('Storage 变化:', {
    key: event.key,
    oldValue: event.oldValue,
    newValue: event.newValue,
    url: event.url,
    storageArea: event.storageArea
  });
});

```

40.1.1.2 sessionStorage

```

JavaScript
// 基本操作（API 与 localStorage 相同）
// 设置
sessionStorage.setItem('sessionId', 'abc123');
sessionStorage.setItem('cart', JSON.stringify(['item1',
'item2']));

// 获取
const sessionId = sessionStorage.getItem('sessionId');

```

```

const cart = JSON.parse(sessionStorage.getItem('cart') || '[]');

// 特性：标签页独立
// 在标签页 1
sessionStorage.setItem('tab', 'tab1');

// 在标签页 2（无法访问标签页 1 的数据）
console.log(sessionStorage.getItem('tab')); // null

// 使用场景：表单草稿保存
class FormDraft {
  constructor(formId) {
    this.formId = formId;
    this.key = `form_draft_${formId}`;
  }

  save(data) {
    sessionStorage.setItem(this.key, JSON.stringify(data));
  }

  load() {
    const data = sessionStorage.getItem(this.key);
    return data ? JSON.parse(data) : null;
  }

  clear() {
    sessionStorage.removeItem(this.key);
  }
}

// 使用
const form = new FormDraft('contact_form');
form.save({ name: 'John', email: 'john@example.com' });

```

40.1.1.3 Cookie

```

JavaScript
// 设置 Cookie（原生）
function setCookie(name, value, options = {}) {
  let cookie =
    `${encodeURIComponent(name)}=${encodeURIComponent(value)}`;

  if (options.expires) {

```

```

    if (typeof options.expires === 'number') {
        const date = new Date();
        date.setTime(date.getTime() + options.expires * 24 * 60 * 60
* 1000);
        cookie += `; expires=${date.toUTCString()}`;
    } else if (options.expires instanceof Date) {
        cookie += `; expires=${options.expires.toUTCString()}`;
    }
}

if (options.path) cookie += `; path=${options.path}`;
if (options.domain) cookie += `; domain=${options.domain}`;
if (options.secure) cookie += `; secure`;
if (options.sameSite) cookie += `;
samesite=${options.sameSite}`;
if (options.httpOnly) {
    // httpOnly 不能在客户端设置，只能在服务器设置
    console.warn('httpOnly 只能在服务器端设置');
}

document.cookie = cookie;
}

// 获取 Cookie
function getCookie(name) {
    const nameEQ = encodeURIComponent(name) + "=";
    const cookies = document.cookie.split(';');

    for (let cookie of cookies) {
        cookie = cookie.trim();
        if (cookie.indexOf(nameEQ) === 0) {
            return decodeURIComponent(cookie.substring(nameEQ.length));
        }
    }
    return null;
}

// 删除 Cookie
function deleteCookie(name, options = {}) {
    setCookie(name, '', {
        ...options,
        expires: new Date(0) // 设置为过去的时间
    });
}

```



```
// 使用示例
setCookie('username', 'john_doe', {
  expires: 7, // 7 天后过期
  path: '/',
  secure: true,
  sameSite: 'strict'
});

const username = getCookie('username');
deleteCookie('username');
```

实际应用场景

场景 1：用户偏好设置

```
JavaScript
// 使用 localStorage 保存用户设置
class UserPreferences {
  constructor() {
    this.key = 'user_preferences';
    this.defaults = {
      theme: 'light',
      language: 'zh-CN',
      fontSize: 'medium',
      notifications: true
    };
  }

  get() {
    const saved = localStorage.getItem(this.key);
    return saved ? { ...this.defaults, ...JSON.parse(saved) } :
this.defaults;
  }

  update(updates) {
    const current = this.get();
    const updated = { ...current, ...updates };
    localStorage.setItem(this.key, JSON.stringify(updated));

    // 触发自定义事件，通知其他组件
    window.dispatchEvent(new CustomEvent('preferencesChange', {
      detail: updated
    }));
  }
}
```

```

    }));

    return updated;
  }

  reset() {
    localStorage.removeItem(this.key);
    return this.defaults;
  }
}

// 使用
const prefs = new UserPreferences();

// 保存用户选择
prefs.update({ theme: 'dark', fontSize: 'large' });

// 应用设置
function applyPreferences() {
  const { theme, fontSize } = prefs.get();
  document.documentElement.setAttribute('data-theme', theme);
  document.body.style.fontSize = fontSize === 'large' ? '18px' :
  '16px';
}

// 监听变化
window.addEventListener('preferencesChange', (event) => {
  console.log('设置已更新:', event.detail);
  applyPreferences();
}));

```

场景 2：购物车状态

```

JavaScript
// 使用 sessionStorage 保存购物车（标签页独立）
class ShoppingCart {
  constructor() {
    this.key = 'shopping_cart';
    this.items = this.load();
  }

  load() {
    const data = sessionStorage.getItem(this.key);

```

```
    return data ? JSON.parse(data) : [];
  }

  save() {
    sessionStorage.setItem(this.key, JSON.stringify(this.items));
  }

  addItem(product, quantity = 1) {
    const existingIndex = this.items.findIndex(item => item.id === product.id);

    if (existingIndex > -1) {
      this.items[existingIndex].quantity += quantity;
    } else {
      this.items.push({
        id: product.id,
        name: product.name,
        price: product.price,
        quantity: quantity
      });
    }

    this.save();
    this.notify();
  }

  removeItem(productId) {
    this.items = this.items.filter(item => item.id !== productId);
    this.save();
    this.notify();
  }

  clear() {
    this.items = [];
    sessionStorage.removeItem(this.key);
    this.notify();
  }

  getTotal() {
    return this.items.reduce((total, item) => {
      return total + (item.price * item.quantity);
    }, 0);
  }
}
```

```
notify() {
  window.dispatchEvent(new CustomEvent('cartUpdated', {
    detail: { items: this.items, total: this.getTotal() }
  }));
}
}

// 使用
const cart = new ShoppingCart();

// 添加商品
cart.addItem({ id: 1, name: '商品 A', price: 100 }, 2);

// 监听购物车变化
window.addEventListener('cartUpdated', (event) => {
  updateCartUI(event.detail);
});
```

场景 3：认证和会话管理

```
JavaScript
// 使用 Cookie 进行认证（配合服务器）
class AuthService {
  constructor() {
    this.tokenKey = 'auth_token';
    this.userKey = 'auth_user';
  }

  async login(credentials) {
    try {
      const response = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(credentials)
      });

      if (!response.ok) throw new Error('登录失败');

      const { token, user } = await response.json();

      // 保存 token 到 Cookie（服务器也可以设置 HttpOnly Cookie）
      setCookie(this.tokenKey, token, {
        expires: 1, // 1 天过期
      });
    }
  }
}
```

```
        secure: true,
        sameSite: 'strict'
    });

    // 保存用户信息到 localStorage
    localStorage.setItem(this.userKey, JSON.stringify(user));

    return { success: true, user };

} catch (error) {
    return { success: false, error: error.message };
}
}

logout() {
    // 清除 Cookie
    deleteCookie(this.tokenKey);

    // 清除 localStorage
    localStorage.removeItem(this.userKey);

    // 清除 sessionStorage (如果使用了)
    sessionStorage.clear();

    // 重定向到登录页
    window.location.href = '/login';
}

isAuthenticated() {
    return !!getCookie(this.tokenKey);
}

getCurrentUser() {
    const userData = localStorage.getItem(this.userKey);
    return userData ? JSON.parse(userData) : null;
}

// 自动附加认证头
getAuthHeaders() {
    const token = getCookie(this.tokenKey);
    return token ? { 'Authorization': `Bearer ${token}` } : {};
}
}
```

```
// 封装带认证的 fetch
const authFetch = async (url, options = {}) => {
  const authService = new AuthService();
  const headers = {
    ...options.headers,
    ...authService.getAuthHeaders()
  };

  const response = await fetch(url, { ...options, headers });

  if (response.status === 401) {
    // Token 过期，跳转到登录页
    authService.logout();
    return null;
  }

  return response;
};
```

安全考虑

40.1.1.1 XSS 防护

```
JavaScript
// 安全地存储和读取数据
class SecureStorage {
  constructor(secret) {
    this.secret = secret;
  }

  // 简单的加密（生产环境应使用更安全的算法）
  encrypt(data) {
    return btoa(encodeURIComponent(JSON.stringify(data)));
  }

  decrypt(encrypted) {
    try {
      return JSON.parse(decodeURIComponent(atob(encrypted)));
    } catch {
      return null;
    }
  }
}
```



```
setItem(key, data) {  
  const encrypted = this.encrypt(data);  
  localStorage.setItem(key, encrypted);  
}  
  
getItem(key) {  
  const encrypted = localStorage.getItem(key);  
  return encrypted ? this.decrypt(encrypted) : null;  
}  
}  
  
// 使用  
const storage = new SecureStorage('my-secret');  
storage.setItem('sensitive_data', { creditCard: '1234-5678-9012-3456' });  
const data = storage.getItem('sensitive_data');
```

40.1.1.2 Cookie 安全设置

```
JavaScript  
// 服务器端设置安全 Cookie (Node.js Express 示例)  
app.post('/login', (req, res) => {  
  // 验证用户...  
  const token = generateToken(user);  
  
  res.cookie('auth_token', token, {  
    httpOnly: true,    // 防止 XSS 攻击  
    secure: true,      // 仅 HTTPS  
    sameSite: 'strict', // 防止 CSRF  
    maxAge: 24 * 60 * 60 * 1000, // 24 小时  
    path: '/'  
  });  
  
  res.json({ success: true, user });  
});  
  
// 客户端读取 HttpOnly Cookie (只能通过服务器)  
app.get('/api/user', authenticateToken, (req, res) => {  
  // authenticateToken 中间件从 Cookie 读取 token  
  res.json({ user: req.user });  
});
```

40.1.1.3 存储限制和清理

```
JavaScript
// 监控存储使用情况
class StorageMonitor {
  static estimateSize(obj) {
    const str = JSON.stringify(obj);
    return new Blob([str]).size;
  }

  static checkLocalStorageLimit() {
    let total = 0;

    for (let i = 0; i < localStorage.length; i++) {
      const key = localStorage.key(i);
      const value = localStorage.getItem(key);
      total += new Blob([key + value]).size;
    }

    const limit = 5 * 1024 * 1024; // 5MB
    const usage = (total / limit) * 100;

    return {
      used: total,
      limit: limit,
      percentage: usage.toFixed(2)
    };
  }

  static cleanupOldItems() {
    const now = Date.now();
    const oneWeekAgo = now - 7 * 24 * 60 * 60 * 1000;

    for (let i = 0; i < localStorage.length; i++) {
      const key = localStorage.key(i);

      if (key.startsWith('cache_')) {
        const item = localStorage.getItem(key);
        try {
          const data = JSON.parse(item);
          if (data.timestamp && data.timestamp < oneWeekAgo) {
            localStorage.removeItem(key);
          }
        } catch {
        }
      }
    }
  }
}
```

```

        // 不是 JSON，跳过
    }
}
}
}
}

// 定期清理
setInterval(() => {
    StorageMonitor.cleanupOldItems();
}, 24 * 60 * 60 * 1000); // 每天清理一次

```

现代替代方案

IndexedDB

```

JavaScript
// 对于大量结构化数据
const dbPromise = indexedDB.open('myDatabase', 1);

dbPromise.onupgradeneeded = (event) => {
    const db = event.target.result;

    // 创建对象存储
    if (!db.objectStoreNames.contains('users')) {
        db.createObjectStore('users', { keyPath: 'id' });
    }

    if (!db.objectStoreNames.contains('products')) {
        const store = db.createObjectStore('products', { keyPath:
'id' });
        store.createIndex('category', 'category', { unique: false });
    }
};

dbPromise.onsuccess = (event) => {
    const db = event.target.result;

    // 添加数据
    const transaction = db.transaction(['users'], 'readwrite');
    const store = transaction.objectStore('users');

    store.add({ id: 1, name: 'John', age: 30 });

```

```
// 查询数据
const getRequest = store.get(1);
getRequest.onsuccess = () => {
  console.log('用户:', getRequest.result);
};
};
```

服务端 Session

```
JavaScript
// 使用服务端 session（更安全）
// 客户端只保存 session ID
const sessionId = generateSessionId();
setCookie('session_id', sessionId, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict'
});

// 服务器端存储 session 数据
const sessions = new Map();

app.post('/login', (req, res) => {
  // 验证用户
  const sessionId = generateSessionId();

  // 在服务器存储 session 数据
  sessions.set(sessionId, {
    userId: user.id,
    username: user.username,
    lastActive: Date.now()
  });

  // 只发送 session ID 到客户端
  res.cookie('session_id', sessionId, {
    httpOnly: true,
    secure: true,
    sameSite: 'strict'
  });

  res.json({ success: true });
});
```

```
// 验证 session 的中间件
function authenticateSession(req, res, next) {
  const sessionId = req.cookies.session_id;

  if (!sessionId || !sessions.has(sessionId)) {
    return res.status(401).json({ error: '未认证' });
  }

  const session = sessions.get(sessionId);

  // 检查 session 是否过期（30 分钟）
  if (Date.now() - session.lastActive > 30 * 60 * 1000) {
    sessions.delete(sessionId);
    return res.status(401).json({ error: '会话已过期' });
  }

  // 更新最后活跃时间
  session.lastActive = Date.now();

  req.user = session;
  next();
}
```

41. instanceof 和 typeof 操作符的区别是什么？

基本区别总览

特性	typeof	instanceof
用途	检查值的类型	检查对象的原型链
返回值	字符串 (<code>'string'</code> , <code>'number'</code> , <code>'boolean'</code> 等)	布尔值 (<code>true</code> / <code>false</code>)
操作数	任意值（原始值或对象）	对象和构造函数

点击图片可查看完整电子表格

详细对比和示例

41.1.1.1 typeof 操作符

```
TypeScript
// 基本类型检测
console.log(typeof 'hello');           // 'string'
console.log(typeof 123);                // 'number'
console.log(typeof true);               // 'boolean'
console.log(typeof undefined);          // 'undefined'
console.log(typeof Symbol('sym'));      // 'symbol'
console.log(typeof 123n);                // 'bigint'

// 特殊值检测
console.log(typeof null);               // 'object' (历史 bug)
console.log(typeof function() {});      // 'function'
console.log(typeof {});                 // 'object'
console.log(typeof []);                 // 'object'
console.log(typeof /regex/);            // 'object'

// 变量未声明
console.log(typeof undeclaredVar);      // 'undefined' (不会报错)

// 实际应用
function processValue(value) {
  const type = typeof value;

  switch(type) {
    case 'string':
      return value.toUpperCase();
    case 'number':
      return value.toFixed(2);
    case 'boolean':
      return value ? '是' : '否';
    case 'undefined':
      return '未定义';
    case 'object':
      if (value === null) return '空值';
      if (Array.isArray(value)) return '数组';
      return '对象';
    default:
      return `其他类型: ${type}`;
  }
}
```



```
}  
}
```

41.1.1.2 instanceof 操作符

```
JavaScript  
// 内置对象检测  
console.log([] instanceof Array);           // true  
console.log({} instanceof Object);           // true  
console.log(new Date() instanceof Date);     // true  
console.log(/regex/ instanceof RegExp);      // true  
  
// 函数检测  
function MyFunc() {}  
console.log(new MyFunc() instanceof MyFunc); // true  
console.log(MyFunc instanceof Function);     // true  
  
// 原型链检测  
console.log([] instanceof Object);           // true (Array 继承自  
Object)  
console.log(new Date() instanceof Object);   // true  
  
// 原始值检测  
console.log('hello' instanceof String);      // false (原始值)  
console.log(new String('hello') instanceof String); // true (对象)  
  
// 边界情况  
console.log(null instanceof Object);         // false  
console.log(undefined instanceof Object);    // false  
console.log(123 instanceof Number);          // false  
console.log(new Number(123) instanceof Number); // true
```

深入原理

instanceof 的实现原理

```
JavaScript  
// 手动实现 instanceof  
function myInstanceOf(obj, constructor) {  
  // 基础检查  
  if (obj === null || typeof obj !== 'object') {
```

```

    return false;
}

// 获取对象的原型
let proto = Object.getPrototypeOf(obj);

// 沿着原型链向上查找
while (proto !== null) {
    if (proto === constructor.prototype) {
        return true;
    }
    proto = Object.getPrototypeOf(proto);
}

return false;
}

// 测试
function Animal() {}
function Dog() {}
Dog.prototype = Object.create(Animal.prototype);

const dog = new Dog();
console.log(myInstanceof(dog, Dog));      // true
console.log(myInstanceof(dog, Animal));    // true
console.log(myInstanceof(dog, Object));    // true
console.log(myInstanceof({}, Array));      // false

```

typeof 的特殊行为

```

JavaScript
// 为什么 typeof null === 'object'?
// 历史原因: JavaScript 最初的类型标签系统
/*
在 V8 引擎中, 对象的低 1-3 位存储类型标签:
000: object
001: integer
010: double
100: string
110: boolean
111: undefined/null

```

null 的机器码是全 0 (00000000), 所以 typeof 看到最后 3 位是 000, 就认

为是对象

```
*/

// 检测 null 的正确方法
function isNull(value) {
  return value === null;
}

// 检测 undefined 的正确方法
function isUndefined(value) {
  return value === undefined;
  // 或者 typeof value === 'undefined'
}

// 综合类型检测函数
function getType(value) {
  // 处理 null
  if (value === null) return 'null';

  // 处理其他类型
  const type = typeof value;

  if (type === 'object') {
    // 进一步区分对象类型
    if (Array.isArray(value)) return 'array';
    if (value instanceof Date) return 'date';
    if (value instanceof RegExp) return 'regexp';
    if (value instanceof Map) return 'map';
    if (value instanceof Set) return 'set';
    if (value instanceof Promise) return 'promise';
  }

  return type;
}
```

实际应用场景

场景 1：类型守卫

```
JavaScript
// TypeScript 中的类型守卫
interface Bird {
```

```
fly(): void;
layEggs(): void;
}

interface Fish {
  swim(): void;
  layEggs(): void;
}

function isFish(pet: Fish | Bird): pet is Fish {
  return (pet as Fish).swim !== undefined;
}

// JavaScript 实现
function getPetType(pet) {
  if (typeof pet.swim === 'function') {
    return 'fish';
  } else if (typeof pet.fly === 'function') {
    return 'bird';
  }
  return 'unknown';
}

// 使用 instanceof 的类型守卫
class Circle {
  constructor(radius) {
    this.radius = radius;
  }

  area() {
    return Math.PI * this.radius ** 2;
  }
}

class Square {
  constructor(side) {
    this.side = side;
  }

  area() {
    return this.side ** 2;
  }
}
```

```
function calculateArea(shape) {  
  if (shape instanceof Circle) {  
    return shape.area();  
  } else if (shape instanceof Square) {  
    return shape.area();  
  }  
  throw new Error('未知的形状类型');  
}
```

场景 2: API 参数验证

```
JavaScript  
// 函数参数类型检查  
function processInput(input) {  
  // 使用 typeof 检查原始类型  
  if (typeof input === 'string') {  
    return input.trim();  
  }  
  
  if (typeof input === 'number') {  
    return Math.round(input);  
  }  
  
  // 使用 instanceof 检查对象类型  
  if (input instanceof Date) {  
    return input.toISOString();  
  }  
  
  if (input instanceof Array) {  
    return input.filter(Boolean);  
  }  
  
  // 使用 constructor 检查  
  if (input && input.constructor === Object) {  
    return Object.keys(input);  
  }  
  
  throw new TypeError('不支持的输入类型');  
}  
  
// 更健壮的验证  
function validateConfig(config) {  
  const errors = [];
```

```

// 检查必需字段
if (typeof config.apiUrl !== 'string') {
  errors.push('apiUrl 必须是字符串');
}

if (typeof config.timeout !== 'number' || config.timeout <= 0) {
  errors.push('timeout 必须是正数');
}

if (!(config.retryOptions instanceof Object)) {
  errors.push('retryOptions 必须是对象');
} else {
  if (typeof config.retryOptions.maxRetries !== 'number') {
    errors.push('maxRetries 必须是数字');
  }
}

if (config.headers && !(config.headers instanceof Object)) {
  errors.push('headers 必须是对象');
}

return errors;
}

```

场景 3：工厂模式

```

TypeScript
// 根据类型创建不同对象
class VehicleFactory {
  static create(type, options) {
    switch(type) {
      case 'car':
        return new Car(options);
      case 'bike':
        return new Bike(options);
      case 'truck':
        return new Truck(options);
      default:
        throw new Error(`未知的车辆类型: ${type}`);
    }
  }
}

```



```
static isVehicle(obj) {
  return obj instanceof Car ||
    obj instanceof Bike ||
    obj instanceof Truck;
}

class Car {
  constructor(options) {
    this.type = 'car';
    this.wheels = 4;
    this.color = options.color || 'white';
  }
}

class Bike {
  constructor(options) {
    this.type = 'bike';
    this.wheels = 2;
    this.color = options.color || 'black';
  }
}

// 使用
const car = VehicleFactory.create('car', { color: 'red' });
console.log(VehicleFactory.isVehicle(car)); // true
console.log(car instanceof Car); // true
```

边界情况和陷阱

41.1.1.1 跨框架/iframe 问题

```
JavaScript
// 不同执行环境的 instanceof 问题
const iframe = document.createElement('iframe');
document.body.appendChild(iframe);

const iframeArray = iframe.contentWindow.Array;
const myArray = new iframeArray(1, 2, 3);

console.log(myArray instanceof Array); // false (不同全局对象)
console.log(Array.isArray(myArray)); // true (推荐方式)
console.log(myArray instanceof iframeArray); // true
```

```
// 解决方案: 使用 Symbol.hasInstance
class MyArray {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance);
  }
}

console.log([] instanceof MyArray); // true
console.log(myArray instanceof MyArray); // true
```

41.1.1.2 构造函数被修改

```
JavaScript
// instanceof 依赖构造函数和原型
function Person(name) {
  this.name = name;
}

const person = new Person('John');

console.log(person instanceof Person); // true

// 修改原型
Person.prototype = {};

console.log(person instanceof Person); // false (原型已被修改)

// 修改构造函数
person.constructor = Object;

console.log(person instanceof Person); // true (不影响 instanceof)
console.log(person.constructor === Person); // false
```

41.1.1.3 原始值的包装对象

```
JavaScript
// 注意原始值和包装对象的区别
const str = 'hello';
const strObj = new String('hello');

console.log(typeof str); // 'string'
```

```
console.log(typeof strObj); // 'object'

console.log(str instanceof String); // false
console.log(strObj instanceof String); // true

// 自动装箱 (autoboxing)
console.log(str.toUpperCase()); // 'HELLO'
// JavaScript 临时将原始值包装为 String 对象

// 检测包装对象
function isPrimitive(value) {
  return (
    typeof value !== 'object' &&
    typeof value !== 'function' ||
    value === null
  );
}

function isWrapperObject(value) {
  return (
    typeof value === 'object' &&
    value !== null &&
    (
      value instanceof String ||
      value instanceof Number ||
      value instanceof Boolean
    )
  );
}
```

现代替代方案

Object.prototype.toString.call()

```
JavaScript
// 更精确的类型检测
console.log(Object.prototype.toString.call('hello')); //
'[object String]'
console.log(Object.prototype.toString.call(123)); //
'[object Number]'
console.log(Object.prototype.toString.call(true)); //
'[object Boolean]'
console.log(Object.prototype.toString.call(null)); //
```

```

'[object Null]'
console.log(Object.prototype.toString.call(undefined)); //
'[object Undefined]'
console.log(Object.prototype.toString.call([])); //
'[object Array]'
console.log(Object.prototype.toString.call({})); //
'[object Object]'
console.log(Object.prototype.toString.call(/regex/)); //
'[object RegExp]'
console.log(Object.prototype.toString.call(new Date())); //
'[object Date]'
console.log(Object.prototype.toString.call(() => {})); //
'[object Function]'
console.log(Object.prototype.toString.call(Symbol())); //
'[object Symbol]'
console.log(Object.prototype.toString.call(123n)); //
'[object BigInt]'

// 封装为工具函数
function getExactType(value) {
  return Object.prototype.toString.call(value)
    .slice(8, -1)
    .toLowerCase();
}

console.log(getExactType([])); // 'array'
console.log(getExactType(null)); // 'null'
console.log(getExactType(new Map())); // 'map'

```

ES6+ 新增的检测方法

```

JavaScript
// Array.isArray() - 检测数组
console.log(Array.isArray([])); // true
console.log(Array.isArray({})); // false

// Number.isNaN() - 检测 NaN（比全局 isNaN() 更安全）
console.log(Number.isNaN(NaN)); // true
console.log(Number.isNaN('NaN')); // false
console.log(isNaN('NaN')); // true（全局 isNaN 会强制转换）

// Number.isFinite() - 检测有限数

```

```
console.log(Number.isFinite(123));    // true
console.log(Number.isFinite(Infinity)); // false
console.log(Number.isFinite('123'));  // false

// Number.isInteger() - 检测整数
console.log(Number.isInteger(123));    // true
console.log(Number.isInteger(123.5));  // false

// Object.is() - 精确相等（比较 NaN 和 ±0）
console.log(Object.is(NaN, NaN));      // true
console.log(Object.is(0, -0));         // false
console.log(NaN === NaN);              // false
console.log(0 === -0);                 // true
```

最佳实践总结

```
TypeScript
// 类型检测工具库
const TypeChecker = {
  // 基本类型检测
  isString(value) {
    return typeof value === 'string';
  },

  isNumber(value) {
    return typeof value === 'number' && !isNaN(value);
  },

  isBoolean(value) {
    return typeof value === 'boolean';
  },

  isUndefined(value) {
    return value === undefined;
  },

  isNull(value) {
    return value === null;
  },

  isNullOrUndefined(value) {
    return value == null; // 使用 == 同时检查 null 和 undefined
  },
```

```
// 对象类型检测
isObject(value) {
  return value !== null && typeof value === 'object';
},

isPlainObject(value) {
  return this.isObject(value) &&
    Object.getPrototypeOf(value) === Object.prototype;
},

isArray(value) {
  return Array.isArray(value);
},

isFunction(value) {
  return typeof value === 'function';
},

isDate(value) {
  return value instanceof Date;
},

isRegExp(value) {
  return value instanceof RegExp;
},

// 综合检测
getType(value) {
  if (value === null) return 'null';
  if (value === undefined) return 'undefined';

  const type = typeof value;

  if (type === 'object') {
    if (Array.isArray(value)) return 'array';
    if (value instanceof Date) return 'date';
    if (value instanceof RegExp) return 'regexp';
    if (value instanceof Map) return 'map';
    if (value instanceof Set) return 'set';
    if (value instanceof Promise) return 'promise';
    return 'object';
  }
}
```



```
    return type;
  },

  // 安全检测（跨框架）
  safeInstanceOf(value, constructor) {
    try {
      return value instanceof constructor;
    } catch {
      return false;
    }
  }
};

// 使用示例
console.log(TypeChecker.getType([])); // 'array'
console.log(TypeChecker.isPlainObject({})); // true
console.log(TypeChecker.safeInstanceOf([], Array)); // true
```

42. 请解释一下 BFC 的概念及其应用。

什么是 BFC?

BFC (Block Formatting Context, 块级格式化上下文) 是 Web 页面 CSS 渲染的一部分, 它是一个独立的渲染区域, 规定了内部块级盒子的布局方式, 并且与外部区域隔离。

BFC 的创建条件

一个元素在满足以下任一条件时会创建 BFC:

1. 根元素 (<html>)
2. 浮动元素 (float 值不为 none)
3. 绝对定位元素 (position 为 absolute 或 fixed)
4. display 值为 inline-block、table-cell、table-caption、flex、inline-flex、grid、inline-grid
5. overflow 值不为 visible (hidden、auto、scroll)
6. contain 值为 layout、content 或 paint
7. 多列容器 (column-count 或 column-width 不为 auto)

BFC 的特性

42.1.1.1 内部盒子垂直排列

```
JavaScript
/* BFC 内部，块级盒子从上到下垂直排列 */
.bfc-container {
  overflow: hidden; /* 创建 BFC */
}

.bfc-container > div {
  margin: 20px 0;
}
```

42.1.1.2 外边距折叠 (Margin Collapse)

```
XML
<style>
.normal-container div {
  margin: 20px 0;
}

.bfc-container {
  overflow: hidden; /* 创建 BFC */
}

.bfc-container div {
  margin: 20px 0;
}
</style>

<!-- 正常流：外边距折叠 -->
<div class="normal-container">
  <div>Box 1</div> <!-- 下边距 20px -->
  <div>Box 2</div> <!-- 上边距 20px -->
  <!-- 实际间距为 20px（折叠），而不是 40px -->
</div>

<!-- BFC：外边距不折叠 -->
<div class="bfc-container">
  <div>Box A</div> <!-- 下边距 20px -->
  <div>Box B</div> <!-- 上边距 20px -->
```

```
<!-- 实际间距为 40px（不折叠） -->
</div>
```

42.1.1.3 包含浮动元素

XML

```
<style>
.container {
  border: 2px solid #333;
  padding: 10px;
}

.float-left {
  float: left;
  width: 100px;
  height: 100px;
  background: #f00;
}

.clearfix::after {
  content: '';
  display: block;
  clear: both;
}

/* 创建 BFC 包含浮动 */
.bfc-container {
  overflow: hidden; /* 或 auto */
}
</style>

<!-- 问题：容器高度塌陷 -->
<div class="container">
  <div class="float-left"></div>
  文字内容文字内容文字内容
</div>

<!-- 解决方案 1：传统 clearfix -->
<div class="container clearfix">
  <div class="float-left"></div>
  文字内容文字内容文字内容
</div>
```

```
<!-- 解决方案 2: 创建 BFC -->
<div class="container bfc-container">
  <div class="float-left"></div>
  文字内容文字内容文字内容
</div>
```

42.1.1.4 阻止元素被浮动覆盖

```
XML
<style>
.float-left {
  float: left;
  width: 200px;
  height: 150px;
  background: #f00;
  opacity: 0.5;
}

.text-content {
  background: #0f0;
}

/* 创建 BFC 防止文字环绕 */
.bfc-text {
  overflow: hidden;
}
</style>

<!-- 文字环绕浮动元素 -->
<div class="float-left"></div>
<div class="text-content">
  这里是文字内容，会环绕在浮动元素周围...
</div>

<!-- 创建 BFC 阻止环绕 -->
<div class="float-left"></div>
<div class="text-content bfc-text">
  这里是文字内容，不会环绕浮动元素...
</div>
```

BFC 的实际应用

应用 1：清除浮动

SQL

```
/* 现代清除浮动方案 */
.clearfix {
    display: flow-root; /* 创建 BFC 的新方法 */
}

/* 传统清除浮动方案 */
.old-clearfix::after {
    content: '';
    display: table;
    clear: both;
}

/* 使用 overflow 创建 BFC */
.overflow-clearfix {
    overflow: hidden; /* 或 auto */
}

/* 使用 display 创建 BFC */
.display-clearfix {
    display: inline-block;
    width: 100%; /* 可能需要设置宽度 */
}
```

应用 2：两栏自适应布局

XML

```
<style>
.container {
    width: 100%;
    max-width: 1200px;
    margin: 0 auto;
}

.sidebar {
    float: left;
    width: 200px;
    background: #f0f0f0;
}
```

```

/* 创建 BFC 实现自适应宽度 */
.content {
  overflow: hidden; /* 创建 BFC */
  background: #e0e0e0;
}
</style>

<div class="container">
  <div class="sidebar">
    侧边栏<br>固定宽度 200px
  </div>
  <div class="content">
    主要内容区域<br>自适应剩余宽度<br>不会与浮动元素重叠
  </div>
</div>

```

应用 3：防止外边距穿透

```

XML
<style>
.parent {
  background: #f0f0f0;
  /* 没有 BFC，子元素外边距会穿透 */
}

.parent-bfc {
  background: #e0e0e0;
  overflow: hidden; /* 创建 BFC */
}

.child {
  margin-top: 50px;
  background: #ccc;
  height: 100px;
}
</style>

<!-- 问题：外边距穿透 -->
<div class="parent">
  <div class="child">子元素</div>
</div>

<!-- 解决方案：父元素创建 BFC -->

```



```
<div class="parent-bfc">
  <div class="child">子元素</div>
</div>
```

应用 4：多栏布局隔离

YAML

```
/* 使用 BFC 实现独立的多栏布局 */
.column-container {
  column-count: 3;
  column-gap: 20px;
}

.bfc-item {
  break-inside: avoid; /* 避免分栏中断 */
  overflow: hidden; /* 创建 BFC */
  margin-bottom: 20px;
  background: #f5f5f5;
  padding: 15px;
}
```

现代布局中的 BFC

42.1.1.1 Flex 布局 and BFC

```
JavaScript
/* Flex 容器创建 BFC */
.flex-container {
  display: flex; /* 创建 BFC */
  gap: 20px; /* 间隙，不会折叠 */
}

.flex-item {
  margin: 10px; /* 不会与相邻元素的外边距折叠 */
}
```

42.1.1.2 Grid 布局 and BFC

```
JavaScript
/* Grid 容器创建 BFC */
```

```

.grid-container {
  display: grid; /* 创建 BFC */
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.grid-item {
  overflow: hidden; /* 每个 grid item 也可以创建自己的 BFC */
}

```

42.1.1.3 使用 `display: flow-root`

```

C++
/* 现代创建 BFC 的方法 */
.clean-bfc {
  display: flow-root; /* 专门用于创建 BFC，没有副作用 */
}

/* 对比 overflow: hidden */
.overflow-bfc {
  overflow: hidden; /* 可能隐藏内容 */
}

.overflow-auto-bfc {
  overflow: auto; /* 可能出现滚动条 */
}

/*
flow-root 的优点：
1. 无副作用（不隐藏内容，不产生滚动条）
2. 语义明确（专门用于创建 BFC）
3. 现代浏览器支持
*/

```

BFC 的性能考虑

```

SQL
/* 创建 BFC 可能影响渲染性能 */
/*
需要考虑：
1. 层叠上下文创建（某些 BFC 触发条件会创建新的层叠上下文）

```

```
2. 布局计算（BFC 需要独立计算布局）
3. 内存占用（额外的渲染层）
*/

/* 性能友好的 BFC 创建方式 */
.performance-friendly {
  /* 优先使用这些方式 */
  display: flow-root; /* 最佳选择 */
  contain: layout; /* 现代 API，性能优化 */
}

/* 可能影响性能的方式 */
.performance-costly {
  transform: translateZ(0); /* 创建层叠上下文和硬件加速 */
  opacity: 0.999; /* 微小的变化也可能触发 */
}

/* 需要权衡的方式 */
.trade-off {
  overflow: hidden; /* 可能隐藏内容 */
  position: absolute; /* 脱离文档流 */
  float: left; /* 影响其他元素 */
}
```

实际案例分析

案例 1：弹窗组件

```
YAML
/* 弹窗组件的 BFC 应用 */
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  z-index: 1000;

  /* 创建 BFC 防止外边距穿透 */
  display: flow-root;
```

```

}

.modal-content {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background: white;
  padding: 30px;
  border-radius: 8px;
  max-width: 500px;
  width: 90%;

  /* 创建 BFC 包含内部浮动 */
  overflow: hidden;
}

.modal-header {
  margin-bottom: 20px;
  padding-bottom: 10px;
  border-bottom: 1px solid #eee;

  /* 创建 BFC 防止外边距折叠 */
  display: flow-root;
}

.modal-footer {
  margin-top: 20px;
  padding-top: 10px;
  border-top: 1px solid #eee;
  text-align: right;

  /* 按钮浮动时创建 BFC 包含 */
  overflow: hidden;
}

```

案例 2：卡片列表布局

```

YAML
/* 卡片列表的 BFC 应用 */
.card-list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
}

```

```
gap: 20px;
padding: 20px;
}

.card {
  background: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);

  /* 每个卡片创建独立的 BFC */
  display: flow-root;

  /* 防止外边距穿透到网格间隙 */
  contain: layout;
}

.card-image {
  float: left;
  width: 100px;
  height: 100px;
  margin-right: 15px;
  margin-bottom: 10px;
  border-radius: 4px;
  object-fit: cover;
}

.card-content {
  /* 文字内容创建 BFC 防止环绕 */
  overflow: hidden;
}

.card-title {
  margin: 0 0 10px 0;

  /* 创建 BFC 防止外边距折叠 */
  display: inline-block;
  width: 100%;
}

.card-description {
  margin: 10px 0;
  line-height: 1.5;
  color: #666;
}
```

案例 3：表单布局

YAML

```
/* 表单中的 BFC 应用 */
.form-group {
  margin-bottom: 20px;

  /* 创建 BFC 包含浮动标签 */
  display: flow-root;
}

.form-label {
  float: left;
  width: 120px;
  padding-top: 8px;
  font-weight: bold;
}

.form-control {
  /* 创建 BFC 防止与浮动标签重叠 */
  overflow: hidden;
  padding-left: 130px; /* 为标签留出空间 */
}

.input-wrapper {
  /* 内部创建 BFC 包含输入框和错误信息 */
  display: flow-root;
}

.form-input {
  width: 100%;
  padding: 8px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 14px;
}

.form-error {
  margin-top: 5px;
  color: #e74c3c;
  font-size: 12px;

  /* 创建 BFC 防止外边距折叠 */
}
```



```
display: inline-block;
}
```

BFC 调试技巧

YAML

```
/* 调试 BFC 的样式 */
.debug-bfc {
  /* 添加轮廓显示 BFC 边界 */
  outline: 2px dashed rgba(255, 0, 0, 0.3);
  outline-offset: -1px;
```

```
  /* 添加背景色区分 */
  background-color: rgba(0, 255, 0, 0.05);
}
```

```
.debug-bfc::before {
  content: 'BFC';
  position: absolute;
  top: 2px;
  left: 2px;
  font-size: 10px;
  color: red;
  background: white;
  padding: 1px 3px;
  border: 1px solid red;
  z-index: 1000;
}
```

```
/* 在开发工具中检查 BFC */
/*
```

Chrome DevTools 检查方法:

1. 选中元素
2. 在 Styles 面板查看 computed 样式
3. 检查是否满足 BFC 创建条件

或者使用扩展:

- CSS Peeper
- Pesticide

```
*/
```

浏览器兼容性和回退方案

```
C++
/* 支持现代浏览器和旧浏览器的 BFC 方案 */
.bfc-fallback {
  /* 现代浏览器使用 flow-root */
  display: flow-root;

  /* 旧浏览器回退方案 */
  display: block; /* 确保块级显示 */
}

@supports not (display: flow-root) {
  .bfc-fallback {
    /* 不支持 flow-root 的浏览器使用 overflow */
    overflow: hidden;

    /* 如果需要考虑 IE，添加 zoom: 1 */
    zoom: 1; /* IE 特有，触发 hasLayout（类似 BFC） */
  }
}

/* 针对不同场景的 BFC 创建方案 */
.bfc-adaptive {
  /* 默认使用无副作用的方式 */
  display: flow-root;
}

/* 需要包含浮动时 */
.bfc-contain-float {
  display: flow-root;

  @supports not (display: flow-root) {
    overflow: hidden;
    /* 或者使用 clearfix */
  }
}

/* 需要防止外边距折叠时 */
.bfc-no-collapse {
  display: flow-root;
  padding-top: 0.1px; /* 微小的 padding 也可以防止折叠 */
}
```

```
/* 需要隔离浮动影响时 */
.bfc-isolate-float {
  display: flow-root;

  @supports not (display: flow-root) {
    overflow: auto;
    width: 100%; /* 确保宽度计算正确 */
  }
}
```

总结和最佳实践

SQL

```
/* BFC 最佳实践指南 */
```

```
/*
```

1. 优先使用 `display: flow-root`

- 语义明确，专门用于创建 BFC
- 无副作用，不会隐藏内容或产生滚动条

2. 谨慎使用 `overflow: hidden/auto`

- 可能意外隐藏内容
- 可能产生不必要的滚动条

3. 避免使用浮动创建 BFC

- 浮动有副作用，会影响布局
- 现代布局有更好的替代方案

4. 在需要时创建 BFC:

- 清除内部浮动
- 阻止外边距折叠
- 防止被浮动元素覆盖
- 创建独立的布局上下文

5. 性能考虑:

- 避免不必要的 BFC 创建
- 使用 `contain: layout` 优化性能
- 注意层叠上下文的创建

```
*/
```

```
/* 实用工具类 */
```

```
.util-bfc {
  /* 创建 BFC 的工具类 */
  display: flow-root;
}

.util-clearfix {
  /* 清除浮动的工具类 */
  display: flow-root;
}

.util-no-collapse {
  /* 防止外边距折叠的工具类 */
  display: flow-root;
  padding-top: 0.1px;
}

.util-isolate {
  /* 布局隔离的工具类 */
  display: flow-root;
  contain: layout;
}

/* 响应式 BFC 方案 */
@media (max-width: 768px) {
  .responsive-bfc {
    /* 在小屏幕上可能需要不同的 BFC 策略 */
    overflow: auto; /* 允许滚动查看隐藏内容 */
  }
}
```

43. 什么是 Service Worker? 它有哪些应用场景?

什么是 Service Worker?

Service Worker 是一个在浏览器后台独立运行的脚本，它充当 Web 应用程序与网络之间的代理。它能够在离线时提供缓存资源，实现推送通知，后台同步等功能。

核心特性

43.1.1.1 独立运行环境

```

JavaScript
// Service Worker 运行在独立线程，不能直接访问 DOM
// 但可以通过 postMessage 与主线程通信

// 注册 Service Worker
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js')
    .then(registration => {
      console.log('Service Worker 注册成功:', registration);
    })
    .catch(error => {
      console.log('Service Worker 注册失败:', error);
    });
}

```

43.1.1.2 生命周期

```

JavaScript
// sw.js - Service Worker 文件
// 1. 安装阶段
self.addEventListener('install', event => {
  console.log('Service Worker 安装中...');

  // 等待直到缓存完成
  event.waitUntil(
    caches.open('v1').then(cache => {
      return cache.addAll([
        '/',
        '/index.html',
        '/styles.css',
        '/app.js',
        '/logo.png'
      ]);
    })
  );
});

// 2. 激活阶段
self.addEventListener('activate', event => {
  console.log('Service Worker 激活中...');

  // 清理旧缓存

```

```

event.waitUntil(
  caches.keys().then(cacheNames => {
    return Promise.all(
      cacheNames.map(cacheName => {
        if (cacheName !== 'v1') {
          console.log('删除旧缓存:', cacheName);
          return caches.delete(cacheName);
        }
      })
    );
  })
);

// 3. 拦截请求 (fetch 事件)
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response => {
      // 返回缓存或网络请求
      return response || fetch(event.request);
    })
  );
});

```

应用场景

场景 1：离线应用 (PWA)

```

JavaScript
// 离线优先策略
self.addEventListener('fetch', event => {
  // 只处理 GET 请求
  if (event.request.method !== 'GET') return;

  event.respondWith(
    caches.match(event.request).then(cachedResponse => {
      // 立即返回缓存, 同时更新缓存
      const fetchPromise =
        fetch(event.request).then(networkResponse => {
          // 克隆响应流 (只能使用一次)
          const responseToCache = networkResponse.clone();

```



```

        caches.open('dynamic-cache').then(cache => {
            cache.put(event.request, responseToCache);
        });

        return networkResponse;
    });

    // 返回缓存或网络请求
    return cachedResponse || fetchPromise;
})
);
});

// 网络优先，失败时使用缓存
self.addEventListener('fetch', event => {
    event.respondWith(
        fetch(event.request).catch(() => {
            return caches.match(event.request);
        })
    );
});

```

场景 2：后台同步

```

JavaScript
// 后台同步 API（当网络恢复时执行）
// 主线程中注册同步任务
async function syncData() {
    const registration = await navigator.serviceWorker.ready;

    try {
        await registration.sync.register('sync-notes');
        console.log('后台同步已注册');
    } catch (error) {
        console.log('后台同步注册失败:', error);
    }
}

// Service Worker 中处理同步事件
self.addEventListener('sync', event => {
    if (event.tag === 'sync-notes') {
        console.log('开始后台同步...');
        event.waitUntil(syncNotes());
    }
}

```

```

    }
  });

  async function syncNotes() {
    // 从 IndexedDB 获取待同步的数据
    const notes = await getPendingNotes();

    for (const note of notes) {
      try {
        await fetch('/api/notes', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(note)
        });

        // 同步成功后从待同步列表中移除
        await removePendingNote(note.id);
      } catch (error) {
        console.error('同步失败:', error);
        // 失败的重试逻辑由浏览器自动处理
      }
    }
  }
}

```

场景 3：推送通知

```

JavaScript
// 请求推送权限
async function requestPushPermission() {
  const permission = await Notification.requestPermission();

  if (permission === 'granted') {
    // 获取推送订阅
    const registration = await navigator.serviceWorker.ready;
    const subscription = await
    registration.pushManager.subscribe({
      userVisibleOnly: true,
      applicationServerKey:
    urlBase64ToUint8Array('PUBLIC_VAPID_KEY')
    });

    // 发送订阅信息到服务器
    await sendSubscriptionToServer(subscription);
  }
}

```

```
}  
}  
  
// Service Worker 处理推送事件  
self.addEventListener('push', event => {  
  console.log('收到推送消息:', event);  
  
  const data = event.data.json();  
  
  const options = {  
    body: data.body,  
    icon: '/icon-192.png',  
    badge: '/badge-72.png',  
    vibrate: [200, 100, 200],  
    data: {  
      url: data.url  
    },  
    actions: [  
      {  
        action: 'open',  
        title: '打开'  
      },  
      {  
        action: 'close',  
        title: '关闭'  
      }  
    ]  
  };  
  
  event.waitUntil(  
    self.registration.showNotification(data.title, options)  
  );  
});  
  
// 处理通知点击  
self.addEventListener('notificationclick', event => {  
  event.notification.close();  
  
  if (event.action === 'open') {  
    // 打开相关页面  
    const url = event.notification.data.url;  
    event.waitUntil(  
      clients.openWindow(url)  
    );  
  }  
});
```

```
    } else if (event.action === 'close') {  
      // 关闭通知，不做操作  
    } else {  
      // 默认点击行为  
      event.waitUntil(  
        clients.openWindow('/')  
      );  
    }  
  });  
});
```

场景 4：预缓存和更新

```
JavaScript  
// 版本化缓存管理  
const CACHE_NAME = 'my-app-v1.2.3';  
  
self.addEventListener('install', event => {  
  // 跳过等待，立即激活新版本  
  self.skipWaiting();  
  
  event.waitUntil(  
    caches.open(CACHE_NAME).then(cache => {  
      return cache.addAll([  
        '/',  
        '/index.html',  
        '/app.js?hash=abc123',  
        '/styles.css?hash=def456',  
        '/manifest.json'  
      ]);  
    })  
  );  
});  
  
self.addEventListener('activate', event => {  
  event.waitUntil(  
    Promise.all([  
      // 清理旧版本缓存  
      caches.keys().then(cacheNames => {  
        return Promise.all(  
          cacheNames.map(cacheName => {  
            if (cacheName !== CACHE_NAME) {  
              return caches.delete(cacheName);  
            }  
          })  
        );  
      })  
    ])  
  );  
});
```

```
        })
    );
  }),

  // 立即控制所有客户端
  clients.claim()
  ])
);
});
```

高级用法

动态缓存策略

```
JavaScript
// 根据请求类型和资源类型采用不同缓存策略
self.addEventListener('fetch', event => {
  const request = event.request;
  const url = new URL(request.url);

  // API 请求：网络优先
  if (url.pathname.startsWith('/api/')) {
    event.respondWith(networkFirst(request));
  }
  // 静态资源：缓存优先
  else if (isStaticAsset(url)) {
    event.respondWith(cacheFirst(request));
  }
  // HTML 页面：网络优先
  else if (request.headers.get('Accept').includes('text/html')) {
    event.respondWith(networkFirst(request));
  }
  // 其他：默认策略
  else {
    event.respondWith(cacheFirst(request));
  }
});

async function networkFirst(request) {
  try {
    // 先尝试网络请求
    const response = await fetch(request);
```

```

    // 缓存响应（除了某些 API）
    if (request.method === 'GET'
    && !request.url.includes('/api/auth')) {
        const cache = await caches.open('dynamic-cache');
        cache.put(request, response.clone());
    }

    return response;
} catch (error) {
    // 网络失败，尝试缓存
    const cached = await caches.match(request);
    return cached || new Response('网络不可用', {
        status: 503,
        headers: { 'Content-Type': 'text/plain' }
    });
}
}

async function cacheFirst(request) {
    const cached = await caches.match(request);

    if (cached) {
        // 同时更新缓存（不阻塞返回）
        updateCache(request);
        return cached;
    }

    return fetch(request).then(response => {
        // 缓存新资源
        if (response.ok) {
            caches.open('static-cache').then(cache => {
                cache.put(request, response.clone());
            });
        }
        return response;
    });
}

function isStaticAsset(url) {
    return
    /\. (css|js|png|jpg|jpeg|gif|svg|woff2?|eot|ttf)$/.test(url.pathname);
}

```



```
}
```

消息通信

```
TypeScript
// Service Worker 与页面通信
// 页面发送消息到 Service Worker
navigator.serviceWorker.controller.postMessage({
  type: 'GET_CACHE_INFO'
});

// Service Worker 接收消息
self.addEventListener('message', event => {
  const { type, data } = event.data;

  switch (type) {
    case 'GET_CACHE_INFO':
      getCacheInfo().then(info => {
        event.ports[0].postMessage(info);
      });
      break;

    case 'CLEAR_CACHE':
      clearCache(data.cacheName).then(() => {
        event.ports[0].postMessage({ success: true });
      });
      break;

    case 'PREFETCH':
      prefetchResources(data.urls);
      break;
  }
});

// Service Worker 主动推送消息到页面
async function broadcastToClients(message) {
  const clients = await self.clients.matchAll();
  clients.forEach(client => {
    client.postMessage(message);
  });
}

// 页面接收消息
```

```
navigator.serviceWorker.addEventListener('message', event => {
  console.log('收到 Service Worker 消息:', event.data);

  if (event.data.type === 'CACHE_UPDATED') {
    showUpdateNotification();
  }
});
```

注意事项和最佳实践

43.1.1.1 HTTPS 要求

```
JavaScript
// Service Worker 只能在 HTTPS 或 localhost 下运行
if (window.location.protocol === 'http:' &&
    window.location.hostname !== 'localhost' &&
    window.location.hostname !== '127.0.0.1') {
  console.warn('Service Worker 需要 HTTPS 环境');
}
```

43.1.1.2 缓存策略选择

```
JavaScript
// 根据应用类型选择合适的缓存策略
const CachingStrategies = {
  // 适合静态网站
  CACHE_FIRST: 'cache-first',

  // 适合实时数据
  NETWORK_FIRST: 'network-first',

  // 适合频繁更新的数据
  STALE_WHILE_REVALIDATE: 'stale-while-revalidate',

  // 适合必须最新的数据
  NETWORK_ONLY: 'network-only',

  // 适合完全离线应用
  CACHE_ONLY: 'cache-only'
};
```

```

// 实现 stale-while-revalidate
self.addEventListener('fetch', event => {
  if (event.request.url.includes('/api/data')) {
    event.respondWith(
      caches.open('data-cache').then(cache => {
        return cache.match(event.request).then(cachedResponse => {
          // 立即返回缓存（即使过期）
          const fetchPromise = fetch(event.request).then(response
=> {
            // 更新缓存
            cache.put(event.request, response.clone());
            return response;
          });

          return cachedResponse || fetchPromise;
        });
      })
    );
  }
});

```

43.1.1.3 调试技巧

```

JavaScript
// Service Worker 调试工具
class ServiceWorkerDebugger {
  static async getRegistrations() {
    const registrations = await
navigator.serviceWorker.getRegistrations();
    console.log('已注册的 Service Worker:', registrations);
    return registrations;
  }

  static async unregisterAll() {
    const registrations = await
navigator.serviceWorker.getRegistrations();
    await Promise.all(registrations.map(r => r.unregister()));
    console.log('所有 Service Worker 已注销');
  }

  static async getCacheContents(cacheName) {
    const cache = await caches.open(cacheName);
    const keys = await cache.keys();
  }
}

```

```

const contents = await Promise.all(
  keys.map(async request => ({
    url: request.url,
    timestamp: await getResponseTimestamp(cache, request)
  })))
);
return contents;
}

static async updateServiceWorker() {
  const registration = await navigator.serviceWorker.ready;
  await registration.update();
  console.log('Service Worker 更新检查完成');
}
}

// Chrome DevTools 中的调试
/*
1. 打开 DevTools → Application → Service Workers
2. 查看 Service Worker 状态和日志
3. 可以手动更新、停止、绕过
4. 查看缓存内容
5. 模拟离线状态
*/

```

44. 请解释 `position` 属性的几个值 (`static`, `relative`, `absolute`, `fixed`, `sticky`) 之间的区别。

概述表格

属性值	脱离文档流	定位基准	滚动行为	z-index 生效	常见应用场景
static	否	正常流	正常滚动	否	默认布局
relative	否	自身原位置	正常滚动	是	微调位置，作为 absolute 的父级
absolute	是	最近的定位祖先	随祖先滚动	是	对话框，下拉菜单，图标定位
					固定导航

点击图片可查看完整电子表格

详细解释和示例

44.1.1.1 position: static (默认值)

SQL

```

/* 元素处于正常的文档流中，忽略 top、right、bottom、left 和 z-index */
.static-element {
  position: static; /* 默认值，通常不需要显式设置 */
  top: 20px; /* 无效 */
  left: 30px; /* 无效 */
  z-index: 10; /* 无效 */
}

/* 实际应用：重置定位 */
.reset-position {
  position: static !important; /* 强制元素回到正常流 */
}

```

44.1.1.2 position: relative

XML

```

<style>
.container {
  border: 2px solid #333;
  padding: 20px;
}

```

```

.box {
  width: 100px;
  height: 100px;
  background: lightblue;
  margin: 10px;
}

.relative-box {
  position: relative;
  top: 20px;    /* 向下移动 20px */
  left: 30px;   /* 向右移动 30px */
  background: lightcoral;
  z-index: 1;   /* 有效，可以设置层叠顺序 */
}
</style>

<div class="container">
  <div class="box">Box 1</div>
  <div class="box relative-box">Box 2 (relative)</div>
  <div class="box">Box 3</div>
</div>
<!-- Box 2 相对自己原来位置偏移，但原位置仍然保留 -->

```

44.1.1.3 position: absolute

```

XML
<style>
.container {
  position: relative; /* 为 absolute 子元素提供定位基准 */
  border: 2px solid #333;
  padding: 20px;
  height: 300px;
  margin: 20px 0;
}

.absolute-box {
  position: absolute;
  top: 50px;
  left: 100px;
  width: 150px;
  height: 150px;
  background: lightgreen;
}

```



```

    z-index: 10;
}

/* 如果没有定位祖先，则相对于视口 */
.no-position-ancestor {
    position: absolute;
    top: 0;
    right: 0;
    background: lightpink;
}

/* 绝对定位的实用案例 */
.dropdown {
    position: relative;
    display: inline-block;
}

.dropdown-menu {
    position: absolute;
    top: 100%; /* 在父元素下方 */
    left: 0;
    display: none;
    min-width: 200px;
}

.dropdown:hover .dropdown-menu {
    display: block;
}
</style>

<div class="container">
    <div class="absolute-box">绝对定位元素</div>
    <p>其他内容...</p>
</div>

<div class="no-position-ancestor">
    没有定位祖先，相对于视口
</div>

```

44.1.1.4 position: fixed

```

XML
<style>

```

```
/* 固定导航栏 */
.fixed-header {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  height: 60px;
  background: rgba(255, 255, 255, 0.95);
  z-index: 1000;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

/* 固定底部按钮 */
.floating-button {
  position: fixed;
  bottom: 30px;
  right: 30px;
  width: 60px;
  height: 60px;
  border-radius: 50%;
  background: #007bff;
  color: white;
  border: none;
  box-shadow: 0 4px 12px rgba(0, 123, 255, 0.3);
  z-index: 999;
}

/* 模态框遮罩 */
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  z-index: 2000;
  display: flex;
  align-items: center;
  justify-content: center;
}

.modal-content {
  position: relative;
  background: white;
```

```
padding: 30px;
border-radius: 8px;
max-width: 500px;
width: 90%;
}

/* 注意事项: fixed 在 transform 祖先中会失效 */
.transform-parent {
  transform: translateX(0); /* 会导致 fixed 子元素表现异常 */
}
</style>

<header class="fixed-header">固定导航栏</header>
<main style="padding-top: 80px;">
  <!-- 需要给内容添加 padding, 避免被 fixed 元素遮挡 -->
  <p>页面内容...</p>
</main>
<button class="floating-button">+</button>
```

44.1.1.5 position: sticky

```
XML
<style>
/* 吸顶导航 */
.sticky-nav {
  position: sticky;
  top: 0; /* 触发吸顶的阈值 */
  background: white;
  z-index: 100;
  padding: 15px 0;
  border-bottom: 1px solid #eee;
}

/* 表格吸顶标题 */
.sticky-table thead th {
  position: sticky;
  top: 0;
  background: #f8f9fa;
  z-index: 10;
}

/* 多级 sticky */
.sticky-container {
```

```

    height: 400px;
    overflow: auto;
}

.sticky-header {
    position: sticky;
    top: 0;
    background: #007bff;
    color: white;
    padding: 10px;
}

.sticky-subheader {
    position: sticky;
    top: 40px; /* 第一个 header 的高度 */
    background: #6c757d;
    color: white;
    padding: 8px;
}

/* 底部 sticky */
.sticky-footer {
    position: sticky;
    bottom: 0;
    background: #28a745;
    color: white;
    padding: 10px;
}
</style>

<nav class="sticky-nav">
  <a href="#section1">章节 1</a>
  <a href="#section2">章节 2</a>
</nav>

<div class="sticky-container">
  <div class="sticky-header">主标题</div>
  <div class="sticky-subheader">子标题</div>
  <div style="height: 1000px;">长内容...</div>
  <div class="sticky-footer">底部工具栏</div>
</div>

```

组合使用和高级技巧

44.1.1.1 层叠上下文 (Stacking Context)

YAML

```
/* 创建层叠上下文的常见属性 */
.layer-context {
  /* 以下任一属性都会创建新的层叠上下文 */
  position: relative | absolute | fixed | sticky;
  z-index: auto; /* 但需要 z-index 不为 auto */
  opacity: 0.99; /* 小于 1 */
  transform: translateZ(0);
  will-change: transform;
  filter: blur(0);
  mix-blend-mode: normal;

  /* flex/grid 容器的子项, 且 z-index 不为 auto */
  display: flex;
}

/* 层叠顺序示例 */
.parent {
  position: relative;
  z-index: 1; /* 创建层叠上下文 */
}

.child {
  position: absolute;
  z-index: 100; /* 只在 parent 的层叠上下文中生效 */
}

.other-element {
  position: relative;
  z-index: 2; /* 这会覆盖整个 parent, 即使 child 的 z-index 更大 */
}
```

44.1.1.2 定位与 transform

SQL

```
/* transform 会影响定位基准 */
.container {
  position: relative;
  transform: translateX(50px); /* 这会成为 absolute/fixed 的包含块 */
}
```

```

    /* 注意: fixed 元素会被 transform 祖先影响 */
}

.absolute-child {
    position: absolute;
    top: 0;
    left: 0;
    /* 相对于 transform 后的 container 定位 */
}

/* 修复 fixed 在 transform 中的问题 */
.fixed-inside-transform {
    position: fixed;
    top: 0;
    left: 0;
    /* 如果父元素有 transform, fixed 会失效 */
    /* 解决方案: 将 fixed 元素移到 transform 元素外部 */
}

```

44.1.1.3 实用布局模式

```

YAML
/* 居中定位 (经典方案) */
.center-absolute {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
}

.center-fixed {
    position: fixed;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
}

/* 使用 flex/grid 的现代方案 */
.center-flex {
    display: flex;
    align-items: center;
    justify-content: center;
}

```



```
}

/* 全屏覆盖 */
.overlay {
  position: fixed; /* 或 absolute, 如果相对于某个容器 */
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
}

/* 粘性页脚 */
.page-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.content {
  flex: 1;
}

.footer {
  position: sticky;
  bottom: 0;
}
```

浏览器兼容性

44.1.1.1 sticky 的兼容性处理

```
JavaScript
.sticky-element {
  position: -webkit-sticky; /* Safari 需要前缀 */
  position: sticky;
  top: 0;
}

/* 降级方案: 使用 JavaScript 模拟 */
@supports not (position: sticky) {
  .sticky-element {
    position: relative;
```

```

}

.sticky-element.stuck {
  position: fixed;
  top: 0;
}
}

/* JavaScript 模拟 sticky */
function polyfillSticky() {
  const elements = document.querySelectorAll('.sticky-element');

  elements.forEach(element => {
    const observer = new IntersectionObserver(
      ([entry]) => {
        element.classList.toggle('stuck', !entry.isIntersecting);
      },
      { threshold: [1] }
    );

    observer.observe(element);
  });
}

```

44.1.1.2 fixed 在移动端的特殊处理

```

JavaScript
/* 移动端 fixed 的常见问题 */
.fixed-mobile {
  position: fixed;

  /* iOS Safari 的 viewport 问题 */
  height: 100vh; /* 可能有问题 */
  height: -webkit-fill-available; /* 更好的方案 */

  /* 避免键盘弹出时的问题 */
  bottom: 0;
  /* 键盘弹出时, iOS 会调整 viewport, 导致 fixed 元素异常 */
}

/* 使用 JavaScript 处理移动端键盘 */
window.addEventListener('resize', () => {
  const isKeyboardOpen = window.innerHeight < initialHeight;

```

```
if (isKeyboardOpen) {
    document.querySelector('.fixed-bottom').style.position =
'absolute';
} else {
    document.querySelector('.fixed-bottom').style.position =
'fixed';
}
});
```

性能考虑

JavaScript

/* 定位对性能的影响 */

/* 1. fixed 和 absolute 会创建新的层叠上下文 */

/* 这可能导致重绘和重排，但现代浏览器优化得很好 */

/* 2. 大量使用 fixed/absolute 可能导致性能问题 */

/* 解决方案：使用 transform 和 opacity 进行动画 */

/* 3. sticky 的性能 */

.sticky-performance {

position: sticky;

top: 0;

/* sticky 在滚动时需要持续计算位置 */

/* 避免在大量元素上使用 */

/* 使用 will-change 提示浏览器 */

will-change: transform;

}

/* 4. 硬件加速 */

.hardware-accelerated {

position: relative;

transform: translateZ(0); /* 触发 GPU 加速 */

/* 或者使用 */

will-change: transform;

}

/* 5. 避免布局抖动 */

/* 不要在滚动事件中修改定位属性 */

/* 使用 requestAnimationFrame 和 transform */

```
function smoothScroll() {
  let lastPosition = 0;

  function update() {
    const currentPosition = window.scrollY;
    const diff = currentPosition - lastPosition;

    // 使用 transform 而不是修改 top/left
    element.style.transform = `translateY(${diff}px)`;

    lastPosition = currentPosition;
    requestAnimationFrame(update);
  }

  requestAnimationFrame(update);
}
```

45. 如何实现一个 JSON.parse 函数?

基本思路

实现一个 JSON 解析器需要处理:

1. 字符串解析
2. 数字解析
3. 布尔值和 null
4. 数组解析
5. 对象解析
6. 转义字符处理

简单实现（不考虑完整特性）

```
JavaScript
function myJSONParse(jsonString) {
  let index = 0;
  const length = jsonString.length;

  // 跳过空白字符
  function skipWhitespace() {
    while (index < length && /\s/.test(jsonString[index])) {
```

```
        index++;
    }
}

// 解析值
function parseValue() {
    skipWhitespace();
    const char = jsonString[index];

    if (char === '"') {
        return parseString();
    } else if (char === '{') {
        return parseObject();
    } else if (char === '[') {
        return parseArray();
    } else if (char === 't' && jsonString.substr(index, 4) ===
'true') {
        index += 4;
        return true;
    } else if (char === 'f' && jsonString.substr(index, 5) ===
'false') {
        index += 5;
        return false;
    } else if (char === 'n' && jsonString.substr(index, 4) ===
'null') {
        index += 4;
        return null;
    } else if (char === '-' || (char >= '0' && char <= '9')) {
        return parseNumber();
    } else {
        throw new Error(`Unexpected character: ${char}`);
    }
}

// 解析字符串
function parseString() {
    index++; // 跳过开头的 "
    let result = '';

    while (index < length) {
        const char = jsonString[index];

        if (char === '"') {
            index++; // 跳过结尾的 "
```

```
    return result;
  } else if (char === '\\') {
    // 处理转义字符
    index++;
    const nextChar = jsonString[index];

    switch (nextChar) {
      case '"': result += '"'; break;
      case '\\': result += '\\'; break;
      case '/': result += '/'; break;
      case 'b': result += '\b'; break;
      case 'f': result += '\f'; break;
      case 'n': result += '\n'; break;
      case 'r': result += '\r'; break;
      case 't': result += '\t'; break;
      case 'u':
        // 简化处理：跳过 Unicode 转义
        index += 4;
        result += '?';
        break;
      default:
        result += nextChar;
    }
  } else {
    result += char;
  }

  index++;
}

throw new Error('Unterminated string');
}

// 解析数字
function parseNumber() {
  let start = index;

  // 处理负号
  if (jsonString[index] === '-') {
    index++;
  }

  // 处理整数部分
  while (index < length && jsonString[index] >= '0' &&
```



```
jsonString[index] <= '9') {
    index++;
}

// 处理小数部分
if (jsonString[index] === '.') {
    index++;
    while (index < length && jsonString[index] >= '0' &&
jsonString[index] <= '9') {
        index++;
    }
}

// 处理指数部分
if (jsonString[index] === 'e' || jsonString[index] === 'E') {
    index++;
    if (jsonString[index] === '+' || jsonString[index] === '-') {
        index++;
    }
    while (index < length && jsonString[index] >= '0' &&
jsonString[index] <= '9') {
        index++;
    }
}

const numberStr = jsonString.substring(start, index);
return parseFloat(numberStr);
}

// 解析数组
function parseArray() {
    index++; // 跳过 [
    const result = [];

    skipWhitespace();

    if (jsonString[index] === ']') {
        index++; // 跳过 ]
        return result;
    }

    while (index < length) {
        // 解析数组元素
```

```
const value = parseValue();
result.push(value);

skipWhitespace();

if (jsonString[index] === ']') {
  index++; // 跳过 ]
  return result;
} else if (jsonString[index] === ',') {
  index++; // 跳过逗号
  skipWhitespace();
} else {
  throw new Error(`Expected ',' or ']' in array`);
}

throw new Error('Unterminated array');
}

// 解析对象
function parseObject() {
  index++; // 跳过 {
  const result = {};

  skipWhitespace();

  if (jsonString[index] === '}') {
    index++; // 跳过 }
    return result;
  }

  while (index < length) {
    // 解析键
    skipWhitespace();
    if (jsonString[index] !== '"') {
      throw new Error(`Expected string key in object`);
    }
    const key = parseString();

    // 解析冒号
    skipWhitespace();
    if (jsonString[index] !== ':') {
      throw new Error(`Expected ':' after key in object`);
    }
  }
}
```

```

    index++; // 跳过 :

    // 解析值
    const value = parseValue();
    result[key] = value;

    skipWhitespace();

    if (jsonString[index] === '}') {
        index++; // 跳过 }
        return result;
    } else if (jsonString[index] === ',') {
        index++; // 跳过逗号
        skipWhitespace();
    } else {
        throw new Error(`Expected ',' or '}' in object`);
    }
}

throw new Error('Unterminated object');
}

const result = parseValue();
skipWhitespace();

if (index !== length) {
    throw new Error(`Extra characters after JSON:
${jsonString.substring(index)}`);
}

return result;
}

```

更完整的实现

```

JavaScript
function JSONParser(str) {
    let i = 0;

    // 错误处理
    function syntaxError(message) {
        const error = new SyntaxError(message);
        error.index = i;
    }
}

```

```
    error.input = str;
    throw error;
}

// 当前字符
function ch() {
    return str[i];
}

// 前进并返回字符
function next() {
    return str[++i];
}

// 期待特定字符
function expect(c) {
    if (ch() !== c) {
        syntaxError(`Expected '${c}' but found '${ch()}'`);
    }
    i++;
}

// 跳过空白
function white() {
    while (ch() && ch() <= ' ') i++;
}

// 解析值
function value() {
    white();
    switch (ch()) {
        case '{': return object();
        case '[': return array();
        case '"': return string();
        case '-': return number();
        case 't': return trueVal();
        case 'f': return falseVal();
        case 'n': return nullVal();
        default: return ch() >= '0' && ch() <= '9' ? number() :
            syntaxError('Unexpected token');
    }
}

// 解析对象
```

```
function object() {
  expect('{');
  const obj = {};
  white();

  if (ch() === '}') {
    expect('}');
    return obj;
  }

  while (ch()) {
    const key = string();
    white();
    expect(':');
    obj[key] = value();
    white();

    if (ch() === '}') {
      expect('}');
      return obj;
    }

    expect(',');
    white();
  }

  syntaxError('Unterminated object');
}
```

// 解析数组

```
function array() {
  expect('[');
  const arr = [];
  white();

  if (ch() === ']') {
    expect(']');
    return arr;
  }

  while (ch()) {
    arr.push(value());
    white();
  }
}
```

```

    if (ch() === ']') {
        expect(']');
        return arr;
    }

    expect(',');
    white();
}

syntaxError('Unterminated array');
}

// 解析字符串
function string() {
    expect('');
    let s = '';

    while (ch() && ch() !== '') {
        if (ch() === '\\') {
            next();
            switch (ch()) {
                case '"': s += '"'; break;
                case '\\': s += '\\'; break;
                case '/': s += '/'; break;
                case 'b': s += '\b'; break;
                case 'f': s += '\f'; break;
                case 'n': s += '\n'; break;
                case 'r': s += '\r'; break;
                case 't': s += '\t'; break;
                case 'u':
                    // Unicode 转义
                    const hex = str.substr(i + 1, 4);
                    if (!/^[0-9a-fA-F]{4}$/.test(hex)) {
                        syntaxError('Invalid Unicode escape');
                    }
                    s += String.fromCharCode(parseInt(hex, 16));
                    i += 4;
                    break;
                default: syntaxError('Invalid escape character');
            }
        } else {
            s += ch();
        }
        next();
    }
}

```



```
}

expect('');
return s;
}

// 解析数字
function number() {
  let start = i;

  if (ch() === '-') next();

  if (ch() === '0') {
    next();
  } else if (ch() >= '1' && ch() <= '9') {
    while (ch() >= '0' && ch() <= '9') next();
  } else {
    syntaxError('Invalid number');
  }

  if (ch() === '.') {
    next();
    if (ch() < '0' || ch() > '9') syntaxError('Invalid number');
    while (ch() >= '0' && ch() <= '9') next();
  }

  if (ch() === 'e' || ch() === 'E') {
    next();
    if (ch() === '+' || ch() === '-') next();
    if (ch() < '0' || ch() > '9') syntaxError('Invalid number');
    while (ch() >= '0' && ch() <= '9') next();
  }

  return parseFloat(str.substring(start, i));
}

// 解析 true
function trueVal() {
  expect('t');
  expect('r');
  expect('u');
  expect('e');
  return true;
}
```

```

// 解析 false
function falseVal() {
  expect('f');
  expect('a');
  expect('l');
  expect('s');
  expect('e');
  return false;
}

// 解析 null
function nullVal() {
  expect('n');
  expect('u');
  expect('l');
  expect('l');
  return null;
}

const result = value();
white();

if (i !== str.length) {
  syntaxError('Extra characters after JSON');
}

return result;
}

```

测试用例

```

JavaScript
// 测试函数
function testJSONParser() {
  console.log('=== 测试 JSON 解析器 ===\n');

  const testCases = [
    // 基本类型
    { input: '123', expected: 123 },
    { input: '-456.78', expected: -456.78 },
    { input: '"hello"', expected: 'hello' },
    { input: 'true', expected: true },

```

```
{ input: 'false', expected: false },
{ input: 'null', expected: null },

// 数组
{ input: '[]', expected: [] },
{ input: '[1, 2, 3]', expected: [1, 2, 3] },
{ input: '["a", "b", "c"]', expected: ['a', 'b', 'c'] },
{ input: '[true, false, null]', expected: [true, false,
null] },
{ input: '[[1, 2], [3, 4]]', expected: [[1, 2], [3, 4]] },

// 对象
{ input: '{}', expected: {} },
{ input: '{"name": "John", "age": 30}', expected: { name:
'John', age: 30 } },
{ input: '{"nested": {"key": "value"}}', expected: { nested:
{ key: 'value' } } },

// 混合
{
  input: '{"users": [{"name": "Alice", "active": true},
{"name": "Bob", "active": false}]}' ,
  expected: {
    users: [
      { name: 'Alice', active: true },
      { name: 'Bob', active: false }
    ]
  }
},

// 转义字符
{ input: '"Line1\\nLine2"', expected: 'Line1\nLine2' },
{ input: '"He said, \\\"Hello\\\""' , expected: 'He said,
"Hello"' },

// 空格处理
{ input: ' { "key" : "value" } ' , expected: { key:
'value' } },
];

let passed = 0;
let failed = 0;

testCases.forEach((testCase, index) => {
```

```

    try {
      const result = JSONParser(testCase.input);
      const success = JSON.stringify(result) ===
JSON.stringify(testCase.expected);

      if (success) {
        console.log(`✓ 测试 ${index + 1} 通过:
${testCase.input}`);
        passed++;
      } else {
        console.log(`✗ 测试 ${index + 1} 失败:
${testCase.input}`);
        console.log(`  期望:
${JSON.stringify(testCase.expected)}`);
        console.log(`  实际: ${JSON.stringify(result)}`);
        failed++;
      }
    } catch (error) {
      console.log(`✗ 测试 ${index + 1} 异常: ${testCase.input}`);
      console.log(`  错误: ${error.message}`);
      failed++;
    }
  });

  console.log(`\n=== 测试结果: ${passed} 通过, ${failed} 失败
===`);
}

// 运行测试
testJSONParser();

```

性能优化版本

```

JavaScript
// 使用状态机的 JSON 解析器
function FastJSONParser(str) {
  const tokens = [];
  let i = 0;
  const len = str.length;

  // 词法分析: 将字符串转换为 token 流
  function tokenize() {

```

```
while (i < len) {
  const ch = str[i];

  if (ch <= ' ') {
    i++; // 跳过空白
    continue;
  }

  if (ch === '{' || ch === '}' || ch === '[' || ch === ']' ||
ch === ':' || ch === ',') {
    tokens.push({ type: 'PUNCTUATOR', value: ch });
    i++;
    continue;
  }

  if (ch === '"') {
    // 字符串
    let value = '';
    i++; // 跳过开头的 "

    while (i < len) {
      const c = str[i];

      if (c === '"') {
        i++; // 跳过结尾的 "
        break;
      }

      if (c === '\\') {
        i++;
        const next = str[i];

        switch (next) {
          case '"': value += '"'; break;
          case '\\': value += '\\'; break;
          case '/': value += '/'; break;
          case 'b': value += '\b'; break;
          case 'f': value += '\f'; break;
          case 'n': value += '\n'; break;
          case 'r': value += '\r'; break;
          case 't': value += '\t'; break;
          case 'u':
            const hex = str.substr(i + 1, 4);
            value += String.fromCharCode(parseInt(hex, 16));
```

```

        i += 4;
        break;
      default: value += next;
    }
  } else {
    value += c;
  }

  i++;
}

tokens.push({ type: 'STRING', value });
continue;
}

if (ch === '-' || (ch >= '0' && ch <= '9')) {
  // 数字
  let start = i;

  if (ch === '-') i++;

  while (i < len && str[i] >= '0' && str[i] <= '9') i++;

  if (str[i] === '.') {
    i++;
    while (i < len && str[i] >= '0' && str[i] <= '9') i++;
  }

  if (str[i] === 'e' || str[i] === 'E') {
    i++;
    if (str[i] === '+' || str[i] === '-') i++;
    while (i < len && str[i] >= '0' && str[i] <= '9') i++;
  }

  const numStr = str.substring(start, i);
  tokens.push({ type: 'NUMBER', value:
parseFloat(numStr) });
  continue;
}

// 字面量: true, false, null
if (ch === 't' && str.substr(i, 4) === 'true') {
  tokens.push({ type: 'TRUE', value: true });
  i += 4;

```



```
        continue;
    }

    if (ch === 'f' && str.substr(i, 5) === 'false') {
        tokens.push({ type: 'FALSE', value: false });
        i += 5;
        continue;
    }

    if (ch === 'n' && str.substr(i, 4) === 'null') {
        tokens.push({ type: 'NULL', value: null });
        i += 4;
        continue;
    }

    throw new SyntaxError(`Unexpected character: ${ch}`);
}

// 语法分析: 将 token 流转换为 AST
let tokenIndex = 0;

function parseTokens() {
    const token = tokens[tokenIndex];

    if (token.type === 'STRING') {
        tokenIndex++;
        return token.value;
    }

    if (token.type === 'NUMBER') {
        tokenIndex++;
        return token.value;
    }

    if (token.type === 'TRUE') {
        tokenIndex++;
        return true;
    }

    if (token.type === 'FALSE') {
        tokenIndex++;
        return false;
    }
}
```

```
if (token.type === 'NULL') {
  tokenIndex++;
  return null;
}

if (token.value === '[') {
  tokenIndex++; // 跳过 [
  const arr = [];

  if (tokens[tokenIndex].value === ']') {
    tokenIndex++; // 跳过 ]
    return arr;
  }

  while (tokenIndex < tokens.length) {
    arr.push(parseTokens());

    if (tokens[tokenIndex].value === ']') {
      tokenIndex++; // 跳过 ]
      return arr;
    }

    if (tokens[tokenIndex].value === ',') {
      tokenIndex++; // 跳过逗号
      continue;
    }

    throw new SyntaxError('Expected "," or "]" in array');
  }

  throw new SyntaxError('Unterminated array');
}

if (token.value === '{') {
  tokenIndex++; // 跳过 {
  const obj = {};

  if (tokens[tokenIndex].value === '}') {
    tokenIndex++; // 跳过 }
    return obj;
  }

  while (tokenIndex < tokens.length) {
```

```
// 键必须是字符串
if (tokens[tokenIndex].type !== 'STRING') {
  throw new SyntaxError('Expected string key in object');
}

const key = tokens[tokenIndex].value;
tokenIndex++;

// 必须有冒号
if (tokens[tokenIndex].value !== ':') {
  throw new SyntaxError('Expected ":" after key');
}
tokenIndex++;

// 值
obj[key] = parseTokens();

if (tokens[tokenIndex].value === '}') {
  tokenIndex++; // 跳过 }
  return obj;
}

if (tokens[tokenIndex].value === ',') {
  tokenIndex++; // 跳过逗号
  continue;
}

throw new SyntaxError('Expected ",", or "]" in object');
}

throw new SyntaxError('Unterminated object');
}

throw new SyntaxError(`Unexpected token: ${token.value}`);
}

// 执行解析
tokenize();
const result = parseTokens();

if (tokenIndex !== tokens.length) {
  throw new SyntaxError('Extra tokens after JSON');
}
```

```
return result;
}
```

46. 谈谈你对 HTTP 缓存机制的理解 (强缓存和协商缓存)。

HTTP 缓存概述

HTTP 缓存是一种在客户端或代理服务器上存储资源副本的技术，用于提高网站性能、减少服务器负载和节省带宽。

缓存位置

强缓存 (Strong Cache)

强缓存直接从本地缓存读取资源，不向服务器发送请求。

46.1.1.1 Cache-Control (HTTP/1.1)

YAML

服务器响应头

Cache-Control: max-age=31536000	# 缓存一年
Cache-Control: public	# 允许所有缓存
Cache-Control: private	# 仅允许浏览器缓存
Cache-Control: no-cache	# 强制协商缓存
Cache-Control: no-store	# 不缓存
Cache-Control: immutable	# 资源永不变
Cache-Control: must-revalidate	# 必须验证

46.1.1.2 Expires (HTTP/1.0)

JavaScript

绝对时间，存在时区问题

Expires: Wed, 21 Oct 2025 07:28:00 GMT

协商缓存 (Conditional Cache)

协商缓存会向服务器验证缓存是否有效。

46.1.1.1 Last-Modified / If-Modified-Since

YAML

第一次请求（服务器响应）

HTTP/1.1 200 OK

Last-Modified: Wed, 21 Oct 2023 07:28:00 GMT

Content-Length: 1024

第二次请求（浏览器发送）

GET /resource.js HTTP/1.1

If-Modified-Since: Wed, 21 Oct 2023 07:28:00 GMT

服务器响应（未修改）

HTTP/1.1 304 Not Modified

46.1.1.2 ETag / If-None-Match

YAML

第一次请求（服务器响应）

HTTP/1.1 200 OK

ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4"

Content-Length: 1024

第二次请求（浏览器发送）

GET /resource.js HTTP/1.1

If-None-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4"

服务器响应（未修改）

HTTP/1.1 304 Not Modified

完整缓存策略示例

静态资源缓存策略

Bash

Nginx 配置示例

server {

location ~*

\.(css|js|png|jpg|jpeg|gif|ico|svg|woff2?|eot|ttf)\$ {

开启强缓存

expires 1y;

add_header Cache-Control "public, immutable, max-

```

age=31536000";

    # 添加文件指纹/哈希
    add_header ETag "";

    # 关闭日志以减少磁盘 IO
    access_log off;
}

location ~* \.(html|htm)$ {
    # HTML 文件不缓存或短期缓存
    expires 1h;
    add_header Cache-Control "public, max-age=3600, must-revalidate";
}

location /api/ {
    # API 接口不缓存或短期缓存
    add_header Cache-Control "no-store, no-cache, must-revalidate";
    expires 0;
}
}

```

实际缓存头分析

```

JavaScript
// 检查资源的缓存状态
async function checkCacheStatus(url) {
    const response = await fetch(url, { method: 'HEAD' });
    const headers = response.headers;

    const cacheInfo = {
        url: url,
        cacheControl: headers.get('Cache-Control'),
        expires: headers.get('Expires'),
        etag: headers.get('ETag'),
        lastModified: headers.get('Last-Modified'),
        date: headers.get('Date'),
        age: headers.get('Age')
    };

    // 分析缓存策略

```



```

    if (cacheInfo.cacheControl?.includes('no-store')) {
      cacheInfo.strategy = '不可缓存';
    } else if (cacheInfo.cacheControl?.includes('no-cache')) {
      cacheInfo.strategy = '协商缓存';
    } else if (cacheInfo.cacheControl?.includes('max-age')) {
      const maxAge = cacheInfo.cacheControl.match(/max-age=(\d+)/)?.[1];
      cacheInfo.strategy = `强缓存, 有效期 ${maxAge} 秒`;
    } else if (cacheInfo.expires) {
      cacheInfo.strategy = '强缓存 (Expires)';
    } else {
      cacheInfo.strategy = '默认缓存策略';
    }

    return cacheInfo;
  }

  // 使用
  const cacheStatus = await checkCacheStatus('/static/app.js');
  console.log(cacheStatus);

```

缓存更新策略

46.1.1.1 文件指纹/哈希

XML

```

<!-- 通过修改文件名强制更新 -->
<script src="/app.a1b2c3d4.js"></script>
<link rel="stylesheet" href="/styles.e5f6g7h8.css">

<!-- 或使用查询参数 -->
<script src="/app.js?v=1.2.3"></script>

```

46.1.1.2 Service Worker 缓存控制

JavaScript

```

// Service Worker 中的缓存版本控制
const CACHE_VERSION = 'v2.0.1';
const STATIC_CACHE = `static-${CACHE_VERSION}`;

self.addEventListener('install', event => {

```

```

event.waitUntil(
  caches.open(STATIC_CACHE).then(cache => {
    return cache.addAll([
      '/',
      '/app.a1b2c3d4.js',
      '/styles.e5f6g7h8.css'
    ]);
  })
);
});

self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.map(cacheName => {
          // 删除旧版本的缓存
          if (cacheName.startsWith('static-') && cacheName !==
STATIC_CACHE) {
            return caches.delete(cacheName);
          }
        })
      );
    })
  );
});
});

```

46.1.1.3 Cache API 控制

```

JavaScript
// 手动管理缓存
class CacheManager {
  constructor() {
    this.cacheName = 'my-app-cache';
  }

  async addToCache(request, response) {
    const cache = await caches.open(this.cacheName);
    await cache.put(request, response);
  }

  async getFromCache(request) {
    const cache = await caches.open(this.cacheName);

```

```

    return await cache.match(request);
  }

  async deleteFromCache(request) {
    const cache = await caches.open(this.cacheName);
    return await cache.delete(request);
  }

  async clearCache() {
    await caches.delete(this.cacheName);
  }

  async updateCache(request) {
    try {
      // 获取最新资源
      const response = await fetch(request);

      if (response.ok) {
        // 更新缓存
        await this.addToCache(request, response.clone());
      }

      return response;
    } catch (error) {
      // 网络失败，返回缓存
      return this.getFromCache(request);
    }
  }
}

```

缓存问题与解决方案

46.1.1.1 缓存污染 (Cache Poisoning)

```

Bash
# 防止缓存污染
location / {
  # 验证用户输入
  if ($args ~* "malicious") {
    return 403;
  }
}

```

```

# 设置合适的缓存键
proxy_cache_key "$scheme$request_method$host$request_uri";

# 只缓存安全的响应
proxy_cache_valid 200 302 10m;
proxy_cache_valid 404 1m;

# 不缓存错误响应
proxy_cache_valid 500 502 503 504 0s;
}

```

46.1.1.2 缓存击穿 (Cache Stampede)

```

JavaScript
// 使用互斥锁防止缓存击穿
class CacheWithMutex {
  constructor() {
    this.cache = new Map();
    this.locks = new Map();
  }

  async get(key, fetchData, ttl = 60000) {
    // 检查缓存
    const cached = this.cache.get(key);

    if (cached && Date.now() - cached.timestamp < ttl) {
      return cached.data;
    }

    // 获取锁
    if (!this.locks.has(key)) {
      this.locks.set(key, Promise.resolve());
    }

    const lock = this.locks.get(key);

    return lock.then(async () => {
      try {
        // 再次检查缓存（其他请求可能已经更新）
        const cachedAgain = this.cache.get(key);
        if (cachedAgain && Date.now() - cachedAgain.timestamp <
ttl) {
          return cachedAgain.data;

```

```

    }

    // 获取数据
    const data = await fetchData();

    // 更新缓存
    this.cache.set(key, {
      data,
      timestamp: Date.now()
    });

    return data;
  } finally {
    // 释放锁
    this.locks.delete(key);
  }
});
}
}

```

46.1.1.3 缓存雪崩 (Cache Avalanche)

```

JavaScript
// 随机过期时间防止雪崩
class CacheWithRandomTTL {
  constructor(baseTTL = 60000, jitter = 0.2) {
    this.cache = new Map();
    this.baseTTL = baseTTL;
    this.jitter = jitter; // 20% 的随机波动
  }

  set(key, value) {
    // 计算随机 TTL
    const randomFactor = 1 + (Math.random() * 2 - 1) *
this.jitter;
    const ttl = this.baseTTL * randomFactor;

    this.cache.set(key, {
      value,
      expires: Date.now() + ttl
    });
  }
}

```

```
get(key) {
  const item = this.cache.get(key);

  if (!item) return null;

  if (Date.now() > item.expires) {
    this.cache.delete(key);
    return null;
  }

  return item.value;
}
```

现代缓存技术

46.1.1.1 HTTP/2 服务器推送

```
JavaScript
# Nginx HTTP/2 服务器推送
server {
  listen 443 ssl http2;

  location = /index.html {
    http2_push /styles.css;
    http2_push /app.js;
    http2_push /logo.png;
  }
}
```

46.1.1.2 CDN 缓存

```
TypeScript
// CDN 缓存策略
const CDN_CONFIG = {
  // 静态资源使用 CDN
  static: {
    domain: 'cdn.example.com',
    cacheTTL: 31536000, // 1 年
    headers: {
      'Cache-Control': 'public, max-age=31536000, immutable'
    }
  }
}
```



```

    },

    // 动态内容使用 CDN
    dynamic: {
      domain: 'dynamic-cdn.example.com',
      cacheTTL: 300, // 5 分钟
      headers: {
        'Cache-Control': 'public, max-age=300, s-maxage=300'
      }
    }
  };

  // 资源 URL 生成
  function getCDNUrl(path, type = 'static') {
    const config = CDN_CONFIG[type];
    return `https://${config.domain}${path}`;
  }

```

46.1.1.3 边缘计算缓存

```

JavaScript
// Cloudflare Workers 缓存示例
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request));
});

async function handleRequest(request) {
  const url = new URL(request.url);

  // 检查缓存
  let response = await caches.default.match(request);

  if (!response) {
    // 从源服务器获取
    response = await fetch(request);

    // 缓存静态资源
    if (url.pathname.match(/\.(css|js|png|jpg)$/)) {
      const cacheControl = 'public, max-age=31536000';
      response = new Response(response.body, response);
      response.headers.set('Cache-Control', cacheControl);

      // 存储到缓存
    }
  }
}

```

```
        event.waitUntil(caches.default.put(request,
response.clone()));
    }
}

return response;
}
```

性能监控和优化

46.1.1.1 缓存命中率监控

```
JavaScript
// 监控缓存性能
class CacheMonitor {
  constructor() {
    this.stats = {
      hits: 0,
      misses: 0,
      size: 0
    };
  }

  trackCacheHit() {
    this.stats.hits++;
    this.logStats();
  }

  trackCacheMiss() {
    this.stats.misses++;
    this.logStats();
  }

  getHitRate() {
    const total = this.stats.hits + this.stats.misses;
    return total > 0 ? (this.stats.hits / total) * 100 : 0;
  }

  logStats() {
    console.log(`缓存命中率: ${this.getHitRate().toFixed(2)}%`);
  }
}

// 使用 Performance API 监控
```

```

monitorResourceTiming() {
  const observer = new PerformanceObserver((list) => {
    list.getEntries().forEach(entry => {
      if (entry.transferSize === 0) {
        this.trackCacheHit();
      } else {
        this.trackCacheMiss();
      }
    });
  });

  observer.observe({ entryTypes: ['resource'] });
}
}

```

46.1.1.2 缓存预热

```

JavaScript
// 预缓存重要资源
async function warmUpCache() {
  const criticalResources = [
    '/',
    '/app.js',
    '/styles.css',
    '/api/config',
    '/api/user'
  ];

  const cache = await caches.open('warm-cache');

  // 并行预加载
  await Promise.all(
    criticalResources.map(async resource => {
      try {
        const response = await fetch(resource, {
          // 低优先级获取
          priority: 'low'
        });

        if (response.ok) {
          await cache.put(resource, response);
          console.log(`预缓存: ${resource}`);
        }
      }
    })
  );
}

```

```

    } catch (error) {
      console.warn(`预缓存失败: ${resource}`, error);
    }
  })
);
}

// 在空闲时预热
if ('requestIdleCallback' in window) {
  requestIdleCallback(() => {
    warmUpCache();
  });
}

```

最佳实践总结

```

Java
const CachingBestPractices = {
  1: '为静态资源设置长的强缓存（一年）',
  2: '使用文件指纹或哈希来更新缓存',
  3: 'HTML 文件使用短缓存或协商缓存',
  4: 'API 响应根据数据变化频率设置缓存',
  5: '使用 ETag 比 Last-Modified 更可靠',
  6: '考虑使用 Service Worker 进行细粒度控制',
  7: '监控缓存命中率并调整策略',
  8: '使用 CDN 进行边缘缓存',
  9: '避免缓存敏感数据',
  10: '为不同的资源类型制定不同的缓存策略'
};

// 完整的缓存配置示例
const optimalCacheConfig = {
  '*.html': {
    cacheControl: 'no-cache', // 协商缓存
    expires: '0'
  },
  '*.css': {
    cacheControl: 'public, max-age=31536000, immutable',
    expires: '1y'
  },
  '*.js': {
    cacheControl: 'public, max-age=31536000, immutable',

```

```

    expires: '1y'
  },
  '*.png,jpg,jpeg,gif,svg,ico': {
    cacheControl: 'public, max-age=31536000, immutable',
    expires: '1y'
  },
  '*.woff2,woff,eot,ttf': {
    cacheControl: 'public, max-age=31536000, immutable',
    expires: '1y'
  },
  '/api/*': {
    cacheControl: 'private, max-age=300, must-revalidate',
    expires: '5m'
  },
  '/static/*': {
    cacheControl: 'public, max-age=31536000, immutable',
    expires: '1y'
  }
};

```

47. 请解释 **interface** 和 **type** 在 **TypeScript** 中的区别。

相同点：

- 都可以用来描述对象类型
- 都可以被继承/扩展（interface 使用 `extends`，type 使用 `&`）

主要区别：

特性	interface	type
声明合并	支持，同名interface会自动合并	不支持，同名type会报错
扩展方式	<code>extends</code>	<code>&</code> （交叉类型）
适用范围	主要用于对象类型	可用于任何类型（联合、
性能	在复杂类型检查中稍快	稍慢，特别是在递归类型
可声明类型	对象、函数、类	对象、函数、元组、联合

点击图片可查看完整电子表格

示例：

TypeScript

```
// interface - 支持声明合并
interface User { name: string; }
interface User { age: number; } // 自动合并

// type - 不支持声明合并
type User2 = { name: string; }
// type User2 = { age: number; } // 错误

// type 可以做更多事情
type ID = string | number; // 联合类型
type Point = [number, number]; // 元组
type Callback = () => void; // 函数类型

// 扩展
interface Animal { name: string; }
interface Dog extends Animal { breed: string; }

type Animal2 = { name: string; }
type Dog2 = Animal2 & { breed: string; }
```

选择建议：

- 定义对象类型优先用 `interface`
- 需要联合类型、元组等用 `type`
- 库/框架开发时用 `interface`（支持扩展）

48. 谈谈你对 MVVM (Model-View-ViewModel) 架构模式的理解。

MVVM (Model-View-ViewModel) 是一种前端架构模式：

核心思想：

1. **Model**（模型）：数据层，包含业务数据和业务逻辑
2. **View**（视图）：UI 层，负责显示数据，响应用户交互
3. **ViewModel**（视图模型）：连接层，将 **Model** 转换为 **View** 可用的数据格式，处理 **View** 的交互逻辑

特点：

- 数据双向绑定：View 和 ViewModel 自动同步

- 关注点分离：各层职责清晰
- 可测试性：ViewModel 不依赖 View，便于单元测试

工作流程：

JavaScript

View <--双向绑定--> ViewModel <--数据操作--> Model
 用户交互 命令/事件 数据变化

优点：

1. 代码结构清晰，易于维护
2. 开发效率高（双向绑定减少 DOM 操作）
3. 便于测试

应用框架：

- Vue.js（最典型的 MVVM）
- Angular（MVVM 变种）
- Knockout.js

49. display: none;, visibility: hidden; 和 opacity: 0; 这三者在表现和行为上有什么区别？

特性	display: none	visibility: hidden
是否占据空间	不占空间	占空间
是否可交互	不可交互	不可交互
是否影响子元素	子元素也隐藏	子元素可设置visibility: visible
性能影响	重排（reflow）+ 重绘（repaint）	重绘（repaint）
动画支持	不支持过渡动画	支持visibility的过渡动画
是否在渲染树中	不在	在

点击图片可查看完整电子表格

示例场景：

JavaScript

```
/* 完全移除元素，不占空间 */
.modal { display: none; }
```

```
/* 隐藏但占位，适合需要保持布局的情况 */
.loading { visibility: hidden; }

/* 透明但可点击，适合淡入淡出效果 */
.tooltip { opacity: 0; transition: opacity 0.3s; }
```

50. Generator 函数有什么用途？

Generator（生成器）是 ES6 引入的特殊函数：

基本语法：

```
JavaScript
function* generator() {
  yield 1;
  yield 2;
  return 3;
}
```

主要用途：

1. 惰性求值：按需生成值

```
JavaScript
function* fibonacci() {
  let [prev, curr] = [0, 1];
  while (true) {
    [prev, curr] = [curr, prev + curr];
    yield curr;
  }
}
```

2. 异步编程：在 `async/await` 之前用于异步控制

```
JavaScript
function* fetchData() {
  const user = yield fetch('/api/user');
  const posts = yield fetch('/api/posts');
  return { user, posts };
}
```

```
// 配合自动执行器
function run(generator) {
  const gen = generator();
  function next(data) {
    const result = gen.next(data);
    if (!result.done) {
      result.value.then(data => next(data));
    }
  }
  next();
}
```

3. 状态机：可以暂停和恢复执行

```
JavaScript
function* trafficLight() {
  while (true) {
    yield 'red';
    yield 'yellow';
    yield 'green';
  }
}
```

4. 迭代器生成：简化迭代器实现

```
JavaScript
const obj = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};
```

5. 数据流处理：处理大文件或流数据

```
JavaScript
function* readLines(file) {
  // 逐行读取大文件
}
```

51. 谈谈你对 Proxy 和 Reflect 的理解和应用场

景。

Proxy (代理)

创建对象的代理，用于拦截和自定义基本操作。

基本用法：

```
JavaScript
const target = {};
const handler = {
  get(target, prop) {
    console.log(`Getting ${prop}`);
    return target[prop] || 'default';
  },
  set(target, prop, value) {
    console.log(`Setting ${prop} to ${value}`);
    target[prop] = value;
    return true;
  }
};

const proxy = new Proxy(target, handler);
```

Reflect (反射)

提供与 Proxy handler 方法对应的默认行为。

```
JavaScript
const obj = { x: 1, y: 2 };

// 等价于 obj.x
Reflect.get(obj, 'x');

// 等价于 obj.z = 3
Reflect.set(obj, 'z', 3);
```

应用场景

1. 数据验证和拦截

```
JavaScript
const validator = {
```

```
set(target, prop, value) {
  if (prop === 'age' && !Number.isInteger(value)) {
    throw new TypeError('Age must be an integer');
  }
  return Reflect.set(target, prop, value);
}
};
```

2. 实现观察者模式

```
JavaScript
function observe(target, callback) {
  return new Proxy(target, {
    set(target, prop, value) {
      const oldValue = target[prop];
      const result = Reflect.set(target, prop, value);
      callback(prop, oldValue, value);
      return result;
    }
  });
}
```

3. 负索引数组

```
JavaScript
function createArray(...elements) {
  return new Proxy(elements, {
    get(target, prop) {
      const index = Number(prop);
      if (index < 0) {
        return target[target.length + index];
      }
      return Reflect.get(target, prop);
    }
  });
}
```

4. 函数调用拦截

```
JavaScript
function trace(fn) {
  return new Proxy(fn, {
    apply(target, thisArg, args) {
      console.log(`Calling ${target.name} with`, args);
    }
  });
}
```

```
        return Reflect.apply(target, thisArg, args);
    }
});
}
```

5. 属性访问控制

```
JavaScript
const user = {
  name: 'John',
  _password: 'secret'
};

const protectedUser = new Proxy(user, {
  get(target, prop) {
    if (prop.startsWith('_')) {
      throw new Error('Access denied');
    }
    return Reflect.get(target, prop);
  }
});
```

总结:

- Proxy 用于拦截和自定义对象操作
- Reflect 提供操作对象的默认方法，与 Proxy 配合使用
- 两者结合可以实现强大的元编程能力

52. 什么是代码分割？在 Webpack (或 Vite) 中如何实现？

什么是代码分割 (Code Splitting)

代码分割是一种将代码分割成多个包 (chunks) 的技术，然后可以按需或并行加载这些包。它的主要目的是减少初始加载时间，提高应用性能。

主要目的：

1. 减小初始包体积：将应用拆分为多个小包，减少首次加载时间
2. 按需加载：只在需要时才加载特定代码

- 3. 并行加载：可以同时加载多个包
- 4. 缓存优化：只更新修改的代码块

代码分割的类型

52.1.1 入口点分割 (Entry Points)

```
JavaScript
// webpack.config.js
module.exports = {
  entry: {
    app: './src/app.js',
    admin: './src/admin.js'
  }
};
```

52.1.2 动态导入 (Dynamic Imports)

```
JavaScript
// 使用 import() 语法
const module = await import('./module.js');
```

52.1.3 防止重复 (Prevent Duplication)

```
JavaScript
// 使用 SplitChunksPlugin 提取公共依赖
```

在 Webpack 中实现代码分割

方法一：入口点分割

```
Java
// webpack.config.js
module.exports = {
  entry: {
    main: './src/index.js',
    vendor: './src/vendor.js'
  },
  output: {
```

```
filename: '[name].[contenthash].bundle.js',
path: path.resolve(__dirname, 'dist')
}
};
```

方法二：动态导入（推荐）

JavaScript

```
// 1. 使用 import() 语法
button.addEventListener('click', async () => {
  const { default: module } = await import('./module.js');
  module.doSomething();
});

// 2. React 中的动态导入
const About = React.lazy(() => import('./About'));
const Contact = React.lazy(() => import('./Contact'));
```

方法三：配置 SplitChunksPlugin

YAML

```
// webpack.config.js
module.exports = {
  optimization: {
    splitChunks: {
      chunks: 'all', // async, initial, all
      minSize: 20000, // 最小尺寸, 20KB
      maxSize: 0, // 最大尺寸, 0 表示不限制
      minChunks: 1, // 最少被引用次数
      maxAsyncRequests: 30, // 最大异步请求数
      maxInitialRequests: 30, // 最大初始请求数
      automaticNameDelimiter: '~', // 命名分隔符
      cacheGroups: {
        // 提取第三方库
        vendors: {
          test: /[\\/]node_modules[\\/]/,
          priority: -10,
          name: 'vendors'
        },
        // 提取公共模块
        commons: {
```

```
    minChunks: 2, // 至少被 2 个入口引用
    priority: -20,
    reuseExistingChunk: true
  }
}
}
}
};
```

方法四：魔法注释 (Magic Comments)

```
Java
// 给代码块命名
import(
  /* webpackChunkName: "my-chunk-name" */
  /* webpackPrefetch: true */ // 空闲时预加载
  /* webpackPreload: true */ // 父块加载时并行加载
  './module'
);
```

完整 Webpack 配置示例：

```
JavaScript
// webpack.config.js
const path = require('path');

module.exports = {
  mode: 'production',
  entry: './src/index.js',
  output: {
    filename: '[name].[contenthash].js',
    path: path.resolve(__dirname, 'dist'),
    clean: true
  },
  optimization: {
    moduleIds: 'deterministic',
    runtimeChunk: 'single', // 提取 runtime 代码
    splitChunks: {
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: 'vendors',

```

```
        chunks: 'all'
      }
    }
  }
};
```

在 Vite 中实现代码分割

Vite 基于 Rollup，代码分割是内置功能且配置更简单。

52.1.1 动态导入（与原生 ES 模块一致）

```
JavaScript
// 直接使用动态导入
const module = await import('./module.js');

// React 中使用
const About = React.lazy(() => import('./About'));
```

52.1.2 配置代码分割策略

```
C++
// vite.config.js
import { defineConfig } from 'vite';

export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        // 手动配置代码分割
        manualChunks: {
          // 将 React 相关库打包到一个 chunk
          'react-vendor': ['react', 'react-dom', 'react-router-dom'],

          // 将工具库打包到一个 chunk
          'utils': ['lodash', 'axios', 'dayjs'],

          // 将 UI 库打包到一个 chunk
          'ui-library': ['antd', '@material-ui/core']
        }
      }
    }
  }
});
```

```

    },

    // 自动代码分割策略
    chunkSizeWarningLimit: 500, // 块大小警告限制
    entryFileNames: 'assets/[name]-[hash].js',
    chunkFileNames: 'assets/[name]-[hash].js',
    assetFileNames: 'assets/[name]-[hash].[ext]'
  }
},

// 禁用或启用代码分割
cssCodeSplit: true, // CSS 代码分割

// 分块策略
terserOptions: {
  compress: {
    drop_console: true
  }
}
});

```

52.1.3 自动依赖分割

```

JavaScript
// vite.config.js
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks(id) {
          // 根据模块路径自动分块
          if (id.includes('node_modules')) {
            if (id.includes('react')) {
              return 'react-vendor';
            }
            if (id.includes('lodash')) {
              return 'lodash';
            }
            // 其他 node_modules 中的依赖
            return 'vendor';
          }
        }
      }
    }
  }
});

```

```
    }  
  }  
}  
});
```

最佳实践和优化建议

52.1.1 按路由分割 (React 示例)

```
JavaScript  
// 使用 React.lazy 和 Suspense  
import React, { Suspense, lazy } from 'react';  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
const Home = lazy(() => import('./pages/Home'));  
const About = lazy(() => import('./pages/About'));  
const Contact = lazy(() => import('./pages/Contact'));  
  
function App() {  
  return (  
    <Router>  
      <Suspense fallback={<div>Loading...</div>}>  
        <Routes>  
          <Route path="/" element={<Home />} />  
          <Route path="/about" element={<About />} />  
          <Route path="/contact" element={<Contact />} />  
        </Routes>  
      </Suspense>  
    </Router>  
  );  
}
```

52.1.2 组件级分割

```
JavaScript  
// 按需加载重组件  
const HeavyComponent = lazy(() => import('./HeavyComponent'));  
  
function MyComponent() {  
  const [showHeavy, setShowHeavy] = useState(false);
```



```

return (
  <div>
    <button onClick={() => setShowHeavy(true)}>
      Load Heavy Component
    </button>
    {showHeavy && (
      <Suspense fallback={<div>Loading component...</div>}>
        <HeavyComponent />
      </Suspense>
    )}
  </div>
);
}

```

52.1.3 库的分割策略

```

Java
// 将大型库单独打包
// webpack
{
  vendor: ['react', 'react-dom', 'moment'],
  charts: ['echarts', 'highcharts'],
  utils: ['lodash', 'axios']
}

// vite
manualChunks: {
  'react-vendor': ['react', 'react-dom'],
  'data-viz': ['echarts', 'd3'],
  'utility': ['lodash', 'dayjs', 'axios']
}

```

52.1.4 预加载/预获取优化

```

JavaScript
// 预加载重要模块
import(
  /* webpackPrefetch: true */
  /* webpackPreload: true */
  './critical-module'
);

```

```
// 在适当时间预加载
useEffect(() => {
  // 当用户可能访问下一页时预加载
  const preloadModule = () => import('./next-page');

  // 例如，在鼠标悬停时预加载
  const link = document.getElementById('next-page-link');
  link.addEventListener('mouseenter', preloadModule);

  return () => link.removeEventListener('mouseenter',
    preloadModule);
}, []);
```

性能监控和优化

52.1.1 分析打包结果

```
Python
# Webpack
npx webpack --profile --json > stats.json

# 使用分析工具
npm install webpack-bundle-analyzer --save-dev
# 配置插件
const BundleAnalyzerPlugin = require('webpack-bundle-analyzer').BundleAnalyzerPlugin;

# Vite
npm install rollup-plugin-visualizer
# vite.config.js
import { visualizer } from 'rollup-plugin-visualizer';

export default defineConfig({
  plugins: [visualizer()]
});
```

52.1.2 监控加载性能

```
JavaScript
// 使用 Performance API 监控
```

```
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(`${entry.name}: ${entry.duration}ms`);
  }
});

observer.observe({ entryTypes: ['resource', 'navigation'] });
```

总结对比

特性	Webpack	Vite
配置复杂度	较复杂，需要手动配置SplitChunksPlugin	较简单，Rollup自动处理
开发体验	热更新较慢	基于ESM，热更新极快
构建速度	较慢	极快
代码分割	需要配置	默认支持，配置简单
Tree Shaking	支持	默认支持，更高效

点击图片可查看完整电子表格

建议：

- Webpack：大型、复杂的项目，需要精细控制打包过程
- Vite：现代前端项目，追求开发体验和构建速度

代码分割是现代前端性能优化的核心手段之一，合理使用可以显著提升应用加载速度和用户体验。

53. new 操作符在创建一个实例时具体做了哪些事情？

当使用 new 操作符创建实例时，JavaScript 会执行以下步骤：

详细执行过程

1. 创建一个空对象

JavaScript

```
const obj = {}; // 创建一个全新的空对象
```

2. 设置原型链

将新对象的 `proto` 指向构造函数的 `prototype` 属性：

```
JavaScript  
obj.__proto__ = Constructor.prototype;
```

3. 绑定 `this` 并执行构造函数

将构造函数的作用域赋给新对象（`this` 指向新对象），然后执行构造函数：

```
JavaScript  
const result = Constructor.call(obj, ...args);
```

4. 返回对象

- 如果构造函数显式返回一个对象，则返回该对象
- 否则返回新创建的对象

手动实现 `new` 操作符

```
JavaScript  
function myNew(Constructor, ...args) {  
  // 1. 创建一个新对象，继承构造函数的原型  
  const obj = Object.create(Constructor.prototype);  
  
  // 2. 调用构造函数，绑定 this 到新对象  
  const result = Constructor.apply(obj, args);  
  
  // 3. 判断返回值类型  
  // 如果构造函数返回对象，则返回该对象，否则返回新创建的对象  
  return result instanceof Object ? result : obj;  
}  
  
// 使用示例  
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

```
const p1 = myNew(Person, 'John', 30);
console.log(p1.name); // 'John'
console.log(p1 instanceof Person); // true
```

特殊情况处理

构造函数返回原始值

```
JavaScript
function Person(name) {
  this.name = name;
  return 123; // 返回原始值会被忽略
}
const p = new Person('John');
console.log(p.name); // 'John' - 仍然返回新创建的对象
```

构造函数返回对象

```
JavaScript
function Person(name) {
  this.name = name;
  return { custom: 'object' }; // 返回对象
}
const p = new Person('John');
console.log(p.custom); // 'object' - 返回的是这个对象
console.log(p.name); // undefined - 新创建的对象被丢弃
```

54. 请解释 Flexbox 布局中的 flex-grow, flex-shrink, 和 flex-basis 属性。

这三个属性控制 flex 项目在主轴上的大小和行为。

1. flex-basis

定义项目在分配多余空间之前的初始大小。

```
CSS
.item {
```

```
flex-basis: 200px;    /* 固定值 */
flex-basis: 30%;      /* 百分比 */
flex-basis: auto;     /* 默认值，基于内容大小 */
flex-basis: 0;        /* 零基础尺寸 */
}
```

2. flex-grow

定义项目的放大比例，默认 0（不放大）。

```
CSS
.item {
  flex-grow: 2; /* 当有剩余空间时，该项目将获得其他项目的 2 倍空间 */
}
```

计算规则：

```
CSS
项目最终大小 = flex-basis + (剩余空间 × (flex-grow / 所有项目的 flex-grow 总和))
```

3. flex-shrink

定义项目的缩小比例，默认 1（空间不足时会缩小）。

```
CSS
.item {
  flex-shrink: 0; /* 空间不足时不缩小 */
  flex-shrink: 2; /* 缩小比例是其他项目的 2 倍 */
}
```

计算规则：

```
CSS
溢出空间 = 总空间 - 所有项目的 flex-basis 总和
每个项目收缩比例 = (flex-basis × flex-shrink) / 所有项目的(flex-basis × flex-shrink)总和
项目收缩大小 = 溢出空间 × 每个项目收缩比例
```

综合示例

XML

```
<div class="container">
  <div class="item item1">Item 1</div>
  <div class="item item2">Item 2</div>
  <div class="item item3">Item 3</div>
</div>
```

CSS

```
.container {
  display: flex;
  width: 600px;
  border: 1px solid #333;
}

.item {
  padding: 20px;
  border: 1px solid #ccc;
}

.item1 {
  flex: 1 1 100px; /* flex-grow: 1, flex-shrink: 1, flex-basis: 100px */
  background: lightblue;
}

.item2 {
  flex: 2 1 200px; /* flex-grow: 2, flex-shrink: 1, flex-basis: 200px */
  background: lightgreen;
}

.item3 {
  flex: 0 0 150px; /* flex-grow: 0, flex-shrink: 0, flex-basis: 150px */
  background: lightcoral;
}
```

缩写形式

CSS

```
.item {
  flex: 1; /* 等价于 flex: 1 1 0% */
}
```

```

flex: auto;          /* 等价于 flex: 1 1 auto */
flex: none;          /* 等价于 flex: 0 0 auto */
flex: 2 1;           /* 等价于 flex: 2 1 0% */
flex: 1 100px;       /* 等价于 flex: 1 1 100px */
flex: 2 0 200px;     /* flex-grow: 2, flex-shrink: 0, flex-basis:
200px */
}

```

55. for...in 和 for...of 循环有什么区别？

特性	for...in	for...of
遍历对象	对象的可枚举属性（包括原型链）	可迭代对象的值
返回值	键名（key/index）	键值（value）
适用对象	普通对象、数组、字符串	数组、字符串、Map、Set、N
原型链	会遍历原型链上的可枚举属性	只遍历对象自身的值
遍历顺序	不一定按插入顺序（对象）	按迭代器顺序
性能	相对较慢	相对较快

点击图片可查看完整电子表格

示例对比

for...in

```

JavaScript
const obj = { a: 1, b: 2, c: 3 };

// 遍历对象的可枚举属性
for (let key in obj) {
  console.log(key); // 'a', 'b', 'c'
  console.log(obj[key]); // 1, 2, 3
}

// 数组使用 for...in（不推荐）
const arr = [10, 20, 30];
arr.customProp = '自定义属性';

for (let index in arr) {
  console.log(index); // '0', '1', '2', 'customProp'
}

```

```
console.log(typeof index); // 'string'
}

// 遍历原型链属性
function Person(name) {
  this.name = name;
}
Person.prototype.age = 30;

const person = new Person('John');
for (let key in person) {
  console.log(key); // 'name', 'age' (来自原型)
}
```

for...of

```
JavaScript
// 遍历数组
const arr = [10, 20, 30];
for (let value of arr) {
  console.log(value); // 10, 20, 30
}

// 遍历字符串
const str = 'hello';
for (let char of str) {
  console.log(char); // 'h', 'e', 'l', 'l', 'o'
}

// 遍历 Set
const set = new Set([1, 2, 3]);
for (let value of set) {
  console.log(value); // 1, 2, 3
}

// 遍历 Map
const map = new Map([[ 'a', 1 ], [ 'b', 2 ]]);
for (let [key, value] of map) {
  console.log(key, value); // 'a' 1, 'b' 2
}

// NodeList
const divs = document.querySelectorAll('div');
```

```
for (let div of divs) {  
  console.log(div);  
}
```

注意事项

1. for...in 遍历数组的问题

```
JavaScript  
const arr = [1, 2, 3];  
arr.customProp = 'test';  
  
for (let i in arr) {  
  console.log(i); // '0', '1', '2', 'customProp'  
  console.log(arr[i]); // 1, 2, 3, 'test'  
}  
  
// 解决方法: 使用 hasOwnProperty 过滤  
for (let i in arr) {  
  if (arr.hasOwnProperty(i)) {  
    console.log(arr[i]); // 1, 2, 3  
  }  
}
```

2. 让普通对象可迭代

```
JavaScript  
const obj = { a: 1, b: 2, c: 3 };  
  
// 添加 Symbol.iterator 方法使对象可迭代  
obj[Symbol.iterator] = function* () {  
  for (let key in this) {  
    if (this.hasOwnProperty(key)) {  
      yield this[key];  
    }  
  }  
};  
  
// 现在可以用 for...of 遍历  
for (let value of obj) {  
  console.log(value); // 1, 2, 3
```

```
}
```

56. 如何在 JavaScript 中实现一个 EventEmitter?

基础实现

```
JavaScript
class EventEmitter {
  constructor() {
    this.events = new Map();
  }

  // 监听事件
  on(eventName, listener) {
    if (!this.events.has(eventName)) {
      this.events.set(eventName, []);
    }
    this.events.get(eventName).push(listener);
    return this; // 支持链式调用
  }

  // 触发事件
  emit(eventName, ...args) {
    const listeners = this.events.get(eventName);
    if (listeners && listeners.length) {
      listeners.forEach(listener => {
        try {
          listener.apply(this, args);
        } catch (error) {
          console.error(`Error in listener for event "${eventName}":`, error);
        }
      });
    }
    return true;
  }

  return false;
}

// 移除指定事件的指定监听器
off(eventName, listenerToRemove) {
```

```

    const listeners = this.events.get(eventName);
    if (listeners && listeners.length) {
      this.events.set(
        eventName,
        listeners.filter(listener => listener !==
listenerToRemove)
      );
    }
    return this;
  }

  // 监听一次
  once(eventName, listener) {
    const onceWrapper = (...args) => {
      listener.apply(this, args);
      this.off(eventName, onceWrapper);
    };
    return this.on(eventName, onceWrapper);
  }

  // 移除所有事件或指定事件的所有监听器
  removeAllListeners(eventName) {
    if (eventName) {
      this.events.delete(eventName);
    } else {
      this.events.clear();
    }
    return this;
  }

  // 获取指定事件的监听器数量
  listenerCount(eventName) {
    const listeners = this.events.get(eventName);
    return listeners ? listeners.length : 0;
  }

  // 获取所有事件名称
  eventNames() {
    return Array.from(this.events.keys());
  }
}

```

使用示例


```
JavaScript
const emitter = new EventEmitter();

// 基本使用
function handleEvent(data) {
  console.log('Event received:', data);
}

emitter.on('data', handleEvent);
emitter.emit('data', 'Hello World'); // Event received: Hello
World

// 链式调用
emitter
  .on('start', () => console.log('Started'))
  .on('end', () => console.log('Ended'))
  .emit('start')
  .emit('end');

// 只监听一次
emitter.once('once', () => console.log('This will only fire
once'));
emitter.emit('once'); // This will only fire once
emitter.emit('once'); // Nothing happens

// 移除监听器
function toBeRemoved() {
  console.log('I will be removed');
}
emitter.on('test', toBeRemoved);
emitter.off('test', toBeRemoved);
emitter.emit('test'); // Nothing happens

// 错误处理
emitter.on('errorEvent', () => {
  throw new Error('Listener error');
});
emitter.emit('errorEvent'); // Error in listener for event
"errorEvent": Error: Listener error
```

高级功能实现

JavaScript

```
class AdvancedEventEmitter extends EventEmitter {
  // 最大监听器数量警告
  constructor(maxListeners = 10) {
    super();
    this.maxListeners = maxListeners;
    this.defaultMaxListeners = 10;
  }

  // 设置最大监听器数量
  setMaxListeners(n) {
    this.maxListeners = n;
    return this;
  }

  // 获取最大监听器数量
  getMaxListeners() {
    return this.maxListeners;
  }

  // 重写 on 方法，添加最大监听器检查
  on(eventName, listener) {
    const listeners = this.events.get(eventName) || [];

    // 检查监听器数量限制
    if (listeners.length >= this.maxListeners) {
      console.warn(
        `Possible EventEmitter memory leak detected. ` +
        `${listeners.length} ${eventName} listeners added. ` +
        `Use emitter.setMaxListeners() to increase limit.`
      );
    }

    return super.on(eventName, listener);
  }

  // 添加前置监听器
  prependListener(eventName, listener) {
    if (!this.events.has(eventName)) {
      this.events.set(eventName, []);
    }
    this.events.get(eventName).unshift(listener);
    return this;
  }
}
```

```

}

// 添加前置一次性监听器
prependOnceListener(eventName, listener) {
  const onceWrapper = (...args) => {
    listener.apply(this, args);
    this.off(eventName, onceWrapper);
  };
  return this.prependListener(eventName, onceWrapper);
}

// 获取所有监听器
listeners(eventName) {
  return this.events.get(eventName) || [];
}

// 异步触发事件
async emitAsync(eventName, ...args) {
  const listeners = this.events.get(eventName);
  if (listeners && listeners.length) {
    const promises = listeners.map(listener =>
      Promise.resolve().then(() => listener.apply(this, args))
    );
    await Promise.all(promises);
    return true;
  }
  return false;
}
}

```

Node.js 中的 EventEmitter

```

JavaScript
const EventEmitter = require('events');

class MyEmitter extends EventEmitter {}

const myEmitter = new MyEmitter();

myEmitter.on('event', () => {
  console.log('an event occurred!');
});

```

```
myEmitter.emit('event');
```

57. 谈谈你对 RESTful API 的理解。

什么是 RESTful API?

REST (Representational State Transfer) 是一种软件架构风格, RESTful API 是基于 REST 原则设计的 Web API。

REST 的六个核心约束

1. 客户端-服务器分离 (Client-Server)

- 关注点分离: 前端专注 UI/UX, 后端专注数据处理
- 独立演化: 客户端和服务端可以独立开发和部署

2. 无状态 (Stateless)

- 每个请求都包含处理所需的所有信息
- 服务器不保存客户端状态
- 优点: 可扩展性好, 易于缓存

3. 可缓存 (Cacheable)

- 响应必须明确是否可以缓存
- 减少客户端-服务器交互, 提高性能

4. 统一接口 (Uniform Interface)

- 资源标识: 使用 URI 标识资源
- 资源操作: 通过 HTTP 方法操作资源
- 自描述消息: 消息包含足够的信息说明如何处理
- 超媒体驱动 (HATEOAS): 响应中包含相关资源的链接

5. 分层系统 (Layered System)

- 系统可以分层构建

- 客户端不知道是否直接连接服务器

6. 按需代码 (Code-On-Demand, 可选)

- 服务器可以传输可执行代码

RESTful API 设计原则

1. 资源导向

- 使用名词 (复数) 表示资源
- 使用 HTTP 方法表示操作

GET /users # 获取用户列表

POST /users # 创建新用户

GET /users/123 # 获取 ID 为 123 的用户

PUT /users/123 # 更新用户 123 (全量更新)

PATCH /users/123 # 更新用户 123 (部分更新)

DELETE /users/123 # 删除用户 123

2. 正确的 HTTP 方法使用

方法	幂等性	安全性	描述
GET	是	是	获取资源
POST	否	否	创建资源
PUT	是	否	全量更新资源
PATCH	否	否	部分更新资源
DELETE	是	否	删除资源

点击图片可查看完整电子表格

3. 正确的状态码返回

200 OK # 成功

201 Created # 创建成功

204 No Content # 成功, 无返回内容

400 Bad Request # 请求错误
401 Unauthorized # 未认证
403 Forbidden # 无权限
404 Not Found # 资源不存在
409 Conflict # 资源冲突

500 Internal Server Error # 服务器错误

4. 版本控制

Markdown
URL 版本控制
GET /api/v1/users
GET /api/v2/users

Header 版本控制
GET /api/users
Accept: application/json; version=1.0

参数版本控制
GET /api/users?version=1

5. 过滤、排序、分页

Markdown
过滤
GET /api/users?age=25&gender=male

排序
GET /api/users?sort=name,-created_at

分页
GET /api/users?page=2&limit=20

6. HATEOAS（超媒体驱动）

JSON


```
{
  "id": 123,
  "name": "John Doe",
  "_links": {
    "self": { "href": "/api/users/123" },
    "orders": { "href": "/api/users/123/orders" },
    "friends": { "href": "/api/users/123/friends" }
  }
}
```

完整的 RESTful API 示例

CSS

// 用户资源 API

```
const userRoutes = {
  // 获取用户列表（支持分页、过滤、排序）
  'GET /api/users': {
    description: '获取用户列表',
    query: {
      page: '页码',
      limit: '每页数量',
      sort: '排序字段',
      filter: '过滤条件'
    },
    response: {
      200: {
        data: [{ id: 1, name: 'John' }],
        pagination: {
          total: 100,
          page: 1,
          limit: 10
        },
        _links: {
          self: '/api/users?page=1',
          next: '/api/users?page=2',
          prev: null
        }
      }
    }
  },
  // 创建用户
```

```
'POST /api/users': {
  description: '创建新用户',
  body: {
    name: '用户名',
    email: '邮箱'
  },
  response: {
    201: {
      data: { id: 123, name: 'John' },
      _links: {
        self: '/api/users/123'
      }
    },
    400: { error: '请求参数错误' }
  }
};
```

RESTful API 最佳实践

1. 命名规范

Markdown

使用名词，复数形式

/users	# ✓ 正确
/user	# ✗ 避免
/getUser	# ✗ 避免（动词）

资源关系嵌套不超过两层

/users/{id}/orders	# ✓ 正确
/users/{id}/orders/{oid}/items	# ✗ 避免（太深）

2. 安全考虑

- 使用 HTTPS
- 认证和授权（JWT/OAuth）
- 输入验证和清理
- 速率限制

3. 性能优化

- 启用缓存
- 支持压缩 (gzip)
- 分页和限制返回字段

4. 文档

- 使用 OpenAPI/Swagger
- 提供完整的 API 文档
- 包含示例和错误代码

58. JWT 通常由哪几部分组成？

JWT (JSON Web Token) 结构

JWT 由三部分组成，用点 (.) 分隔：

```
CSS
Header.Payload.Signature
```

格式：

```
CSS
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

1. Header (头部)

包含令牌类型和签名算法。

```
JSON
{
  "alg": "HS256", // 签名算法：HS256、RS256 等
  "typ": "JWT"    // 令牌类型
}
```

Base64Url 编码后:

```
JSON
// 原始
{"alg": "HS256", "typ": "JWT"}
// 编码后
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

2. Payload (载荷)

包含声明 (claims), 有三种类型:

a) 注册声明 (Registered claims)

预定义的标准声明:

```
JSON
{
  "iss": "example.com",           // Issuer - 签发者
  "sub": "1234567890",           // Subject - 主题 (用户 ID)
  "aud": "client_id",            // Audience - 受众
  "exp": 1516239022,              // Expiration Time - 过期时间
  "nbf": 1516238022,              // Not Before - 生效时间
  "iat": 1516237022,              // Issued At - 签发时间
  "jti": "unique_token_id"        // JWT ID - 唯一标识
}
```

b) 公共声明 (Public claims)

可自定义, 但要避免冲突:

```
JSON
{
  "name": "John Doe",
  "email": "john@example.com",
  "roles": ["admin", "user"]
}
```

c) 私有声明 (Private claims)

自定义的声明:

```
CSS
{
  "company": "ABC Inc.",
  "department": "Engineering"
}
```

Base64Url 编码后:

```
JSON
// 原始
{"sub":"1234567890","name":"John Doe","iat":1516239022}
// 编码后
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ
```

3. Signature (签名)

用于验证消息完整性, 防止篡改。

```
CSS
// 签名生成过程
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key
)
```

完整的 JWT 示例

```
JavaScript
const jwt = require('jsonwebtoken');

// 创建 JWT
const token = jwt.sign(
  {
    userId: 123,
    username: 'john_doe',
    roles: ['user', 'admin']
  },
  'your-secret-key',
  {
    expiresIn: '1h', // 1 小时后过期
  }
);
```

```

    issuer: 'my-app',          // 签发者
    audience: 'web-client'     // 受众
  }
);

console.log(token);
// 输出:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJyMywidXNlcm5hbWUiOiJqb2huX2RvZSIsInJvbGVzIjpbInVzZXIiLCJhZG1pbiJdLCJpYXQiOiJlcm5hbWUu...signature

// 验证 JWT
try {
  const decoded = jwt.verify(token, 'your-secret-key');
  console.log(decoded);
} catch (error) {
  console.error('Invalid token:', error.message);
}

```

JWT 在 HTTP 中的使用

```

JavaScript
// 前端：发送请求时携带 JWT
fetch('/api/protected', {
  headers: {
    'Authorization': `Bearer ${jwtToken}`
  }
});

// 后端：验证 JWT
const express = require('express');
const jwt = require('jsonwebtoken');

const app = express();

// 验证中间件
const authenticateJWT = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (authHeader) {
    const token = authHeader.split(' ')[1];

```



```

    jwt.verify(token, 'your-secret-key', (err, user) => {
      if (err) {
        return res.sendStatus(403); // Forbidden
      }

      req.user = user;
      next();
    });
  } else {
    res.sendStatus(401); // Unauthorized
  }
};

// 保护路由
app.get('/api/protected', authenticateJWT, (req, res) => {
  res.json({ message: 'Protected data', user: req.user });
});

```

JWT 的安全考虑

1. 存储位置

```

JavaScript
// 不安全的存储方式
localStorage.setItem('jwt', token); // ✗ XSS 攻击可读取

// 相对安全的存储方式
document.cookie = `jwt=${token}; HttpOnly; Secure; SameSite=Strict`;

```

2. 令牌刷新

```

JSON
// 短期访问令牌 + 长期刷新令牌
{
  "access_token": "eyJ...", // 有效期短（15 分钟）
  "refresh_token": "eyJ...", // 有效期长（7 天）
  "expires_in": 900
}

```

3. 黑名单机制

```
JavaScript
// 实现令牌注销
const tokenBlacklist = new Set();

function logout(token) {
  // 将令牌加入黑名单
  tokenBlacklist.add(token);

  // 设置较短的黑名单过期时间
  setTimeout(() => {
    tokenBlacklist.delete(token);
  }, 15 * 60 * 1000); // 15 分钟
}

// 验证时检查黑名单
function verifyToken(token) {
  if (tokenBlacklist.has(token)) {
    throw new Error('Token revoked');
  }
  return jwt.verify(token, 'secret');
}
```

JWT 的优缺点

优点：

1. 无状态：服务器不需要存储会话信息
2. 可扩展：适用于分布式系统和微服务
3. 自包含：包含所有必要信息
4. 跨域友好：可用于跨域认证

缺点：

1. 无法立即失效：令牌在过期前一直有效
2. 令牌大小：数据越多，令牌越大
3. 安全性依赖：密钥泄露会导致安全问题

59. 浏览器的垃圾回收机制是如何工作的？常见的内存泄漏场景有哪些？

浏览器的垃圾回收机制

主要算法

1. 标记-清除算法 (Mark and Sweep)

这是现代浏览器主要的垃圾回收算法。

工作原理：

```
JavaScript
function markAndSweep() {
  // 1. 标记阶段：从根对象开始标记所有可达对象
  mark(root);

  // 2. 清除阶段：回收未被标记的对象
  sweep();
}

// 简化示例
function simulateMarkAndSweep() {
  const root = { name: 'window' };

  // 创建对象
  const obj1 = { data: 'obj1' };
  const obj2 = { data: 'obj2' };
  const obj3 = { data: 'obj3' };

  // 建立引用关系
  root.ref1 = obj1; // obj1 可达
  obj1.ref2 = obj2; // obj2 通过 obj1 可达

  // obj3 不可达 - 将被回收
  // 这里没有引用指向 obj3

  // 手动断开引用模拟垃圾回收
  root.ref1 = null; // obj1 和 obj2 变为不可达
}
```

2. 引用计数算法 (Reference Counting)

较老的算法，存在循环引用问题。

```
Java
// 引用计数示例
let objA = { data: 'A' };
let objB = { data: 'B' };

objA.ref = objB; // objB 引用计数: 1
objB.ref = objA; // objA 引用计数: 1

// 循环引用问题
objA = null; // objA 引用计数减 1, 但 objB.ref 还引用着, 所以不是 0
objB = null; // objB 引用计数减 1, 但 objA.ref 还引用着, 所以不是 0
// 两者都无法被回收 - 内存泄漏!
```

3. 分代收集 (Generational Collection)

V8 引擎使用的优化策略:

代	描述	回收频率
新生代 (Young Generation)	新创建的对象	频繁
老生代 (Old Generation)	存活时间长的对象	较少
大对象区 (Large Object Space)	大的对象	单独处理

点击图片可查看完整电子表格

垃圾回收工作流程

```
JavaScript
// 模拟垃圾回收过程
class GarbageCollector {
  constructor() {
    this.heap = new Map(); // 模拟堆内存
    this.nextId = 1;
    this.roots = new Set(); // 根引用
  }

  allocate(data) {
    const id = this.nextId++;
    this.heap.set(id, {
```

```
    data,
    marked: false,
    references: new Set()
  });
  return id;
}

addReference(fromId, toId) {
  const obj = this.heap.get(fromId);
  if (obj) {
    obj.references.add(toId);
  }
}

markFromRoots() {
  // 重置标记
  for (const [id, obj] of this.heap) {
    obj.marked = false;
  }

  // 从根开始标记
  const stack = [...this.roots];
  while (stack.length > 0) {
    const id = stack.pop();
    const obj = this.heap.get(id);

    if (obj && !obj.marked) {
      obj.marked = true;
      // 递归标记引用的对象
      for (const refId of obj.references) {
        stack.push(refId);
      }
    }
  }
}

sweep() {
  const toDelete = [];
  for (const [id, obj] of this.heap) {
    if (!obj.marked) {
      toDelete.push(id);
    }
  }
}
```

```
// 删除未标记的对象
for (const id of toDelete) {
  this.heap.delete(id);
  console.log(`回收对象 ${id}`);
}

return toDelete.length;
}

collect() {
  console.log('开始垃圾回收...');
  this.markFromRoots();
  const collected = this.sweep();
  console.log(`回收了 ${collected} 个对象`);
  return collected;
}
}
```

常见的内存泄漏场景

1. 意外的全局变量

```
JavaScript
// 情况 1: 忘记声明变量
function leak1() {
  leak = '意外全局变量'; // 没有 var/let/const
}

// 情况 2: this 指向全局
function leak2() {
  this.leak = '意外全局'; // this 指向 window
}

// 预防: 使用严格模式
'use strict';
function safe() {
  // leak = 'error'; // 严格模式下会报错
}
```

2. 被遗忘的定时器和回调


```

JavaScript
// 泄漏示例
function createTimerLeak() {
  const data = new Array(1000000).fill('*');

  setInterval(() => {
    console.log(data.length); // data 一直无法被回收
  }, 1000);
}

// 正确做法
function createTimer() {
  const data = new Array(1000000).fill('*');
  const timerId = setInterval(() => {
    console.log(data.length);
  }, 1000);

  // 需要时清除
  setTimeout(() => {
    clearInterval(timerId);
    console.log('定时器已清除');
  }, 5000);
}

```

3. DOM 引用

```

JavaScript
// 泄漏示例
const elements = {
  button: document.getElementById('button'),
  image: document.getElementById('image')
};

// 即使从 DOM 中移除，elements 对象仍然引用着
document.body.removeChild(document.getElementById('button'));
// elements.button 仍然存在，无法被回收

// 正确做法
const elements = new WeakMap(); // 使用 WeakMap

function storeElement(id, element) {
  elements.set(id, element);
}

```

```
}  
  
function removeElement(id) {  
  elements.delete(id);  
}
```

4. 闭包

```
JavaScript  
// 泄漏示例  
function createClosureLeak() {  
  const hugeData = new Array(1000000).fill('*');  
  
  return function() {  
    console.log('闭包使用了大量数据');  
    // hugeData 一直被闭包引用，无法回收  
  };  
}  
  
// 优化：及时释放  
function createOptimizedClosure() {  
  const hugeData = new Array(1000000).fill('*');  
  const result = processData(hugeData); // 处理数据  
  
  // 处理完后置为 null  
  hugeData = null;  
  
  return function() {  
    console.log(result); // 只保留需要的结果  
  };  
}
```

5. 事件监听器

```
JavaScript  
// 泄漏示例  
class Component {  
  constructor() {  
    this.data = new Array(100000).fill('*');  
    this.handleClick = this.handleClick.bind(this);  
    document.addEventListener('click', this.handleClick);  
  }  
}
```

```

    }

    handleClick() {
      console.log('点击事件', this.data.length);
    }

    // 忘记移除事件监听器
  }

  // 正确做法
  class SafeComponent {
    constructor() {
      this.data = new Array(100000).fill('*');
      this.handleClick = this.handleClick.bind(this);
      document.addEventListener('click', this.handleClick);
    }

    handleClick() {
      console.log('点击事件', this.data.length);
    }

    destroy() {
      document.removeEventListener('click', this.handleClick);
      this.data = null; // 帮助垃圾回收
    }
  }

```

6. 缓存不当

```

JavaScript
// 泄漏示例：无限增长的缓存
const cache = new Map();

function getExpensiveData(key) {
  if (!cache.has(key)) {
    const data = computeExpensiveData(key);
    cache.set(key, data); // 可能无限增长
  }
  return cache.get(key);
}

// 改进：使用 LRU 缓存

```

```

class LRUCache {
  constructor(maxSize = 100) {
    this.maxSize = maxSize;
    this.cache = new Map();
  }

  get(key) {
    if (!this.cache.has(key)) return null;

    const value = this.cache.get(key);
    // 刷新使用顺序
    this.cache.delete(key);
    this.cache.set(key, value);

    return value;
  }

  set(key, value) {
    if (this.cache.has(key)) {
      this.cache.delete(key);
    }

    this.cache.set(key, value);

    // 如果超出大小，删除最久未使用的
    if (this.cache.size > this.maxSize) {
      const firstKey = this.cache.keys().next().value;
      this.cache.delete(firstKey);
    }
  }
}

```

7. Web API 泄漏

```

JavaScript
// Observer 泄漏
function createObserverLeak() {
  const target = document.createElement('div');
  const observer = new MutationObserver(() => {
    console.log('变化了');
  });

  observer.observe(target, { attributes: true });
}

```

```
// 移除元素但忘记断开观察者
target.remove();
// observer 仍然引用着 target
}

// 正确做法
function createObserver() {
  const target = document.createElement('div');
  const observer = new MutationObserver(() => {
    console.log('变化了');
  });

  observer.observe(target, { attributes: true });

  // 需要时断开连接
  function cleanup() {
    observer.disconnect();
  }

  return { target, cleanup };
}
```

内存泄漏检测和预防

1. 使用开发者工具

```
JavaScript
// Chrome DevTools 内存分析步骤:
// 1. 打开 Performance Monitor
// 2. 使用 Memory 面板的 Heap snapshot
// 3. 使用 Allocation instrumentation on timeline

// 示例: 监控内存使用
function monitorMemory() {
  if (window.performance && performance.memory) {
    const used = performance.memory.usedJSHeapSize;
    const total = performance.memory.totalJSHeapSize;
    const limit = performance.memory.jsHeapSizeLimit;

    console.log(`已使用: ${Math.floor(used / 1024 / 1024).toFixed(2)}MB`);
  }
}
```

```
    console.log(`总内存: ${total / 1024 / 1024}.toFixed(2)}MB`);
    console.log(`限制: ${limit / 1024 / 1024}.toFixed(2)}MB`);
  }
}

// 定期检查
setInterval(monitorMemory, 10000);
```

2. 使用 WeakRef 和 FinalizationRegistry

```
JavaScript
// ES2021 WeakRef 和 FinalizationRegistry
function useWeakReferences() {
  const registry = new FinalizationRegistry((heldValue) => {
    console.log(`${heldValue} 被回收了`);
  });

  let obj = { data: new Array(10000).fill('*') };
  const weakRef = new WeakRef(obj);

  // 注册回调，当 obj 被回收时调用
  registry.register(obj, '大型对象');

  // 手动解除引用
  obj = null;

  // 稍后检查
  setTimeout(() => {
    const deref = weakRef.deref();
    if (deref) {
      console.log('对象还在');
    } else {
      console.log('对象已被回收');
    }
  }, 1000);
}
```

3. 代码规范建议

```
TypeScript
// 内存管理最佳实践
```



```
class MemorySafeApp {
  constructor() {
    this.listeners = new Map();
    this.timers = new Set();
    this.cache = new WeakMap(); // 优先使用 WeakMap
  }

  addEventListener(element, type, handler) {
    element.addEventListener(type, handler);
    const key = `${type}-${Date.now()}`;
    this.listeners.set(key, { element, type, handler });
    return key;
  }

  removeEventListener(key) {
    const { element, type, handler } = this.listeners.get(key) ||
    {};
    if (element && handler) {
      element.removeEventListener(type, handler);
    }
    this.listeners.delete(key);
  }

  addTimer(callback, delay) {
    const id = setTimeout(callback, delay);
    this.timers.add(id);
    return id;
  }

  clearTimer(id) {
    clearTimeout(id);
    this.timers.delete(id);
  }

  cleanup() {
    // 清理所有监听器
    for (const [key] of this.listeners) {
      this.removeEventListener(key);
    }

    // 清理所有定时器
    for (const id of this.timers) {
      this.clearTimer(id);
    }
  }
}
```

```
// 清除缓存
this.cache = new WeakMap();
}
}
```

60. SSR 与 CSR 相比有什么优缺点？

SSR (Server-Side Rendering，服务端渲染)

工作流程：

CSS

客户端请求 → 服务器获取数据 → 服务器渲染 HTML → 返回完整 HTML 页面

CSR (Client-Side Rendering，客户端渲染)

工作流程：

CSS

客户端请求 → 服务器返回空 HTML 框架 → 客户端下载 JS → 客户端获取数据 → 客户端渲染页面

详细对比

维度	SSR	CSR
首次加载速度	✔ 快（返回完整HTML）	✘ 慢（需下载JS）
SEO友好度	✔ 优秀（搜索引擎直接抓取）	✘ 差（需要JS执行）
服务器压力	✘ 高（每次请求都要渲染）	✔ 低（只提供静态资源）
用户体验	⚠ 首屏快，但页面切换可能白屏	✔ 首次慢，后续快
开发复杂度	✘ 高（需考虑同构、hydration等）	✔ 低（传统前端开发）
缓存策略	⚠ 页面级缓存，但数据更新复杂	✔ 灵活的资源缓存
可扩展性	⚠ 需要服务器扩展	✔ 易于扩展（CDN）
技术栈	Next.js, Nuxt.js, Angular Universal	React, Vue, Angular

点击图片可查看完整电子表格

实际应用场景

SSR 适合的场景：

```
JavaScript
// Next.js SSR 示例
export async function getServerSideProps(context) {
  // 服务器端获取数据
  const res = await fetch('https://api.example.com/data');
  const data = await res.json();

  return {
    props: { data }, // 传递给页面组件
  };
}

function Page({ data }) {
  // 服务器渲染时已经有数据
  return (
    <div>
      <h1>SSR 页面</h1>
      <ul>
        {data.items.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    </div>
  );
}
```

CSR 适合的场景：

```
JavaScript
// React CSR 示例
function App() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // 客户端获取数据
    fetch('https://api.example.com/data')
      .then(res => res.json())
      .then(data => {
        setData(data);
      });
  });
}
```

```
        setLoading(false);
    });
}, []);

if (loading) return <div>加载中...</div>;

return (
    <div>
        <h1>CSR 页面</h1>
        { /* 客户端渲染内容 */ }
    </div>
);
}
```

混合方案：SSG + CSR

JavaScript

// Next.js 静态生成 + 客户端渲染

```
export async function getStaticProps() {
    // 构建时获取数据 (SSG)
    const res = await fetch('https://api.example.com/static-data');
    const staticData = await res.json();

    return {
        props: { staticData },
        revalidate: 60, // 60 秒后重新验证
    };
}

function HybridPage({ staticData }) {
    const [dynamicData, setDynamicData] = useState(null);

    useEffect(() => {
        // 客户端获取动态数据 (CSR)
        fetch('https://api.example.com/dynamic-data')
            .then(res => res.json())
            .then(setDynamicData);
    }, []);

    return (
        <div>
            <h1>混合渲染页面</h1>
```

```
    { /* 静态内容 */  
      <div>{staticData.title}</div>  
  
      { /* 动态内容 */  
        {dynamicData && (  
          <div>{dynamicData.content}</div>  
        )}  
      </div>  
    );  
  }  
}
```

性能对比分析

```
Java  
// 性能指标对比  
const performanceComparison = {  
  ssr: {  
    fcp: '1.2s',    // First Contentful Paint  
    lcp: '2.5s',    // Largest Contentful Paint  
    tti: '3.0s',    // Time to Interactive  
    seo: '100/100' // SEO 评分  
  },  
  csr: {  
    fcp: '0.8s',    // 空 HTML 很快  
    lcp: '3.5s',    // 需要等 JS 执行  
    tti: '4.0s',    // 交互需要等 JS 加载  
    seo: '50/100'   // SEO 较差  
  },  
  hybrid: {  
    fcp: '1.0s',  
    lcp: '2.0s',  
    tti: '2.5s',  
    seo: '90/100'  
  }  
};
```

现代框架的解决方案

1. Next.js (React)

JavaScript

```
// 多种渲染策略
export default function Page() {
  return <div>页面内容</div>;
}

// SSG: 静态生成
export async function getStaticProps() { /* ... */ }

// SSR: 服务器渲染
export async function getServerSideProps() { /* ... */ }

// ISR: 增量静态再生
export async function getStaticProps() {
  return {
    props: { /* ... */ },
    revalidate: 10, // 10 秒重新生成
  };
}
```

2. Nuxt.js (Vue)

```
C++
// nuxt.config.js
export default {
  target: 'static', // 'server' 或 'static'
  ssr: true, // 启用 SSR

  // 渲染模式
  render: {
    ssr: true,
    static: {
      // 静态生成配置
    }
  }
};
```

3. 选择建议

```
Java
const renderingStrategy = {
  // 选择 SSR 的情况
```



```
useSSR: [
  '需要 SEO 的页面（博客、电商、内容站）',
  '首屏速度要求高的应用',
  '社交媒体分享频繁的页面',
  '对爬虫友好的需求'
],

// 选择 CSR 的情况
useCSR: [
  '后台管理系统',
  '需要丰富交互的应用',
  '单页面应用（SPA）',
  '移动端 Web 应用'
],

// 选择混合方案的情况
useHybrid: [
  '既有静态内容又有动态交互',
  '电商网站（商品页 SSR，购物车 CSR）',
  '需要部分预渲染的应用',
  '追求最佳性能体验'
]
};
```

61. ARIA 的作用是什么？

ARIA (Accessible Rich Internet Applications)

ARIA 是一组属性，用于增强 Web 应用的可访问性，特别是对于使用辅助技术（屏幕阅读器等）的用户。

核心作用

1. 补充语义信息

```
XML
<!-- 传统 div 没有语义 -->
<div onclick="toggleMenu()">菜单</div>
```

```
<!-- 使用 ARIA 添加语义 -->
<div
  role="button"
  aria-label="打开菜单"
  tabindex="0"
  onclick="toggleMenu()"
  onkeypress="handleKeyPress(event)"
>
  菜单
</div>
```

2. 描述元素状态

```
XML
<!-- 自定义复选框 -->
<div
  role="checkbox"
  aria-checked="false"
  aria-labelledby="checkbox-label"
  tabindex="0"
  onclick="toggleCheckbox()"
>
  <span id="checkbox-label">接受条款</span>
</div>
```

3. 提供实时通知

```
XML
<!-- 动态内容更新 -->
<div
  aria-live="polite"
  aria-atomic="true"
>
  新消息数量: <span id="message-count">3</span>
</div>

<!-- 紧急通知 -->
<div
  aria-live="assertive"
  role="alert"
>
```

错误：保存失败！

</div>

主要 ARIA 属性分类

1. 角色 (Roles)

XML

```
<!-- 地标角色 -->
<nav role="navigation">
  <!-- 导航内容 -->
</nav>

<main role="main">
  <!-- 主要内容 -->
</main>

<aside role="complementary">
  <!-- 补充内容 -->
</aside>

<!-- 小部件角色 -->
<div role="button">按钮</div>
<div role="checkbox">复选框</div>
<div role="slider">滑块</div>
<div role="tablist">
  <div role="tab">标签 1</div>
  <div role="tabpanel">内容 1</div>
</div>

<!-- 文档结构角色 -->
<div role="heading" aria-level="2">二级标题</div>
<div role="list">
  <div role="listitem">列表项</div>
</div>
```

2. 属性 (Attributes)

XML

```
<!-- 标签和描述 -->
```

```
<button aria-label="关闭对话框">X</button>

<div aria-labelledby="title-id">
  <h2 id="title-id">对话框标题</h2>
</div>

<div aria-describedby="desc-id">
  <p id="desc-id">详细描述信息</p>
</div>

<!-- 状态和属性 -->
<input
  type="text"
  aria-required="true"
  aria-invalid="false"
  aria-describedby="error-message"
>

<div
  role="progressbar"
  aria-valuenow="75"
  aria-valuemin="0"
  aria-valuemax="100"
>
  75%
</div>

<!-- 关系 -->
<div role="menu">
  <div role="menuitem" aria-haspopup="true">文件</div>
  <div role="menu" aria-labelledby="file-menu">
    <div role="menuitem">新建</div>
  </div>
</div>
```

实际应用示例

示例 1：自定义下拉菜单

```
XML
<div class="custom-select">
  <button
```

```
id="select-button"
aria-haspopup="listbox"
aria-expanded="false"
aria-controls="select-list"
>
  选择选项
</button>

<ul
  id="select-list"
  role="listbox"
  aria-labelledby="select-button"
  tabindex="-1"
  style="display: none;"
>
  <li
    role="option"
    aria-selected="true"
    tabindex="0"
  >
    选项 1
  </li>
  <li
    role="option"
    aria-selected="false"
    tabindex="0"
  >
    选项 2
  </li>
</ul>
</div>

<script>
document.getElementById('select-button').addEventListener('click',
function() {
  const list = document.getElementById('select-list');
  const isExpanded = this.getAttribute('aria-expanded') ===
'true';

  list.style.display = isExpanded ? 'none' : 'block';
  this.setAttribute('aria-expanded', !isExpanded);
});
</script>
```

示例 2：模态对话框

XML

```
<button
  aria-haspopup="dialog"
  aria-controls="modal-dialog"
  onclick="openModal()"
>
  打开对话框
</button>
```

```
<div
  id="modal-dialog"
  role="dialog"
  aria-modal="true"
  aria-labelledby="modal-title"
  aria-describedby="modal-description"
  style="display: none;"
>
  <h2 id="modal-title">对话框标题</h2>

  <p id="modal-description">
    这是一个重要的对话框，包含了一些信息。
  </p>
```

```
  <button onclick="closeModal()">关闭</button>
</div>
```

```
<script>
```

```
let previousActiveElement;
```

```
function openModal() {
  const modal = document.getElementById('modal-dialog');
  modal.style.display = 'block';
```

```
  // 保存当前焦点元素
```

```
  previousActiveElement = document.activeElement;
```

```
  // 将焦点移动到对话框
```

```
  modal.focus();
```

```
  // 阻止背景内容被访问
```

```
  document.querySelector('main').setAttribute('aria-hidden',
```



```

'true');
}

function closeModal() {
  const modal = document.getElementById('modal-dialog');
  modal.style.display = 'none';

  // 恢复背景内容
  document.querySelector('main').removeAttribute('aria-hidden');

  // 恢复焦点
  if (previousActiveElement) {
    previousActiveElement.focus();
  }
}
</script>

```

示例 3：表单验证

```

XML
<form id="signup-form">
  <div>
    <label for="email">邮箱</label>
    <input
      type="email"
      id="email"
      aria-required="true"
      aria-describedby="email-error"
      aria-invalid="false"
    >
    <div
      id="email-error"
      role="alert"
      aria-live="polite"
      style="color: red; display: none;"
    ></div>
  </div>

  <button type="submit">注册</button>
</form>

<script>
document.getElementById('signup-form').addEventListener('submit',

```

```
function(e) {
  e.preventDefault();

  const emailInput = document.getElementById('email');
  const errorElement = document.getElementById('email-error');

  if (!emailInput.value.includes('@')) {
    // 显示错误
    emailInput.setAttribute('aria-invalid', 'true');
    errorElement.textContent = '请输入有效的邮箱地址';
    errorElement.style.display = 'block';

    // 焦点移动到错误字段
    emailInput.focus();
  } else {
    // 清除错误
    emailInput.setAttribute('aria-invalid', 'false');
    errorElement.style.display = 'none';

    // 提交表单
    console.log('表单提交成功');
  }
});
</script>
```

ARIA 使用规则

规则 1：优先使用原生 HTML 语义

XML

<!-- ✖ 不推荐：用 div 模拟按钮 -->

<div role="button" onclick="handleClick()">点击</div>

<!-- ✔ 推荐：使用原生 button -->

<button onclick="handleClick()">点击</button>

规则 2：确保可交互元素可访问

XML

<!-- ✖ 不可访问 -->

```
<div class="button" onclick="submit()">提交</div>

<!-- ✔ 可访问 -->
<div
  role="button"
  tabindex="0"
  onclick="submit()"
  onkeypress="(e) => e.key === 'Enter' && submit()"
>
  提交
</div>
```

规则 3：管理焦点

```
JavaScript
class AccessibleComponent {
  constructor(element) {
    this.element = element;
    this.trapFocus = false;
  }

  // 焦点管理
  manageFocus() {
    // 获取所有可聚焦元素
    const focusableElements = this.element.querySelectorAll(
      'button, [href], input, select, textarea,
      [tabindex]:not([tabindex="-1"])'
    );

    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length
- 1];

    // 焦点捕获
    if (this.trapFocus) {
      firstElement.addEventListener('keydown', (e) => {
        if (e.key === 'Tab' && e.shiftKey) {
          e.preventDefault();
          lastElement.focus();
        }
      });
    }

    lastElement.addEventListener('keydown', (e) => {
```

```
        if (e.key === 'Tab' && !e.shiftKey) {
            e.preventDefault();
            firstElement.focus();
        }
    });
}
}
```

测试和验证工具

JavaScript

// 自动化测试示例

```
function testAccessibility() {
    // 1. 检查图片的 alt 属性
    const images = document.querySelectorAll('img');
    images.forEach(img => {
        if (!img.alt && !img.getAttribute('aria-label')) {
            console.warn('图片缺少 alt 或 aria-label:', img);
        }
    });

    // 2. 检查表单标签
    const inputs = document.querySelectorAll('input, textarea, select');
    inputs.forEach(input => {
        if (!input.id && !input.getAttribute('aria-label')) {
            console.warn('表元素缺少关联标签:', input);
        }
    });

    // 3. 检查 ARIA 属性有效性
    const ariaElements = document.querySelectorAll('[role]');
    ariaElements.forEach(el => {
        const role = el.getAttribute('role');
        const requiredAttrs = getRequiredAriaAttributes(role);

        requiredAttrs.forEach(attr => {
            if (!el.hasAttribute(attr)) {
                console.warn(`元素 role="${role}"缺少必要属性${attr}:`, el);
            }
        });
    });
}
```

```

    });
}

// 使用 axe-core 进行测试
if (typeof axe !== 'undefined') {
  axe.run(document, {
    rules: {
      'color-contrast': { enabled: true },
      'image-alt': { enabled: true },
      'button-name': { enabled: true },
      'link-name': { enabled: true }
    }
  }, (err, results) => {
    if (err) throw err;
    console.log('无障碍测试结果:', results);
  });
}

```

62. CSS-in-JS 方案与传统的 CSS 预处理器相比，有什么不同？

核心概念对比

传统 CSS 预处理器（Sass/Less/Stylus）

```

CSS
// styles.scss - 基于文件的作用域
$primary-color: #1890ff;
$font-size: 14px;

.container {
  padding: 20px;
  background: $primary-color;

  .title {
    font-size: $font-size * 1.5;
    color: white;
  }

  &:hover {

```

```
background: darken($primary-color, 10%);
}
}
```

CSS-in-JS (Styled-components/Emotion)

```
CSS
// styles.scss - 基于文件的作用域
$primary-color: #1890ff;
$font-size: 14px;

.container {
  padding: 20px;
  background: $primary-color;

  .title {
    font-size: $font-size * 1.5;
    color: white;
  }

  &:hover {
    background: darken($primary-color, 10%);
  }
}
```

详细对比分析

1. 作用域机制

方案	作用域方式	样式冲突风险
传统CSS	全局作用域，需手动管理	高，容易冲突
CSS预处理器	全局作用域，但支持嵌套	中等，依赖命名约定
CSS-in-JS	组件级别作用域	极低，自动生成唯一类名

点击图片可查看完整电子表格

```
JavaScript
// CSS Modules (介于两者之间)
// Button.module.css
.button {
  background: blue;
}
```



```

}

// Button.js
import styles from './Button.module.css';
function Button() {
  return <button className={styles.button}>按钮</button>;
}
// 编译为: <button class="Button_button_1a2b3c">按钮</button>

```

2. 动态样式能力

```

CSS
// CSS-in-JS: 运行时动态样式
const Button = styled.button`
  background: ${props =>
    props.primary ? '#1890ff' :
    props.danger ? '#ff4d4f' :
    '#f0f0f0'}
  };
  color: ${props => props.primary ? 'white' : '#333'};
  padding: ${props => props.size === 'large' ? '12px 24px' : '8px
16px'};
  opacity: ${props => props.disabled ? 0.5 : 1};
  cursor: ${props => props.disabled ? 'not-allowed' : 'pointer'};

  &:hover {
    transform: ${props => !props.disabled && 'translateY(-2px)'};
  }
`;

// 传统 CSS: 静态样式, 需要多个类名
// button.css
.button { padding: 8px 16px; }
.button-primary { background: #1890ff; color: white; }
.button-danger { background: #ff4d4f; color: white; }
.button-disabled { opacity: 0.5; cursor: not-allowed; }
.button-large { padding: 12px 24px; }

```

3. 主题和变量管理

```

CSS
// CSS-in-JS: 动态主题切换

```

```

import { ThemeProvider } from 'styled-components';

const theme = {
  colors: {
    primary: '#1890ff',
    secondary: '#52c41a',
    danger: '#ff4d4f'
  },
  spacing: {
    small: '8px',
    medium: '16px',
    large: '24px'
  }
};

const Button = styled.button`
  background: ${props => props.theme.colors.primary};
  padding: ${props => props.theme.spacing.medium};
  color: white;
`;

function App() {
  const [darkMode, setDarkMode] = useState(false);

  return (
    <ThemeProvider theme={darkMode ? darkTheme : lightTheme}>
      <Button>切换主题</Button>
    </ThemeProvider>
  );
}

// 传统方案: CSS 变量
// theme.css
:root {
  --color-primary: #1890ff;
  --color-secondary: #52c41a;
  --spacing-medium: 16px;
}

[data-theme="dark"] {
  --color-primary: #177ddc;
}

.button {

```

```
background: var(--color-primary);
padding: var(--spacing-medium);
}
```

4. 性能影响

方面	CSS-in-JS	传统CSS
运行时开销	有（解析、注入样式）	无
包大小	较大（包含运行时库）	较小
样式切换	快（动态更新）	需要加载新CSS文件
关键CSS提取	复杂，需要额外配置	简单，已有成熟方案

点击图片可查看完整电子表格

```
JavaScript
// CSS-in-JS 性能优化
import { css, keyframes } from '@emotion/react';
import styled from '@emotion/styled';

// 1. 使用 css prop 代替 styled-component
const styles = css`
  color: hotpink;
  &:hover {
    color: red;
  }
`;

// 2. 提取关键样式
const CriticalStyles = () => (
  <Global
    styles={css`
      body { margin: 0; }
      .above-the-fold { display: block; }
    `}
  />
);

// 3. 服务端渲染优化
export async function getServerSideProps() {
  const { extractCritical } = require('@emotion/server');
  const { html, css } = extractCritical(renderToString(<App />));
```

```
return {
  props: {
    html,
    css
  }
};
}
```

5. 开发体验

TypeScript

// CSS-in-JS: 类型安全 (TypeScript)

```
interface ButtonProps {
  primary?: boolean;
  size?: 'small' | 'medium' | 'large';
  disabled?: boolean;
}
```

```
const Button = styled.button<ButtonProps>`
  background: ${props => props.primary ? 'blue' : 'gray'};
  padding: ${props => {
    switch (props.size) {
      case 'small': return '4px 8px';
      case 'large': return '12px 24px';
      default: return '8px 16px';
    }
  }};
  cursor: ${props => props.disabled ? 'not-allowed' : 'pointer'};

  // TypeScript 会检查 props 是否正确使用
`;
```

// 传统 CSS: 需要单独文件, 无类型检查
// 容易拼写错误, 难以重构

现代解决方案对比

流行库比较

JavaScript

// 1. Styled-components (最流行)

```

import styled from 'styled-components';
const Button = styled.button`color: palevioletred;`;

// 2. Emotion (性能更好)
import { css } from '@emotion/react';
<div css={css`color: hotpink;`} />;

// 3. Stitches (零运行时)
import { styled } from '@stitches/react';
const Button = styled('button', {
  backgroundColor: 'gainsboro'
});

// 4. Linaria (零运行时, 编译时提取)
import { css } from '@linaria/core';
const title = css`color: blue;`;

// 5. Vanilla Extract (类型安全, 编译时)
import { style } from '@vanilla-extract/css';
export const button = style({
  backgroundColor: 'blue'
});

```

性能基准对比

```

CSS
const performanceComparison = {
  // 运行时性能
  runtime: {
    styledComponents: {
      bundleSize: '15.7kB',
      renderTime: '45ms',
      memoryUsage: '中等'
    },
    emotion: {
      bundleSize: '11.5kB',
      renderTime: '38ms',
      memoryUsage: '较低'
    },
    stitches: {
      bundleSize: '6.2kB',
      renderTime: '22ms',

```

```

        memoryUsage: '低'
      },
      vanillaExtract: {
        bundleSize: '0kB',
        renderTime: '18ms',
        memoryUsage: '很低'
      }
    },

    // 开发者体验
    developerExperience: {
      styledComponents: {
        learningCurve: '低',
        typescriptSupport: '好',
        debugging: '好'
      },
      emotion: {
        learningCurve: '中等',
        typescriptSupport: '优秀',
        debugging: '优秀'
      }
    }
  }
};

```

选择建议

```

CSS
const selectionGuide = {
  // 选择 CSS-in-JS 的场景
  chooseCSSinJS: {
    reason: [
      '项目重度依赖 JavaScript 运行时逻辑',
      '需要高度动态的样式（如主题切换）',
      '组件库开发，需要样式隔离',
      '团队熟悉 JavaScript 生态'
    ],
    recommendation: [
      '大型应用: Emotion',
      '设计系统: Stitches/Vanilla Extract',
      '快速原型: Styled-components'
    ]
  }
};

```



```

    },

    // 选择传统 CSS 的场景
    chooseTraditionalCSS: {
      reason: [
        '对性能要求极高，需要最小化 JS 包',
        '项目中有大量静态页面',
        '需要良好的 SEO 和首次加载速度',
        '团队成员 CSS 技能强'
      ],
      recommendation: [
        '搭配 CSS Modules',
        '使用 PostCSS 处理现代 CSS',
        'CSS 变量管理主题'
      ]
    },

    // 混合方案
    hybridApproach: {
      description: '根据页面类型使用不同方案',
      examples: [
        '营销页面: 传统 CSS + Tailwind',
        '应用界面: CSS-in-JS',
        '组件库: Vanilla Extract + CSS-in-JS'
      ]
    }
  }
};

```

实际项目配置示例

```

JavaScript
// 现代 React 项目配置
// package.json
{
  "dependencies": {
    "@emotion/react": "^11.0.0",
    "@emotion/styled": "^11.0.0",
    "postcss": "^8.0.0",
    "autoprefixer": "^10.0.0"
  }
}

```

```
// vite.config.js (Vite 项目)
export default {
  css: {
    // 对传统 CSS 使用 PostCSS
    postcss: {
      plugins: [
        require('autoprefixer'),
        require('postcss-preset-env')
      ]
    },

    // CSS-in-JS 由相应库处理
    modules: {
      localsConvention: 'camelCase'
    }
  },

  // 优化构建
  build: {
    cssCodeSplit: true,
    rollupOptions: {
      output: {
        manualChunks: {
          'emotion': ['@emotion/react', '@emotion/styled']
        }
      }
    }
  }
};
```

63. 请实现一个函数 `flatten`，将一个嵌套的多维数组拍平成一维数组。

方案 1：递归实现（基础版）

```
JavaScript
function flatten(arr) {
  const result = [];

  function flattenHelper(array) {
    for (let i = 0; i < array.length; i++) {
```

```

    if (Array.isArray(array[i])) {
      flattenHelper(array[i]);
    } else {
      result.push(array[i]);
    }
  }
}

flattenHelper(arr);
return result;
}

// 简化版递归
function flattenSimple(arr) {
  return arr.reduce((result, item) => {
    return result.concat(Array.isArray(item) ?
flattenSimple(item) : item);
  }, []);
}

```

方案 2：迭代实现（使用栈）

```

C++
function flattenIterative(arr) {
  const result = [];
  const stack = [...arr];

  while (stack.length) {
    const item = stack.pop();

    if (Array.isArray(item)) {
      // 将数组元素推入栈中
      stack.push(...item);
    } else {
      result.push(item);
    }
  }

  // 由于使用 pop，顺序会反转，需要反转回来
  return result.reverse();
}

// 保持顺序的迭代版本

```

```
function flattenIterativeOrdered(arr) {
  const result = [];
  const stack = [...arr];

  while (stack.length) {
    const item = stack.shift(); // 使用 shift 保持顺序

    if (Array.isArray(item)) {
      stack.unshift(...item); // 在开头插入
    } else {
      result.push(item);
    }
  }

  return result;
}
```

方案 3: Generator 实现

```
JavaScript
function* flattenGenerator(arr) {
  for (const item of arr) {
    if (Array.isArray(item)) {
      yield* flattenGenerator(item); // 递归 yield
    } else {
      yield item;
    }
  }
}

// 使用 Generator
function flattenUsingGenerator(arr) {
  return [...flattenGenerator(arr)];
}

// 异步版本的 Generator
async function* asyncFlattenGenerator(arr) {
  for (const item of arr) {
    if (Array.isArray(item)) {
      yield* asyncFlattenGenerator(item);
    } else if (item && typeof item.then === 'function') {
      // 处理 Promise
      yield await item;
    }
  }
}
```

```
    } else {  
      yield item;  
    }  
  }  
}
```

方案 4：使用 ES2019 flat 方法

JavaScript

```
// 现代浏览器/Node.js 内置方法  
function flattenUsingFlat(arr) {  
  // Infinity 表示无限深度拍平  
  return arr.flat(Infinity);  
}  
  
// 自定义深度拍平  
function flattenDepth(arr, depth = 1) {  
  if (depth <= 0) return arr;  
  
  return arr.reduce((result, item) => {  
    if (Array.isArray(item) && depth > 0) {  
      return result.concat(flattenDepth(item, depth - 1));  
    }  
    return result.concat(item);  
  }, []);  
}
```

方案 5：TypeScript 类型安全版本

TypeScript

```
// TypeScript 实现，支持类型推断  
type Flatten<T extends any[]> = T extends (infer U)[]  
  ? U extends any[]  
    ? Flatten<U>  
    : U  
  : never;  
  
function flattenTS<T extends any[]>(arr: T): Flatten<T>[] {  
  const result: any[] = [];  
  
  function flattenHelper(array: any[]): void {  
    for (const item of array) {
```

```

        if (Array.isArray(item)) {
            flattenHelper(item);
        } else {
            result.push(item);
        }
    }
}

flattenHelper(arr);
return result as Flatten<T>[];
}

```

// 使用示例

```

const nested = [1, [2, [3, 4], 5], 6];
const flattened = flattenTS(nested);
// flattened 类型为 number[]

```

方案 6：性能优化版本

C++

// 避免递归调用栈溢出（尾递归优化）

```

function flattenOptimized(arr) {
    const result = [];
    const stack = [{ array: arr, index: 0 }];

    while (stack.length) {
        const current = stack[stack.length - 1];

        if (current.index >= current.array.length) {
            stack.pop();
            continue;
        }

        const item = current.array[current.index++];

        if (Array.isArray(item)) {
            stack.push({ array: item, index: 0 });
        } else {
            result.push(item);
        }
    }

    return result;
}

```



```
}
```

功能增强版本

版本 1：支持自定义处理函数

```
JavaScript
function flattenWithCallback(arr, callback = x => x) {
  const result = [];

  function flattenHelper(array) {
    for (const item of array) {
      if (Array.isArray(item)) {
        flattenHelper(item);
      } else {
        result.push(callback(item));
      }
    }
  }

  flattenHelper(arr);
  return result;
}

// 使用示例
const arr = [1, [2, [3, 4]]];
const flattened = flattenWithCallback(arr, x => x * 2);
// 输出: [2, 4, 6, 8]
```

版本 2：支持过滤和去重

```
JavaScript
function flattenWithOptions(arr, options = {}) {
  const {
    depth = Infinity,
    filter = null,
    unique = false
  } = options;

  const result = [];
  const seen = new Set();
```

```

function flattenHelper(array, currentDepth) {
  if (currentDepth >= depth) {
    result.push(...array);
    return;
  }

  for (const item of array) {
    if (Array.isArray(item)) {
      flattenHelper(item, currentDepth + 1);
    } else {
      // 应用过滤函数
      if (filter && !filter(item)) continue;

      // 去重处理
      if (unique) {
        const key = JSON.stringify(item);
        if (seen.has(key)) continue;
        seen.add(key);
      }

      result.push(item);
    }
  }
}

flattenHelper(arr, 0);
return result;
}

// 使用示例
const complexArr = [1, [2, [3, 1]], 4, [5, [6]]];
const result = flattenWithOptions(complexArr, {
  depth: 2,
  filter: x => x > 2,
  unique: true
});
// 输出: [3, 4, 5, 6] (深度 2, 过滤>2, 去重)

```

版本 3：异步拍平

```

JavaScript
async function asyncFlatten(arr) {

```

```

const result = [];

async function flattenHelper(array) {
  for (const item of array) {
    if (Array.isArray(item)) {
      await flattenHelper(item);
    } else if (item && typeof item.then === 'function') {
      // 等待 Promise 解析
      const resolved = await item;
      if (Array.isArray(resolved)) {
        await flattenHelper(resolved);
      } else {
        result.push(resolved);
      }
    } else {
      result.push(item);
    }
  }
}

await flattenHelper(arr);
return result;
}

// 使用示例
async function example() {
  const asyncArr = [
    1,
    Promise.resolve([2, Promise.resolve(3)]),
    [4, Promise.resolve(5)]
  ];

  const result = await asyncFlatten(asyncArr);
  console.log(result); // [1, 2, 3, 4, 5]
}

```

版本 4：流式处理（大数据集）

```

JavaScript
function* flattenStream(arr) {
  const stack = [arr[Symbol.iterator]()];

  while (stack.length) {

```

```

const current = stack[stack.length - 1];
const { value, done } = current.next();

if (done) {
  stack.pop();
  continue;
}

if (Array.isArray(value)) {
  stack.push(value[Symbol.iterator]());
} else {
  yield value;
}
}
}

// 使用示例（处理大数据集）
function processLargeArray() {
  // 模拟大数据集
  const largeArray = new Array(1000000).fill(0).map((_, i) =>
    i % 100 === 0 ? new Array(10).fill(i) : i
  );

  let count = 0;
  for (const item of flattenStream(largeArray)) {
    // 逐个处理，避免内存爆炸
    processItem(item);
    count++;

    if (count % 10000 === 0) {
      console.log(`已处理 ${count} 个元素`);
    }
  }
}

function processItem(item) {
  // 处理每个元素
}

```

测试用例

JavaScript

```
// 测试函数
function testFlatten() {
  const testCases = [
    {
      input: [1, [2, [3, 4], 5], 6],
      expected: [1, 2, 3, 4, 5, 6],
      name: '基本嵌套数组'
    },
    {
      input: [],
      expected: [],
      name: '空数组'
    },
    {
      input: [[[]], [[]]],
      expected: [],
      name: '嵌套空数组'
    },
    {
      input: [1, 'a', true, null, undefined],
      expected: [1, 'a', true, null, undefined],
      name: '混合类型'
    },
    {
      input: [[1], [[2]], [[[3]]]],
      expected: [1, 2, 3],
      name: '多层嵌套'
    },
    {
      input: [{ a: 1 }, [2, 3]],
      expected: [{ a: 1 }, 2, 3],
      name: '包含对象'
    }
  ];

  // 测试所有实现
  const implementations = [
    { name: '递归', fn: flattenSimple },
    { name: '迭代', fn: flattenIterativeOrdered },
    { name: 'Generator', fn: flattenUsingGenerator },
    { name: 'flat 方法', fn: flattenUsingFlat }
  ];
}
```

```

for (const impl of implementations) {
  console.log(`\n 测试实现: ${impl.name}`);
  let passed = 0;

  for (const testCase of testCases) {
    try {
      const result = impl.fn(testCase.input);
      const success = JSON.stringify(result) ===
JSON.stringify(testCase.expected);

      if (success) {
        passed++;
        console.log(`✓ ${testCase.name}`);
      } else {
        console.log(`✗ ${testCase.name}`);
        console.log(`  期望:
${JSON.stringify(testCase.expected)}`);
        console.log(`  实际: ${JSON.stringify(result)}`);
      }
    } catch (error) {
      console.log(`✗ ${testCase.name} 出错: ${error.message}`);
    }
  }

  console.log(`通过: ${passed}/${testCases.length}`);
}
}

// 运行测试
testFlatten();

```

性能对比

```

JavaScript
// 性能测试
function benchmarkFlatten() {
  const implementations = {
    '递归': flattenSimple,
    '迭代（栈）': flattenIterative,
    '迭代（队列）': flattenIterativeOrdered,
    'flat 方法': flattenUsingFlat,
    'Generator': flattenUsingGenerator
  }
}

```



```

};

// 生成测试数据
function generateTestData(depth, width) {
  function generate(d, w) {
    if (d <= 0) return Array(w).fill(0).map((_, i) => i);
    return Array(w).fill(0).map(() => generate(d - 1, w));
  }
  return generate(depth, width);
}

const testData = generateTestData(4, 5); // 4 层深度，每层 5 个元素

console.log('性能测试开始...');
console.log(`测试数据：深度 4 层，每层 5 个元素，共${5**4}个元素`);

for (const [name, fn] of Object.entries(implementations)) {
  const startTime = performance.now();

  // 运行多次取平均值
  const iterations = 1000;
  for (let i = 0; i < iterations; i++) {
    fn(testData);
  }

  const endTime = performance.now();
  const avgTime = (endTime - startTime) / iterations;

  console.log(`${name}: 平均 ${avgTime.toFixed(3)}ms`);
}

// 运行性能测试
benchmarkFlatten();

```

实际应用场景

```

JavaScript
// 场景 1: 处理 API 返回的嵌套数据
async function processNestedAPI() {
  // 模拟 API 返回的嵌套数据
  const apiResponse = {

```

```

users: [
  {
    id: 1,
    posts: [
      { id: 101, tags: ['js', 'web'] },
      { id: 102, tags: ['node', 'backend'] }
    ]
  },
  {
    id: 2,
    posts: [
      { id: 201, tags: ['react', 'frontend'] }
    ]
  }
]
};

// 提取所有标签
const allTags = flattenSimple(
  apiResponse.users.map(user =>
    user.posts.map(post => post.tags)
  )
);

console.log('所有标签:', allTags);
// 输出: ['js', 'web', 'node', 'backend', 'react', 'frontend']
}

// 场景 2: 树形结构转平铺列表
function treeToFlatList(tree, key = 'children') {
  const result = [];

  function traverse(node) {
    const { [key]: children, ...rest } = node;
    result.push(rest);

    if (children && children.length) {
      children.forEach(traverse);
    }
  }

  if (Array.isArray(tree)) {
    tree.forEach(traverse);
  } else {

```

```

    traverse(tree);
  }

  return result;
}

// 使用示例
const treeData = [
  {
    id: 1,
    name: 'Node 1',
    children: [
      { id: 2, name: 'Node 1.1' },
      {
        id: 3,
        name: 'Node 1.2',
        children: [{ id: 4, name: 'Node 1.2.1' }]
      }
    ]
  }
];

const flatList = treeToFlatList(treeData);
console.log(flatList);
// 输出: [{id:1,name:'Node 1'}, {id:2,name:'Node 1.1'}, ...]

```

64. 什么是可选链操作符和空值合并操作符?

可选链操作符 (Optional Chaining) `?.`

```

CSS
// 传统写法需要多层判断
const street = user && user.address && user.address.street;

// 使用可选链
const street = user?.address?.street; // 遇到 null/undefined 立即返回 undefined

```

空值合并操作符 (Nullish Coalescing) `??`

```

Java
// 逻辑或 || 的问题: 会过滤所有假值 (0, '', false 等)

```

```
const count = 0;
const result = count || 10; // 10 (但期望的是 0)

// 空值合并只针对 null/undefined
const result = count ?? 10; // 0 (符合预期)
```

两者结合使用：

```
CSS
const price = product?.inventory?.price ?? 99;
// product 不存在 -> undefined
// inventory 不存在 -> undefined
// price 不存在 -> undefined
// 最终使用默认值 99
```

65. HOC 在 React Hooks 出现后，它在哪些场景下仍然适用？

HOC 仍然适用的场景：

1. Class 组件增强：遗留项目或必须使用 Class 组件时
2. 第三方库集成：如 `connect()(React-Redux)`、`withRouter()(React-Router)`
3. 条件渲染封装：

```
JavaScript
const withLoading = (Component) => (props) => {
  if (props.loading) return <Spinner />;
  return <Component {...props} />;
};
```

4. 跨组件逻辑复用：多个组件需要相同逻辑时（Hooks 也可用自定义 Hook 实现）
5. 反向继承模式：需要操作组件生命周期时

```
JavaScript
const withLogging = (WrappedComponent) =>
  class extends WrappedComponent {
    componentDidMount() {
      console.log('Component mounted');
      super.componentDidMount?.();
    }
  }
```

```
};
```

推荐原则：新项目优先使用 Hooks，遇到上述场景再考虑 HOC。

66. 谈谈你对 Core Web Vitals 的理解，它包含哪三项指标？

Core Web Vitals 是 Google 定义的网页用户体验核心指标，用于衡量真实用户感知的性能和体验。

三项核心指标：

1. LCP (Largest Contentful Paint) - 最大内容绘制
 - 衡量加载速度
 - 标准：≤2.5 秒
 - 优化：优化关键资源、CDN 加速、服务端渲染
2. FID (First Input Delay) - 首次输入延迟
 - 衡量交互性
 - 标准：≤100 毫秒
 - 优化：拆分长任务、优化 JavaScript 执行
3. (注意：FID 将被 INP 取代)
4. CLS (Cumulative Layout Shift) - 累计布局偏移
 - 衡量视觉稳定性
 - 标准：≤0.1
 - 优化：设置尺寸属性、预留空间、避免动态插入内容

工具：Lighthouse、PageSpeed Insights、Chrome DevTools

67. Shadow DOM 与虚拟 DOM 有什么关系？

两者本质完全不同：

维度	Shadow DOM	虚拟 DOM
所属范畴	浏览器原生API (Web标准)	库/框架的实现 (如React)
解决的问题	CSS样式隔离、DOM封装	提升DOM操作性能
工作层面	浏览器渲染引擎层级	JavaScript应用层级
使用场景	Web Components	React、Vue等前端框架
核心特性	封装、样式隔离	差异比较、批量更新

点击图片可查看完整电子表格

关系：

- 可结合使用：React 组件可以封装 Web Components (含 Shadow DOM)
- 目的不同：Shadow DOM 解决封装隔离，虚拟 DOM 解决性能优化

68. requestAnimationFrame 与 setTimeout/setInterval 相比，用于实现动画有什么优势？

requestAnimationFrame 的优势：

1. 性能优化

```
JavaScript
// rAF: 与浏览器刷新率同步 (通常 60fps)
function animate() {
  element.style.left = (pos++) + 'px';
  requestAnimationFrame(animate); // 下次重绘前执行
}

// setTimeout: 固定时间间隔, 可能掉帧
setInterval(() => {
  element.style.left = (pos++) + 'px';
}, 16.67); // 试图模拟 60fps
```

2. 节能特性

- 标签页隐藏时自动暂停
- 减少 CPU/GPU 消耗

3. 时间精度

- 传入的时间戳是 `performance.now()`, 精度更高

- 适合基于时间的动画计算

4. 批量渲染

- 多个 rAF 回调会在同一帧执行
- 避免布局抖动

对比总结:

- `setTimeout/setInterval`: 固定间隔, 不考虑渲染时机
- `requestAnimationFrame`: 浏览器优化调度, 适合流畅动画

69. 请解释 ES6 中的 Symbol 类型有什么用处。

Symbol 的核心特性

Symbol 是 ES6 引入的一种新的原始数据类型, 表示独一无二的值。

基本用法

```
JavaScript
// 创建 Symbol
const sym1 = Symbol();
const sym2 = Symbol('description'); // 可选的描述字符串
const sym3 = Symbol('description');

console.log(sym2 === sym3); // false - 每个 Symbol 都是唯一的
console.log(sym2.description); // "description" - ES2019 新增
```

Symbol 的主要用途

1. 避免属性名冲突

```
JavaScript
const user = {
  id: 1,
  name: 'John'
};

// 第三方库可能会添加 id 属性, 造成冲突
```

```
// 使用 Symbol 可以避免这个问题
const id = Symbol('id');
user[id] = 'unique_id_123';

console.log(user.id); // 1 - 原来的属性
console.log(user[id]); // 'unique_id_123' - Symbol 属性
```

2. 定义对象的私有属性

```
JavaScript
const _privateData = Symbol('privateData');

class User {
  constructor(name, secret) {
    this.name = name;
    this[_privateData] = secret; // "私有"属性
  }

  getSecret() {
    return this[_privateData];
  }
}

const user = new User('Alice', 'password123');

console.log(user.name); // 'Alice'
console.log(user[_privateData]); // undefined - 无法直接访问
console.log(user.getSecret()); // 'password123'

// 但还是可以通过 Object.getOwnPropertySymbols 获取
const symbols = Object.getOwnPropertySymbols(user);
console.log(user[symbols[0]]); // 'password123'
```

3. 实现内置协议 (Well-known Symbols)

```
JavaScript
// Symbol.iterator - 使对象可迭代
const myIterable = {
  data: [1, 2, 3, 4, 5],
  [Symbol.iterator]: function* () {
    for (let item of this.data) {
      yield item * 2;
    }
  }
}
```

```

    }
  }
};

for (let value of myIterable) {
  console.log(value); // 2, 4, 6, 8, 10
}

// Symbol.toStringTag - 自定义 toString 行为
class MyClass {
  get [Symbol.toStringTag]() {
    return 'MyClass';
  }
}

const obj = new MyClass();
console.log(Object.prototype.toString.call(obj)); // "[object MyClass]"

// Symbol.hasInstance - 自定义 instanceof 行为
class MyArray {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance);
  }
}

console.log([] instanceof MyArray); // true
console.log({} instanceof MyArray); // false

```

4. 全局 Symbol 注册表

```

JavaScript
// 创建全局 Symbol
const globalSym1 = Symbol.for('global.key');
const globalSym2 = Symbol.for('global.key');

console.log(globalSym1 === globalSym2); // true - 相同的 key 返回相同的 Symbol

// 获取已注册 Symbol 的 key
console.log(Symbol.keyFor(globalSym1)); // "global.key"

// 使用场景：跨模块共享 Symbol

```

```
// module1.js
export const EVENT_CLICK = Symbol.for('app.event.click');

// module2.js
import { EVENT_CLICK } from './module1.js';
const sameSymbol = Symbol.for('app.event.click');
console.log(EVENT_CLICK === sameSymbol); // true
```

实际应用示例

示例 1：React 中的 Symbol 使用

```
JavaScript
// React 内部使用 Symbol 来标识 React 元素类型
const REACT_ELEMENT_TYPE = Symbol.for('react.element');
const REACT_FRAGMENT_TYPE = Symbol.for('react.fragment');

// 检查是否为 React 元素
function isReactElement(element) {
  return element.$$typeof === REACT_ELEMENT_TYPE;
}
```

示例 2：枚举类型的实现

```
JavaScript
// 使用 Symbol 实现类型安全的枚举
const Colors = Object.freeze({
  RED: Symbol('red'),
  GREEN: Symbol('green'),
  BLUE: Symbol('blue')
});

function getColorName(color) {
  switch (color) {
    case Colors.RED:
      return '红色';
    case Colors.GREEN:
      return '绿色';
    case Colors.BLUE:
      return '蓝色';
    default:

```

```
        throw new Error('未知颜色');
    }
}

console.log(getColorName(Colors.RED)); // "红色"
```

示例 3：元编程

```
JavaScript
// 使用 Symbol 进行元编程
const DEBUG = Symbol('debug');
const PERF = Symbol('perf');

class Calculator {
  constructor() {
    this[DEBUG] = false;
    this[PERF] = [];
  }

  add(a, b) {
    const start = performance.now();
    const result = a + b;
    const end = performance.now();

    if (this[DEBUG]) {
      console.log(`add(${a}, ${b}) = ${result}`);
    }

    this[PERF].push({
      operation: 'add',
      time: end - start
    });

    return result;
  }

  enableDebug() {
    this[DEBUG] = true;
  }

  getPerformance() {
    return this[PERF];
  }
}
```

```
}

```

70. GraphQL 与 RESTful API 相比有哪些优缺点？

GraphQL 概述

GraphQL 是一种 API 查询语言，由 Facebook 开发，允许客户端精确指定需要的数据结构。

详细对比

维度	RESTful API	GraphQL
数据获取	多个端点，多次请求	单一端点，一次请求
数据返回	固定结构，可能过多或不足	客户端指定，精确返回
版本管理	URL版本(v1, v2)或Header	Schema演化，无版本号
类型系统	无强制类型，靠文档	强类型Schema
错误处理	HTTP状态码	始终返回200，错误在响应体中
缓存	HTTP缓存机制完善	需要自定义缓存策略
学习曲线	简单，基于HTTP	较陡峭，需要学习新概念

点击图片可查看完整电子表格

RESTful API 示例

```
JSON
// 传统 RESTful API
// 获取用户信息
GET /api/v1/users/123
Response:
{
  "id": 123,
  "name": "John",
  "email": "john@example.com",
  "address": {
    "street": "123 Main St",
    "city": "NYC",
    "country": "USA"
  },
  // 还包括了很多不需要的字段
}
```



```
}

// 获取用户的订单
GET /api/v1/users/123/orders
Response:
[
  { "id": 1, "product": "Book", "price": 20 },
  { "id": 2, "product": "Pen", "price": 5 }
]

// 问题：需要 2 次请求才能获取完整信息
```

GraphQL 示例

```
JavaScript
// GraphQL 查询
query GetUserWithOrders($userId: ID!) {
  user(id: $userId) {
    name
    email
    orders {
      product
      price
    }
  }
}

// 一次请求，精确获取所需数据
Response:
{
  "data": {
    "user": {
      "name": "John",
      "email": "john@example.com",
      "orders": [
        { "product": "Book", "price": 20 },
        { "product": "Pen", "price": 5 }
      ]
    }
  }
}
```

GraphQL 核心概念

1. Schema 定义

YAML

```
type User {  
  id: ID!  
  name: String!  
  email: String!  
  age: Int  
  posts: [Post!]  
}  
  
type Post {  
  id: ID!  
  title: String!  
  content: String!  
  author: User!  
}  
  
type Query {  
  user(id: ID!): User  
  posts(limit: Int = 10): [Post!]  
}  
  
type Mutation {  
  createUser(input: CreateUserInput!): User!  
}  
  
input CreateUserInput {  
  name: String!  
  email: String!  
  age: Int  
}
```

2. Resolver 实现

Python

```
// GraphQL Resolvers  
const resolvers = {  
  Query: {  
    user: async (_, { id }) => {
```

```

    return await UserModel.findById(id);
  },
  posts: async (_, { limit }) => {
    return await PostModel.find().limit(limit);
  }
},

User: {
  posts: async (user) => {
    return await PostModel.find({ authorId: user.id });
  }
},

Mutation: {
  createUser: async (_, { input }) => {
    const user = new UserModel(input);
    return await user.save();
  }
}
};

```

实际项目比较

RESTful API 项目结构

```

Bash
/api
/v1
  /users
    GET /          # 获取用户列表
    POST /         # 创建用户
    GET /:id       # 获取用户详情
    PUT /:id       # 更新用户
    DELETE /:id    # 删除用户
  /posts
    GET /          # 获取文章列表
    POST /         # 创建文章
/v2
...

```

GraphQL 项目结构

```
Bash
/graphql
  /schema
    user.graphql      # 用户类型定义
    post.graphql      # 文章类型定义
    query.graphql     # 查询定义
    mutation.graphql  # 变更定义
  /resolvers
    user.js           # 用户解析器
    post.js           # 文章解析器
    index.js          # 主入口文件
```

性能考虑

RESTful 缓存策略

```
JavaScript
// 利用 HTTP 缓存
app.get('/api/users/:id', (req, res) => {
  const userId = req.params.id;
  const user = getUserFromCache(userId);

  if (user) {
    // 设置缓存头
    res.setHeader('Cache-Control', 'public, max-age=3600');
    res.setHeader('ETag', generateETag(user));
    res.json(user);
  } else {
    // 从数据库获取
  }
});
```

GraphQL 缓存挑战

```
JavaScript
// GraphQL 缓存需要更复杂的策略
const responseCache = new Map();

async function executeQuery(query, variables) {
  const cacheKey = generateCacheKey(query, variables);
```

```

if (responseCache.has(cacheKey)) {
  return responseCache.get(cacheKey);
}

const result = await graphql(schema, query, null, null,
variables);
responseCache.set(cacheKey, result);

// 要实现缓存失效策略
setTimeout(() => {
  responseCache.delete(cacheKey);
}, 10000);

return result;
}

```

选择建议

CSS

```

const decisionMatrix = {
  chooseGraphQL: [
    '移动端应用需要减少网络请求',
    '多平台应用（Web、iOS、Android）需要不同数据',
    '复杂的数据关系需要一次获取',
    '需要强类型 API 契约',
    '前端开发希望有更多控制权'
  ],

  chooseREST: [
    '简单的 CRUD 应用',
    '需要利用 HTTP 缓存',
    '项目团队熟悉 REST',
    '需要更好的工具链支持',
    '微服务架构中服务间通信'
  ],

  hybridApproach: {
    description: '在实际项目中，可以混合使用',
    examples: [
      '公开 API 使用 REST，内部 API 使用 GraphQL',
      '简单资源用 REST，复杂查询用 GraphQL',

```

```
    '迁移过程中逐步替换'  
  ]  
}  
};
```

71. 在 Vue 中，v-if 和 v-show 各自的适用场景是什么？

核心区别

特性	v-if	v-show
DOM操作	条件为false时元素从DOM中移除	始终在DOM中，只切
初始渲染成本	低（不渲染为false的元素）	高（所有元素都会渲
切换成本	高（需要销毁/重建组件）	低（只切换CSS属性）
生命周期	切换时会触发组件的生命周期钩子	不会触发生命周期钩
适用场景	不频繁切换，或条件很可能为false	频繁切换显示/隐藏

点击图片可查看完整电子表格

详细示例

v-if 示例

```
XML  
<template>  
  <div>  
    <!-- v-if: 条件为 false 时元素不存在于 DOM 中 -->  
    <div v-if="isAdmin">  
      <AdminPanel />  
    </div>  
  
    <!-- v-else-if 和 v-else -->  
    <div v-else-if="isUser">  
      普通用户界面  
    </div>  
    <div v-else>  
      请先登录  
    </div>  
  </div>  
</template>
```



```

<!-- 与 template 一起使用 -->
<template v-if="hasData">
  <h2>数据标题</h2>
  <p>数据内容</p>
</template>
</div>
</template>

<script>
export default {
  data() {
    return {
      isAdmin: false,
      isUser: true,
      hasData: true
    };
  },

  components: {
    AdminPanel: {
      created() {
        console.log('AdminPanel 组件创建');
      },
      destroyed() {
        console.log('AdminPanel 组件销毁');
      },
      template: '<div>管理员面板</div>'
    }
  }
};
</script>

```

v-show 示例

```

XML
<template>
  <div>
    <!-- v-show: 元素始终在 DOM 中，通过 CSS 控制显示 -->
    <div v-show="isVisible">
      这个元素始终在 DOM 中
    </div>
  </div>

```

```

<!-- 多个元素需要相同控制逻辑 -->
<div class="controls">
  <button v-show="!isPlaying" @click="play">播放</button>
  <button v-show="isPlaying" @click="pause">暂停</button>
</div>

<!-- 与 v-for 结合 -->
<ul>
  <li
    v-for="item in items"
    :key="item.id"
    v-show="item.isActive"
  >
    {{ item.name }}
  </li>
</ul>
</div>
</template>

<script>
export default {
  data() {
    return {
      isVisible: true,
      isPlaying: false,
      items: [
        { id: 1, name: 'Item 1', isActive: true },
        { id: 2, name: 'Item 2', isActive: false },
        { id: 3, name: 'Item 3', isActive: true }
      ]
    };
  },

  methods: {
    play() {
      this.isPlaying = true;
      // 播放逻辑
    },

    pause() {
      this.isPlaying = false;
      // 暂停逻辑
    }
  }
}

```

```
}  
};  
</script>
```

性能分析

CSS

// 性能测试组件

```
const PerformanceTest = {  
  template: `  
    <div>  
      <h3>性能测试</h3>  
  
      <button @click="toggle('v-if')">  
        v-if 切换测试 ({{ count }}次)  
      </button>  
  
      <button @click="toggle('v-show')">  
        v-show 切换测试 ({{ count }}次)  
      </button>  
  
      <div v-if="mode === 'v-if' && showContent" class="content">  
        v-if 内容区  
      </div>  
  
      <div v-show="mode === 'v-show' && showContent"  
class="content">  
        v-show 内容区  
      </div>  
  
      <p>切换模式: {{ mode }}</p>  
      <p>显示内容: {{ showContent }}</p>  
    </div>  
  `,  
  
  data() {  
    return {  
      mode: 'v-if',  
      showContent: true,  
      count: 0  
    };  
  },  
};
```

```

methods: {
  toggle(type) {
    const startTime = performance.now();
    this.mode = type;

    // 快速切换 100 次
    for (let i = 0; i < 100; i++) {
      this.showContent = !this.showContent;
      this.count++;
    }

    const endTime = performance.now();
    console.log(`${type} 切换 100 次耗时: ${endTime -
    startTime}ms`);
  }
};

```

实际应用场景

场景 1：权限控制（适合 v-if）

```

XML
<template>
  <div class="dashboard">
    <!-- 管理员才有的功能 -->
    <AdminTools v-if="user.role === 'admin'" />

    <!-- 需要特定权限的功能 -->
    <template v-if="hasPermission('edit')">
      <EditButton @click="handleEdit" />
      <DeleteButton @click="handleDelete" />
    </template>

    <!-- 根据环境显示调试信息 -->
    <DebugInfo v-if="process.env.NODE_ENV === 'development'" />
  </div>
</template>

```

场景 2：标签页/选项卡（适合 v-show）

XML

```
<template>
  <div class="tabs">
    <div class="tab-headers">
      <button
        v-for="tab in tabs"
        :key="tab.id"
        :class="{ active: activeTab === tab.id }"
        @click="activeTab = tab.id"
      >
        {{ tab.title }}
      </button>
    </div>

    <div class="tab-contents">
      <!-- 使用 v-show 保持组件状态 -->
      <div
        v-for="tab in tabs"
        :key="tab.id"
        v-show="activeTab === tab.id"
        class="tab-content"
      >
        <component :is="tab.component" />
      </div>
    </div>
  </div>
</template>

<script>
export default {
  data() {
    return {
      activeTab: 'info',
      tabs: [
        { id: 'info', title: '基本信息', component: 'UserInfo' },
        { id: 'orders', title: '订单记录', component:
'OrderList' },
        { id: 'settings', title: '设置', component: 'Settings' }
      ]
    };
  },
  components: {
    UserInfo: {
```

```

data() {
  return {
    // 表单数据，切换标签页时保持状态
    form: {
      name: '',
      email: ''
    }
  };
},
template: `
  <div>
    <input v-model="form.name" placeholder="姓名">
    <input v-model="form.email" placeholder="邮箱">
  </div>
`,
  OrderList: { /* ... */ },
  Settings: { /* ... */ }
}
};
</script>

```

场景 3：加载状态（v-if 更合适）

```

XML
<template>
  <div>
    <!-- 加载中显示 loading -->
    <div v-if="isLoading" class="loading">
      加载中...
    </div>

    <!-- 加载成功显示数据 -->
    <div v-else-if="data" class="content">
      <h2>{{ data.title }}</h2>
      <p>{{ data.content }}</p>
    </div>

    <!-- 加载失败显示错误 -->
    <div v-else class="error">
      加载失败
      <button @click="retry">重试</button>
    </div>
  </div>

```



```

    </div>
  </div>
</template>

<script>
export default {
  data() {
    return {
      isLoading: false,
      data: null,
      error: null
    };
  },

  async created() {
    await this.loadData();
  },

  methods: {
    async loadData() {
      this.isLoading = true;
      try {
        const response = await fetch('/api/data');
        this.data = await response.json();
      } catch (error) {
        this.error = error;
      } finally {
        this.isLoading = false;
      }
    },

    retry() {
      this.loadData();
    }
  }
};
</script>

```

最佳实践建议

```

Java
const bestPractices = {
  useVIf: [

```

```

    '初始条件很可能为 false 的情况',
    '需要触发组件生命周期的场景',
    '包含重量级组件, 不希望初始渲染',
    '条件切换不频繁的场景'
  ],

  useVShow: [
    '频繁切换显示/隐藏的场景',
    '需要保持组件内部状态',
    '简单的 CSS 显示隐藏',
    '性能要求高的动画切换'
  ],

  avoidBoth: [
    '避免在同一元素上同时使用 v-if 和 v-show',
    'v-for 和 v-if 同时使用时, 考虑使用计算属性过滤数据'
  ],

  optimizationTips: [
    '对于长列表, 使用虚拟滚动替代 v-show',
    '复杂条件使用计算属性封装',
    '考虑使用<template>包裹多个元素'
  ]
};

```

72. CSS 的 Grid 布局和 Flexbox 布局有什么区别？

核心概念对比

维度	Grid (网格布局)	Flexbox (弹性盒子)
维度	二维布局 (行和列)	一维布局 (水平或垂直)
布局方向	同时控制行和列	单一方向 (row或column)
容器控制	定义网格轨道, 放置项目	定义主轴方向, 分配空间
内容优先	网格优先, 先定义结构	内容优先, 根据内容伸缩
对齐方式	强大的网格对齐系统	灵活的项目对齐方式

点击图片可查看完整电子表格

Grid 布局详解

基础示例

CSS

```
/* Grid 容器 */
.container {
  display: grid;
  /* 定义 3 列: 第一列 100px, 第二列自动, 第三列 1fr */
  grid-template-columns: 100px auto 1fr;
  /* 定义 3 行: 第一行 50px, 第二行自动, 第三行 100px */
  grid-template-rows: 50px auto 100px;
  /* 行间距和列间距 */
  gap: 10px 20px;
  /* 显式网格区域命名 */
  grid-template-areas:
    "header header header"
    "sidebar content content"
    "footer footer footer";
}

/* Grid 项目 */
.item {
  /* 项目在网格中的位置 */
  grid-column: 1 / 3; /* 跨越第 1 到第 3 列线 */
  grid-row: 2 / 4; /* 跨越第 2 到第 4 行线 */

  /* 使用命名区域 */
  grid-area: content;
}

/* 响应式网格 */
@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "content"
      "sidebar"
      "footer";
  }
}
```

Grid 实际应用

XML

```
<!-- 圣杯布局 -->
```

```
<div class="holy-grail">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main Content</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
</div>
```

```
<style>
```

```
.holy-grail {
  display: grid;
  grid-template-rows: auto 1fr auto;
  grid-template-columns: 200px 1fr 200px;
  grid-template-areas:
    "header header header"
    "nav main aside"
    "footer footer footer";
  min-height: 100vh;
  gap: 10px;
}
```

```
header { grid-area: header; background: #333; color: white;
padding: 20px; }
nav { grid-area: nav; background: #f4f4f4; padding: 20px; }
main { grid-area: main; background: white; padding: 20px; }
aside { grid-area: aside; background: #f4f4f4; padding: 20px; }
footer { grid-area: footer; background: #333; color: white;
padding: 20px; }
```

```
@media (max-width: 768px) {
  .holy-grail {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "nav"
      "main"
      "aside"
      "footer";
  }
}
```

```
</style>
```

Flexbox 布局详解

基础示例

CSS

```
/* Flex 容器 */
.container {
  display: flex;
  /* 主轴方向 */
  flex-direction: row; /* row | row-reverse | column | column-reverse */
  /* 换行 */
  flex-wrap: nowrap; /* nowrap | wrap | wrap-reverse */
  /* 主轴对齐 */
  justify-content: flex-start; /* flex-start | flex-end | center | space-between | space-around | space-evenly */
  /* 交叉轴对齐 */
  align-items: stretch; /* stretch | flex-start | flex-end | center | baseline */
  /* 多行对齐 */
  align-content: stretch; /* 类似于 align-items, 但针对多行 */
}

/* Flex 项目 */
.item {
  /* 放大比例 */
  flex-grow: 0; /* 默认不放大 */
  /* 缩小比例 */
  flex-shrink: 1; /* 默认可缩小 */
  /* 基础大小 */
  flex-basis: auto; /* 初始大小 */

  /* 简写 */
  flex: 1; /* 相当于 flex: 1 1 0% */
  flex: auto; /* 相当于 flex: 1 1 auto */
  flex: none; /* 相当于 flex: 0 0 auto */

  /* 单个项目对齐 */
  align-self: auto; /* 覆盖 align-items */
  order: 0; /* 排序 */
}
```

Flexbox 实际应用

XML

```
<!-- 导航栏 -->
<nav class="navbar">
  <div class="logo">Logo</div>
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Services</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
  <div class="user-menu">
    <button>Login</button>
    <button>Sign Up</button>
  </div>
</nav>
```

```
<style>
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 20px;
  background: #333;
  color: white;
}

.nav-links {
  display: flex;
  list-style: none;
  margin: 0;
  padding: 0;
  gap: 20px;
}

.nav-links a {
  color: white;
  text-decoration: none;
  padding: 10px;
}

.user-menu {
  display: flex;
```



```
gap: 10px;
}

/* 卡片布局 */
.card-container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  justify-content: center;
}

.card {
  flex: 1 1 300px; /* 基础 300px, 可以伸缩 */
  max-width: 400px;
  background: white;
  border-radius: 8px;
  padding: 20px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
</style>
```

选择指南

何时使用 Grid?

```
CSS
/* 1. 整体页面布局 */
.page-layout {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr auto;
  height: 100vh;
}

/* 2. 复杂网格系统 */
.photo-gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 10px;
}

/* 3. 精确控制行列 */
```

```
.calendar {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  grid-template-rows: repeat(6, 100px);
}

/* 4. 重叠元素 */
.hero-section {
  display: grid;
}

.hero-image, .hero-text {
  grid-area: 1 / 1; /* 重叠放置 */
}
```

何时使用 Flexbox?

CSS

```
/* 1. 导航栏和菜单 */
.navigation {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* 2. 列表项目布局 */
.product-list {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

/* 3. 居中元素 */
.center-container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

/* 4. 响应式图片库 */
.image-row {
  display: flex;
```

```
flex-wrap: wrap;
gap: 10px;
}

.image-row img {
  flex: 1 1 200px;
  max-width: 100%;
}
```

混合使用示例

XML

```
<div class="dashboard">
  <header class="header">
    <!-- 使用 Flexbox 水平布局 -->
    <div class="logo">Dashboard</div>
    <nav class="main-nav">
      <a href="#">Home</a>
      <a href="#">Analytics</a>
      <a href="#">Reports</a>
    </nav>
  </header>

  <div class="main-content">
    <!-- 使用 Grid 创建主布局 -->
    <aside class="sidebar">
      <!-- 使用 Flexbox 垂直布局 -->
      <div class="widget">Widget 1</div>
      <div class="widget">Widget 2</div>
      <div class="widget">Widget 3</div>
    </aside>

    <main class="content">
      <!-- 使用 Grid 创建卡片网格 -->
      <div class="card-grid">
        <div class="card">Card 1</div>
        <div class="card">Card 2</div>
        <div class="card">Card 3</div>
        <div class="card">Card 4</div>
      </div>
    </main>
  </div>
</div>
```

```
<style>
.dashboard {
  display: grid;
  grid-template-rows: auto 1fr;
  height: 100vh;
}

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 20px;
  background: #333;
  color: white;
}

.main-nav {
  display: flex;
  gap: 20px;
}

.main-content {
  display: grid;
  grid-template-columns: 250px 1fr;
  gap: 20px;
  padding: 20px;
}

.sidebar {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.widget {
  flex: 1;
  background: #f4f4f4;
  padding: 20px;
  border-radius: 5px;
}

.card-grid {
  display: grid;
}
```

```
grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
gap: 20px;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
</style>
```

73. 请解释 React 中的 Fiber 架构，它解决了什么问题？

Fiber 架构解决的问题

React 16 之前版本的协调（Reconciliation）算法是递归的、不可中断的，这导致：

1. 主线程阻塞：大型组件树渲染会阻塞主线程
2. 丢帧：动画卡顿，用户交互响应延迟
3. 无法优先处理高优先级任务

Fiber 核心概念

Fiber 节点结构

```
JavaScript
// 简化版 Fiber 节点结构
class FiberNode {
  constructor(tag, pendingProps, key, mode) {
    // 标识类型
    this.tag = tag; // FunctionComponent, ClassComponent,
    HostComponent 等
    this.key = key;
    this.elementType = null;
    this.type = null;

    // 状态节点
```

```

    this.stateNode = null;

    // 链表结构
    this.return = null; // 父节点
    this.child = null; // 第一个子节点
    this.sibling = null; // 兄弟节点

    // 更新相关
    this.pendingProps = pendingProps;
    this.memoizedProps = null;
    this.updateQueue = null;
    this.memoizedState = null;

    // 副作用
    this.flags = NoFlags;
    this.subtreeFlags = NoFlags;
    this.deletions = null;

    // 优先级
    this.lanes = NoLanes;
    this.childLanes = NoLanes;

    // 用于复用
    this.alternate = null;

    // 渲染阶段
    this.index = 0;
    this.ref = null;
  }
}

```

Fiber 工作流程

```

JavaScript
// 简化的 Fiber 工作循环
function workLoop(deadline) {
  // 检查是否有剩余时间
  while (nextUnitOfWork && !shouldYield()) {
    nextUnitOfWork = performUnitOfWork(nextUnitOfWork);
  }

  // 如果还有工作但时间到了，让出控制权

```



```

    if (nextUnitOfWork) {
      requestIdleCallback(workLoop);
    } else if (pendingCommit) {
      // 提交阶段（不可中断）
      commitAllWork(pendingCommit);
    }
  }
}

function performUnitOfWork(fiber) {
  // 开始阶段：创建 Fiber 的子节点
  beginWork(fiber);

  if (fiber.child) {
    return fiber.child;
  }

  // 如果没有子节点，处理兄弟节点
  let nextFiber = fiber;
  while (nextFiber) {
    // 完成阶段：收集副作用
    completeWork(nextFiber);

    if (nextFiber.sibling) {
      return nextFiber.sibling;
    }

    // 返回父节点
    nextFiber = nextFiber.return;
  }

  return null;
}

```

关键特性

1. 时间切片（Time Slicing）

```

JavaScript
// 启用并发模式
ReactDOM.createRoot(rootElement).render(<App />);

// 时间切片示例

```

```
function ExpensiveComponent() {
  const items = useMemo(() => {
    // 昂贵的计算
    return computeExpensiveItems();
  }, []);

  return (
    <React.Suspense fallback={<Spinner />}>
      <List items={items} />
    </React.Suspense>
  );
}
```

2. 优先级调度

```
JavaScript
// React 内部优先级
const PriorityLevels = {
  NoPriority: 0,          // 无优先级
  ImmediatePriority: 1,   // 立即执行（用户输入）
  UserBlockingPriority: 2, // 用户阻塞（用户交互）
  NormalPriority: 3,      // 正常（数据获取）
  LowPriority: 4,         // 低（分析数据）
  IdlePriority: 5         // 空闲（非必要更新）
};

// 使用 startTransition API
import { startTransition, useState } from 'react';

function SearchBox() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  function handleChange(e) {
    const value = e.target.value;
    setQuery(value);

    // 标记为过渡更新，避免阻塞用户输入
    startTransition(() => {
      fetchResults(value).then(setResults);
    });
  }
}
```

```
return (  
  <div>  
    <input value={query} onChange={handleChange} />  
    <ResultsList results={results} />  
  </div>  
)  
};  
}
```

3. 可中断的渲染

```
JavaScript  
// Fiber 之前的递归渲染（不可中断）  
function reconcileChildren(current, workInProgress, nextChildren)  
{  
  // 如果是递归，一旦开始就无法中断  
  reconcileChildFibers(workInProgress, current.child,  
nextChildren);  
}  
  
// Fiber 的链表遍历（可中断）  
function reconcileChildren(current, workInProgress, nextChildren)  
{  
  // 创建子 Fiber 节点  
  const newChild = createChild(workInProgress, nextChildren);  
  
  // 可以在这里中断  
  if (shouldYield()) {  
    return; // 让出控制权  
  }  
  
  // 继续处理下一个节点  
  if (newChild) {  
    newChild.return = workInProgress;  
    workInProgress.child = newChild;  
  }  
}
```

实际应用

Suspense 和懒加载

```

JavaScript
// 基于 Fiber 的 Suspense 实现
const ProfilePage = React.lazy(() => import('./ProfilePage'));

function App() {
  return (
    <ErrorBoundary>
      <Suspense fallback={<Spinner />}>
        <Router>
          <Route path="/profile">
            <ProfilePage />
          </Route>
        </Router>
      </Suspense>
    </ErrorBoundary>
  );
}

```

并发特性使用

```

JavaScript
import { useState, useTransition } from 'react';

function ImageGallery() {
  const [images, setImages] = useState([]);
  const [isPending, startTransition] = useTransition();

  function loadMoreImages() {
    startTransition(() => {
      // 这个更新会被标记为可中断
      fetchNewImages().then(newImages => {
        setImages(prev => [...prev, ...newImages]);
      });
    });
  }

  return (
    <div>
      {isPending && <Spinner />}
      <ImageList images={images} />
      <button onClick={loadMoreImages}>加载更多</button>
    </div>
  );
}

```

```
}
```

性能对比

JavaScript

// Fiber 前后的性能对比

```
const performanceComparison = {
  beforeFiber: {
    rendering: '同步，不可中断',
    largeUpdates: '阻塞主线程',
    priority: '无优先级调度',
    maxDepth: '递归深度限制'
  },

  afterFiber: {
    rendering: '异步，可中断',
    largeUpdates: '时间切片处理',
    priority: '基于 lanes 的优先级',
    features: 'Suspense、并发渲染'
  }
};
```

// 实际性能测试

```
function PerformanceTest() {
  const [count, setCount] = useState(0);
  const items = Array(10000).fill(null).map((_, i) => i);

  // 不使用时间切片
  const handleSyncUpdate = () => {
    setCount(prev => prev + 1);
  };

  // 使用时间切片
  const handleAsyncUpdate = () => {
    startTransition(() => {
      setCount(prev => prev + 1);
    });
  };

  return (
    <div>
      <button onClick={handleSyncUpdate}>同步更新</button>
    </div>
  );
}
```

```

    <button onClick={handleAsyncUpdate}>异步更新</button>

    {/* 大量 DOM 节点 */}
    <div>
      {items.map(item => (
        <div key={item}>Item {item}</div>
      ))}
    </div>
  </div>
);
}

```

架构演进

从 Stack Reconciler 到 Fiber

```

JavaScript
// Stack Reconciler (React 15)
function reconcile(prevInstance, nextInstance) {
  // 递归遍历整棵树
  if (prevInstance.type !== nextInstance.type) {
    // 类型不同，卸载整个子树
    unmountTree(prevInstance);
    return mountTree(nextInstance);
  }

  // 更新属性
  updateProperties(prevInstance, nextInstance);

  // 递归处理子节点
  reconcileChildren(prevInstance.children, nextInstance.children);
}

// Fiber Reconciler (React 16+)
function workLoopConcurrent() {
  // 可以中断的循环
  while (workInProgress !== null && !shouldYield()) {
    performUnitOfWork(workInProgress);
  }

  // 如果还有工作，稍后继续
  if (workInProgress !== null) {

```



```
    return workLoopConcurrent;  
  }  
  
  // 提交更新  
  commitRoot();  
}
```

总结

Fiber 架构带来的主要改进:

1. 增量渲染: 将渲染工作分成小块
2. 优先级调度: 高优先级任务优先处理
3. 并发渲染: 多个任务可以同时进行
4. 更好的错误边界: 更细粒度的错误处理

```
Java  
// 现代 React 应用的最佳实践  
const bestPractices = {  
  concurrentFeatures: [  
    '使用 React.lazy 进行代码分割',  
    '利用 Suspense 处理异步状态',  
    '使用 startTransition 标记非紧急更新',  
    '考虑使用 useDeferredValue 处理延迟值'  
  ],  
  performanceOptimizations: [  
    '避免不必要的 re-render',  
    '使用 React.memo、useMemo、useCallback',  
    '批量状态更新',  
    '虚拟化长列表'  
  ],  
  futureReady: [  
    '为并发模式准备组件',  
    '使用新的 API 如 useTransition',  
    '考虑服务器组件架构',  
    '保持依赖更新'  
  ]  
};
```

74. Monorepo 与传统的多仓库 相比有什么优劣？

Monorepo（单一仓库）概述

Monorepo 是一种将所有相关项目放在同一个代码仓库中的开发策略。

对比分析

维度	Monorepo	多仓库（Polyrepo）
代码共享	✅ 容易，直接引用	❌ 需要发布包或git submodule
依赖管理	⚠️ 统一管理，可能臃肿	✅ 独立管理，更清晰
版本控制	⚠️ 单一版本号或独立版本	✅ 每个项目独立版本
构建工具	✅ 统一配置，共享脚本	⚠️ 每个项目单独配置
团队协作	✅ 跨项目修改容易	❌ 跨项目修改需要协调
权限控制	❌ 较粗粒度，整个仓库	✅ 细粒度，按仓库控制
CI/CD	✅ 统一流水线，依赖清晰	⚠️ 需要单独配置每个仓库

点击图片可查看完整电子表格

实际项目结构对比

多仓库项目结构

```
CSS
company/
├── web-app/           # 单独仓库
│   ├── package.json
│   └── src/
├── mobile-app/       # 单独仓库
│   ├── package.json
│   └── src/
├── shared-ui/        # 单独仓库
│   ├── package.json
│   └── src/
└── api-server/       # 单独仓库
    ├── package.json
    └── src/
```

Monorepo 项目结构

```
Python
company-monorepo/
├── packages/
│   ├── web-app/      # 子包
│   │   ├── package.json
│   │   └── src/
│   ├── mobile-app/   # 子包
│   │   ├── package.json
│   │   └── src/
│   ├── shared-ui/    # 子包
│   │   ├── package.json
│   │   └── src/
│   └── api-server/    # 子包
│       ├── package.json
│       └── src/
├── package.json      # 根 package.json
├── pnpm-workspace.yaml # 工作空间配置
└── turbo.json        # Turborepo 配置
```

现代 Monorepo 工具

1. pnpm Workspaces

```
CSS
# pnpm-workspace.yaml
packages:
  - 'packages/*'
  - 'apps/*'
  - '!**/test/**'
```

```
JSON
// 根目录 package.json
{
  "name": "company-monorepo",
  "private": true,
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "lint": "turbo run lint"
  },
  "devDependencies": {
```

```
"turbo": "^1.10.0"
}
}
```

2. Turborepo 配置

```
JSON
// turbo.json
{
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "lint": {
      "outputs": []
    },
    "test": {
      "dependsOn": ["build"],
      "outputs": ["coverage/**"]
    }
  }
}
```

3. Nx 工作空间

```
JSON
// nx.json
{
  "affected": {
    "defaultBase": "main"
  },
  "tasksRunnerOptions": {
    "default": {
      "runner": "nx/tasks-runners/default",
      "options": {
        "cacheableOperations": ["build", "lint", "test"]
      }
    }
  }
}
```

```
    }  
  },  
  "targetDefaults": {  
    "build": {  
      "dependsOn": ["^build"]  
    }  
  }  
}
```

实际应用示例

跨包引用示例

```
JavaScript  
// packages/shared-ui/src/Button.jsx  
export const Button = ({ children }) => (  
  <button className="btn">{children}</button>  
);  
  
// packages/web-app/src/App.jsx  
import { Button } from '@company/shared-ui'; // 直接引用内部包  
  
function App() {  
  return (  
    <div>  
      <Button>Click me</Button>  
    </div>  
  );  
}
```

依赖管理示例

```
JSON  
// 根目录 package.json  
{  
  "name": "my-monorepo",  
  "workspaces": ["packages/*"],  
  "devDependencies": {  
    "typescript": "^5.0.0",  
    "jest": "^29.0.0",  
    "eslint": "^8.0.0"  
  }  
}
```

```

    }
  }

  // packages/web-app/package.json
  {
    "name": "@company/web-app",
    "version": "1.0.0",
    "dependencies": {
      "@company/shared-ui": "workspace:*", // 引用工作空间内的包
      "react": "^18.0.0",
      "react-dom": "^18.0.0"
    },
    "scripts": {
      "dev": "vite",
      "build": "tsc && vite build"
    }
  }
}

```

优势场景分析

适合 Monorepo 的场景：

```

CSS
const suitableForMonorepo = {
  microservices: '多个相互依赖的服务',
  componentLibraries: '共享组件的设计系统',
  fullstackApps: '前后端紧密耦合的应用',
  mobileApps: '共享业务逻辑的 iOS/Android 应用',
  tools: '相互依赖的开发工具链'
};

```

适合多仓库的场景：

```

CSS
const suitableForPolyrepo = {
  independentProducts: '完全独立的产品线',
  openSourceLibraries: '独立的开源库',
  clientProjects: '不同客户的独立项目',
  legacySystems: '遗留系统集成',
  securityCritical: '需要严格权限隔离'
};

```



```
};
```

75. 响应式原理 Vue 2 和 Vue 3 的实现有什么不同?

Vue 2 响应式原理

核心: Object.defineProperty

```
JavaScript
// Vue 2 响应式简化实现
class Vue2Reactive {
  constructor(data) {
    this.data = data;
    this.walk(data);
  }

  walk(obj) {
    const keys = Object.keys(obj);
    for (let key of keys) {
      this.defineReactive(obj, key, obj[key]);
    }
  }

  defineReactive(obj, key, val) {
    const dep = new Dep(); // 依赖收集器

    // 递归处理嵌套对象
    if (typeof val === 'object' && val !== null) {
      this.walk(val);
    }

    Object.defineProperty(obj, key, {
      enumerable: true,
      configurable: true,
      get() {
        // 收集依赖
        if (Dep.target) {
          dep.addSub(Dep.target);
        }
      }
    });
  }
}
```

```

        return val;
    },
    set(newVal) {
        if (newVal === val) return;
        val = newVal;
        // 触发更新
        dep.notify();
    }
    });
}
}

// 依赖收集器
class Dep {
    constructor() {
        this.subs = [];
    }

    addSub(sub) {
        this.subs.push(sub);
    }

    notify() {
        this.subs.forEach(sub => sub.update());
    }
}

```

Vue 2 的限制

```

Java
// 1. 无法检测属性的添加/删除
const vm = new Vue({
    data: {
        a: 1
    }
});

vm.b = 2; // 不是响应式的
Vue.set(vm, 'b', 2); // 需要使用 Vue.set

// 2. 数组索引和长度变化
const vm2 = new Vue({
    data: {

```

```
    items: ['a', 'b', 'c']
  }
});

vm2.items[1] = 'x'; // 不是响应式的
vm2.items.length = 1; // 不是响应式的

// 需要特殊处理
vm2.$set(vm2.items, 1, 'x'); // 使用$set
vm2.items.splice(1, 1, 'x'); // 使用变异方法
```

Vue 3 响应式原理

核心: Proxy + Reflect

```
JavaScript
// Vue 3 响应式简化实现
function reactive(target) {
  return createReactiveObject(target);
}

function createReactiveObject(target) {
  if (!isObject(target)) {
    return target;
  }

  const proxy = new Proxy(target, {
    get(target, key, receiver) {
      const res = Reflect.get(target, key, receiver);

      // 收集依赖
      track(target, key);

      // 递归处理嵌套对象
      if (isObject(res)) {
        return reactive(res);
      }

      return res;
    },
    set(target, key, value, receiver) {
```

```
const oldValue = target[key];
const result = Reflect.set(target, key, value, receiver);

// 触发更新
if (oldValue !== value) {
  trigger(target, key);
}

return result;
},

deleteProperty(target, key) {
  const hadKey = hasOwn(target, key);
  const result = Reflect.deleteProperty(target, key);

  if (hadKey) {
    trigger(target, key);
  }

  return result;
}
});

return proxy;
}

// 依赖收集系统
const targetMap = new WeakMap();
let activeEffect = null;

function track(target, key) {
  if (!activeEffect) return;

  let depsMap = targetMap.get(target);
  if (!depsMap) {
    targetMap.set(target, (depsMap = new Map()));
  }

  let dep = depsMap.get(key);
  if (!dep) {
    depsMap.set(key, (dep = new Set()));
  }

  dep.add(activeEffect);
```

```

}

function trigger(target, key) {
  const depsMap = targetMap.get(target);
  if (!depsMap) return;

  const effects = depsMap.get(key);
  effects && effects.forEach(effect => effect());
}

```

关键差异对比

1. API 设计差异

```

JavaScript
// Vue 2
const vm = new Vue({
  data() {
    return {
      count: 0,
      user: {
        name: 'John',
        age: 25
      }
    };
  }
});

// Vue 3 Composition API
import { reactive, ref, computed, watch } from 'vue';

export default {
  setup() {
    const count = ref(0);
    const user = reactive({
      name: 'John',
      age: 25
    });

    const doubleCount = computed(() => count.value * 2);

    watch(count, (newVal, oldVal) => {
      console.log(`Count changed: ${oldVal} -> ${newVal}`);
    });
  }
}

```

```
    });  
  
    return { count, user, doubleCount };  
  }  
};
```

2. 响应式类型对比

TypeScript

// Vue 3 的多种响应式 API

```
import { reactive, ref, readonly, shallowReactive, shallowRef }  
from 'vue';
```

// 完全响应式对象

```
const state = reactive({  
  nested: {  
    deep: {  
      value: 1  
    }  
  }  
});
```

// 响应式基本类型

```
const count = ref(0);
```

// 只读响应式

```
const readonlyState = readonly(state);
```

// 浅层响应式

```
const shallowState = shallowReactive({  
  nested: { value: 1 } // nested 变化不会触发更新  
});
```

// 浅层 ref

```
const shallowValue = shallowRef({ value: 1 });
```

3. 性能优化

JavaScript

// Vue 3 编译时优化示例

```
const _hoisted_1 = { id: "app" }; // 静态节点提升
```



```

// Patch flag
export function render(_ctx, _cache) {
  return (_openBlock(), _createBlock("div", _hoisted_1, [
    _createVNode("h1", null, _toDisplayString(_ctx.title), 1 /*
TEXT */),
    _createVNode("p", null, "静态内容"), // 无 flag
    _ctx.show ?
      (_openBlock(), _createBlock("div", { key: 0 }, "动态内容")) :
      _createCommentVNode("v-if", true)
  ]))
}

// 树结构拍平
const vnode = {
  type: 'div',
  children: [
    { type: 'p', children: 'static' }, // 静态
    { type: 'p', children: ctx.dynamic } // 动态
  ],
  // 只包含动态子节点的引用
  dynamicChildren: [
    { type: 'p', children: ctx.dynamic }
  ]
};

```

实际迁移示例

Vue 2 组件迁移

```

XML
<!-- Vue 2 Options API -->
<template>
  <div>
    <p>{{ fullName }}</p>
    <button @click="increment">Count: {{ count }}</button>
  </div>
</template>

<script>
export default {
  data() {

```

```

    return {
      firstName: 'John',
      lastName: 'Doe',
      count: 0
    };
  },

  computed: {
    fullName() {
      return `${this.firstName} ${this.lastName}`;
    }
  },

  methods: {
    increment() {
      this.count++;
    }
  },

  watch: {
    count(newVal) {
      console.log('Count changed:', newVal);
    }
  }
};
</script>

```

Vue 3 Composition API

XML

```

<!-- Vue 3 Composition API -->
<template>
  <div>
    <p>{{ fullName }}</p>
    <button @click="increment">Count: {{ count }}</button>
  </div>
</template>

<script setup>
import { ref, computed, watch } from 'vue';

const firstName = ref('John');
const lastName = ref('Doe');
const count = ref(0);

```

```
const fullName = computed(() =>
  `${firstName.value} ${lastName.value}`
);

function increment() {
  count.value++;
}

watch(count, (newVal) => {
  console.log('Count changed:', newVal);
}));
</script>
```

响应式工具函数对比

```
TypeScript
// Vue 2 响应式工具
Vue.set(object, 'newProp', 123);
Vue.delete(object, 'prop');
Vue.nextTick(() => {
  // DOM 更新后执行
});

// Vue 3 响应式工具
import { set, del, nextTick } from 'vue';

set(object, 'newProp', 123);
del(object, 'prop');
nextTick(() => {
  // DOM 更新后执行
});
```

76. CDN 的工作原理和优势是什么？

CDN 工作原理

基本架构

CSS

用户请求 → DNS 解析 → CDN 智能 DNS → 最近边缘节点
↓
边缘节点检查缓存 → 有缓存：直接返回
↓
无缓存：回源站获取 → 缓存到边缘节点 → 返回给用户

详细工作流程

```
JavaScript
// 模拟 CDN 请求流程
class CDNRequest {
  constructor() {
    this.edgeNodes = new Map(); // 边缘节点缓存
    this.originServer = 'https://origin.example.com';
  }

  async request(userLocation, resourceUrl) {
    // 1. 查找最近的边缘节点
    const edgeNode = this.findNearestEdgeNode(userLocation);

    // 2. 检查边缘节点缓存
    const cached = edgeNode.getCache(resourceUrl);

    if (cached && !this.isExpired(cached)) {
      // 3. 缓存命中，直接返回
      console.log('CDN 缓存命中');
      return this.enhanceResponse(cached);
    }

    // 4. 缓存未命中，回源站获取
    console.log('CDN 缓存未命中，回源站');
    const originResponse = await
    this.fetchFromOrigin(resourceUrl);

    // 5. 缓存到边缘节点
    edgeNode.setCache(resourceUrl, originResponse);

    // 6. 返回给用户
    return this.enhanceResponse(originResponse);
  }

  findNearestEdgeNode(location) {
```

```

// 基于用户位置选择最优节点
const nodes = [
  { id: 'us-east', location: 'New York', latency: 20 },
  { id: 'eu-west', location: 'London', latency: 40 },
  { id: 'ap-south', location: 'Singapore', latency: 100 }
];

// 简化的最近节点选择
return nodes.sort((a, b) => a.latency - b.latency)[0];
}
}

```

CDN 核心优势

1. 性能优化

```

JavaScript
// 性能对比数据
const performanceStats = {
  withoutCDN: {
    latency: '200-500ms',
    throughput: '10-50 Mbps',
    availability: '99%',
    timeToFirstByte: '300ms'
  },
  withCDN: {
    latency: '20-100ms',
    throughput: '100-1000 Mbps',
    availability: '99.9%',
    timeToFirstByte: '50ms'
  }
};

// 实际性能提升
function measurePerformance() {
  const resources = [
    'logo.png',
    'styles.css',
    'app.js',
    'api/data.json'
  ];
}

```

```
resources.forEach(resource => {
  const start = performance.now();

  // 模拟 CDN 请求
  fetch(`https://cdn.example.com/${resource}`)
    .then(() => {
      const end = performance.now();
      console.log(`${resource}: ${end - start}ms`);
    });
});
}
```

2. 缓存策略示例

YAML

CDN 缓存控制头

GET /image.jpg HTTP/1.1

Host: cdn.example.com

响应头示例

HTTP/1.1 200 OK

Content-Type: image/jpeg

Cache-Control: public, max-age=31536000

ETag: "xyz123"

Last-Modified: Tue, 15 Jan 2023 12:00:00 GMT

CDN-Cache-Control: max-age=86400

X-Cache: HIT from cache-node-nyc

3. 负载均衡

JavaScript

// CDN 负载均衡策略

```
class CDNLoadBalancer {
```

```
  constructor() {
```

```
    this.nodes = [
```

```
      { id: 'node1', capacity: 1000, currentLoad: 300, location: 'us-east' },
```

```
      { id: 'node2', capacity: 800, currentLoad: 500, location: 'eu-west' },
```

```
      { id: 'node3', capacity: 1200, currentLoad: 200, location: 'ap-south' }
    ]
  }
```



```

    ];
  }

  selectNode(userLocation, resourceType) {
    const strategies = {
      // 基于地理位置
      geoBased: () => {
        return this.nodes
          .filter(node => this.isNearby(node.location,
            userLocation))
          .sort((a, b) => a.currentLoad / a.capacity -
            b.currentLoad / b.capacity)[0];
      },

      // 基于负载
      loadBased: () => {
        return this.nodes
          .sort((a, b) => a.currentLoad - b.currentLoad)[0];
      },

      // 基于资源类型
      resourceBased: () => {
        const videoNodes = this.nodes.filter(n =>
          n.hasVideoOptimization);
        return videoNodes.sort((a, b) => a.currentLoad -
          b.currentLoad)[0];
      }
    };

    return strategies.geoBased();
  }
}

```

实际配置示例

Cloudflare Workers 配置

```

JavaScript
// CDN 边缘计算示例
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request));
});

```

```
async function handleRequest(request) {
  const url = new URL(request.url);

  // 1. 静态资源缓存
  if (url.pathname.startsWith('/static/')) {
    const cache = caches.default;
    let response = await cache.match(request);

    if (!response) {
      response = await fetch(request);

      // 设置缓存策略
      response = new Response(response.body, response);
      response.headers.set('Cache-Control', 'public, max-age=31536000');

      event.waitUntil(cache.put(request, response.clone()));
    }

    return response;
  }

  // 2. API 请求优化
  if (url.pathname.startsWith('/api/')) {
    // 添加边缘计算逻辑
    const userAgent = request.headers.get('User-Agent');
    const country = request.cf.country;

    // 基于地理位置定制响应
    if (country === 'CN') {
      return new Response('中国用户专属内容');
    }

    // 缓存 API 响应
    const cacheKey = new Request(url.toString(), request);
    const cache = caches.default;
    let response = await cache.match(cacheKey);

    if (!response) {
      response = await fetch(request);
      response = new Response(response.body, response);
      response.headers.set('Cache-Control', 'public, max-age=60');
      event.waitUntil(cache.put(cacheKey, response.clone()));
    }
  }
}
```

```
}

return response;
}

// 3. 动态内容加速
return fetch(request);
}
```

Nginx CDN 配置

```
Bash
# CDN 边缘节点配置示例
server {
    listen 80;
    server_name cdn.example.com;

    # 开启 gzip 压缩
    gzip on;
    gzip_types text/plain text/css application/json
    application/javascript;

    location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
        # 设置缓存
        expires 1y;
        add_header Cache-Control "public, immutable";

        # 代理到源站
        proxy_pass https://origin.example.com;

        # 缓存配置
        proxy_cache cdn_cache;
        proxy_cache_key $scheme$proxy_host$request_uri;
        proxy_cache_valid 200 302 1h;
        proxy_cache_valid 404 1m;

        # 添加 CDN 头部
        add_header X-CDN-Node $hostname;
        add_header X-Cache-Status $upstream_cache_status;
    }

    # API 请求处理
```

```
location /api/ {
    proxy_pass https://origin.example.com;
    proxy_set_header X-Real-IP $remote_addr;

    # 限制请求频率
    limit_req zone=api burst=10 nodelay;

    # 开启 keepalive
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}
}
```

现代 CDN 特性

1. 边缘计算

```
JavaScript
// 边缘函数示例: A/B 测试
export default {
  async fetch(request, env) {
    const cookie = request.headers.get('Cookie') || '';
    const url = new URL(request.url);

    // 在边缘进行 A/B 测试分配
    let variant = 'control';
    if (!cookie.includes('ab_test=')) {
      variant = Math.random() > 0.5 ? 'variant-a' : 'variant-b';
    } else {
      variant = cookie.match(/ab_test=([^\;]+)/)[1];
    }

    // 修改响应
    const response = await fetch(request);
    const modified = new HTMLRewriter()
      .on('h1', {
        element(element) {
          if (variant === 'variant-a') {
            element.setInnerContent('版本 A 的标题');
          } else {
            element.setInnerContent('版本 B 的标题');
          }
        }
      })
      .rewrite(response);
    return modified.toResponse(response.headers);
  }
}
```

```

    }
  })
  .transform(response);

  // 设置 cookie
  modified.headers.set('Set-Cookie', `ab_test=${variant}; Max-Age=86400`);

  return modified;
}
};

```

2. 安全防护

```

JavaScript
// CDN 安全功能示例
class CDNSecurity {
  constructor() {
    this.blockedIPs = new Set();
    this.rateLimits = new Map();
  }

  async checkRequest(request) {
    const ip = request.headers.get('CF-Connecting-IP');
    const userAgent = request.headers.get('User-Agent');
    const path = new URL(request.url).pathname;

    // 1. IP 黑名单检查
    if (this.blockedIPs.has(ip)) {
      return { allow: false, reason: 'IP blocked' };
    }

    // 2. 速率限制检查
    const rateKey = `${ip}:${path}`;
    const now = Date.now();
    const windowStart = now - 60000; // 1 分钟窗口

    if (!this.rateLimits.has(rateKey)) {
      this.rateLimits.set(rateKey, []);
    }

    const requests = this.rateLimits.get(rateKey);
    const recent = requests.filter(time => time > windowStart);

```

```

    if (recent.length > 100) { // 每分钟最多 100 次
      return { allow: false, reason: 'Rate limit exceeded' };
    }

    recent.push(now);
    this.rateLimits.set(rateKey, recent);

    // 3. DDoS 防护
    if (this.isDDOSAttack(request)) {
      this.blockedIPs.add(ip);
      return { allow: false, reason: 'DDoS detected' };
    }

    return { allow: true };
  }
}

```

77. 如何实现一个图片的懒加载？

基础实现方案

方案 1: IntersectionObserver API

```

JavaScript
class LazyLoader {
  constructor(options = {}) {
    this.options = {
      root: null,
      rootMargin: '0px',
      threshold: 0.1,
      ...options
    };
  }

  this.observer = null;
  this.images = new Map();
  this.init();
}

init() {
  // 创建观察器

```



```
this.observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      this.loadImage(entry.target);
      this.observer.unobserve(entry.target);
    }
  });
}, this.options);

// 查找所有需要懒加载的图片
document.querySelectorAll('img[data-src]').forEach(img => {
  this.observer.observe(img);
  this.images.set(img, {
    src: img.dataset.src,
    srcset: img.dataset.srcset,
    loaded: false
  });
});
}

loadImage(img) {
  const data = this.images.get(img);
  if (!data || data.loaded) return;

  // 设置占位背景色
  img.style.backgroundColor = '#f0f0f0';

  // 创建新的 Image 对象预加载
  const tempImage = new Image();

  tempImage.onload = () => {
    // 实际设置图片源
    if (data.src) img.src = data.src;
    if (data.srcset) img.srcset = data.srcset;

    // 添加加载完成样式
    img.style.opacity = '0';
    img.style.transition = 'opacity 0.3s';

    setTimeout(() => {
      img.style.opacity = '1';
      img.style.backgroundColor = 'transparent';
    }, 10);
  }
}
```

```

        data.loaded = true;
        img.dispatchEvent(new Event('lazyload:loaded'));
    };

    tempImage.onerror = () => {
        console.error('图片加载失败:', data.src);
        img.dispatchEvent(new Event('lazyload:error'));
    };

    tempImage.src = data.src;
}

// 手动触发加载
loadAll() {
    document.querySelectorAll('img[data-src]').forEach(img => {
        this.loadImage(img);
        this.observer.unobserve(img);
    });
}

// 添加新图片
addImage(img) {
    if (img.dataset.src) {
        this.observer.observe(img);
        this.images.set(img, {
            src: img.dataset.src,
            srcset: img.dataset.srcset,
            loaded: false
        });
    }
}
}

```

方案 2：传统滚动监听

```

JavaScript
class ScrollLazyLoader {
    constructor() {
        this.images = [];
        this.throttleTimer = null;
        this.init();
    }
}

```

```
init() {
  // 收集所有懒加载图片
  this.collectImages();

  // 监听滚动事件
  window.addEventListener('scroll',
this.handleScroll.bind(this));
  window.addEventListener('resize',
this.handleScroll.bind(this));

  // 初始检查
  this.checkImages();
}

collectImages() {
  this.images = Array.from(document.querySelectorAll('img[data-
src]'))
    .map(img => ({
      element: img,
      src: img.dataset.src,
      srcset: img.dataset.srcset,
      loaded: false
    }));
}

handleScroll() {
  // 节流处理
  if (this.throttleTimer) return;

  this.throttleTimer = setTimeout(() => {
    this.checkImages();
    this.throttleTimer = null;
  }, 100);
}

checkImages() {
  const viewport = {
    top: window.scrollY,
    bottom: window.scrollY + window.innerHeight,
    left: window.scrollX,
    right: window.scrollX + window.innerWidth
  };

  this.images.forEach(item => {
```

```

    if (item.loaded) return;

    const rect = item.element.getBoundingClientRect();
    const windowRect = {
      top: rect.top + window.scrollY,
      bottom: rect.bottom + window.scrollY,
      left: rect.left + window.scrollX,
      right: rect.right + window.scrollX
    };

    // 检查是否在视口中
    if (this.isInViewPort(viewport, windowRect)) {
      this.loadImage(item);
    }
  });
}

isInViewPort(viewport, rect) {
  return (
    rect.bottom >= viewport.top &&
    rect.top <= viewport.bottom &&
    rect.right >= viewport.left &&
    rect.left <= viewport.right
  );
}

loadImage(item) {
  item.loaded = true;

  // 预加载
  const tempImage = new Image();
  tempImage.onload = () => {
    if (item.src) item.element.src = item.src;
    if (item.srcset) item.element.srcset = item.srcset;
    item.element.classList.add('lazyloaded');
  };
  tempImage.src = item.src;
}
}

```

高级功能实现

响应式图片懒加载

XML

```
<!-- HTML 结构 -->


<!-- 背景图片懒加载 -->
<div
  class="lazy-bg"
  data-bg="url('background.jpg')"
  data-bg-mobile="url('background-mobile.jpg')"
  data-bg-retina="url('background@2x.jpg')"
>
  内容
</div>
```

JavaScript

```
class AdvancedLazyLoader {
  constructor() {
    this.observer = new
IntersectionObserver(this.handleIntersect.bind(this), {
  rootMargin: '50px 0px', // 提前 50px 加载
  threshold: 0
});

    this.init();
  }

  init() {
    // 图片元素
```

```
document.querySelectorAll('.lazy-image').forEach(img => {
  this.observer.observe(img);
});

// 背景图片元素
document.querySelectorAll('.lazy-bg').forEach(el => {
  this.observer.observe(el);
});

// iframe 懒加载
document.querySelectorAll('.lazy-iframe').forEach(iframe => {
  this.observer.observe(iframe);
});
}

handleIntersect(entries) {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const element = entry.target;

      if (element.classList.contains('lazy-image')) {
        this.loadImage(element);
      } else if (element.classList.contains('lazy-bg')) {
        this.loadBackground(element);
      } else if (element.classList.contains('lazy-iframe')) {
        this.loadIframe(element);
      }

      this.observer.unobserve(element);
    }
  });
}

loadImage(img) {
  // 处理响应式图片
  if (img.dataset.srcset) {
    img.srcset = img.dataset.srcset;
  }

  if (img.dataset.sizes) {
    img.sizes = img.dataset.sizes;
  }

  // 设置 src (作为 fallback)
```



```
if (img.dataset.src) {
  img.src = img.dataset.src;
}

// 添加加载动画
img.classList.add('loading');

img.onload = () => {
  img.classList.remove('loading');
  img.classList.add('loaded');

  // 触发自定义事件
  img.dispatchEvent(new CustomEvent('lazyload:complete', {
    detail: { element: img }
  }));
};

img.onerror = () => {
  console.error('图片加载失败:', img.dataset.src);
  img.classList.add('error');
  img.dispatchEvent(new CustomEvent('lazyload:error'));
};
}

loadBackground(element) {
  // 获取设备像素比
  const dpr = window.devicePixelRatio || 1;

  // 根据设备选择图片
  let bgUrl = element.dataset.bg;

  if (dpr >= 2 && element.dataset.bgRetina) {
    bgUrl = element.dataset.bgRetina;
  } else if (window.innerWidth <= 768 &&
element.dataset.bgMobile) {
    bgUrl = element.dataset.bgMobile;
  }

  // 预加载
  const tempImage = new Image();
  tempImage.onload = () => {
    element.style.backgroundImage = bgUrl;
    element.classList.add('bg-loaded');
  };
}
```

```

    tempImage.src = bgUrl.replace(/url\(['"]?(.*?)[']?\)/, '$1');
  }

  loadIframe(iframe) {
    if (iframe.dataset.src) {
      iframe.src = iframe.dataset.src;
      iframe.classList.add('iframe-loaded');
    }
  }
}

```

性能优化版本

JavaScript

```

class OptimizedLazyLoader {
  constructor() {
    this.images = new Set();
    this.priorityQueue = new Map(); // 优先级队列
    this.isLoading = false;
    this.concurrentLimit = 3; // 并发加载限制
    this.currentLoading = 0;

    this.init();
  }

  init() {
    // 使用 requestIdleCallback 进行初始化
    if ('requestIdleCallback' in window) {
      requestIdleCallback(() => this.setupObserver());
    } else {
      setTimeout(() => this.setupObserver(), 0);
    }
  }

  setupObserver() {
    this.observer = new IntersectionObserver(
      this.handleIntersect.bind(this),
      {
        root: null,
        rootMargin: this.calculateRootMargin(),
        threshold: this.calculateThresholds()
      }
    );
  }
}

```

```

    // 收集图片并设置优先级
    this.collectImages();
  }

  calculateRootMargin() {
    // 根据网络速度调整预加载距离
    const connection = navigator.connection;
    let margin = '500px';

    if (connection) {
      switch (connection.effectiveType) {
        case 'slow-2g':
        case '2g':
          margin = '100px';
          break;
        case '3g':
          margin = '300px';
          break;
        case '4g':
          margin = '500px';
          break;
      }
    }

    return `${margin} 0px ${margin} 0px`;
  }

  calculateThresholds() {
    // 根据设备性能调整阈值
    const isLowEndDevice =
      navigator.hardwareConcurrency <= 4 ||
      /Android|webOS|iPhone|iPad|iPod|BlackBerry|IEMobile|Opera
      Mini/i.test(navigator.userAgent);

    return isLowEndDevice ? [0.5] : [0, 0.25, 0.5, 0.75, 1];
  }

  collectImages() {
    const images = document.querySelectorAll('img[data-src]');

    images.forEach((img, index) => {
      // 根据位置设置优先级（越靠前优先级越高）
      const priority = 1 / (index + 1);

```

```
this.images.add({
  element: img,
  src: img.dataset.src,
  priority,
  loaded: false
});

// 设置低质量占位图
if (img.dataset.lqip) {
  img.src = img.dataset.lqip;
}

this.observer.observe(img);
});
}

handleIntersect(entries) {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      const imageData = this.findImageData(img);

      if (imageData && !imageData.loaded) {
        this.addToQueue(imageData);
        this.observer.unobserve(img);
      }
    }
  });
}

this.processQueue();
}

addToQueue(imageData) {
  this.priorityQueue.set(imageData.element, imageData);
}

processQueue() {
  if (this.isLoading || this.priorityQueue.size === 0) return;

  // 按优先级排序
  const sortedQueue = Array.from(this.priorityQueue.values())
    .sort((a, b) => b.priority - a.priority);
```

```
// 加载前 N 个
const toLoad = sortedQueue.slice(0, this.concurrentLimit);

toLoad.forEach(imageData => {
  this.loadImage(imageData);
  this.priorityQueue.delete(imageData.element);
});
}

async loadImage(imageData) {
  this.currentLoading++;

  try {
    // 使用 preload link
    const preload = document.createElement('link');
    preload.rel = 'preload';
    preload.as = 'image';
    preload.href = imageData.src;
    document.head.appendChild(preload);

    // 实际加载
    await new Promise((resolve, reject) => {
      const img = new Image();
      img.onload = resolve;
      img.onerror = reject;
      img.src = imageData.src;
    });

    // 设置到实际元素
    imageData.element.src = imageData.src;
    imageData.element.classList.add('lazyloaded');
    imageData.loaded = true;

    // 移除 preload
    preload.remove();

  } catch (error) {
    console.error('Failed to load image:', imageData.src);
    imageData.element.classList.add('lazyload-error');
  } finally {
    this.currentLoading--;

    // 继续处理队列
    if (this.priorityQueue.size > 0) {
```

```
        this.processQueue();
    }
}

findImageData(img) {
    return Array.from(this.images).find(data => data.element ===
img);
}
}
```

实际应用场景

电商网站图片懒加载

XML

```
<div class="product-grid">
  <div class="product-card">
    
    <div class="product-info">
      <h3>商品名称</h3>
      <p>价格: ¥199</p>
    </div>
  </div>
  <!-- 更多商品卡片 -->
</div>

<!-- 商品详情大图 -->
<div class="product-gallery">
  <img
    class="lazy-gallery-image active"
```



```
    data-src="/products/large/1-1.jpg"
    data-index="0"
    alt="商品大图 1"
  >
  
</div>
```

JavaScript

// 电商专用懒加载

```
class EcommerceLazyLoader {
  constructor() {
    this.observerOptions = {
      root: null,
      rootMargin: '200px 0px',
      threshold: 0.1
    };

    this.init();
  }

  init() {
    // 商品列表图片
    this.initProductImages();

    // 商品详情图
    this.initGalleryImages();

    // 用户评论图片
    this.initReviewImages();
  }

  initProductImages() {
    const productObserver = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          const img = entry.target;
          this.loadProductImage(img);
        }
      });
    });
  }
}
```

```

        productObserver.unobserve(img);
    }
  });
}, this.observerOptions);

document.querySelectorAll('.lazy-product-image').forEach(img
=> {
  // 设置占位色和加载动画
  img.style.backgroundColor =
this.getDominantColor(img.dataset.lqip);
  img.classList.add('loading');

  productObserver.observe(img);
});
}

initGalleryImages() {
  // 画廊图片延迟加载
  const galleryImages = document.querySelectorAll('.lazy-
gallery-image');

  // 预加载第一张
  if (galleryImages[0]) {
    this.loadImage(galleryImages[0]);
  }

  // 监听画廊切换
  document.querySelector('.product-
gallery').addEventListener('slideChange', (e) => {
    const nextIndex = e.detail.nextIndex;
    const nextImage = galleryImages[nextIndex];

    if (nextImage && !nextImage.src) {
      this.loadImage(nextImage);
    }
  });
}

loadProductImage(img) {
  // 优先加载缩略图
  const thumbnail = new Image();
  thumbnail.src = img.dataset.src;

  thumbnail.onload = () => {

```

```

img.src = img.dataset.src;
img.classList.remove('loading');
img.classList.add('loaded');

// 如果用户停留时间较长，加载高清图
setTimeout(() => {
  if (this.isElementInViewport(img)) {
    this.loadHighQualityImage(img);
  }
}, 1000);
};
}

loadHighQualityImage(img) {
  if (img.dataset.srcset && !img.classList.contains('hq-loaded')) {
    const hqImage = new Image();
    hqImage.srcset = img.dataset.srcset;
    hqImage.sizes = '(min-width: 768px) 400px, 100vw';

    hqImage.onload = () => {
      img.srcset = img.dataset.srcset;
      img.classList.add('hq-loaded');
    };
  }
}

isElementInViewport(el) {
  const rect = el.getBoundingClientRect();
  return (
    rect.top >= 0 &&
    rect.left >= 0 &&
    rect.bottom <= (window.innerHeight ||
document.documentElement.clientHeight) &&
    rect.right <= (window.innerWidth ||
document.documentElement.clientWidth)
  );
}
}

```

78. 谈谈 HTTP/2 相比 HTTP/1.1 有哪些重要的改进？

HTTP/2 核心特性

1. 二进制分帧层

```
Java
// HTTP/1.1 vs HTTP/2 对比
const httpComparison = {
  http11: {
    protocol: '文本协议',
    framing: '基于换行符的文本帧',
    parsing: '需要按字符解析',
    efficiency: '低'
  },

  http2: {
    protocol: '二进制协议',
    framing: '二进制分帧层',
    frameTypes: [
      'HEADERS',      // 头部帧
      'DATA',          // 数据帧
      'PRIORITY',      // 优先级帧
      'RST_STREAM',    // 流终止帧
      'SETTINGS',      // 设置帧
      'PUSH_PROMISE',  // 推送承诺帧
      'PING',          // PING 帧
      'GOAWAY',        // 断开连接帧
      'WINDOW_UPDATE'  // 窗口更新帧
    ],
    efficiency: '高'
  }
};
```

2. 多路复用

```
JavaScript
// HTTP/1.1 的问题: 队头阻塞
class Http11Request {
  constructor() {
    this.maxConnections = 6; // 浏览器限制
    this.activeConnections = 0;
```

```

    this.queue = [];
  }

  async makeRequests(urls) {
    // 串行或有限并行请求
    const results = [];

    for (let i = 0; i < urls.length; i++) {
      // 等待可用连接
      while (this.activeConnections >= this.maxConnections) {
        await this.wait();
      }

      this.activeConnections++;
      fetch(urls[i])
        .then(response => {
          results[i] = response;
          this.activeConnections--;
        });
    }

    return results;
  }
}

// HTTP/2 多路复用
class Http2Request {
  constructor() {
    this.streams = new Map();
    this.nextStreamId = 1;
  }

  async makeRequests(urls) {
    // 所有请求通过单一连接并行发送
    const promises = urls.map(url => {
      const streamId = this.nextStreamId++;
      return this.sendRequest(streamId, url);
    });

    return Promise.all(promises);
  }

  sendRequest(streamId, url) {
    return new Promise((resolve) => {

```

```

// 在同一连接上创建新流
this.streams.set(streamId, { resolve });

// 发送 HEADERS 帧
this.sendFrame({
  type: 'HEADERS',
  streamId,
  headers: {
    ':method': 'GET',
    ':path': new URL(url).pathname,
    ':authority': new URL(url).hostname
  }
});
});
}
}

```

HTTP/2 实际改进

头部压缩 (HPACK)

```

JavaScript
// HPACK 头部压缩示例
class HPACKCompressor {
  constructor() {
    // 静态表（预定义的常见头部）
    this.staticTable = [
      [':authority', ''],
      [':method', 'GET'],
      [':method', 'POST'],
      [':path', '/'],
      [':path', '/index.html'],
      // ... 61 个预定义条目
    ];

    // 动态表（维护最近发送的头部）
    this.dynamicTable = [];
    this.dynamicTableSize = 0;
    this.maxTableSize = 4096;
  }

  encode(headers) {

```



```

const encoded = [];

headers.forEach((value, name) => {
  // 查找完全匹配
  const staticIndex = this.findInStaticTable(name, value);
  if (staticIndex !== -1) {
    encoded.push(this.encodeIndexed(staticIndex));
    return;
  }

  // 查找名称匹配
  const nameIndex = this.findNameInTables(name);
  if (nameIndex !== -1) {
    encoded.push(this.encodeLiteralWithIndexing(nameIndex,
value));
  } else {
    encoded.push(this.encodeLiteralNewName(name, value));
  }

  // 添加到动态表
  this.addToDynamicTable(name, value);
});

return Buffer.concat(encoded);
}

decode(buffer) {
  // 解码过程
  const headers = new Map();
  let offset = 0;

  while (offset < buffer.length) {
    const firstByte = buffer[offset];

    if (firstByte & 0x80) {
      // 索引表示法
      const index = this.decodeInteger(buffer, offset, 7);
      const [name, value] = this.getFromTable(index);
      headers.set(name, value);
    } else if (firstByte & 0x40) {
      // 字面量头部字段带索引
      const index = this.decodeInteger(buffer, offset, 6);
      const name = this.getNameFromTable(index);
      const value = this.decodeString(buffer, offset);
    }
  }
}

```

```
        headers.set(name, value);
    }
    // ... 其他编码模式
}

return headers;
}
}
```

服务器推送

```
JavaScript
// HTTP/2 服务器推送实现
const http2 = require('http2');
const fs = require('fs');

const server = http2.createSecureServer({
  key: fs.readFileSync('server.key'),
  cert: fs.readFileSync('server.crt')
});

server.on('stream', (stream, headers) => {
  const path = headers[':path'];

  // 处理主请求
  if (path === '/index.html') {
    // 推送相关资源
    stream.pushStream({ ':path': '/styles.css' }, (err,
    pushStream) => {
      if (!err) {
        pushStream.respondWithFile('./public/styles.css', {
          'content-type': 'text/css'
        });
      }
    });
  }

  stream.pushStream({ ':path': '/app.js' }, (err, pushStream) =>
  {
    if (!err) {
      pushStream.respondWithFile('./public/app.js', {
        'content-type': 'application/javascript'
      });
    }
  }
});
```

```
});

// 响应主请求
stream.respondWithFile('./public/index.html', {
  'content-type': 'text/html'
});
}
});
```

性能对比数据

```
JavaScript
// 性能测试结果
const benchmarkResults = {
  http11: {
    pageLoadTime: '3.2s',
    totalRequests: '100',
    totalBytes: '2.1MB',
    waterfallTime: '2.8s',
    connectionCount: '6'
  },

  http2: {
    pageLoadTime: '1.8s',          // 提升 43%
    totalRequests: '100',
    totalBytes: '1.9MB',          // 头部压缩节省 10%
    waterfallTime: '0.9s',        // 减少 68%
    connectionCount: '1'          // 单一连接
  }
};

// 实际性能测试函数
async function testPerformance() {
  const testUrls = Array(50).fill().map((_, i) =>
    `/api/data/${i}`
  );

  // HTTP/1.1 测试
  console.time('HTTP/1.1');
  const http11Results = await Promise.all(
    testUrls.map(url => fetch(url))
  );
```

```
console.timeEnd('HTTP/1.1');

// HTTP/2 测试（实际需要支持 HTTP/2 的服务器）
console.time('HTTP/2');
const http2Results = await Promise.all(
  testUrls.map(url => fetch(url, {
    // 强制使用 HTTP/2
    headers: { 'X-Use-HTTP2': 'true' }
  })))
);
console.timeEnd('HTTP/2');
}
```

实际配置示例

Nginx HTTP/2 配置

Markdown

Nginx HTTP/2 配置

```
server {
    listen 443 ssl http2; # 启用 HTTP/2
    server_name example.com;

    # SSL 配置
    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    # HTTP/2 优化配置
    http2_push_preload on;
    http2_max_concurrent_streams 100;
    http2_streams_index_size 64;
    http2_recv_timeout 30s;

    # 头部压缩
    gzip on;
    gzip_types text/plain text/css application/json
    application/javascript;

    # 资源推送
    location = /index.html {
        http2_push /style.css;
    }
}
```

```
    http2_push /app.js;
    root /var/www/html;
}

# 静态资源缓存
location ~* \.(css|js|jpg|jpeg|png|gif|ico)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
}
```

Node.js HTTP/2 服务器

```
JavaScript
const http2 = require('http2');
const fs = require('fs');

const server = http2.createSecureServer({
  key: fs.readFileSync('localhost-privkey.pem'),
  cert: fs.readFileSync('localhost-cert.pem'),

  // HTTP/2 设置
  settings: {
    enablePush: true,
    initialWindowSize: 65535,
    maxFrameSize: 16384,
    maxConcurrentStreams: 100,
    maxHeaderListSize: 65536
  }
});

// 请求处理
server.on('stream', (stream, headers) => {
  const method = headers[':method'];
  const path = headers[':path'];

  // 处理不同请求
  switch (path) {
    case '/':
      handleHomePage(stream);
      break;
    case '/api/data':
      handleAPI(stream, headers);
```

```
        break;
      default:
        handleStatic(stream, path);
      }
    });

function handleHomePage(stream) {
  // 推送关键资源
  pushResource(stream, '/styles.css');
  pushResource(stream, '/bundle.js');

  // 响应 HTML
  stream.respond({
    'status': 200,
    'content-type': 'text/html; charset=utf-8'
  });

  stream.end(`
    <!DOCTYPE html>
    <html>
      <head>
        <link rel="stylesheet" href="/styles.css">
      </head>
      <body>
        <script src="/bundle.js"></script>
      </body>
    </html>
  `);
}

function pushResource(stream, path) {
  stream.pushStream({ 'path': path }, (err, pushStream) => {
    if (err) {
      console.error('Push failed:', err);
      return;
    }

    const filePath = `./public${path}`;
    const stat = fs.statSync(filePath);

    pushStream.respondWithFD(fs.openSync(filePath), {
      'content-type': getContentType(path),
      'content-length': stat.size
    });
  });
}
```



```
});  
}
```

HTTP/3 展望

JavaScript

// HTTP/3 基于 QUIC 协议

```
const http3Comparison = {  
  transportLayer: {  
    http2: 'TCP + TLS',  
    http3: 'QUIC (UDP)'  
  },  
  
  handshake: {  
    http2: '3-RTT (TCP+TLS)',  
    http3: '0-1 RTT'  
  },  
  
  headOfLineBlocking: {  
    http2: '应用层解决, TCP 层仍有',  
    http3: '完全解决'  
  },  
  
  connectionMigration: {  
    http2: '不支持',  
    http3: '支持 (切换网络不断连)'  
  }  
};
```

// 未来兼容性考虑

```
function setupProtocol() {  
  // 检测支持的协议  
  const protocol = window.performance  
    .getEntriesByType('navigation')[0]  
    .nextHopProtocol;  
  
  console.log('当前协议:', protocol);  
  
  // 根据协议优化  
  if (protocol === 'h2') {  
    // HTTP/2 优化策略  
    enableHTTP2Features();  
  }  
}
```

```
} else if (protocol === 'h3') {  
  // HTTP/3 优化策略  
  enableHTTP3Features();  
} else {  
  // HTTP/1.1 降级策略  
  enableFallbackFeatures();  
}  
}  
  
function enableHTTP2Features() {  
  // 启用服务器推送  
  // 使用多路复用  
  // 头部压缩优化  
}  
  
function enableHTTP3Features() {  
  // 更激进的资源推送  
  // 0-RTT 优化  
  // 连接迁移支持  
}
```

79. Content Security Policy 有助于防范哪些类型的攻击？

CSP 简介

Content Security Policy（内容安全策略）是一个 HTTP 响应头，用于限制页面可以加载哪些资源，从而减少 XSS 等攻击。

CSP 防范的攻击类型

1. 跨站脚本攻击（XSS）

```
XML  
<!-- 没有 CSP 的情况 -->  
<script>  
  // 攻击者注入恶意脚本  
  eval(location.hash.substring(1));  
</script>
```

```
<!-- 有 CSP 的情况 -->
<!-- 响应头: Content-Security-Policy: script-src 'self' -->
<!-- 此时外部脚本和 eval 将被阻止 -->
```

2. 点击劫持 (Clickjacking)

```
CSS
# 阻止点击劫持
Content-Security-Policy: frame-ancestors 'self';
# 或
Content-Security-Policy: frame-ancestors none;
```

3. 数据注入攻击

```
CSS
# 阻止内联脚本执行
Content-Security-Policy: script-src 'self';
# 这样注入的<script>alert('xss')</script>将不会执行
```

4. 恶意资源加载

```
CSS
# 限制图片只能从信任域加载
Content-Security-Policy: img-src https://cdn.example.com;
# 防止加载恶意图片或窃取信息的图片
```

5. 混合内容攻击

```
CSS
# 强制使用 HTTPS
Content-Security-Policy: upgrade-insecure-requests;
# 将所有 HTTP 请求升级为 HTTPS
```

CSP 指令详解

```
Java
// 完整的 CSP 配置示例
```

```
const cspConfig = {
  defaultSrc: ["'self'"], // 默认策略
  scriptSrc: [
    "'self'",
    "'unsafe-inline'", // 谨慎使用
    "'unsafe-eval'", // 谨慎使用
    "https://cdn.example.com",
    "https://www.google-analytics.com"
  ],
  styleSrc: [
    "'self'",
    "'unsafe-inline'", // 允许内联样式
    "https://fonts.googleapis.com"
  ],
  imgSrc: [
    "'self'",
    "data:", // 允许 data URL
    "https://*.example.com"
  ],
  fontSrc: [
    "'self'",
    "https://fonts.gstatic.com"
  ],
  connectSrc: [
    "'self'",
    "https://api.example.com",
    "wss://ws.example.com"
  ],
  frameSrc: [
    "'self'",
    "https://youtube.com"
  ],
  mediaSrc: [
    "'self'",
    "https://cdn.example.com"
  ],
  objectSrc: ["'none'"], // 禁止 Flash 等插件
  frameAncestors: ["'self'"], // 防止点击劫持
  reportUri: "/csp-violation-report", // 违规报告
  upgradeInsecureRequests: true
};
```

CSP 实际应用

开发环境配置

```
JavaScript
// Express.js 中配置 CSP
const helmet = require('helmet');
const app = express();

// 开发环境: 允许内联样式和 eval
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ['self'],
    scriptSrc: ['self', "'unsafe-inline'", "'unsafe-eval'"],
    styleSrc: ['self', "'unsafe-inline'"],
    imgSrc: ['self', "data:"],
    reportUri: '/report-violation'
  },
  reportOnly: true // 仅报告不阻止
}));
```

生产环境配置

```
CSS
# Nginx 生产环境 CSP 配置
add_header Content-Security-Policy "
  default-src 'self';
  script-src 'self' 'sha256-abc123...' https://cdn.example.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https://*.example.com;
  font-src 'self' https://fonts.gstatic.com;
  connect-src 'self' https://api.example.com;
  frame-ancestors 'self';
  upgrade-insecure-requests;
" always;
```

React 应用 CSP 配置

```
CSS
// create-react-app CSP 配置
// public/index.html
<meta
  http-equiv="Content-Security-Policy"
```

```
content="
  default-src 'self';
  script-src 'self' 'unsafe-inline' 'unsafe-eval';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https;;
  font-src 'self' data;;
  connect-src 'self' https://api.example.com;
"
>
```

CSP 违规报告

```
JavaScript
// 接收 CSP 违规报告的端点
app.post('/csp-violation-report', (req, res) => {
  const report = req.body['csp-report'];

  console.error('CSP 违规:', {
    violatedDirective: report['violated-directive'],
    blockedURI: report['blocked-uri'],
    documentURI: report['document-uri'],
    originalPolicy: report['original-policy'],
    referrer: report['referrer'],
    userAgent: req.headers['user-agent']
  });

  // 发送到监控服务
  sendToMonitoringService(report);

  res.status(204).end();
});
```

CSP 最佳实践

```
JavaScript
const cspBestPractices = {
  1: "使用 report-only 模式进行测试",
  2: "从不使用 'unsafe-inline' 和 'unsafe-eval'",
  3: "使用 nonce 或 hash 允许特定内联脚本",
  4: "定期审查和更新 CSP 策略",
  5: "集成到 CI/CD 流程中",
}
```



```

6: "监控违规报告并及时响应"
};

// 生成 nonce 示例
function generateNonceMiddleware(req, res, next) {
  res.locals.nonce = crypto.randomBytes(16).toString('base64');

  // 设置 CSP 头
  res.setHeader(
    'Content-Security-Policy',
    `script-src 'self' 'nonce-${res.locals.nonce}';`
  );

  next();
}

// 在模板中使用 nonce
app.get('/', generateNonceMiddleware, (req, res) => {
  res.render('index', {
    nonce: res.locals.nonce
  });
});

```

80. Vue 3 中的 ref 和 reactive 在定义响应式数据时应如何选择？

ref vs reactive 对比

特性	ref	reactive
数据类型	基本类型和对象	仅对象
访问方式	.value访问	直接访问
响应式原理	包装对象为{ value: ... }	Proxy代理对象
模板中使用	自动解包	直接使用
TypeScript支持	更好	需要类型定义

点击图片可查看完整电子表格

使用场景指南

场景 1：基本类型数据 - 使用 ref

```
JavaScript
import { ref } from 'vue';

// 基本类型必须用 ref
const count = ref(0); // number
const name = ref(''); // string
const isLoading = ref(false); // boolean
const price = ref(9.99); // number

// 在 setup 函数中
setup() {
  const count = ref(0);

  const increment = () => {
    count.value++; // 通过.value 访问
  };

  return {
    count, // 模板中自动解包，直接使用 count
    increment
  };
}
```

场景 2：对象数据 - 根据情况选择

```
JavaScript
import { ref, reactive } from 'vue';

// 情况 1：对象作为整体替换 - 使用 ref
const userRef = ref({
  name: 'John',
  age: 30
});

// 替换整个对象
userRef.value = {
  name: 'Jane',
  age: 25
};
```

```
// 情况 2: 对象属性频繁修改 - 使用 reactive
const userReactive = reactive({
  name: 'John',
  age: 30,
  profile: {
    bio: 'Developer',
    skills: ['JavaScript', 'Vue']
  }
});

// 直接修改属性
userReactive.name = 'Jane';
userReactive.profile.skills.push('React');
```

场景 3: 表单数据 - 使用 reactive

```
JavaScript
import { reactive } from 'vue';

setup() {
  // 表单对象使用 reactive 更直观
  const form = reactive({
    username: '',
    password: '',
    rememberMe: false,
    preferences: {
      theme: 'dark',
      notifications: true
    }
  });

  const submitForm = () => {
    // 直接访问属性, 无需.value
    console.log(form.username, form.password);
  };

  return { form, submitForm };
}
```

实际应用示例

示例 1：复杂状态管理

```
JavaScript
import { reactive, ref, computed, watch } from 'vue';

export function useUserStore() {
  // 用户基本信息用 reactive
  const state = reactive({
    user: null,
    isAuthenticated: false,
    loading: false,
    error: null
  });

  // 单个值用 ref
  const retryCount = ref(0);
  const lastLoginTime = ref(null);

  // computed 计算属性
  const userName = computed(() =>
    state.user ? state.user.name : 'Guest'
  );

  // 方法
  async function login(credentials) {
    state.loading = true;
    state.error = null;

    try {
      const response = await api.login(credentials);
      state.user = response.user;
      state.isAuthenticated = true;
      lastLoginTime.value = new Date();
      retryCount.value = 0;
    } catch (error) {
      state.error = error.message;
      retryCount.value++;
    } finally {
      state.loading = false;
    }
  }

  return {
```

```
// 导出 state 属性
...toRefs(state),
// 导出 ref
retryCount,
lastLoginTime,
// 导出 computed
userName,
// 导出方法
login
};
}
```

示例 2：组合式函数

```
JavaScript
import { ref, reactive, watchEffect } from 'vue';

// 使用 ref 的场景
export function useCounter(initialValue = 0) {
  const count = ref(initialValue);

  const increment = () => count.value++;
  const decrement = () => count.value--;
  const reset = () => count.value = initialValue;

  return {
    count,
    increment,
    decrement,
    reset
  };
}

// 使用 reactive 的场景
export function useForm(initialValues = {}) {
  const form = reactive({ ...initialValues });
  const errors = reactive({});
  const isSubmitting = ref(false);

  const validate = () => {
    Object.keys(errors).forEach(key => delete errors[key]);

    if (!form.username) {
```

```

    errors.username = '用户名不能为空';
  }

  if (!form.email.includes('@')) {
    errors.email = '邮箱格式不正确';
  }

  return Object.keys(errors).length === 0;
};

const submit = async () => {
  if (!validate()) return;

  isSubmitting.value = true;
  try {
    await api.submitForm(form);
  } finally {
    isSubmitting.value = false;
  }
};

return {
  form,
  errors,
  isSubmitting,
  submit,
  validate
};
}

```

TypeScript 中的最佳实践

```

TypeScript
import { ref, reactive, Ref, ComputedRef } from 'vue';

// 1. 明确类型定义
interface User {
  id: number;
  name: string;
  email: string;
}

interface Product {

```



```
id: number;
name: string;
price: number;
inStock: boolean;
}

// 2. 使用泛型
const user = ref<User | null>(null);
const products = ref<Product[]>([]);
const loading = ref<boolean>(false);

// 3. reactive 的类型推断
const cart = reactive<{
  items: Product[];
  total: number;
}>({
  items: [],
  total: 0
});

// 4. 组合式函数的类型
export function useProductFilter(products: Ref<Product[]>) {
  const searchQuery = ref<string>('');
  const minPrice = ref<number>(0);
  const maxPrice = ref<number>(1000);

  const filteredProducts = computed<Product[]>(() => {
    return products.value.filter(product => {
      const matchesSearch = product.name
        .toLowerCase()
        .includes(searchQuery.value.toLowerCase());

      const matchesPrice = product.price >= minPrice.value &&
        product.price <= maxPrice.value;

      return matchesSearch && matchesPrice;
    });
  });

  return {
    searchQuery,
    minPrice,
    maxPrice,
    filteredProducts
  }
}
```

```
};  
}
```

性能考虑

```
JavaScript  
import { ref, reactive, shallowRef, shallowReactive, markRaw }  
from 'vue';  
  
// 1. 浅层响应式 - 性能优化  
const largeList = shallowRef([]); // 只跟踪.value 的变化  
const config = shallowReactive({ // 只跟踪第一层属性  
  apiUrl: 'https://api.example.com',  
  timeout: 5000,  
  headers: { // 嵌套对象不会自动响应式  
    'Content-Type': 'application/json'  
  }  
});  
  
// 2. 避免不必要的响应式  
const constants = markRaw({  
  PI: 3.14159,  
  MAX_ITEMS: 1000,  
  COLORS: ['red', 'green', 'blue']  
});  
  
// 3. 大对象使用 ref 替代 reactive  
// 如果对象很大且不经常修改属性，用 ref 性能更好  
const largeData = ref({  
  // 大数据对象  
});  
  
// 4. 批量更新  
const state = reactive({  
  items: [],  
  loading: false,  
  error: null  
});  
  
function updateItems(newItems) {  
  // 一次性更新，减少触发次数  
  state.items = newItems;
```

```
state.loading = false;
state.error = null;
}
```

81. 微前端架构旨在解决什么问题，同时会引入哪些新的挑战？

微前端解决的问题

1. 单体应用拆分

CSS

单体巨石应用问题：

- 代码库庞大，构建时间长
- 团队协作困难
- 技术栈难以升级
- 发布风险高

微前端解决方案：

Portal（主应用）		
团队 A (React)	团队 B (Vue)	团队 C (Ang)

2. 团队自治

CSS

// 各团队独立开发部署

```
const teamConfigs = {
  teamA: {
    framework: 'React 18',
    ciCd: '独立流水线',
    domain: 'products.example.com',
    deploy: '随时部署'
  },
  teamB: {
```

```
framework: 'Vue 3',
ciCd: '独立流水线',
domain: 'orders.example.com',
deploy: '按需发布'
}
};
```

3. 技术栈多样性

XML

```
<!-- 同一页面混合不同技术栈 -->
<div id="app">
  <!-- React 组件 -->
  <div id="product-list"></div>

  <!-- Vue 组件 -->
  <div id="shopping-cart"></div>

  <!-- Angular 组件 -->
  <div id="user-profile"></div>

  <!-- 原生 Web 组件 -->
  <search-widget></search-widget>
</div>
```

微前端实现方案

方案 1：基座模式（Single-SPA）

JavaScript

```
// single-spa 配置
import { registerApplication, start } from 'single-spa';

// 注册应用
registerApplication({
  name: 'product-app',
  app: () => System.import('@team-a/product-app'),
  activeWhen: '/products'
});

registerApplication({
```

```
name: 'order-app',
app: () => System.import('@team-b/order-app'),
activeWhen: '/orders'
});

// 启动
start();
```

方案 2: Web Components

```
JavaScript
// 使用 Web Components 封装微前端
class ProductWidget extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
  }

  connectedCallback() {
    this.render();
    this.loadApp();
  }

  async loadApp() {
    // 动态加载应用
    const { default: initApp } = await import(
      'https://cdn.team-a.com/product-app.js'
    );
    initApp(this.shadowRoot);
  }

  render() {
    this.shadowRoot.innerHTML = `
      <style>
        :host {
          display: block;
          border: 1px solid #ccc;
          padding: 20px;
        }
      </style>
      <div id="app-container"></div>
    `;
  }
}
```

```
}  
  
customElements.define('product-widget', ProductWidget);
```

方案 3：模块联邦（Module Federation）

```
Java  
// webpack.config.js - 主应用  
const ModuleFederationPlugin =  
require('webpack/lib/container/ModuleFederationPlugin');  
  
module.exports = {  
  plugins: [  
    new ModuleFederationPlugin({  
      name: 'host',  
      remotes: {  
        productApp: 'productApp@https://cdn.team-a.com/remoteEntry.js',  
        orderApp: 'orderApp@https://cdn.team-b.com/remoteEntry.js'  
      },  
      shared: ['react', 'react-dom', 'react-router-dom']  
    })  
  ]  
};  
  
// 使用远程模块  
import('productApp/ProductList').then(module => {  
  const ProductList = module.default;  
  // 使用 ProductList 组件  
});
```

微前端引入的挑战

挑战 1：样式隔离

```
JavaScript  
/* 解决方案 1: CSS-in-JS */  
const styles = css`  
  .button {  
    /* 作用域样式 */  
  }  
`
```



```

`;

/* 解决方案 2: Shadow DOM */
class MicroApp extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
  }
}

/* 解决方案 3: 命名约定 */
.mfe-product__button { /* 产品团队按钮 */ }
.mfe-order__button { /* 订单团队按钮 */ }

```

挑战 2：状态管理

```

JavaScript
// 跨应用状态管理方案
class GlobalStateManager {
  constructor() {
    this.state = {};
    this.listeners = new Map();
  }

  // 发布-订阅模式
  subscribe(key, callback) {
    if (!this.listeners.has(key)) {
      this.listeners.set(key, new Set());
    }
    this.listeners.get(key).add(callback);

    return () => {
      this.listeners.get(key).delete(callback);
    };
  }

  setState(key, value) {
    this.state[key] = value;

    if (this.listeners.has(key)) {
      this.listeners.get(key).forEach(callback => {
        callback(value);
      });
    }
  }
}

```

```
    }  
  }  
  
  getState(key) {  
    return this.state[key];  
  }  
}  
  
// 或使用 Redux + 事件总线  
const eventBus = new EventEmitter();
```

挑战 3：路由管理

```
JavaScript  
// 统一路由管理  
class RouterManager {  
  constructor() {  
    this.routes = new Map();  
    this.currentApp = null;  
  }  
  
  registerApp(name, routerConfig) {  
    this.routes.set(name, routerConfig);  
  }  
  
  navigate(path) {  
    // 解析路径属于哪个应用  
    const appName = this.resolveApp(path);  
  
    if (appName !== this.currentApp) {  
      // 卸载当前应用  
      this.unmountCurrentApp();  
  
      // 加载新应用  
      this.loadApp(appName);  
  
      this.currentApp = appName;  
    }  
  
    // 传递路由到子应用  
    this.routes.get(appName).navigate(path);  
  }  
}
```

```
}
```

挑战 4：性能优化

JavaScript

// 资源加载优化策略

```
class ResourceManager {
```

```
  constructor() {
```

```
    this.preloadedApps = new Set();
```

```
    this.cache = new Map();
```

```
  }
```

// 预加载非活跃应用

```
preloadApp(appName) {
```

```
  if (this.preloadedApps.has(appName)) return;
```

```
  const link = document.createElement('link');
```

```
  link.rel = 'preload';
```

```
  link.as = 'script';
```

```
  link.href = this.getAppUrl(appName);
```

```
  document.head.appendChild(link);
```

```
  this.preloadedApps.add(appName);
```

```
}
```

// 应用卸载时保留缓存

```
unloadApp(appName) {
```

```
  const app = this.getApp(appName);
```

// 保留 DOM 快照

```
  const snapshot = app.container.innerHTML;
```

```
  this.cache.set(appName, {
```

```
    snapshot,
```

```
    timestamp: Date.now()
```

```
  });
```

// 清理过期的快照

```
this.cleanupOldSnapshots();
```

```
}
```

// 应用切换动画

```
async switchApp(fromApp, toApp, animation = 'fade') {
```

```
  // 显示加载状态
```

```
this.showLoading();

// 并行执行：卸载旧应用 + 加载新应用
await Promise.all([
  this.unmountApp(fromApp),
  this.loadApp(toApp)
]);

// 隐藏加载状态
this.hideLoading();
}
}
```

微前端监控和调试

```
JavaScript
// 微前端监控系统
class MicroFrontendMonitor {
  constructor() {
    this.metrics = {
      appLoadTimes: {},
      errorCount: 0,
      activeApps: new Set()
    };

    this.initMonitoring();
  }

  initMonitoring() {
    // 性能监控
    const observer = new PerformanceObserver((list) => {
      list.getEntries().forEach(entry => {
        if (entry.entryType === 'resource') {
          this.trackResourceLoad(entry);
        }
      });
    });

    observer.observe({ entryTypes: ['resource'] });

    // 错误监控
    window.addEventListener('error', (event) => {
      this.trackError(event.error);
    });
  }
}
```

```
});

// 自定义事件
window.addEventListener('microfrontend:app-mounted', (event)
=> {
  this.metrics.activeApps.add(event.detail.appName);
  this.emitMetrics();
});
}

trackResourceLoad(entry) {
  // 记录各应用资源加载时间
  console.log(`Resource loaded: ${entry.name}, time:
${entry.duration}ms`);
}

trackError(error) {
  this.metrics.errorCount++;

  // 发送到错误监控服务
  fetch('/api/errors', {
    method: 'POST',
    body: JSON.stringify({
      error: error.message,
      stack: error.stack,
      timestamp: new Date().toISOString()
    })
  });
}

emitMetrics() {
  // 定期发送指标
  setInterval(() => {
    const metrics = {
      timestamp: Date.now(),
      ...this.metrics
    };

    // 发送到监控平台
    this.sendToMonitoringPlatform(metrics);
  }, 60000);
}
}
```

82. 谈谈你对 React 并发模式 (Concurrent Mode) 的理解，以及 useTransition 和 useDeferredValue 的作用。

React 并发模式 (Concurrent Mode)

核心概念

```
JavaScript
// 传统渲染（同步，不可中断）
function syncRender(element, container) {
  // 一旦开始渲染，必须完成
  // 如果组件树很大，会阻塞主线程
}

// 并发渲染（可中断）
function concurrentRender(element, container) {
  // 将渲染工作分成小单元
  // 可以在单元之间中断，让出主线程
  // 优先级高的更新可以插队
}
```

启用并发模式

```
JavaScript
// React 18 之前
ReactDOM.render(<App />, document.getElementById('root'));

// React 18 之后
import { createRoot } from 'react-dom/client';

const root = createRoot(document.getElementById('root'));
root.render(<App />);

// 或使用并发特性
import { unstable_createRoot as createRoot } from 'react-dom';

const root = createRoot(document.getElementById('root'));
root.render(
```



```
<React.unstable_ConcurrentMode>
  <App />
</React.unstable_ConcurrentMode>
);
```

useTransition

基本用法

```
JavaScript
import { useState, useTransition } from 'react';

function SearchBox() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    const value = e.target.value;

    // 紧急更新: 用户输入
    setQuery(value);

    // 非紧急更新: 搜索结果
    startTransition(() => {
      fetchResults(value).then(setResults);
    });
  }

  return (
    <div>
      <input
        value={query}
        onChange={handleChange}
        placeholder="搜索..."
      />

      {/* 显示加载状态 */}
      {isPending && <div className="spinner">搜索中...</div>}

      {/* 结果列表 */}
      <ul>
```

```

        {results.map(result => (
            <li key={result.id}>{result.name}</li>
        ))}
    </ul>
</div>
);
}

```

实际应用场景

JavaScript

```

function ProductList() {
    const [filter, setFilter] = useState('');
    const [filteredProducts, setFilteredProducts] = useState([]);
    const [isPending, startTransition] = useTransition();

    // 昂贵的计算
    const expensiveFilter = (products, filter) => {
        // 模拟大量计算
        return products.filter(product =>
            product.name.toLowerCase().includes(filter.toLowerCase())
        );
    };

    useEffect(() => {
        // 将昂贵计算包装在 transition 中
        startTransition(() => {
            const filtered = expensiveFilter(allProducts, filter);
            setFilteredProducts(filtered);
        });
    }, [filter]);

    return (
        <div>
            <input
                value={filter}
                onChange={(e) => setFilter(e.target.value)}
                placeholder="过滤产品..."
            />

            {/* 使用 isPending 优化用户体验 */}
            <div style={{ opacity: isPending ? 0.5 : 1 }}>
                {filteredProducts.map(product => (

```

```
        <ProductItem key={product.id} product={product} />
      )}}
    </div>
  </div>
);
}
```

useDeferredValue

基本用法

```
JavaScript
import { useState, useDeferredValue, memo } from 'react';

function SearchApp() {
  const [query, setQuery] = useState('');
  const deferredQuery = useDeferredValue(query);

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="输入搜索词..."
      />

      {/* 结果列表使用延迟的值 */}
      <SearchResults query={deferredQuery} />
    </div>
  );
}

const SearchResults = memo(({ query }) => {
  // 这个组件只在 deferredQuery 变化时重新渲染
  // 即使 query 频繁变化，也不会频繁重新渲染

  const results = useMemo(() => {
    return expensiveSearch(query);
  }, [query]);

  return (
    <ul>
```

```

      {results.map(result => (
        <li key={result.id}>{result.name}</li>
      ))}
    </ul>
  );
});

```

与 useTransition 对比

JavaScript

// 方案 1: useTransition

```

function WithTransition() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    const value = e.target.value;
    setQuery(value);

    startTransition(() => {
      setResults(expensiveSearch(value));
    });
  }

  return (
    <div>
      <input value={query} onChange={handleChange} />
      {isPending ? '搜索中...' : <ResultsList results={results} />}
    </div>
  );
}

```

// 方案 2: useDeferredValue

```

function WithDeferredValue() {
  const [query, setQuery] = useState('');
  const deferredQuery = useDeferredValue(query);

  const results = useMemo(() => {
    return expensiveSearch(deferredQuery);
  }, [deferredQuery]);

  return (

```

```

    <div>
      <input value={query} onChange={(e) =>
setQuery(e.target.value)} />
      <ResultsList results={results} />
    </div>
  );
}

// 选择指南:
// - useTransition: 控制状态更新的时机, 有明确的"pending"状态
// - useDeferredValue: 延迟某个值, 自动处理新旧值切换

```

并发渲染特性

Suspense 集成

```

JavaScript
import { Suspense, lazy } from 'react';

// 延迟加载组件
const ProductDetails = lazy(() => import('./ProductDetails'));

function ProductPage({ productId }) {
  return (
    <div>
      <ProductHeader />

      {/* Suspense 边界 */}
      <Suspense fallback={<ProductSkeleton />}>
        <ProductDetails productId={productId} />
      </Suspense>

      <ProductReviews />
    </div>
  );
}

// 数据获取的 Suspense
function UserProfile({ userId }) {
  // 假设 useUser 是一个支持 Suspense 的 hook
  const user = useUser(userId);
}

```

```

return (
  <div>
    <h1>{user.name}</h1>
    <p>{user.bio}</p>
  </div>
);
}

// 在组件树中使用
function App() {
  return (
    <Suspense fallback={<GlobalLoading />}>
      <ErrorBoundary>
        <Router>
          <Route path="/user/:id" element={<UserProfile />} />
        </Router>
      </ErrorBoundary>
    </Suspense>
  );
}

```

流式 SSR

```

JavaScript
// React 18 流式服务端渲染
import { renderToPipeableStream } from 'react-dom/server';

app.get('*', (req, res) => {
  const { pipe } = renderToPipeableStream(
    <App />,
    {
      // 立即发送 HTML 骨架
      bootstrapScripts: ['/main.js'],

      // 所有内容加载完成后执行
      onShellReady() {
        res.setHeader('Content-type', 'text/html');
        pipe(res);
      },

      // 部分内容加载完成后执行（流式）
      onShellError(error) {
        console.error(error);
      }
    }
  );
});

```



```
    res.status(500).send('出错了');
  },

  // 渐进式加载
  onAllReady() {
    console.log('所有内容已加载');
  }
}
);
});
```

性能优化模式

82.1.1 防抖模式

```
JavaScript
function DebouncedSearch() {
  const [query, setQuery] = useState('');
  const deferredQuery = useDeferredValue(query);
  const [results, setResults] = useState([]);

  // 使用 useEffect 和防抖
  useEffect(() => {
    const timer = setTimeout(() => {
      fetchResults(deferredQuery).then(setResults);
    }, 300);

    return () => clearTimeout(timer);
  }, [deferredQuery]);

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <SearchResults results={results} />
    </div>
  );
}
```

82.1.2 优先级调度

```
JavaScript
function InteractiveApp() {
  const [urgentState, setUrgentState] = useState('');
  const [backgroundState, setBackgroundState] = useState('');
  const [isPending, startTransition] = useTransition();

  // 紧急更新（用户交互）
  const handleUrgentAction = () => {
    setUrgentState('updated');
  };

  // 非紧急更新（后台任务）
  const handleBackgroundAction = () => {
    startTransition(() => {
      setBackgroundState(computeExpensiveValue());
    });
  };

  return (
    <div>
      {/* 紧急更新立即显示 */}
      <button onClick={handleUrgentAction}>
        立即响应
      </button>

      {/* 非紧急更新显示加载状态 */}
      <button onClick={handleBackgroundAction}>
        后台处理 {isPending && '...'}
      </button>

      <div>
        <UrgentComponent value={urgentState} />
        <BackgroundComponent value={backgroundState} />
      </div>
    </div>
  );
}
```

实际案例分析

JavaScript

// 大型列表渲染优化

```
function VirtualizedList({ items }) {  
  const [visibleRange, setVisibleRange] = useState({ start: 0,  
end: 20 });  
  const deferredRange = useDeferredValue(visibleRange);  
  const [isScrolling, startTransition] = useTransition();
```

// 监听滚动

```
useEffect(() => {  
  const handleScroll = () => {  
    const start = Math.floor(window.scrollY / 50);  
    const end = start + 20;
```

// 滚动更新使用 transition

```
startTransition(() => {  
  setVisibleRange({ start, end });  
});  
};
```

```
  window.addEventListener('scroll', handleScroll, { passive:  
true });  
  return () => window.removeEventListener('scroll',  
handleScroll);  
}, []);
```

// 只渲染可见项

```
const visibleItems = useMemo(() => {  
  return items.slice(  
    deferredRange.start,  
    deferredRange.end  
  );  
}, [items, deferredRange]);
```

```
return (  
  <div style={{ height: items.length * 50 }}>  
    { /* 滚动指示器 */ }  
    { isScrolling && (  
      <div className="scroll-indicator">  
        滚动中...  
      </div>  
    ) }  
  )
```

```

    {/* 虚拟列表 */}
    <div style={{ position: 'relative' }}>
      {visibleItems.map((item, index) => (
        <div
          key={item.id}
          style={{
            position: 'absolute',
            top: (deferredRange.start + index) * 50,
            height: 50
          }}
        >
          {item.content}
        </div>
      ))}
    </div>
  </div>
);
}

```

83. 抽象语法树 AST 在前端领域有哪些应用？

AST 基础概念

```

CSS
// 源代码
const code = `function add(a, b) {
  return a + b;
}`;

// 对应的 AST 结构
const ast = {
  type: 'Program',
  body: [{
    type: 'FunctionDeclaration',
    id: {
      type: 'Identifier',
      name: 'add'
    },
    params: [
      { type: 'Identifier', name: 'a' },
      { type: 'Identifier', name: 'b' }
    ],
  }],
};

```

```

body: {
  type: 'BlockStatement',
  body: [{
    type: 'ReturnStatement',
    argument: {
      type: 'BinaryExpression',
      operator: '+',
      left: { type: 'Identifier', name: 'a' },
      right: { type: 'Identifier', name: 'b' }
    }
  }]
}
}]
};

```

AST 在前端的应用

1. 代码转换 (Babel 插件)

```

JavaScript
// Babel 插件: 将箭头函数转为普通函数
const babelPlugin = {
  visitor: {
    ArrowFunctionExpression(path) {
      const { node } = path;

      // 创建函数表达式
      const functionExpression = {
        type: 'FunctionExpression',
        id: null,
        params: node.params,
        body: node.body,
        generator: false,
        async: false
      };

      // 替换 AST 节点
      path.replaceWith(functionExpression);
    }
  }
};

```

```
// 使用示例
const babel = require('@babel/core');
const code = `const add = (a, b) => a + b;`;

const result = babel.transform(code, {
  plugins: [babelPlugin]
});

console.log(result.code);
// 输出: const add = function(a, b) { return a + b; };
```

2. 代码格式化 (Prettier)

```
JavaScript
// 简化的格式化逻辑
function formatCode(ast) {
  // 遍历 AST, 添加缩进和换行
  function formatNode(node, indent = 0) {
    const spaces = ' '.repeat(indent * 2);

    switch (node.type) {
      case 'FunctionDeclaration':
        return `${spaces}function
${node.id.name}(${node.params.map(p => p.name).join(', ')} ) {\n` +
          node.body.body.map(stmt => formatNode(stmt, indent
+ 1)).join('\n') +
          `\n${spaces}}`;

      case 'ReturnStatement':
        return `${spaces}return ${formatNode(node.argument, 0)};`;

      case 'BinaryExpression':
        return `${formatNode(node.left, 0)} ${node.operator}
${formatNode(node.right, 0)}`;

      case 'Identifier':
        return node.name;
    }
  }

  return ast.body.map(node => formatNode(node)).join('\n');
}
```


3. 代码检查 (ESLint)

```
JavaScript
// ESLint 规则: 禁止使用 console.log
module.exports = {
  create(context) {
    return {
      MemberExpression(node) {
        if (node.object.name === 'console' &&
            node.property.name === 'log') {
          context.report({
            node,
            message: '禁止使用 console.log'
          });
        }
      }
    };
  }
};
```

// AST 遍历查找特定模式

```
function findConsoleLogs(ast) {
  const issues = [];

  function traverse(node) {
    if (node.type === 'MemberExpression' &&
        node.object.type === 'Identifier' &&
        node.object.name === 'console' &&
        node.property.type === 'Identifier' &&
        node.property.name === 'log') {

      issues.push({
        type: 'console.log',
        line: node.loc.start.line,
        column: node.loc.start.column
      });
    }
  }
}
```

// 递归遍历子节点

```
for (const key in node) {
  if (node[key] && typeof node[key] === 'object') {
    if (Array.isArray(node[key])) {
      node[key].forEach(child => {
        if (child && typeof child === 'object') {
```

```

        traverse(child);
    }
    });
    } else if (node[key].type) {
        traverse(node[key]);
    }
    }
    }
    }
    }

    traverse(ast);
    return issues;
}

```

4. 代码压缩 (UglifyJS/Terser)

```

JavaScript
// 简化的变量名压缩
function compressVariables(ast) {
    const variableMap = new Map();
    let nextId = 0;

    function generateName(id) {
        // 生成短变量名: a, b, c, ... aa, ab, ...
        let name = '';
        do {
            name = String.fromCharCode(97 + (id % 26)) + name;
            id = Math.floor(id / 26) - 1;
        } while (id >= 0);
        return name;
    }

    function renameVariables(node) {
        if (node.type === 'VariableDeclarator' &&
            node.id.type === 'Identifier') {

            const oldName = node.id.name;
            const newName = generateName(nextId++);

            variableMap.set(oldName, newName);
            node.id.name = newName;
        }
    }
}

```

```

    // 更新所有标识符引用
    if (node.type === 'Identifier' && variableMap.has(node.name))
    {
        node.name = variableMap.get(node.name);
    }
}

// 遍历 AST 并重命名
traverse(ast, renameVariables);
return ast;
}

```

高级应用场景

1. 代码分析工具

```

JavaScript
// 计算代码复杂度
function calculateComplexity(ast) {
    let complexity = 0;

    function traverse(node) {
        // 增加复杂度的结构
        const complexityNodes = [
            'IfStatement',
            'ForStatement',
            'WhileStatement',
            'DoWhileStatement',
            'SwitchStatement',
            'ConditionalExpression',
            'LogicalExpression' // && 和 ||
        ];

        if (complexityNodes.includes(node.type)) {
            complexity++;
        }

        // 递归遍历
        for (const key in node) {
            if (node[key] && typeof node[key] === 'object') {
                if (Array.isArray(node[key])) {
                    node[key].forEach(child => {

```

```

        if (child && typeof child === 'object') {
            traverse(child);
        }
    });
    } else if (node[key].type) {
        traverse(node[key]);
    }
    }
}
}

traverse(ast);
return complexity;
}

```

2. 代码生成工具

```

JavaScript
// 从AST 生成代码
function generateCode(ast) {
    const generators = {
        Program(node) {
            return node.body.map(generate).join('\n');
        },

        FunctionDeclaration(node) {
            const params = node.params.map(p => p.name).join(', ');
            const body = generate(node.body);
            return `function ${node.id.name}(${params}) ${body}`;
        },

        BlockStatement(node) {
            const body = node.body.map(generate).join('\n');
            return `${'\n${body}\n'}`;
        },

        ReturnStatement(node) {
            return `return ${generate(node.argument)};`;
        },

        BinaryExpression(node) {
            return `${generate(node.left)} ${node.operator} ${generate(node.right)}`;
        }
    };
}

```

```

    },

    Identifier(node) {
        return node.name;
    }
};

function generate(node) {
    if (generators[node.type]) {
        return generators[node.type](node);
    }
    throw new Error(`Unknown node type: ${node.type}`);
}

return generate(ast);
}

```

3. 代码重构工具

```

Go
// 自动重构: 提取函数
function extractFunction(ast, startLine, endLine, newFunctionName)
{
    // 1. 找到指定行数的代码块
    const targetNodes = [];

    function findNodesInRange(node) {
        if (node.loc &&
            node.loc.start.line >= startLine &&
            node.loc.end.line <= endLine) {
            targetNodes.push(node);
            return false; // 不继续遍历子节点
        }

        // 继续遍历...
        return true;
    }

    traverse(ast, findNodesInRange);

    // 2. 创建新函数
    const newFunction = {
        type: 'FunctionDeclaration',

```

```

    id: { type: 'Identifier', name: newFunctionName },
    params: extractParameters(targetNodes),
    body: {
      type: 'BlockStatement',
      body: targetNodes
    }
  };

// 3. 替换原代码为函数调用
const functionCall = {
  type: 'ExpressionStatement',
  expression: {
    type: 'CallExpression',
    callee: { type: 'Identifier', name: newFunctionName },
    arguments: extractArguments(targetNodes)
  }
};

// 4. 修改 AST
// ... 实现 AST 修改逻辑

return ast;
}

```

实际工具实现

1. 自定义 Babel 插件

```

JavaScript
// 插件：自动添加错误边界
module.exports = function({ types: t }) {
  return {
    visitor: {
      ClassDeclaration(path) {
        // 检查是否是 React 组件
        const isReactComponent = path.node.superClass &&
          path.node.superClass.name === 'Component';

        if (!isReactComponent) return;

        // 检查是否已经定义了 componentDidCatch
        const hasErrorBoundary = path.node.body.body.some(

```



```

        method => method.key.name === 'componentDidCatch'
    );

    if (!hasErrorBoundary) {
        // 添加 componentDidCatch 方法
        const componentDidCatch = t.classMethod(
            'method',
            t.identifier('componentDidCatch'),
            [
                t.identifier('error'),
                t.identifier('errorInfo')
            ],
            t.blockStatement([
                t.expressionStatement(
                    t.callExpression(
                        t.memberExpression(
                            t.identifier('console'),
                            t.identifier('log')
                        ),
                        [t.identifier('error'),
                            t.identifier('errorInfo')]
                    )
                )
            ])
        );

        path.node.body.body.push(componentDidCatch);
    }
}
};
};

```

2. 代码依赖分析

```

JavaScript
// 分析文件依赖关系
function analyzeDependencies(ast) {
    const dependencies = new Set();

    const visitor = {
        ImportDeclaration(path) {
            dependencies.add(path.node.source.value);
        }
    };
}

```

```

    },

    CallExpression(path) {
      if (path.node.callee.type === 'Identifier' &&
        path.node.callee.name === 'require') {
        const modulePath = path.node.arguments[0].value;
        dependencies.add(modulePath);
      }
    }
  };

  // 遍历 AST
  traverse(ast, visitor);

  return Array.from(dependencies);
}

```

3. 代码安全扫描

```

JavaScript
// 检测安全漏洞
function securityScan(ast) {
  const vulnerabilities = [];

  const rules = [
    {
      name: 'eval 使用',
      check: (node) =>
        node.type === 'CallExpression' &&
        node.callee.type === 'Identifier' &&
        node.callee.name === 'eval'
    },
    {
      name: 'innerHTML 直接赋值',
      check: (node) =>
        node.type === 'AssignmentExpression' &&
        node.left.type === 'MemberExpression' &&
        node.left.property.name === 'innerHTML'
    },
    {
      name: '未经验证的 URL 跳转',
      check: (node) =>
        node.type === 'CallExpression' &&

```

```

        node.callee.type === 'MemberExpression' &&
        node.callee.property.name === 'open' &&
        node.arguments.length > 0 &&
        node.arguments[0].type === 'Identifier'
    }
];

function scanNode(node) {
    rules.forEach(rule => {
        if (rule.check(node)) {
            vulnerabilities.push({
                rule: rule.name,
                location: node.loc,
                severity: 'high'
            });
        }
    });
}

traverse(ast, scanNode);
return vulnerabilities;
}

```

现代前端工具链中的应用

Vite 的 HMR 实现

```

JavaScript
// 简化的模块热替换
class HMRManager {
    constructor() {
        this.modules = new Map();
        this.acceptHandlers = new Map();
    }

    // 分析模块的 AST，找到导出和依赖
    analyzeModule(moduleId, code) {
        const ast = parse(code);
        const exports = [];
        const imports = [];

        traverse(ast, {

```

```

    ExportNamedDeclaration(path) {
      exports.push(...path.node.specifiers.map(s =>
s.exported.name));
    },
    ImportDeclaration(path) {
      imports.push({
        source: path.node.source.value,
        specifiers: path.node.specifiers.map(s => s.local.name)
      });
    }
  });

  return { exports, imports, ast };
}

// 处理模块更新
handleUpdate(moduleId, newCode) {
  const oldModule = this.modules.get(moduleId);
  const newAnalysis = this.analyzeModule(moduleId, newCode);

  // 比较 AST 差异
  const changes = this.diffAst(oldModule.ast, newAnalysis.ast);

  // 执行热更新
  if (this.acceptHandlers.has(moduleId)) {
    this.acceptHandlers.get(moduleId)(changes);
  }

  // 更新依赖模块
  this.updateDependents(moduleId, changes);
}
}

```

Next.js 的代码分割

```

JavaScript
// 基于 AST 的代码分割分析
class CodeSplitter {
  analyzeRouteComponents(ast) {
    const components = new Map();

    traverse(ast, {
      ImportDeclaration(path) {

```

```

const source = path.node.source.value;

// 检查是否是动态导入
if (source.startsWith('.')) {
  path.node.specifiers.forEach(specifier => {
    components.set(specifier.local.name, {
      source,
      isDynamic: false
    });
  });
}
},

CallExpression(path) {
  // 检测 React.lazy 或动态 import()
  if (path.node.callee.type === 'Import' ||
    (path.node.callee.type === 'MemberExpression' &&
      path.node.callee.object.name === 'React' &&
      path.node.callee.property.name === 'lazy')) {

    const componentName = this.getComponentName(path);
    const importSource = this.getImportSource(path);

    components.set(componentName, {
      source: importSource,
      isDynamic: true
    });
  }
}
});

return components;
}

generateSplitPoints(components) {
  const splitPoints = [];

  components.forEach((info, name) => {
    if (info.isDynamic) {
      splitPoints.push({
        component: name,
        chunkName: `component_${name.toLowerCase()}`,
        importPath: info.source
      });
    }
  });
}

```

```
    }  
  });  
  
  return splitPoints;  
}  
}
```

84. Vite 在开发环境下的启动和热更新速度为什么通常比 Webpack 快，其底层原理是什么？

核心原理对比

Webpack 开发模式

```
Java  
// Webpack 启动流程  
1. 读取所有入口文件  
2. 构建依赖图（递归解析 import）  
3. 将模块打包成 bundle  
4. 启动开发服务器  
5. 文件变化 → 重新打包 → 刷新页面  
  
// 问题：项目越大，启动和 HMR 越慢  
const webpackPerformance = {  
  coldStart: '20-60s (大型项目)',  
  hmrUpdate: '1-5s',  
  memoryUsage: '高',  
  rebuild: '全量打包'  
};
```

Vite 开发模式

```
CSS  
// Vite 启动流程  
1. 启动服务器（秒级）  
2. 浏览器请求模块时，按需编译和返回  
3. 文件变化 → 仅编译单个模块 → 通过 ESM 热更新
```



```
const vitePerformance = {
  coldStart: '<1s (任何规模)',
  hmrUpdate: '50-200ms',
  memoryUsage: '低',
  rebuild: '按需编译'
};
```

Vite 快速原理详解

1. 原生 ESM

XML

```
<!-- 传统打包: 所有模块打包成一个文件 -->
<script src="/dist/bundle.js"></script>

<!-- Vite: 直接使用原生 ESM -->
<script type="module" src="/src/main.js"></script>

<!-- 浏览器按需请求 -->
<!-- 请求 /src/main.js -->
<!-- 遇到 import 时请求 /src/foo.js -->
<!-- 遇到 import 时请求 /node_modules/vue.js -->
```

2. 预构建优化

```
C++
// vite.config.js
export default {
  optimizeDeps: {
    // 预构建依赖
    include: [
      'vue',
      'vue-router',
      'lodash-es',
      'axios'
    ],
    // 排除不需要预构建的
    exclude: ['某些不需要转换的包']
  }
}
```

```
};

// 预构建过程:
// 1. 扫描 package.json 的 dependencies
// 2. 将 CommonJS 转换为 ESM
// 3. 合并多个小文件减少请求
// 4. 缓存到 node_modules/.vite
```

3. 按需编译

```
JavaScript
// Vite 服务器中间件
async function viteTransformMiddleware(req, res, next) {
  if (req.url.endsWith('.js') || req.url.endsWith('.ts')) {
    // 只编译请求的模块
    const filePath = urlToFilePath(req.url);

    // 检查缓存
    if (cache.has(filePath)) {
      return cache.get(filePath);
    }

    // 按需转换
    const code = await fs.readFile(filePath, 'utf-8');
    const transformed = await transformModule(code, filePath);

    // 缓存结果
    cache.set(filePath, transformed);

    res.setHeader('Content-Type', 'application/javascript');
    res.end(transformed);
    return;
  }

  next();
}
```

4. 热更新机制

```
JavaScript
// Vite HMR 实现
```

```

class ViteHMR {
  constructor() {
    this watcher = chokidar.watch('src');
    this.wsConnections = new Set();
    this.moduleGraph = new ModuleGraph();
  }

  handleFileChange(filePath) {
    // 1. 找到受影响的模块
    const affectedModules =
this.moduleGraph.getModulesByFile(filePath);

    // 2. 只重新编译这些模块
    const updates = affectedModules.map(async module => {
      const transformed = await transformModule(module.code,
module.filePath);
      return {
        type: 'update',
        path: module.url,
        timestamp: Date.now(),
        content: transformed
      };
    });

    // 3. 通过 WebSocket 发送更新
    this.wsConnections.forEach(ws => {
      ws.send(JSON.stringify({
        type: 'update',
        updates: await Promise.all(updates)
      }));
    });
  }
}

```

实际性能对比

CSS

// 项目配置对比

```

const projectConfig = {
  webpack: {
    entry: './src/index.js',
    output: { filename: 'bundle.js' },
    plugins: [

```

```

    new webpack.HotModuleReplacementPlugin()
  ],
  devServer: {
    hot: true,
    port: 3000
  }
},

vite: {
  server: {
    port: 3000,
    hmr: true
  }
}
};

// 性能测试结果
const benchmarkResults = {
  smallProject: {
    webpack: { start: '3.2s', hmr: '1.5s' },
    vite: { start: '0.8s', hmr: '0.2s' }
  },

  mediumProject: {
    webpack: { start: '12.5s', hmr: '3.8s' },
    vite: { start: '1.2s', hmr: '0.3s' }
  },

  largeProject: {
    webpack: { start: '45.8s', hmr: '8.2s' },
    vite: { start: '2.1s', hmr: '0.5s' }
  }
};

```

生产构建差异

```

C++
// Vite 生产构建仍然使用 Rollup
export default {
  build: {
    // Rollup 配置
    rollupOptions: {
      input: 'src/main.js',

```

```

    output: {
      dir: 'dist',
      format: 'es',
      chunkFileNames: '[name]-[hash].js'
    }
  },

  // 构建优化
  minify: 'terser',
  cssCodeSplit: true,
  reportCompressedSize: true,

  // 现代浏览器支持
  target: 'es2015',
  polyfillModulePreload: false
}
};

```

85. WebSocket 和 Server-Sent Events 之间有什么区别和各自的适用场景？

核心区别对比

特性	WebSocket	Server-Sent Events
通信方向	双向（全双工）	单向（服务器→客
协议	独立的ws/wss协议	基于HTTP
连接	持久连接	持久连接
数据格式	二进制或文本	仅文本
重连机制	需手动实现	自动重连
CORS	需要处理	遵循HTTP CORS
浏览器支持	IE10+	除IE外都支持

点击图片可查看完整电子表格

WebSocket 实现

```

JavaScript
// 客户端

```

```
class WebSocketClient {
  constructor(url) {
    this.ws = new WebSocket(url);
    this.setupEventListeners();
  }

  setupEventListeners() {
    this.ws.onopen = () => {
      console.log('WebSocket 连接已建立');
      // 发送消息
      this.send({ type: 'join', room: 'chat' });
    };

    this.ws.onmessage = (event) => {
      const data = JSON.parse(event.data);
      this.handleMessage(data);
    };

    this.ws.onclose = () => {
      console.log('连接关闭, 5 秒后重连...');
      setTimeout(() => this.reconnect(), 5000);
    };

    this.ws.onerror = (error) => {
      console.error('WebSocket 错误:', error);
    };
  }

  send(data) {
    if (this.ws.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify(data));
    }
  }

  handleMessage(data) {
    switch (data.type) {
      case 'chat':
        this.displayMessage(data.message);
        break;
      case 'notification':
        this.showNotification(data.text);
        break;
      case 'system':
        this.handleSystemMessage(data);
    }
  }
}
```



```

        break;
    }
}

// 服务器端 (Node.js + ws)
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
    console.log('新客户端连接');

    // 发送欢迎消息
    ws.send(JSON.stringify({
        type: 'welcome',
        message: '欢迎加入聊天室'
    }));

    // 接收消息
    ws.on('message', (message) => {
        const data = JSON.parse(message);

        // 广播给所有客户端
        wss.clients.forEach((client) => {
            if (client !== ws && client.readyState === WebSocket.OPEN) {
                client.send(JSON.stringify({
                    type: 'chat',
                    user: data.user,
                    message: data.message,
                    timestamp: Date.now()
                }));
            }
        });

        ws.on('close', () => {
            console.log('客户端断开连接');
        });
    });
});

```

SSE 实现

TypeScript

// 客户端

```
class SSEClient {
  constructor(url) {
    this.eventSource = null;
    this.url = url;
    this.reconnectDelay = 1000;
    this.maxReconnectDelay = 30000;
    this.connect();
  }

  connect() {
    this.eventSource = new EventSource(this.url);

    this.eventSource.onopen = () => {
      console.log('SSE 连接已建立');
      this.reconnectDelay = 1000; // 重置重连延迟
    };

    // 监听特定事件
    this.eventSource.addEventListener('message', (event) => {
      this.handleMessage(JSON.parse(event.data));
    });

    this.eventSource.addEventListener('stock-update', (event) => {
      this.handleStockUpdate(JSON.parse(event.data));
    });

    this.eventSource.addEventListener('notification', (event) => {
      this.showNotification(JSON.parse(event.data));
    });

    // 自定义事件
    this.eventSource.addEventListener('custom-event', (event) => {
      console.log('自定义事件:', event.data);
    });

    this.eventSource.onerror = (error) => {
      console.error('SSE 错误:', error);
      this.eventSource.close();

      // 指数退避重连
      setTimeout(() => {
```

```
        this.reconnectDelay = Math.min(
            this.reconnectDelay * 1.5,
            this.maxReconnectDelay
        );
        this.connect();
    }, this.reconnectDelay);
    };
}

close() {
    if (this.eventSource) {
        this.eventSource.close();
    }
}
}

// 服务器端 (Node.js + Express)
const express = require('express');
const app = express();

app.get('/events', (req, res) => {
    // 设置 SSE 头
    res.writeHead(200, {
        'Content-Type': 'text/event-stream',
        'Cache-Control': 'no-cache',
        'Connection': 'keep-alive',
        'Access-Control-Allow-Origin': '*'
    });

    // 发送初始数据
    res.write(`data: ${JSON.stringify({ type: 'connected', time:
Date.now() })}\n\n`);

    // 定期发送数据
    const stockInterval = setInterval(() => {
        const stockData = {
            symbol: 'AAPL',
            price: (150 + Math.random() * 10).toFixed(2),
            timestamp: Date.now()
        };

        // 发送特定事件类型
        res.write(`event: stock-update\n`);
        res.write(`data: ${JSON.stringify(stockData)}\n\n`);
```

```

    }, 3000);

    // 系统消息
    const systemInterval = setInterval(() => {
      const systemStatus = {
        users: Math.floor(Math.random() * 1000),
        uptime: process.uptime(),
        timestamp: Date.now()
      };

      res.write(`event: system-status\n`);
      res.write(`data: ${JSON.stringify(systemStatus)}\n\n`);
    }, 10000);

    // 客户端断开连接时清理
    req.on('close', () => {
      clearInterval(stockInterval);
      clearInterval(systemInterval);
      console.log('客户端断开连接');
    });

    // 保持连接活跃
    const keepAlive = setInterval(() => {
      res.write(': keepalive\n\n');
    }, 30000);

    req.on('close', () => {
      clearInterval(keepAlive);
    });
  });
}

```

选择指南

使用 WebSocket 的场景：

```

Java
const useWebSocket = {
  1: '实时聊天应用',
  2: '多人协作编辑器',
  3: '在线游戏',
  4: '股票交易平台',
  5: '视频会议',

```

```
6: '需要双向通信的任何场景'
};
```

使用 SSE 的场景：

```
Java
const useSSE = {
  1: '实时新闻推送',
  2: '股票价格更新',
  3: '服务器日志流',
  4: '社交媒体动态',
  5: '体育比赛比分',
  6: '只需要服务器推送的场景'
};
```

混合方案示例

```
JavaScript
// 同时使用 WebSocket 和 SSE
class HybridRealtimeClient {
  constructor() {
    // 使用 SSE 接收广播消息
    this.sse = new EventSource('/api/broadcast');
    this.sse.onmessage = this.handleBroadcast.bind(this);

    // 使用 WebSocket 进行双向通信
    this.ws = new WebSocket('wss://api.example.com/chat');
    this.ws.onmessage = this.handleChatMessage.bind(this);
  }

  handleBroadcast(event) {
    const data = JSON.parse(event.data);

    switch (data.type) {
      case 'announcement':
        this.showAnnouncement(data.message);
        break;
      case 'system-alert':
        this.showSystemAlert(data.alert);
        break;
      case 'stats':
```

```

        this.updateStats(data.stats);
        break;
    }
}

handleChatMessage(event) {
    const message = JSON.parse(event.data);
    this.addToChat(message);
}

sendMessage(text) {
    this.ws.send(JSON.stringify({
        type: 'message',
        text: text,
        userId: this.userId
    }));
}

// 服务器端实现
const hybridServer = {
    // HTTP 服务器处理 SSE
    setupSSE: (app) => {
        app.get('/api/broadcast', (req, res) => {
            res.writeHead(200, {
                'Content-Type': 'text/event-stream',
                'Cache-Control': 'no-cache',
                'Connection': 'keep-alive'
            });

            // 广播消息给所有连接
            broadcastSystem.addClient(req, res);
        });
    },

    // WebSocket 服务器
    setupWebSocket: () => {
        const wss = new WebSocket.Server({ port: 8081 });

        wss.on('connection', (ws) => {
            // 处理聊天消息
            ws.on('message', (data) => {
                const message = JSON.parse(data);
                chatManager.broadcast(message);
            });
        });
    }
};

```



```
    });  
  });  
}  
};
```

86. 什么是“注水”？它在服务器端渲染或静态站点生成的框架中扮演什么角色？

什么是注水（Hydration）？

```
CSS  
// SSR 渲染的 HTML  
const serverHTML = `  
  <div id="app">  
    <h1>Hello World</h1>  
    <button>点击我</button>  
  </div>  
`;  
  
// 客户端注水过程  
1. 下载 JavaScript bundle  
2. 解析服务器 HTML  
3. 重建 VNode 树  
4. 将事件处理程序附加到现有 DOM  
5. 接管后续交互
```

注水详细过程

```
JavaScript  
// React 注水示例  
import { hydrateRoot } from 'react-dom/client';  
  
// 服务器渲染的 HTML  
const htmlFromServer = `  
  <div id="root">  
    <h1 data-reactroot="">Hello SSR</h1>  
    <button>Click me</button>  
  </div>  
`;
```

```

// 客户端代码
const rootElement = document.getElementById('root');

// 注水过程
hydrateRoot(
  rootElement,
  <App />, // 相同的组件树
  {
    // 注水选项
    onRecoverableError: (error) => {
      console.error('注水错误:', error);
    }
  }
);

// 注水后的 DOM
// 服务器 HTML + 客户端交互能力 = 完整应用

```

Vue 3 的注水实现

```

XML
<!-- 服务器渲染 -->
<template>
  <div id="app">
    <h1>{{ message }}</h1>
    <button @click="count++">点击: {{ count }}</button>
  </div>
</template>

<script setup>
import { ref } from 'vue';
const message = 'Hello SSR';
const count = ref(0);
</script>

<!-- 生成的 HTML -->
<div id="app">
  <h1>Hello SSR</h1>
  <button>点击: 0</button>
</div>

```

```
<!-- 客户端注水 -->
<script>
import { createSSRApp } from 'vue';

const app = createSSRApp({
  // 与服务器相同的组件选项
});

// 注水：将 Vue 实例附加到现有 HTML
app.mount('#app');
```

注水的重要性

1. 性能优化

```
Java
// 无注水：白屏时间
const noHydration = {
  timeline: [
    '0ms: 请求页面',
    '100ms: 接收 HTML（显示内容）',
    '500ms: 下载 JS',
    '800ms: 执行 JS',
    '1000ms: 渲染完成（可交互）'
  ],
  fcp: '100ms', // First Contentful Paint
  tti: '1000ms' // Time to Interactive
};

// 有注水：快速交互
const withHydration = {
  timeline: [
    '0ms: 请求页面',
    '100ms: 接收 HTML（显示内容）',
    '500ms: 下载 JS + 并行注水',
    '600ms: 可交互'
  ],
  fcp: '100ms',
  tti: '600ms' // 提升 40%
};
```

2. SEO 友好

```
Java
// 搜索引擎爬取过程
const seoCrawling = {
  step1: '请求 HTML 页面',
  step2: '解析静态内容',
  step3: '执行 JavaScript (可选)',
  step4: '索引内容'
};

// 有 SSR + 注水: 爬虫立即看到内容
// 无 SSR: 爬虫可能只看到空 div
```

注水的挑战和解决方案

1. 注水不匹配

```
JavaScript
// 常见问题: 服务器和客户端渲染结果不同
const hydrationMismatch = {
  cause: [
    '不同时区的日期格式化',
    '浏览器特有的 API 使用',
    '随机数生成',
    '异步数据获取时机不同'
  ],

  solution: [
    '使用 useEffect 延迟浏览器特定逻辑',
    '避免在渲染中使用 Math.random()',
    '确保数据获取的一致性',
    '使用注水安全的钩子'
  ]
};

// React 解决方案
function HydrationSafeComponent() {
  const [isClient, setIsClient] = useState(false);
```

```
useEffect(() => {
  setIsClient(true);
}, []);

// 只在客户端执行
if (!isClient) {
  return <div>Loading...</div>;
}

return <div>客户端内容</div>;
}
```

2. 部分注水

```
JavaScript
// 只注水关键部分，延迟非关键部分
import { hydrateLazy } from 'react-lazy-hydration';

function MyPage() {
  return (
    <div>
      {/* 立即注水 */}
      <CriticalContent />

      {/* 延迟注水 */}
      <hydrateLazy>
        <NonCriticalComponent />
      </hydrateLazy>

      {/* 视口内才注水 */}
      <hydrateLazy whenVisible>
        <HeavyComponent />
      </hydrateLazy>
    </div>
  );
}
```

现代框架的注水优化

Next.js 的流式注水

```

JavaScript
// app/page.js - Next.js 13+
export default function Page() {
  return (
    <div>
      {/* 立即渲染 */}
      <Header />

      {/* Suspense 边界 */}
      <Suspense fallback={<Spinner />}>
        {/* 流式渲染 */}
        <ProductList />
      </Suspense>

      <Footer />
    </div>
  );
}

```

// 服务器渲染流程

1. 立即发送 HTML 骨架
2. 流式发送<ProductList>的内容
3. 客户端边接收边注水

Vue 3 的 Islands 架构

```

XML
<!-- Islands 架构: 只在需要交互的地方注水 -->
<template>
  <div>
    <!-- 静态内容, 无注水 -->
    <article v-html="articleContent"></article>

    <!-- 交互式组件, 需要注水 -->
    <CommentsSection :article-id="articleId" />
  </div>
</template>

<!-- 生成的 HTML -->
<div>
  <article>...</article>
  <!-- 只有这个部分需要 JavaScript -->

```



```
<div data-island="comments">
  <!-- 客户端注水这个岛 -->
</div>
</div>
```

注水性能监控

JavaScript

// 注水性能指标

```
class HydrationMetrics {
  constructor() {
    this.metrics = {
      hydrationStart: null,
      hydrationEnd: null,
      componentCount: 0,
      errorCount: 0
    };
  }

  startTracking() {
    // 监听注水开始
    performance.mark('hydration-start');
    this.metrics.hydrationStart = performance.now();

    // 监听 React 注水事件
    if (window.__REACT_DEVTOOLS_GLOBAL_HOOK__) {
      this.setupReactTracking();
    }
  }

  setupReactTracking() {
    // 注水完成回调
    const originalOnCommitRoot =
      window.__REACT_DEVTOOLS_GLOBAL_HOOK__.onCommitFiberRoot;

    window.__REACT_DEVTOOLS_GLOBAL_HOOK__.onCommitFiberRoot =
      (...args) => {
        const [fiberRoot] = args;

        // 检查是否完成注水
        if (fiberRoot.current.alternate === null) {
          performance.mark('hydration-end');
        }
      };
  }
}
```

```

        this.metrics.hydratedEnd = performance.now();

        this.reportMetrics();
    }

    return originalOnCommitRoot?.(...args);
};
}

reportMetrics() {
    const hydrationTime = this.metrics.hydratedEnd -
this.metrics.hydratedStart;

    // 发送到分析平台
    sendToAnalytics({
        metric: 'hydration_time',
        value: hydrationTime,
        timestamp: Date.now(),
        userAgent: navigator.userAgent
    });

    console.log(`注水完成，耗时: ${hydrationTime}ms`);
}
}

```

87. @container 和 @media 的主要区别是什么？

核心区别

特性	@media	@container
查询目标	视口/设备	容器元素
依赖关系	独立于内容	基于容器尺寸
响应单位	px, em, rem	容器相对单位(cqw, cqh)
典型用途	全局布局	组件级响应
支持状态	广泛支持	现代浏览器

点击图片可查看完整电子表格

@media 查询示例

CSS

```
/* 基于视口的响应式设计 */
@media (max-width: 768px) {
  .header {
    flex-direction: column;
  }

  .sidebar {
    display: none;
  }

  .content {
    padding: 10px;
  }
}

@media (min-width: 769px) and (max-width: 1024px) {
  .grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1025px) {
  .container {
    max-width: 1200px;
    margin: 0 auto;
  }
}

/* 设备特性查询 */
@media (hover: hover) {
  /* 支持悬停的设备 */
  .button:hover {
    background-color: #007bff;
  }
}

@media (prefers-color-scheme: dark) {
  body {
    background-color: #121212;
    color: #ffffff;
  }
}
```

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation-duration: 0.01ms !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

@container 查询示例

CSS

```
/* 定义容器 */  
.card-container {  
  container-type: inline-size;  
  container-name: card-container;  
}  
  
/* 基于容器宽度的样式 */  
@container card-container (max-width: 400px) {  
  .card {  
    flex-direction: column;  
  }  
  
  .card-image {  
    width: 100%;  
    height: 200px;  
  }  
  
  .card-content {  
    padding: 1rem;  
  }  
}  
  
@container card-container (min-width: 401px) and (max-width:  
600px) {  
  .card {  
    flex-direction: row;  
  }  
  
  .card-image {  
    width: 150px;  
    height: 150px;  
  }  
}
```

```
@container card-container (min-width: 601px) {
  .card {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }

  .card-title {
    font-size: 1.5rem;
  }
}

/* 容器相对单位 */
.component {
  /* 相对于容器宽度 */
  font-size: 5cqw; /* 容器宽度的 5% */

  /* 相对于容器高度 */
  padding: 2cqh; /* 容器高度的 2% */

  /* 相对于容器尺寸（较小者） */
  margin: 3cqi; /* 容器内联尺寸的 3% */

  /* 相对于容器尺寸（较大者） */
  border-radius: 1cqb; /* 容器块尺寸的 1% */
}
```

实际应用场景对比

场景 1：侧边栏布局

```
CSS
/* 使用@media - 基于视口 */
@media (min-width: 1024px) {
  .layout {
    display: grid;
    grid-template-columns: 250px 1fr;
  }
}

/* 使用@container - 基于父容器 */
```

```
.sidebar-container {
  container-type: inline-size;
}

@container (min-width: 800px) {
  .layout {
    display: grid;
    grid-template-columns: 250px 1fr;
  }
}
```

场景 2：卡片组件

XML

```
<!-- 在不同容器中的卡片 -->
<div class="narrow-container">
  <div class="card">
    <!-- 在窄容器中显示为垂直布局 -->
  </div>
</div>

<div class="wide-container">
  <div class="card">
    <!-- 在宽容器中显示为水平布局 -->
  </div>
</div>
```

CSS

```
.card-container {
  container-type: inline-size;
}

@container (max-width: 400px) {
  .card {
    /* 窄布局 */
    flex-direction: column;
  }
}

@container (min-width: 401px) {
  .card {
    /* 宽布局 */
```



```
    flex-direction: row;
  }
}
```

混合使用示例

CSS

```
/* 全局布局使用@media */
@media (max-width: 768px) {
  .page {
    padding: 1rem;
  }
}

/* 组件内部使用@container */
.widget {
  container-type: inline-size;
  container-name: widget;
}

/* 组件根据自身尺寸响应 */
@container widget (max-width: 300px) {
  .widget-content {
    font-size: 0.875rem;
  }

  .widget-actions {
    flex-direction: column;
  }
}

@container widget (min-width: 301px) and (max-width: 500px) {
  .widget-content {
    font-size: 1rem;
  }
}

@container widget (min-width: 501px) {
  .widget {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
}
```

```

/* 嵌套容器查询 */
.dashboard {
  container-type: inline-size;
  container-name: dashboard;
}

@container dashboard (min-width: 1024px) {
  .dashboard-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 2rem;
  }

  /* 内部组件有自己的容器查询 */
  .chart-container {
    container-type: inline-size;
  }

  @container (min-width: 300px) {
    .chart {
      height: 250px;
    }
  }
}

```

性能考虑

```

JavaScript
// 容器查询性能优化
const containerQueryBestPractices = {
  1: '避免过度嵌套容器查询',
  2: '使用适当的 container-type',
  3: '考虑使用 CSS Containment',
  4: '避免在容器查询中频繁改变布局'
};

// 使用 CSS Containment 优化
.optimized-container {
  /* 启用 containment */
  contain: layout style size;
}

```

```

/* 定义容器类型 */
container-type: inline-size;

/* 性能提示 */
content-visibility: auto;
contain-intrinsic-size: 400px;
}

// JavaScript 中的容器查询
const container = document.querySelector('.container');
const containerQuery = window.matchMedia('(min-width: 400px)');

// 监听容器尺寸变化
const resizeObserver = new ResizeObserver((entries) => {
  for (const entry of entries) {
    const width = entry.contentRect.width;

    if (width >= 400) {
      entry.target.classList.add('large-container');
      entry.target.classList.remove('small-container');
    } else {
      entry.target.classList.add('small-container');
      entry.target.classList.remove('large-container');
    }
  }
});

resizeObserver.observe(container);

```

88. pnpm 与 npm/yarn 在 node_modules 管理上的主要不同之处。

核心区别：node_modules 结构

npm/yarn 的 node_modules

```

CSS
# npm/yarn 的扁平结构
node_modules/
├─ react/

```

```
├─ react-dom/
├─ lodash/
├─ package-a/
│   └─ node_modules/
│       └─ lodash@1.0.0/ # 重复依赖
└─ package-b/
    └─ node_modules/
        └─ lodash@2.0.0/ # 版本冲突
```

pnpm 的 node_modules

Bash

pnpm 的符号链接结构

node_modules/

```
├─ .pnpm/ # 虚拟存储目录
│   ├── lodash@1.0.0/
│   ├── lodash@2.0.0/
│   └─ react@18.2.0/
├─ react -> .pnpm/react@18.2.0/node_modules/react
├─ react-dom -> .pnpm/react-dom@18.2.0/node_modules/react-dom
└─ package-a -> .pnpm/package-a@1.0.0/node_modules/package-a
```

详细对比

安装速度对比

CSS

```
const benchmark = {
  cleanInstall: {
    npm: '45.2s',
    yarn: '32.8s',
    pnpm: '18.5s' // 快 2-3 倍
  },

  withCache: {
    npm: '12.3s',
    yarn: '8.7s',
    pnpm: '3.2s' // 几乎瞬间完成
  },

  diskUsage: {
```

```
npm: '1.2GB',
yarn: '1.1GB',
pnpm: '650MB' // 节省 40-50%空间
}
};
```

pnpm 核心特性

```
Java
// pnpm 的存储机制
const pnpmStore = {
  location: '~/.pnpm-store/v3',
  structure: {
    files: '所有包的 actual 文件（硬链接）',
    metadata: '包版本和依赖信息',
    locks: '安装锁文件'
  },
  benefits: [
    '多个项目共享相同包版本',
    '节省磁盘空间',
    '安装速度快'
  ]
};

// 检查存储状态
$ pnpm store status
// 输出：已使用存储：2.1GB
//          可回收空间：500MB

// 清理不需要的包
$ pnpm store prune
```

2. 严格的 node_modules

```
YAML
// package.json
{
  "dependencies": {
    "express": "^4.18.0",
    "lodash": "^4.17.0"
```

```
}  
}  
  
// pnpm-lock.yaml  
dependencies:  
  express: 4.18.2  
  lodash: 4.17.21  
packages:  
  /express/4.18.2:  
    dependencies:  
      accepts: 1.3.8  
      array-flatten: 1.1.1  
      # 所有依赖都精确声明
```

3. Monorepo 支持

```
Python  
# pnpm-workspace.yaml  
packages:  
  - 'packages/*'  
  - 'apps/*'  
  - '!**/test/**'  
  
# 过滤特定包  
$ pnpm --filter @project/web-app add react  
  
# 在所有包中运行命令  
$ pnpm -r run build  
  
# 只对变更的包运行  
$ pnpm --filter ...changed test
```

实际使用示例

基本使用

```
Markdown  
# 初始化项目  
$ pnpm init  
  
# 安装依赖
```



```
$ pnpm add react react-dom typescript

# 安装开发依赖
$ pnpm add -D jest @types/node

# 全局安装
$ pnpm add -g pnpm

# 更新依赖
$ pnpm update
$ pnpm update --latest # 更新到最新版本

# 删除依赖
$ pnpm remove lodash
```

高级功能

Markdown

```
# 选择性安装
$ pnpm install --prod # 只安装生产依赖
$ pnpm install --no-optional # 跳过可选依赖

# 离线模式
$ pnpm install --offline

# 严格模式（确保 lock 文件一致）
$ pnpm install --frozen-lockfile

# 审核依赖
$ pnpm audit
$ pnpm audit fix

# 列出依赖树
$ pnpm list
$ pnpm list --depth 1
$ pnpm list --prod # 只显示生产依赖

# 查看过时依赖
$ pnpm outdated
```

Monorepo 管理

JSON

```
// 根目录 package.json
{
  "name": "my-monorepo",
  "private": true,
  "scripts": {
    "build": "turbo run build",
    "dev": "turbo run dev --parallel",
    "lint": "turbo run lint",
    "test": "turbo run test"
  },
  "devDependencies": {
    "turbo": "latest",
    "typescript": "^5.0.0"
  }
}

// 子包 package.json
{
  "name": "@project/ui",
  "version": "1.0.0",
  "dependencies": {
    "react": "workspace:*", # 引用工作空间内的包
    "@project/utils": "workspace:*"
  }
}
```

Markdown

```
# 工作空间命令
$ pnpm install # 安装所有工作空间依赖

# 运行特定包的脚本
$ pnpm --filter @project/web-app dev

# 运行多个包的脚本
$ pnpm --filter @project/* test

# 依赖关系图
$ pnpm -r list --graph
```

迁移指南

从 npm 迁移

Markdown

1. 安装 pnpm

```
$ npm install -g pnpm
```

2. 删除现有 node_modules

```
$ rm -rf node_modules
```

```
$ rm package-lock.json
```

3. 使用 pnpm 安装

```
$ pnpm install
```

4. 更新 package.json 脚本（如果需要）

```
{  
  "scripts": {  
    "preinstall": "npx only-allow pnpm"  
  }  
}
```

5. 配置 CI/CD

.github/workflows/ci.yml

```
- name: Setup pnpm  
  uses: pnpm/action-setup@v2  
  with:  
    version: 8
```

从 yarn 迁移

Markdown

1. 删除 yarn.lock 和 node_modules

```
$ rm yarn.lock
```

```
$ rm -rf node_modules
```

2. 转换 lock 文件

```
$ npx synp --source-file yarn.lock
```

这会生成 pnpm-lock.yaml

3. 安装

```
$ pnpm install
```

4. 更新工作流

```
# 将 yarn 命令替换为 pnpm:
# yarn add → pnpm add
# yarn install → pnpm install
# yarn run → pnpm run
```

常见问题和解决方案

1. 幽灵依赖问题

```
JavaScript
// npm/yarn 的问题：可以访问未声明的依赖
// package.json
{
  "dependencies": {
    "express": "^4.18.0"
  }
  // 没有声明 lodash
}

// 代码中（可能工作，但不可靠）
const _ = require('lodash'); // 幽灵依赖！

// pnpm 的解决方案：严格依赖
// 如果没有声明 lodash，这个 require 会失败
// 必须明确声明所有依赖
```

2. 依赖提升问题

```
Markdown
# npm/yarn 会自动提升某些依赖
# 这可能导致：
# 1. 版本冲突
# 2. 不可预测的行为
# 3. 调试困难

# pnpm：无依赖提升
# 每个包只能访问自己声明的依赖
# 更可预测，更安全
```

3. 磁盘空间优化

Markdown

```
# 查看存储使用情况
$ pnpm store path
# ~/.pnpm-store/v3

$ pnpm store status
# 检查可清理的空间

# 清理不需要的包
$ pnpm store prune

# 强制清理所有
$ pnpm store prune --force
```

4. CI/CD 优化

YAML

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup pnpm
        uses: pnpm/action-setup@v2
        with:
          version: 8

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install --frozen-lockfile
```

- name: Run tests
run: pnpm test
- name: Build
run: pnpm build

选择建议

CSS

```
const selectionGuide = {
  choosePnpm: [
    '对安装速度有要求',
    '磁盘空间有限',
    '需要严格的依赖管理',
    '使用 Monorepo',
    '关注安全性'
  ],

  chooseNpm: [
    '简单项目',
    '不需要高级功能',
    '团队习惯 npm',
    '兼容性要求'
  ],

  chooseYarn: [
    '已经使用 Yarn 工作流',
    '需要 workspaces 功能',
    '喜欢 yarn 的 UI'
  ],

  allSupport: {
    lockFiles: {
      npm: 'package-lock.json',
      yarn: 'yarn.lock',
      pnpm: 'pnpm-lock.yaml'
    },
    commands: {
      install: 'npm install / yarn / pnpm install',
      add: 'npm add / yarn add / pnpm add',
      run: 'npm run / yarn run / pnpm run'
    }
  }
}
```



```
    }  
  }  
};
```

89. 什么是纯函数和副作用？。

纯函数

纯函数是指满足以下两个条件的函数：

1. 相同的输入总是返回相同的输出
2. 没有副作用

```
JavaScript  
// 纯函数示例  
function add(a, b) {  
  return a + b;  
}  
  
function multiply(x, y) {  
  return x * y;  
}  
  
// 数组操作纯函数  
function pureMap(array, fn) {  
  return array.map(fn);  
}  
  
// 对象操作纯函数  
function pureUpdate(obj, key, value) {  
  return { ...obj, [key]: value };  
}
```

纯函数的特性

```
JavaScript  
const pureFunctionProperties = {  
  1: "可预测性：给定相同输入，总是返回相同输出",  
  2: "无副作用：不修改外部状态",  
  3: "可缓存性（记忆化）",  
  4: "易于测试：不需要 mock 外部依赖",  
}
```

```
5: "引用透明：函数调用可以被其返回值替换"
};

// 记忆化示例
function memoize(fn) {
  const cache = new Map();
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      return cache.get(key);
    }
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

const memoizedAdd = memoize(add);
console.log(memoizedAdd(2, 3)); // 5
console.log(memoizedAdd(2, 3)); // 从缓存读取
```

副作用

副作用是指函数除了返回值之外，对外部环境产生的可观察的变化。

```
JavaScript
// 副作用示例
let counter = 0;

// 有副作用的函数
function increment() {
  counter++; // 修改外部状态
}

function logMessage(message) {
  console.log(message); // 控制台输出
}

function updateDOM(elementId, content) {
  document.getElementById(elementId).innerHTML = content; // 修改DOM
}
```

```
async function fetchData(url) {  
  const response = await fetch(url); // 网络请求  
  return response.json();  
}
```

副作用分类

```
Java  
const sideEffectTypes = {  
  // 状态修改  
  stateMutation: [  
    '修改全局变量',  
    '修改函数参数',  
    '修改外部数据结构'  
  ],  
  
  // 输入/输出  
  ioOperations: [  
    '文件系统操作',  
    '网络请求',  
    '数据库查询',  
    '控制台输出'  
  ],  
  
  // 外部系统交互  
  systemInteraction: [  
    'DOM 操作',  
    '定时器设置',  
    '事件监听器',  
    '浏览器 API 调用'  
  ]  
};
```

实践中的平衡

函数式编程实践

```
JavaScript  
// 减少副作用的方法  
class PureOperations {
```

```
// 1. 提取纯函数
static calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price *
item.quantity, 0);
}

// 2. 将副作用推到边界
static async processOrder(order) {
  // 纯计算部分
  const total = this.calculateTotal(order.items);
  const tax = total * 0.1;
  const grandTotal = total + tax;

  // 副作用集中处理
  return {
    total,
    tax,
    grandTotal,
    async save() {
      await database.save({ ...order, total: grandTotal });
    }
  };
}

// 3. 使用不可变数据
static updateUser(user, updates) {
  return { ...user, ...updates }; // 返回新对象
}

// 4. 依赖注入
static createLogger(logFn = console.log) {
  return function log(message) {
    logFn(`[${new Date().toISOString()}] ${message}`);
  };
}

// 使用示例
const items = [
  { name: 'Book', price: 20, quantity: 2 },
  { name: 'Pen', price: 5, quantity: 3 }
];
```

```
console.log(PureOperations.calculateTotal(items)); // 纯函数
```

React 中的纯函数组件

```
JavaScript
// 纯函数组件
function PureComponent({ value, onChange }) {
  return (
    <div>
      <input
        value={value}
        onChange={onChange}
      />
      <p>当前值: {value}</p>
    </div>
  );
}

// 使用 React.memo 避免不必要的重新渲染
const MemoizedComponent = React.memo(PureComponent);

// 不纯的组件（有副作用）
function ImpureComponent({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // 副作用：获取数据
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(setUser);
  }, [userId]);

  return <div>{user?.name || '加载中...'}</div>;
}
```

纯函数的优势

```
JavaScript
const pureFunctionAdvantages = {
  // 1. 可测试性
  testability: {
```

```

example: `
  // 纯函数容易测试
  test('add function', () => {
    expect(add(2, 3)).toBe(5);
    expect(add(-1, 1)).toBe(0);
  });
`,

// 2. 可缓存性
memoization: {
  example: `
    // 昂贵的计算可以缓存
    function expensiveCalculation(x) {
      // 耗时计算
      return x * x;
    }

    const memoized = memoize(expensiveCalculation);
    memoized(5); // 计算并缓存
    memoized(5); // 直接从缓存读取
  `,

// 3. 可并行性
parallelism: {
  example: `
    // 纯函数可以安全地并行执行
    const results = await Promise.all([
      computeHeavyTask(data1), // 无数据竞争
      computeHeavyTask(data2),
      computeHeavyTask(data3)
    ]);
  `,

// 4. 引用透明
referentialTransparency: {
  example: `
    // 函数调用可以被其返回值替换
    const x = add(2, 3); // 5
    const y = add(x, 4); // 9
  `
}
}

```



```
// 等同于
const y = add(5, 4); // 9
}
};
```

90. TypeScript 中的泛型解决了什么问题？

泛型解决的问题

1. 代码重用

```
TypeScript
// 没有泛型：需要为每种类型写一个函数
function identityString(arg: string): string {
    return arg;
}

function identityNumber(arg: number): number {
    return arg;
}

// 使用泛型：一个函数支持多种类型
function identity<T>(arg: T): T {
    return arg;
}

// 使用
const str = identity<string>("hello"); // string
const num = identity<number>(42); // number
const bool = identity<boolean>(true); // boolean
```

2. 类型安全

```
TypeScript
// 没有泛型：any 类型不安全
function unsafeIdentity(arg: any): any {
    return arg;
}
```

```
const result = unsafeIdentity(123);
result.toUpperCase(); // 运行时错误，但 TypeScript 不会报错

// 使用泛型：类型安全
function safeIdentity<T>(arg: T): T {
  return arg;
}

const safeResult = safeIdentity(123);
// safeResult.toUpperCase(); // TypeScript 编译时错误
```

3. 约束类型

```
TypeScript
// 泛型约束
interface Lengthwise {
  length: number;
}

function loggingIdentity<T extends Lengthwise>(arg: T): T {
  console.log(arg.length); // 安全访问 length 属性
  return arg;
}

// 正确使用
loggingIdentity("hello"); // 字符串有 length
loggingIdentity([1, 2, 3]); // 数组有 length

// 错误使用
// loggingIdentity(123); // 数字没有 length，编译错误
```

泛型的实际应用

1. 数组和集合操作

```
TypeScript
// 泛型数组
type GenericArray<T> = T[];

// 泛型队列
```

```
class Queue<T> {
  private items: T[] = [];

  enqueue(item: T): void {
    this.items.push(item);
  }

  dequeue(): T | undefined {
    return this.items.shift();
  }

  peek(): T | undefined {
    return this.items[0];
  }
}

// 使用
const numberQueue = new Queue<number>();
numberQueue.enqueue(1);
numberQueue.enqueue(2);
console.log(numberQueue.dequeue()); // 1

const stringQueue = new Queue<string>();
stringQueue.enqueue("hello");
console.log(stringQueue.peek()); // "hello"
```

2. API 响应类型

```
TypeScript
// 泛型 API 响应类型
interface ApiResponse<T> {
  success: boolean;
  data: T;
  error?: string;
  timestamp: Date;
}

// 用户 API 响应
interface User {
  id: number;
  name: string;
  email: string;
}
```

```
// 使用
async function fetchUser(id: number): Promise<ApiResponse<User>> {
  const response = await fetch(`/api/users/${id}`);
  return response.json();
}

// 通用 API 函数
async function fetchData<T>(url: string): Promise<ApiResponse<T>>
{
  const response = await fetch(url);
  return response.json();
}

// 类型安全的 API 调用
const userResponse = await fetchData<User>('/api/users/1');
const productResponse = await
fetchData<Product>('/api/products/1');
```

3. React 组件

```
TypeScript
// 泛型组件
import React, { useState } from 'react';

interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string | number;
}

function GenericList<T>({ items, renderItem, keyExtractor }:
ListProps<T>) {
  return (
    <div>
      {items.map(item => (
        <div key={keyExtractor(item)}>
          {renderItem(item)}
        </div>
      ))}
    </div>
  );
}
```

```

// 使用
interface User {
  id: number;
  name: string;
}

function UserList({ users }: { users: User[] }) {
  return (
    <GenericList
      items={users}
      keyExtractor={(user) => user.id}
      renderItem={(user) => <div>{user.name}</div>}
    />
  );
}

```

高级泛型特性

1. 条件类型

```

TypeScript
// 条件类型
type IsString<T> = T extends string ? true : false;

type A = IsString<string>; // true
type B = IsString<number>; // false

// 实用示例：提取数组元素类型
type ArrayElement<T> = T extends Array<infer U> ? U : never;

type StringArrayElement = ArrayElement<string[]>; // string
type NumberArrayElement = ArrayElement<number[]>; // number

// 过滤类型
type FilterString<T> = T extends string ? T : never;
type Result = FilterString<string | number | boolean>; // string

```

2. 映射类型

```

TypeScript

```

```

// 映射类型
interface User {
  id: number;
  name: string;
  email: string;
}

// 将所有属性变为可选
type PartialUser = Partial<User>;

// 将所有属性变为只读
type ReadonlyUser = Readonly<User>;

// 自定义映射类型
type Optional<T> = {
  [K in keyof T]?: T[K];
};

type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};

// 添加前缀
type WithPrefix<T, P extends string> = {
  [K in keyof T as `${P}${Capitalize<string & K>}`]: T[K];
};

type PrefixedUser = WithPrefix<User, 'user'>;
// { userId: number, userName: string, userEmail: string }

```

3. 泛型约束与默认值

```

TypeScript
// 泛型约束
interface DatabaseEntity {
  id: number;
  createdAt: Date;
}

// 泛型参数默认值
interface PaginatedResponse<T = any> {
  items: T[];
  total: number;
}

```



```

    page: number;
    pageSize: number;
}

// 多个泛型参数
function mergeObjects<T extends object, U extends object>(
    obj1: T,
    obj2: U
): T & U {
    return { ...obj1, ...obj2 };
}

// 键值约束
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const user = { name: 'John', age: 30 };
const name = getProperty(user, 'name'); // string
const age = getProperty(user, 'age'); // number
// const email = getProperty(user, 'email'); // 错误: email 不是
// user 的键

```

实用工具类型

```

TypeScript
// 自定义工具类型
type DeepPartial<T> = {
    [P in keyof T]?: T[P] extends object ? DeepPartial<T[P]> : T[P];
};

type DeepReadonly<T> = {
    readonly [P in keyof T]: T[P] extends object ?
    DeepReadonly<T[P]> : T[P];
};

type UnionToIntersection<U> =
    (U extends any ? (k: U) => void : never) extends ((k: infer I)
    => void) ? I : never;

// 实用示例
interface ComplexObject {

```

```
id: number;
name: string;
address: {
  street: string;
  city: string;
  country: string;
};
metadata: {
  tags: string[];
  createdBy: string;
};
}
```

```
type PartialComplex = DeepPartial<ComplexObject>;
// 所有属性，包括嵌套的，都是可选的

type ReadonlyComplex = DeepReadonly<ComplexObject>;
// 所有属性，包括嵌套的，都是只读的
```

91. IndexedDB 与 localStorage 相比有什么优势和不同？

核心对比

特性	localStorage	IndexedDB
存储容量	5-10MB	至少250MB，浏览器通常更多
数据类型	字符串（需JSON序列化）	结构化数据（对象、数组等）
查询能力	简单的键值查找	强大的索引查询、范围查询
事务支持	无	ACID事务支持
性能	同步（阻塞主线程）	异步（不阻塞）
存储结构	扁平键值对	对象存储（类似表）
浏览器支持	IE8+	IE10+（部分）

点击图片可查看完整电子表格

IndexedDB 的优势

1. 大容量存储

JavaScript

// IndexedDB 可以存储大量数据

```
const indexedDBUsage = {  
  chrome: '通常占磁盘空间的 60%',  
  firefox: '无明确上限, 但会警告',  
  safari: '默认 1GB, 可请求更多',  
  edge: '通常 250MB+'  
};
```

// 请求更多存储空间

```
navigator.storage.estimate().then(estimate => {  
  console.log(`可用: ${estimate.quota} bytes`);  
  console.log(`已用: ${estimate.usage} bytes`);  
});
```

// 请求持久存储 (避免浏览器清理)

```
if (navigator.storage && navigator.storage.persist) {  
  navigator.storage.persist().then(persistent => {  
    console.log(persistent ? '存储将被保留' : '存储可能被清理');  
  });  
}
```

2. 复杂数据结构

JavaScript

// IndexedDB 可以存储复杂对象

```
const complexData = {  
  id: 1,  
  name: '用户数据',  
  metadata: {  
    tags: ['重要', '个人'],  
    created: new Date(),  
    version: 2  
  },  
  files: [  
    { name: 'document.pdf', size: 1024 * 1024 },  
    { name: 'image.jpg', size: 500 * 1024 }  
  ],  
  // 可以存储 Blob、File 等  
  avatar: new Blob([/* 图片数据 */], { type: 'image/jpeg' })  
};
```

```
// localStorage 需要序列化
localStorage.setItem('complexData', JSON.stringify(complexData));
// 注意: JSON.stringify 会丢失 Date 对象类型和函数
```

3. 强大查询能力

```
JavaScript
// 创建索引查询
const dbPromise = indexedDB.open('myDatabase', 1);

dbPromise.onupgradeneeded = (event) => {
  const db = event.target.result;

  // 创建对象存储
  const store = db.createObjectStore('users', { keyPath: 'id' });

  // 创建索引
  store.createIndex('name', 'name', { unique: false });
  store.createIndex('age', 'age', { unique: false });
  store.createIndex('email', 'email', { unique: true });
  store.createIndex('created', 'created', { unique: false });
};

// 复杂查询示例
async function queryUsers() {
  const db = await dbPromise;
  const transaction = db.transaction(['users'], 'readonly');
  const store = transaction.objectStore('users');

  // 1. 范围查询 (年龄 20-30 岁)
  const ageIndex = store.index('age');
  const ageRange = IDBKeyRange.bound(20, 30);

  // 2. 前缀查询 (名字以 J 开头)
  const nameIndex = store.index('name');
  const nameRange = IDBKeyRange.bound('J', 'J\uffff');

  // 3. 复合查询 (需要遍历)
  const results = [];
  const cursor = await new Promise((resolve, reject) => {
    const request = ageIndex.openCursor(ageRange);
```

```
request.onsuccess = (event) => {
  const cursor = event.target.result;
  if (cursor) {
    // 手动过滤
    if (cursor.value.name.startsWith('J')) {
      results.push(cursor.value);
    }
    cursor.continue();
  } else {
    resolve(results);
  }
};
request.onerror = reject;
});

return results;
}
```

4. 事务支持

```
JavaScript
// ACID 事务示例
async function transferFunds(fromId, toId, amount) {
  const db = await dbPromise;
  const transaction = db.transaction(['accounts'], 'readwrite');
  const store = transaction.objectStore('accounts');

  try {
    // 原子操作：要么全部成功，要么全部失败
    const fromAccount = await getRecord(store, fromId);
    const toAccount = await getRecord(store, toId);

    if (fromAccount.balance < amount) {
      throw new Error('余额不足');
    }

    // 更新余额
    fromAccount.balance -= amount;
    toAccount.balance += amount;

    // 同时保存两个账户
    await Promise.all([
      updateRecord(store, fromAccount),
```

```

        updateRecord(store, toAccount)
    ]);

    // 事务自动提交
    return { success: true };
} catch (error) {
    // 事务自动回滚
    console.error('转账失败:', error);
    return { success: false, error: error.message };
}
}

// 辅助函数
function getRecord(store, key) {
    return new Promise((resolve, reject) => {
        const request = store.get(key);
        request.onsuccess = () => resolve(request.result);
        request.onerror = reject;
    });
}

function updateRecord(store, record) {
    return new Promise((resolve, reject) => {
        const request = store.put(record);
        request.onsuccess = () => resolve();
        request.onerror = reject;
    });
}

```

实际应用场景

场景 1：离线应用数据存储

```

JavaScript
class OfflineDataManager {
    constructor() {
        this.dbName = 'offlineAppDB';
        this.version = 2;
        this.initDB();
    }

    async initDB() {

```



```

return new Promise((resolve, reject) => {
  const request = indexedDB.open(this.dbName, this.version);

  request.onupgradeneeded = (event) => {
    const db = event.target.result;

    // 创建存储结构
    if (!db.objectStoreNames.contains('syncQueue')) {
      const queueStore = db.createObjectStore('syncQueue', {
        keyPath: 'id',
        autoIncrement: true
      });
      queueStore.createIndex('status', 'status', { unique:
false });
      queueStore.createIndex('created', 'created', { unique:
false });
    }

    if (!db.objectStoreNames.contains('cache')) {
      const cacheStore = db.createObjectStore('cache', {
        keyPath: 'url'
      });
      cacheStore.createIndex('expires', 'expires', { unique:
false });
    }
  };

  request.onsuccess = (event) => {
    this.db = event.target.result;
    resolve(this.db);
  };

  request.onerror = reject;
});

  async queueForSync(data) {
    const transaction = this.db.transaction(['syncQueue'],
'readwrite');
    const store = transaction.objectStore('syncQueue');

    const item = {
      data,
      status: 'pending',

```

```

        created: new Date(),
        attempts: 0
    };

    return new Promise((resolve, reject) => {
        const request = store.add(item);
        request.onsuccess = () => resolve(request.result);
        request.onerror = reject;
    });
}

async getCacheResponse(url) {
    const transaction = this.db.transaction(['cache'],
'readonly');
    const store = transaction.objectStore('cache');

    return new Promise((resolve, reject) => {
        const request = store.get(url);
        request.onsuccess = () => {
            const item = request.result;
            if (item && item.expires > Date.now()) {
                resolve(item.data);
            } else {
                resolve(null);
            }
        };
        request.onerror = reject;
    });
}
}

```

场景 2：文件存储

```

JavaScript
class FileStorageManager {
    constructor() {
        this.dbName = 'fileStorageDB';
        this.init();
    }

    async init() {
        this.db = await new Promise((resolve, reject) => {
            const request = indexedDB.open(this.dbName, 1);

```

```

request.onupgradeneeded = (event) => {
  const db = event.target.result;

  if (!db.objectStoreNames.contains('files')) {
    const filesStore = db.createObjectStore('files', {
      keyPath: 'id'
    });
    filesStore.createIndex('name', 'name', { unique:
false });
    filesStore.createIndex('type', 'type', { unique:
false });
    filesStore.createIndex('size', 'size', { unique:
false });
  }
};

request.onsuccess = (event) => resolve(event.target.result);
request.onerror = reject;
});
}

async storeFile(file) {
  const transaction = this.db.transaction(['files'],
'readwrite');
  const store = transaction.objectStore('files');

  const fileRecord = {
    id: Date.now() + '-' + Math.random(),
    name: file.name,
    type: file.type,
    size: file.size,
    lastModified: file.lastModified,
    data: file,
    uploaded: new Date()
  };

  return new Promise((resolve, reject) => {
    const request = store.add(fileRecord);
    request.onsuccess = () => resolve(fileRecord.id);
    request.onerror = reject;
  });
}

```

```

    async getFile(id) {
      const transaction = this.db.transaction(['files'],
        'readonly');
      const store = transaction.objectStore('files');

      return new Promise((resolve, reject) => {
        const request = store.get(id);
        request.onsuccess = () => resolve(request.result?.data);
        request.onerror = reject;
      });
    }

    async listFiles(filter = {}) {
      const transaction = this.db.transaction(['files'],
        'readonly');
      const store = transaction.objectStore('files');

      return new Promise((resolve, reject) => {
        const request = store.getAll();
        request.onsuccess = () => {
          let files = request.result;

          // 应用过滤器
          if (filter.type) {
            files = files.filter(f =>
              f.type.startsWith(filter.type));
          }
          if (filter.minSize) {
            files = files.filter(f => f.size >= filter.minSize);
          }
          if (filter.maxSize) {
            files = files.filter(f => f.size <= filter.maxSize);
          }

          resolve(files);
        };
        request.onerror = reject;
      });
    }
  }
}

```

92. React Testing Library 的核心设计哲学是什

么？为什么它鼓励测试用户行为而不是实现细节？

核心哲学：测试用户行为而非实现细节

1. 用户中心测试

```
JavaScript
// ✗ 不好的测试：测试实现细节
test('should call setState when button is clicked', () => {
  const mockSetState = jest.fn();
  const useStateSpy = jest.spyOn(React, 'useState');
  useStateSpy.mockReturnValue([0, mockSetState]);

  render(<Counter />);
  fireEvent.click(screen.getByTestId('increment-button'));

  expect(mockSetState).toHaveBeenCalled();
});

// ✔ 好的测试：测试用户行为
test('should increment counter when button is clicked', () => {
  render(<Counter />);

  // 用户看到初始计数为 0
  expect(screen.getByText('Count: 0')).toBeInTheDocument();

  // 用户点击按钮
  const button = screen.getByRole('button', { name:
/increment/i });
  userEvent.click(button);

  // 用户看到计数变为 1
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

2. 查询优先原则

```
JavaScript
// 查询优先级（从高到低）
const queryPriority = {
  // 1. 语义化查询（最推荐）
```

```

semantic: [
  'getByRole',      // 按角色查询
  'getByLabelText', // 按标签文本
  'getByText',      // 按文本内容
  'getByDisplayValue' // 按显示值
],

// 2. 测试 ID (当语义查询不可用时)
testId: [
  'getByTestId'      // 需要添加 data-testid 属性
],

// 3. 不推荐的查询
discouraged: [
  'container.querySelector', // 直接 DOM 操作
  'getByClassName',          // 实现细节
  'getBySelector'            // CSS 选择器
]
};

// 示例: 正确的查询方式
test('should submit form', () => {
  render(<LoginForm />);

  // 使用语义化查询
  const emailInput = screen.getByRole('textbox', { name:
/email/i });
  const passwordInput = screen.getByLabelText(/password/i);
  const submitButton = screen.getByRole('button', { name: /sign
in/i });

  // 模拟用户操作
  userEvent.type(emailInput, 'test@example.com');
  userEvent.type(passwordInput, 'password123');
  userEvent.click(submitButton);

  // 验证结果
  expect(screen.getByText('登录成功')).toBeInTheDocument();
});

```

3. 鼓励可访问性


```

JavaScript
// 通过测试促进可访问性
test('form should be accessible', () => {
  render(<ContactForm />);

  // 检查必要的 ARIA 属性
  const nameInput = screen.getByRole('textbox', { name:
/name/i });
  expect(nameInput).toHaveAttribute('aria-required', 'true');

  const errorMessage = screen.queryByRole('alert');
  if (errorMessage) {
    expect(errorMessage).toHaveAttribute('aria-live', 'polite');
  }

  // 验证键盘导航
  userEvent.tab();
  expect(nameInput).toHaveFocus();
});

```

最佳实践示例

1. 异步测试

```

JavaScript
// 正确使用异步查询
test('should load and display data', async () => {
  render(<UserList />);

  // 初始状态
  expect(screen.getByText(/loading/i)).toBeInTheDocument();

  // 等待数据加载
  const userItems = await screen.findAllByRole('listitem');
  expect(userItems).toHaveLength(3);

  // 验证内容
  expect(screen.getByText('John Doe')).toBeInTheDocument();
});

// 错误示例：使用 setTimeout

```

```
test('should not use setTimeout', async () => {
  render(<AsyncComponent />);

  // ✗ 不要这样做
  await new Promise(resolve => setTimeout(resolve, 1000));

  // ✔ 这样做
  await screen.findByText('Loaded');
});
```

2. 模拟 HTTP 请求

```
JavaScript
// 使用 MSW (Mock Service Worker)
import { rest } from 'msw';
import { setupServer } from 'msw/node';

const server = setupServer(
  rest.get('/api/users', (req, res, ctx) => {
    return res(
      ctx.json([
        { id: 1, name: 'John Doe' },
        { id: 2, name: 'Jane Smith' }
      ])
    );
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

test('should fetch and display users', async () => {
  render(<UserList />);

  // 等待用户列表加载
  const userItems = await screen.findAllByRole('listitem');
  expect(userItems).toHaveLength(2);

  // 验证数据
  expect(screen.getByText('John Doe')).toBeInTheDocument();
  expect(screen.getByText('Jane Smith')).toBeInTheDocument();
});
```

```
});
```

3. 测试用户交互流程

JavaScript

// 测试完整的工作流

```
test('complete shopping flow', async () => {
```

// 1. 浏览商品

```
  render(<ProductList />);
```

```
  const products = await screen.findAllByRole('article');
```

```
  expect(products).toHaveLength(5);
```

// 2. 查看商品详情

```
  const firstProduct = screen.getAllByRole('link', { name: /view  
  details/i })[0];
```

```
  userEvent.click(firstProduct);
```

// 验证导航到详情页

```
  expect(screen.getByRole('heading', { name: /product  
  details/i })))
```

```
    .toBeInTheDocument();
```

// 3. 添加到购物车

```
  const addToCartButton = screen.getByRole('button', { name: /add  
  to cart/i });
```

```
  userEvent.click(addToCartButton);
```

// 验证购物车更新

```
  expect(screen.getByRole('status')).toHaveTextContent(/added to  
  cart/i);
```

// 4. 查看购物车

```
  const cartLink = screen.getByRole('link', { name: /cart/i });
```

```
  userEvent.click(cartLink);
```

// 验证购物车内容

```
  expect(screen.getByRole('heading', { name: /shopping cart/i })))  
    .toBeInTheDocument();
```

```
  expect(screen.getByText('1 item')).toBeInTheDocument();
```

```
});
```

常见反模式

JavaScript

// 反模式 1: 直接测试状态

```
test('should update state correctly', () => {  
  // ✗ 直接测试组件的状态  
  const { result } = renderHook(() => useCounter());  
  act(() => {  
    result.current.increment();  
  });  
  expect(result.current.count).toBe(1);  
  
  // ✔ 通过 UI 测试状态变化  
  render(<Counter />);  
  userEvent.click(screen.getByRole('button', { name:  
/increment/i }));  
  expect(screen.getByText('Count: 1')).toBeInTheDocument();  
});
```

// 反模式 2: 过度使用 test-id

```
test('should not overuse test-id', () => {  
  render(<LoginForm />);  
  
  // ✗ 过度依赖 data-testid  
  const emailInput = screen.getByTestId('email-input');  
  const passwordInput = screen.getByTestId('password-input');  
  
  // ✔ 使用语义化查询  
  const emailInput = screen.getByRole('textbox', { name:  
/email/i });  
  const passwordInput = screen.getByLabelText(/password/i);  
});
```

// 反模式 3: 测试内部实现

```
test('should not test implementation', () => {  
  // ✗ 测试内部方法调用  
  const mockOnSubmit = jest.fn();  
  render(<Form onSubmit={mockOnSubmit} />);  
  
  userEvent.click(screen.getByRole('button', { name:  
/submit/i }));  
  expect(mockOnSubmit).toHaveBeenCalledTimes(1);  
  expect(mockOnSubmit).toHaveBeenCalledWith({
```

```
    email: '',
    password: ''
  });

  // ✔ 测试用户可见的结果
  render(<Form />);
  const emailInput = screen.getByRole('textbox', { name:
/email/i });
  const submitButton = screen.getByRole('button', { name:
/submit/i });

  userEvent.type(emailInput, 'test@example.com');
  userEvent.click(submitButton);

  // 验证错误消息或成功消息
  expect(screen.getByText(/email is
required/i)).toBeInTheDocument();
});
```

93. 请解释 CSS 中的层叠上下文是如何形成的，以及 z-index 属性在其中的作用。

层叠上下文形成条件

自动创建层叠上下文的元素

```
SQL
/* 1. 根元素 (HTML) */
html {
  /* 根层叠上下文 */
}

/* 2. position 值不为 static, 且 z-index 不为 auto */
.element {
  position: relative;
  z-index: 1; /* 创建新的层叠上下文 */
}

/* 3. 特定 CSS 属性 */
.create-stacking-context {
```

```
/* opacity 小于 1 */
opacity: 0.99;

/* transform 不为 none */
transform: translateZ(0);

/* filter 不为 none */
filter: blur(0);

/* will-change 指定特定属性 */
will-change: transform;

/* isolation 属性 */
isolation: isolate;

/* mix-blend-mode 不为 normal */
mix-blend-mode: multiply;

/* flex 或 grid 容器的子项，且 z-index 不为 auto */
.container {
  display: flex;
}
.child {
  z-index: 1; /* 创建层叠上下文 */
}
}
```

层叠顺序（从后到前）

```
SQL
/* 层叠顺序示例 */
.stacking-order {
  /* 1. 层叠上下文的背景和边框 */
  background: red;
  border: 2px solid blue;

  /* 2. z-index 为负的子元素 */
  .negative-z-index {
    z-index: -1;
  }

  /* 3. 正常流中的块级元素 */
}
```



```
.block-level {
  display: block;
}

/* 4. 浮动元素 */
.float {
  float: left;
}

/* 5. 正常流中的内联元素 */
.inline {
  display: inline;
}

/* 6. z-index 为 0 或 auto 的定位元素 */
.positioned-zero {
  position: relative;
  z-index: 0;
}

/* 7. z-index 为正的子元素 */
.positive-z-index {
  position: relative;
  z-index: 1;
}
}
```

z-index 的工作原理

基本规则

XML

```
<div class="container1">
  <div class="box1">Box 1 (z-index: 5)</div>
</div>

<div class="container2">
  <div class="box2">Box 2 (z-index: 10)</div>
</div>
```

YAML

```
.container1, .container2 {
  position: relative;
}

.container1 {
  z-index: 2; /* 创建层叠上下文 */
}

.container2 {
  z-index: 1; /* 创建层叠上下文 */
}

.box1 {
  position: absolute;
  z-index: 5; /* 在 container1 的上下文中比较 */
}

.box2 {
  position: absolute;
  z-index: 10; /* 在 container2 的上下文中比较 */
}

/* 结果: box1 在 box2 上面
   因为 container1 的 z-index(2) > container2 的 z-index(1)
   box1 和 box2 的 z-index 只在各自父容器的上下文中比较
*/
```

实际示例

```
XML
<div class="stack-demo">
  <div class="parent1">
    <div class="child1">Child 1</div>
  </div>
  <div class="parent2">
    <div class="child2">Child 2</div>
  </div>
</div>
```

```
YAML
.stack-demo {
  position: relative;
```

```
height: 200px;
}

.parent1 {
  position: relative;
  z-index: 1; /* 创建层叠上下文 1 */
  background: lightblue;
  height: 100px;
}

.parent2 {
  position: relative;
  z-index: 2; /* 创建层叠上下文 2，在上下文 1 之上 */
  background: lightgreen;
  height: 100px;
  margin-top: -50px;
}

.child1 {
  position: absolute;
  z-index: 100; /* 在父级上下文 1 中 */
  background: red;
  top: 40px;
  left: 20px;
  width: 100px;
  height: 100px;
}

.child2 {
  position: absolute;
  z-index: 1; /* 在父级上下文 2 中 */
  background: blue;
  top: 40px;
  left: 40px;
  width: 100px;
  height: 100px;
}

/* 结果:
   child2 在 child1 上面, 因为:
   parent2(z-index:2)在 parent1(z-index:1)上面
   无论 child1 和 child2 的 z-index 值是多少, child2 都在 child1 上面
*/
```

常见问题和解决方案

问题 1: z-index 不生效

```
JavaScript
/* ✗ z-index 不生效的原因 */
.z-index-not-working {
  position: static; /* 定位元素才能用 z-index */
  z-index: 9999; /* 无效! */

  /* 解决方案 */
  position: relative; /* 或 absolute/fixed/sticky */
  z-index: 1;
}
```

问题 2: 父元素截断子元素

```
JSON
/* 父元素创建层叠上下文会限制子元素 */
.parent {
  position: relative;
  z-index: 1; /* 创建层叠上下文 */
  overflow: hidden; /* 也可能创建层叠上下文 */

  .child {
    position: absolute;
    z-index: 9999; /* 不能超出父元素的层叠上下文 */
    /* 这个 z-index 只在 parent 的上下文中有效 */
  }
}

/* 解决方案: 避免不必要的层叠上下文 */
.solution-parent {
  position: relative;
  /* 不设置 z-index, 除非必要 */
  /* 不设置 opacity 小于 1 */
  /* 不设置 transform */
}
```

问题 3: 模态框被遮挡

SQL

```
/* 模态框的最佳实践 */
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  z-index: 1000; /* 高 z-index */
  background: rgba(0, 0, 0, 0.5);

  /* 确保创建层叠上下文 */
  /* opacity: 0.999; 如果需要 */
}

.modal-content {
  position: relative;
  z-index: 1001; /* 在 overlay 之上 */
  /* 模态框内容 */
}

/* 确保没有其他元素创建更高的层叠上下文 */
body.modal-open {
  /* body 可能被添加了 transform 或其他属性 */
  /* 重置可能创建层叠上下文的属性 */
  transform: none !important;
}
```

调试技巧

JavaScript

```
// JavaScript 调试层叠上下文
function debugStackingContext(element) {
  const contexts = [];
  let current = element;

  while (current && current !== document.documentElement) {
    const style = window.getComputedStyle(current);

    // 检查是否创建层叠上下文
    const createsContext =
      style.zIndex !== 'auto' &&
```

```
    ['relative', 'absolute', 'fixed',  
'sticky'].includes(style.position) ||  
    parseFloat(style.opacity) < 1 ||  
    style.transform !== 'none' ||  
    style.filter !== 'none' ||  
    style.willChange.includes('transform') ||  
    style.willChange.includes('opacity') ||  
    style.isolation === 'isolate';
```

```
if (createsContext) {  
    contexts.push({  
        element: current,  
        zIndex: style.zIndex,  
        position: style.position,  
        opacity: style.opacity,  
        transform: style.transform  
    });  
}
```

```
current = current.parentElement;  
}
```

```
return contexts.reverse(); // 从最外层到最内层  
}
```

```
// 使用
```

```
const element = document.querySelector('.problem-element');  
console.log(debugStackingContext(element));
```

```
// CSS 自定义属性记录 z-index
```

```
:root {  
    --z-index-dropdown: 100;  
    --z-index-sticky: 200;  
    --z-index-modal-backdrop: 300;  
    --z-index-modal: 400;  
    --z-index-popover: 500;  
    --z-index-tooltip: 600;  
}
```

```
.modal {  
    z-index: var(--z-index-modal);  
}
```


94. 如何使用 JavaScript 实现一个队列数据结构，并包含入队 和出队 方法？

基础队列实现

数组实现

```
JavaScript
class Queue {
  constructor() {
    this.items = [];
  }

  // 入队
  enqueue(element) {
    this.items.push(element);
    return this;
  }

  // 出队
  dequeue() {
    if (this.isEmpty()) {
      return undefined;
    }
    return this.items.shift();
  }

  // 查看队首元素
  peek() {
    if (this.isEmpty()) {
      return undefined;
    }
    return this.items[0];
  }

  // 查看队尾元素
  rear() {
    if (this.isEmpty()) {
      return undefined;
    }
    return this.items[this.items.length - 1];
  }
}
```

```

}

// 判断队列是否为空
isEmpty() {
  return this.items.length === 0;
}

// 获取队列长度
size() {
  return this.items.length;
}

// 清空队列
clear() {
  this.items = [];
  return this;
}

// 遍历队列
forEach(callback) {
  this.items.forEach(callback);
}

// 转换为数组
toArray() {
  return [...this.items];
}

// 打印队列
toString(separator = ', ') {
  return this.items.join(separator);
}
}

```

链表实现（更高效）

```

JavaScript
class QueueNode {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}

```

```
class LinkedListQueue {
  constructor() {
    this.head = null;
    this.tail = null;
    this.length = 0;
  }

  enqueue(value) {
    const newNode = new QueueNode(value);

    if (this.isEmpty()) {
      this.head = newNode;
      this.tail = newNode;
    } else {
      this.tail.next = newNode;
      this.tail = newNode;
    }

    this.length++;
    return this;
  }

  dequeue() {
    if (this.isEmpty()) {
      return undefined;
    }

    const removedNode = this.head;

    if (this.length === 1) {
      this.head = null;
      this.tail = null;
    } else {
      this.head = this.head.next;
      removedNode.next = null;
    }

    this.length--;
    return removedNode.value;
  }

  peek() {
    if (this.isEmpty()) {
```

```
        return undefined;
    }
    return this.head.value;
}

rear() {
    if (this.isEmpty()) {
        return undefined;
    }
    return this.tail.value;
}

isEmpty() {
    return this.length === 0;
}

size() {
    return this.length;
}

clear() {
    this.head = null;
    this.tail = null;
    this.length = 0;
    return this;
}

forEach(callback) {
    let current = this.head;
    let index = 0;

    while (current) {
        callback(current.value, index, this);
        current = current.next;
        index++;
    }
}

toArray() {
    const result = [];
    this.forEach(value => result.push(value));
    return result;
}
```

```

toString(separator = ', ') {
  return this.toArray().join(separator);
}

// 生成器支持迭代
*[Symbol.iterator]() {
  let current = this.head;
  while (current) {
    yield current.value;
    current = current.next;
  }
}
}

```

高级队列实现

循环队列

```

JavaScript
class CircularQueue {
  constructor(k) {
    this.capacity = k;
    this.items = new Array(k);
    this.head = 0;
    this.tail = 0;
    this.size = 0;
  }

  enqueue(value) {
    if (this.isFull()) {
      return false;
    }

    this.items[this.tail] = value;
    this.tail = (this.tail + 1) % this.capacity;
    this.size++;
    return true;
  }

  dequeue() {
    if (this.isEmpty()) {
      return undefined;
    }
  }
}

```

```
    }

    const value = this.items[this.head];
    this.items[this.head] = undefined; // 可选: 清除引用
    this.head = (this.head + 1) % this.capacity;
    this.size--;
    return value;
}

peek() {
    if (this.isEmpty()) {
        return undefined;
    }
    return this.items[this.head];
}

rear() {
    if (this.isEmpty()) {
        return undefined;
    }
    const rearIndex = (this.tail - 1 + this.capacity) %
this.capacity;
    return this.items[rearIndex];
}

isEmpty() {
    return this.size === 0;
}

isFull() {
    return this.size === this.capacity;
}

getSize() {
    return this.size;
}

clear() {
    this.items = new Array(this.capacity);
    this.head = 0;
    this.tail = 0;
    this.size = 0;
}
```



```

toArray() {
  const result = [];
  for (let i = 0; i < this.size; i++) {
    const index = (this.head + i) % this.capacity;
    result.push(this.items[index]);
  }
  return result;
}
}

```

优先队列

```

JavaScript
class PriorityQueue {
  constructor(comparator = (a, b) => a - b) {
    this.items = [];
    this.comparator = comparator;
  }

  enqueue(value, priority = 0) {
    const element = { value, priority };
    let added = false;

    for (let i = 0; i < this.items.length; i++) {
      if (this.comparator(element.priority,
this.items[i].priority) < 0) {
        this.items.splice(i, 0, element);
        added = true;
        break;
      }
    }

    if (!added) {
      this.items.push(element);
    }

    return this;
  }

  dequeue() {
    if (this.isEmpty()) {
      return undefined;
    }
  }
}

```

```
    return this.items.shift().value;
  }

  peek() {
    if (this.isEmpty()) {
      return undefined;
    }
    return this.items[0].value;
  }

  peekPriority() {
    if (this.isEmpty()) {
      return undefined;
    }
    return this.items[0].priority;
  }

  isEmpty() {
    return this.items.length === 0;
  }

  size() {
    return this.items.length;
  }

  clear() {
    this.items = [];
    return this;
  }

  // 更新优先级
  updatePriority(value, newPriority) {
    const index = this.items.findIndex(item => item.value === value);

    if (index !== -1) {
      // 移除元素
      this.items.splice(index, 1);
      // 重新插入
      this.enqueue(value, newPriority);
    }
  }

  // 转换为数组（按优先级排序）
```

```
toArray() {  
  return this.items.map(item => item.value);  
}  
}
```

实用队列扩展

延迟队列

```
JavaScript  
class DelayedQueue {  
  constructor() {  
    this.queue = new PriorityQueue((a, b) => a.delay - b.delay);  
    this.timer = null;  
    this.callbacks = new Map();  
  }  
  
  // 延迟执行  
  delay(value, delayMs, callback) {  
    const executeAt = Date.now() + delayMs;  
    const id = Symbol();  
  
    this.queue.enqueue({ id, value, delay: executeAt });  
  
    if (callback) {  
      this.callbacks.set(id, callback);  
    }  
  
    this.scheduleNext();  
  
    return id;  
  }  
  
  // 取消延迟任务  
  cancel(id) {  
    const index = this.queue.items.findIndex(item => item.id ===  
id);  
    if (index !== -1) {  
      this.queue.items.splice(index, 1);  
      this.callbacks.delete(id);  
      this.scheduleNext();  
      return true;  
    }  
  }  
}
```

```
}
return false;
}

scheduleNext() {
  if (this.timer) {
    clearTimeout(this.timer);
    this.timer = null;
  }

  if (this.queue.isEmpty()) {
    return;
  }

  const next = this.queue.peek();
  const delay = Math.max(0, next.delay - Date.now());

  if (delay > 0) {
    this.timer = setTimeout(() => {
      const item = this.queue.dequeue();
      const callback = this.callbacks.get(item.id);

      if (callback) {
        callback(item.value);
        this.callbacks.delete(item.id);
      }

      this.scheduleNext();
    }, delay);
  } else {
    // 立即执行
    const item = this.queue.dequeue();
    const callback = this.callbacks.get(item.id);

    if (callback) {
      callback(item.value);
      this.callbacks.delete(item.id);
    }

    this.scheduleNext();
  }
}

clear() {
```

```
    if (this.timer) {  
      clearTimeout(this.timer);  
      this.timer = null;  
    }  
    this.queue.clear();  
    this.callbacks.clear();  
  }  
}
```

批量处理队列

JavaScript

```
class BatchQueue {  
  constructor(options = {}) {  
    this.items = [];  
    this.processing = false;  
  
    this.batchSize = options.batchSize || 10;  
    this.processDelay = options.processDelay || 100; // 毫秒  
    this.processor = options.processor || null;  
  
    this.successCallbacks = [];  
    this.errorCallbacks = [];  
  }  
  
  // 添加项目到队列  
  enqueue(item, onSuccess, onError) {  
    this.items.push(item);  
  
    if (onSuccess) {  
      this.successCallbacks.push(onSuccess);  
    }  
  
    if (onError) {  
      this.errorCallbacks.push(onError);  
    }  
  
    // 如果达到批量大小，立即处理  
    if (this.items.length >= this.batchSize) {  
      this.process();  
    }  
    // 否则设置延迟处理  
    else if (!this.processing) {
```

```
        setTimeout(() => this.process(), this.processDelay);
    }
}

// 处理队列
async process() {
    if (this.processing || this.items.length === 0) {
        return;
    }

    this.processing = true;

    try {
        // 取出当前批次
        const batch = this.items.splice(0, this.batchSize);
        const callbacks = {
            success: this.successCallbacks.splice(0, batch.length),
            error: this.errorCallbacks.splice(0, batch.length)
        };

        // 处理批次
        const results = await this.processor(batch);

        // 调用成功回调
        results.forEach((result, index) => {
            if (callbacks.success[index]) {
                callbacks.success[index](result);
            }
        });

    } catch (error) {
        // 调用错误回调
        const callbacks = this.errorCallbacks.splice(0,
this.batchSize);
        callbacks.forEach(callback => {
            if (callback) {
                callback(error);
            }
        });
    } finally {
        this.processing = false;

        // 如果还有剩余项目，继续处理
    }
}
```



```

        if (this.items.length > 0) {
            setTimeout(() => this.process(), 0);
        }
    }
}

// 手动触发处理
flush() {
    if (this.items.length > 0) {
        return this.process();
    }
}

// 获取队列长度
size() {
    return this.items.length;
}

// 清空队列
clear() {
    this.items = [];
    this.successCallbacks = [];
    this.errorCallbacks = [];
}
}

```

实际应用示例

任务队列系统

```

JavaScript
class TaskQueue {
    constructor(concurrency = 1) {
        this.queue = [];
        this.running = 0;
        this.concurrency = concurrency;
        this.paused = false;
    }

    // 添加任务
    add(task, priority = 0) {
        return new Promise((resolve, reject) => {

```

```
const taskWrapper = {
  task,
  priority,
  resolve,
  reject
};

// 按优先级插入
let inserted = false;
for (let i = 0; i < this.queue.length; i++) {
  if (this.queue[i].priority < priority) {
    this.queue.splice(i, 0, taskWrapper);
    inserted = true;
    break;
  }
}

if (!inserted) {
  this.queue.push(taskWrapper);
}

this.next();
});
}

// 执行下一个任务
async next() {
  if (this.paused || this.running >= this.concurrency ||
this.queue.length === 0) {
    return;
  }

  this.running++;
  const taskWrapper = this.queue.shift();

  try {
    const result = await taskWrapper.task();
    taskWrapper.resolve(result);
  } catch (error) {
    taskWrapper.reject(error);
  } finally {
    this.running--;
    this.next();
  }
}
```

```
}

// 暂停队列
pause() {
  this.paused = true;
}

// 恢复队列
resume() {
  this.paused = false;
  this.next();
}

// 清空队列
clear() {
  this.queue = [];
}

// 获取队列状态
getStatus() {
  return {
    pending: this.queue.length,
    running: this.running,
    paused: this.paused,
    concurrency: this.concurrency
  };
}
}

// 使用示例
async function example() {
  const taskQueue = new TaskQueue(3); // 并发数为 3

  // 添加任务
  const tasks = [
    taskQueue.add(() => fetchData('/api/1'), 1),
    taskQueue.add(() => fetchData('/api/2'), 2),
    taskQueue.add(() => fetchData('/api/3'), 3),
    taskQueue.add(() => fetchData('/api/4'), 1)
  ];

  // 获取结果
  const results = await Promise.all(tasks);
```

```
console.log('所有任务完成:', results);  
}
```